

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



KHOA VIỄN THÔNG 1

BÀI GIẢNG

NHẬP MÔN KỸ THUẬT DỮ LIỆU

Nhóm tác giả: PGS.TS Lê Hải Châu

.

ThS. Nguyễn Hữu Cầm

HÀ NỘI, 2025

THƯ VIỆN PTIT

LỜI MỞ ĐẦU

Trong thời đại số hóa, dữ liệu đã trở thành “nguồn tài nguyên” then chốt, quyết định sự thành công của mọi tổ chức, từ doanh nghiệp, giáo dục, y tế cho đến các hệ thống viễn thông, IoT và trí tuệ nhân tạo. Tuy nhiên, dữ liệu thô tự nó không tạo ra giá trị nếu không được thu thập, xử lý, lưu trữ và phân tích một cách bài bản. Kỹ thuật dữ liệu chính là ngành học cung cấp những công cụ, phương pháp và quy trình để biến dữ liệu thành thông tin hữu ích, từ đó hỗ trợ ra quyết định và xây dựng các hệ thống thông minh.

Bài giảng này được thiết kế để trang bị cho sinh viên chương trình đào tạo kỹ thuật dữ liệu những kiến thức cốt lõi nhất về kỹ thuật dữ liệu, từ các khái niệm cơ bản, vòng đời dữ liệu, đến những công cụ và kỹ thuật phân tích hiện đại. Cấu trúc bài giảng bao gồm bốn chương chính:

- **Chương 1: Tổng quan về Kỹ thuật Dữ liệu** — Hiểu được bối cảnh lớn, vị trí then chốt của kỹ thuật dữ liệu trong hệ sinh thái dữ liệu, và vòng đời đầy đủ của một đơn vị dữ liệu.
- **Chương 2: Công cụ & Nền tảng** — Làm chủ những “cờ lê, tua-vít” của thời đại số: từ Python, R, SQL đến các hệ điều hành và nguyên lý quản trị cơ sở dữ liệu.
- **Chương 3: Học máy & Ứng dụng** — Bước vào thế giới của trí tuệ nhân tạo, nơi dữ liệu được biến thành các mô hình dự đoán thông minh, từ cây quyết định đến mạng nơ-ron, và thấu hiểu cách kỹ thuật dữ liệu hỗ trợ toàn bộ quá trình này.
- **Chương 4: Thu thập, Thử nghiệm & Đánh giá** — Nghệ thuật biến thế giới thực thành dữ liệu có cấu trúc, và các phương pháp khoa học để đánh giá chất lượng, độ tin cậy của dữ liệu và mô hình.

Bài giảng không chỉ cung cấp lý thuyết mà còn kèm theo nhiều bài tập thực hành, ví dụ minh họa và tình huống ứng dụng giúp người đọc từng bước hình thành tư duy kỹ thuật và kỹ năng xử lý dữ liệu. Đây cũng là tài liệu hỗ trợ đắc lực cho sinh viên các ngành Công nghệ thông tin, Viễn thông, Khoa học dữ liệu và những ai mong muốn khởi đầu sự nghiệp trong lĩnh vực dữ liệu.

Chúng tôi tin rằng, với cách tiếp cận hệ thống, ngôn ngữ rõ ràng và nội dung cập nhật, cuốn bài giảng này sẽ là người bạn đồng hành đáng tin cậy trên hành trình chinh phục thế giới dữ liệu của bạn. Đây cũng là phiên bản đầu tiên được viết trong thời gian ngắn do vậy sẽ không tránh khỏi những sai sót. Chúng tôi rất mong nhận được ý kiến

đóng góp của các quý thầy cô, các bạn sinh viên và những người quan tâm để hoàn thiện hơn cuốn tài liệu này.

Nhóm tác giả

THƯ VIỆN PTIT

MỤC LỤC

THUẬT NGỮ & VIẾT TẮT.....	i
CHƯƠNG I: TỔNG QUAN VỀ KỸ THUẬT DỮ LIỆU.....	1
1.1. Giới thiệu chung.....	1
1.1.1. Các khái niệm cơ bản.....	1
1.1.1.1. Khái niệm về dữ liệu và thông tin.....	1
1.1.1.2. Khái niệm về kỹ thuật dữ liệu và khoa học dữ liệu.....	2
1.1.2. Kỹ thuật dữ liệu, khoa học dữ liệu và các lĩnh vực liên quan.....	3
1.1.2.1. Khoa học dữ liệu và thống kê.....	4
1.1.2.2. Khoa học dữ liệu và kỹ thuật.....	5
1.1.2.3. Khoa học dữ liệu và phân tích kinh doanh.....	5
1.1.2.4. Khoa học dữ liệu và khoa học xã hội.....	6
1.1.3. Một số vấn đề và xu hướng của dữ liệu.....	7
1.2. Dữ liệu.....	8
1.2.1. Giới thiệu chung.....	8
1.2.2. Các kiểu dữ liệu.....	9
1.2.2.1. Dữ liệu có cấu trúc.....	9
1.2.2.2. Dữ liệu không có cấu trúc.....	12
1.2.3. Thu thập dữ liệu.....	14
1.2.3.1. Dữ liệu mở.....	14
1.2.3.2. Dữ liệu truyền thông xã hội.....	15
1.2.3.3. Dữ liệu đa phương thức.....	16

1.2.3.4.	Lưu trữ và trình bày dữ liệu.....	17
1.2.4.	Tiền xử lí dữ liệu.....	23
1.2.4.1.	Làm sạch dữ liệu.....	24
1.2.4.2.	Tích hợp dữ liệu.....	26
1.2.4.3.	Biến đổi dữ liệu.....	27
1.2.4.4.	Giảm dữ liệu.....	27
1.2.4.5.	Rời rạc hóa dữ liệu.....	28
1.3.	Các kĩ thuật cơ bản.....	36
1.3.1.	Giới thiệu chung.....	36
1.3.2.	Truyền thông và lưu trữ dữ liệu.....	38
1.3.3.	Phân tích dữ liệu.....	41
1.3.4.	Trực quan hoá dữ liệu.....	44
1.4.	Câu hỏi và bài tập Chương 1.....	47
CHƯƠNG II: CÔNG CỤ VÀ NỀN TẢNG.....		50
2.1.	Ngôn ngữ lập trình Python.....	50
2.1.1.	Giới thiệu chung.....	50
2.1.2.	Cấu trúc điều khiển.....	50
2.1.3.	Thống kê với Python.....	53
2.1.3.1.	Nhập dữ liệu.....	56
2.1.3.2.	Vẽ biểu đồ.....	57
2.1.3.3.	Các mối tương quan.....	57
2.1.3.4.	Hồi quy tuyến tính.....	58
2.1.3.5.	Hồi quy tuyến tính đa biến.....	61
2.1.4.	Học máy với Python.....	66

2.1.4.1.	Học máy là gì?.....	66
2.1.4.2.	Phân loại (Học có giám sát).....	68
2.1.4.3.	Phân cụm (Học không có giám sát).....	71
2.1.4.4.	Ước lượng Mật độ (Học không có giám sát).....	75
2.2.	Ngôn ngữ R.....	77
2.2.1.	Giới thiệu chung.....	77
2.2.2.	Truy cập vào R.....	78
2.2.3.	Bắt đầu với R.....	79
2.2.3.1.	Tính toán cơ bản với R.....	79
2.2.3.2.	Cấu trúc điều khiển.....	81
2.2.3.3.	Hàm.....	83
2.2.3.4.	Nhập dữ liệu.....	84
2.2.4.	Đồ hoạ và trực quan hoá dữ liệu.....	85
2.2.4.1.	Cài đặt ggplot2.....	85
2.2.4.2.	Tải dữ liệu.....	86
2.2.4.3.	Vẽ biểu đồ dữ liệu.....	86
2.2.5.	Thống kê và học máy.....	91
2.2.5.1.	Thống kê cơ bản.....	92
2.2.5.2.	Hồi quy.....	93
2.2.5.3.	Phân loại.....	96
2.2.5.4.	Phân cụm.....	99
2.3.	Cơ sở dữ liệu.....	101
2.3.1.	Giới thiệu chung.....	101
2.3.2.	Phân loại cơ sở dữ liệu.....	102

2.3.3.	Cơ sở dữ liệu có cấu trúc.....	105
2.3.4.	Cơ sở dữ liệu bán cấu trúc.....	107
2.3.5.	Cơ sở dữ liệu phi cấu trúc.....	109
2.4.	Hệ điều hành.....	110
2.4.1.	Giới thiệu chung.....	110
2.4.2.	Unix.....	112
2.4.2.1.	Truy cập UNIX.....	113
2.4.2.2.	Kết nối với máy chủ UNIX.....	116
2.4.2.3.	Các lệnh cơ bản.....	120
2.4.2.4.	Phím tắt.....	123
2.4.3.	Windows.....	124
2.4.3.1.	Khái niệm về ổ đĩa.....	125
2.4.3.2.	Khái niệm về tập tin.....	126
2.4.3.3.	Khái niệm về thư mục.....	127
2.4.3.4.	Khái niệm về đường dẫn.....	129
2.5.	Câu hỏi và bài tập Chương 2.....	130
CHƯƠNG III: HỌC MÁY VÀ ỨNG DỤNG.....		132
3.1.	Giới thiệu chung.....	132
3.2.	Phân loại các kĩ thuật học máy.....	137
3.3.	Học có giám sát.....	138
3.3.1.	Giới thiệu chung.....	138
3.3.2.	Hồi quy Logistic.....	139
3.3.3.	Hồi quy Softmax.....	147
3.3.4.	Phân loại với kNN.....	151

3.3.5.	Phân loại với cây quyết định.....	156
3.3.5.1.	Quy tắc quyết định.....	160
3.3.5.2.	Quy tắc phân loại.....	161
3.3.5.3.	Quy tắc kết hợp.....	162
3.3.6.	Phân loại với phương pháp rừng ngẫu nhiên.....	165
3.3.7.	Phân loại với Naïve Bayes.....	172
3.3.8.	Phân loại với Máy vector hỗ trợ (SVM).....	178
3.4.	Học không giám sát.....	186
3.4.1.	Giới thiệu.....	186
3.4.2.	Phân cụm tích hợp.....	186
3.4.3.	Phân cụm phân chia.....	191
3.4.4.	Tối đa hóa kỳ vọng.....	195
3.4.5.	Học tăng cường.....	204
3.5.	Câu hỏi và bài tập Chương 3.....	208
CHƯƠNG IV: THU THẬP, THỬ NGHIỆM VÀ ĐÁNH GIÁ DỮ LIỆU.....		210
4.1.	Giới thiệu chung.....	210
4.2.	Các phương pháp thu thập dữ liệu.....	210
4.2.1.	Khảo sát.....	210
4.2.2.	Các loại câu hỏi khảo sát.....	211
4.2.3.	Đối tượng khảo sát.....	213
4.2.4.	Các dịch vụ khảo sát.....	214
4.2.5.	Phân tích dữ liệu khảo sát.....	215
4.2.6.	Ưu điểm và nhược điểm của khảo sát.....	216
4.2.7.	Phỏng vấn và nhóm trọng tâm.....	216

4.2.8.	Tại sao nên thực hiện phỏng vấn?.....	217
4.2.9.	Tại sao nên thực hiện nhóm tập trung?.....	217
4.2.10.	Quy trình phỏng vấn.....	218
4.2.11.	Phân tích dữ liệu phỏng vấn.....	219
4.2.12.	Ưu điểm và nhược điểm của phỏng vấn và nhóm trọng tâm.....	219
4.2.13.	Thu thập dữ liệu thông qua nhật ký hoặc nhật trình.....	219
4.2.14.	Các nghiên cứu trong phòng thí nghiệm.....	221
4.3.	Các phương pháp phân tích.....	223
4.3.1.	Giới thiệu chung.....	223
4.3.2.	Phương pháp phân tích định lượng.....	223
4.3.3.	Phương pháp phân tích định tính.....	227
4.3.4.	Kết hợp cả hai phương pháp.....	230
4.4.	Đánh giá.....	231
4.4.1.	So sánh mô hình.....	231
4.4.2.	Huấn luyện – kiểm thử và kiểm thử A/B.....	235
4.4.3.	Xác thực chéo.....	236
4.5.	Câu hỏi và bài tập Chương 4.....	237
TÀI LIỆU THAM KHẢO.....		239

THUẬT NGỮ & VIẾT TẮT

Thuật ngữ	Nghĩa tiếng Anh	Nghĩa tiếng Việt
A/B Testing	A/B Testing	Kiểm thử A/B
API	Application Programming Interface	Giao diện lập trình ứng dụng
Big Data	Big Data	Dữ liệu lớn
BI	Business Intelligence	Trí tuệ doanh nghiệp
CA	Computer Aided	Dựa trên nền tảng máy tính
Cloud Computing	Cloud Computing	Điện toán đám mây
CV	Cross-Validation	Xác minh chéo
CSV	Comma-Separated Values	Giá trị phân tách bằng dấu phẩy
Data Analyst	Data Analyst	Chuyên viên phân tích dữ liệu
Data Cleaning	Data Cleaning	Làm sạch dữ liệu
Data Engineering	Data Engineering	Kỹ thuật dữ liệu
Data Engineer	Data Engineer	Kỹ sư dữ liệu
Data Lake	Data Lake	Hồ dữ liệu
Data Mining	Data Mining	Khai phá dữ liệu
Data Pipeline	Data Pipeline	Luồng dữ liệu
Data Preprocessing	Data Preprocessing	Tiền xử lý dữ liệu
Data Science	Data Science	Khoa học dữ liệu

Data Scientist	Data Scientist	Nhà khoa học dữ liệu
Data Visualization	Data Visualization	Trực quan hóa dữ liệu
Data Warehouse	Data Warehouse	Kho dữ liệu
Dataset	Dataset	Tập dữ liệu
DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu
Decision Tree	Decision Tree	Cây quyết định
EM	Expectation-Maximization	Tối đa kỳ vọng
ETL	Extract, Transform, Load	Trích xuất, Chuyển đổi, Tải
Git	Git	Hệ thống quản lý phiên bản Git
IoT	Internet of Things	Internet vạn vật
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu JSON
k-NN	k-Nearest Neighbors	k Láng giềng gần nhất
Linux/Unix	Linux/Unix	Hệ điều hành Linux/Unix
Logistic Regression	Logistic Regression	Hồi quy logistic
Machine Learning (ML)	Machine Learning	Học máy
Naïve Bayes	Naïve Bayes	Thuật toán Naïve Bayes
NoSQL	Not Only SQL	Cơ sở dữ liệu không chỉ SQL
Operating System (OS)	Operating System	Hệ điều hành

Python	Python	Ngôn ngữ lập trình Python
Qualitative Method	Qualitative Method	Phương pháp định tính
Quantitative Method	Quantitative Method	Phương pháp định lượng
R	R	Ngôn ngữ lập trình R
Random Forest	Random Forest	Rừng ngẫu nhiên
Reinforcement Learning (RL)	Reinforcement Learning	Học tăng cường
SDN	Software-Defined Networking	Mạng định nghĩa bằng phần mềm
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
Supervised Learning	Supervised Learning	Học có giám sát
SVM	Support Vector Machine	Máy vector hỗ trợ
Training-Testing Split	Training-Testing Split	Chia tập huấn luyện - kiểm tra
Unsupervised Learning	Unsupervised Learning	Học không giám sát
Windows	Windows	Hệ điều hành Windows

CHƯƠNG I: TỔNG QUAN VỀ KỸ THUẬT DỮ LIỆU

1.1. Giới thiệu chung

1.1.1. Các khái niệm cơ bản

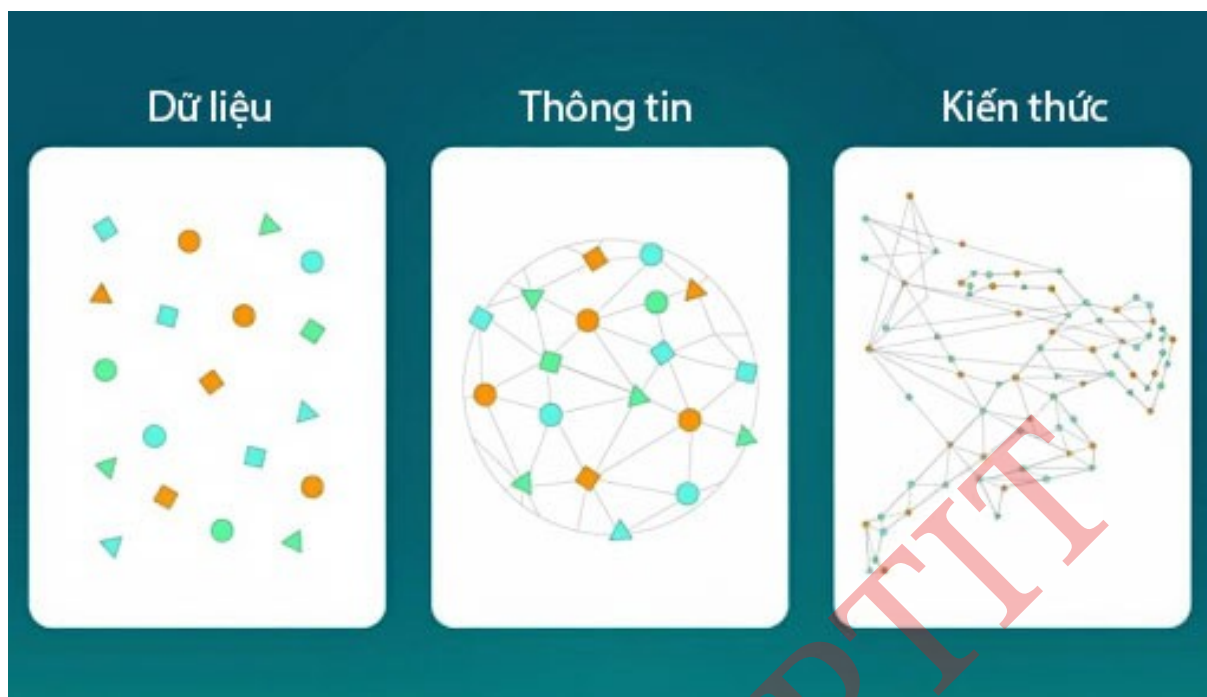
1.1.1.1. Khái niệm về dữ liệu và thông tin

Dữ liệu (Data) là tập hợp các số liệu, ký hiệu, sự kiện hoặc quan sát thô chưa được xử lý, có thể tồn tại ở nhiều dạng khác nhau, từ các con số và văn bản đơn giản đến hình ảnh, âm thanh và video phức tạp. Bản thân dữ liệu không mang ý nghĩa cụ thể nào cho đến khi được xử lý và diễn giải. Các đặc điểm của dữ liệu bao gồm

- **Thô:** Dữ liệu chưa được xử lý và chưa mang ý nghĩa cụ thể.
- **Đa dạng:** Dữ liệu có thể xuất hiện dưới nhiều hình thức khác nhau, từ dạng số liệu, văn bản, hình ảnh cho đến âm thanh và video, tùy thuộc vào cách thức chúng được thu thập và sử dụng.
- **Khách quan:** Dữ liệu thường được thu thập một cách khách quan, không bị ảnh hưởng bởi ý kiến chủ quan.
- **Có thể đo lường:** Dữ liệu thường có thể được đo lường và định lượng.

Trong khi đó, thông tin (Information) là những gì mang lại hiểu biết cho con người về thế giới xung quanh. Đây là kết quả của quá trình xử lý, phân tích và diễn giải dữ liệu giúp người nhận có thể nắm bắt được ý nghĩa cụ thể và giá trị từ dữ liệu đó. Thông tin có thể được biểu diễn dưới nhiều dạng khác nhau, bao gồm văn bản, số liệu, hình ảnh, âm thanh, video, v.v. Thông tin bao gồm các đặc điểm sau:

- **Có ý nghĩa:** Thông tin phải mang một ý nghĩa cụ thể và dễ hiểu đối với người nhận.
- **Có giá trị:** Thông tin phải hữu ích và có thể được sử dụng để đưa ra quyết định hoặc giải quyết vấn đề.
- **Được tổ chức:** Thông tin thường được sắp xếp và cấu trúc theo một logic nhất định để dễ dàng tiếp nhận và xử lý.
- **Kịp thời:** Thông tin cần được cung cấp đúng thời điểm để mang lại giá trị cao nhất.
- **Chính xác:** Thông tin cần phải có độ chính xác cao và đáng tin cậy.



Hình 1: Minh họa dữ liệu (data), thông tin và kiến thức

Điểm khác biệt cốt lõi giữa thông tin và dữ liệu nằm ở ý nghĩa và mục đích sử dụng. Dữ liệu được coi là nguyên liệu thô, trong khi thông tin là sản phẩm đã được xử lý. Dưới đây là bảng so sánh tóm tắt sự khác biệt giữa hai khái niệm này:

Bảng 1: Bảng so sánh giữa dữ liệu và thông tin

Đặc điểm	Dữ liệu	Thông tin
Ý nghĩa	Có tính thô, chưa được xử lý và không mang ý nghĩa cụ thể	Đã được xử lý, có ý nghĩa, mang lại ý nghĩa
Mục đích	Lưu trữ, thu thập	Giúp đưa ra quyết định, giải quyết vấn đề
Hình thức	Con số, văn bản, hình ảnh, âm thanh, video	Báo cáo, biểu đồ, phân tích, tóm tắt
Tính chất	Khách quan	Chủ quan (phụ thuộc vào người nhận)

1.1.1.2. Khái niệm về kĩ thuật dữ liệu và khoa học dữ liệu

Khoa học dữ liệu (Data science) là một lĩnh vực **ngiên cứu** liên quan đến việc thu thập, lưu trữ và xử lý dữ liệu để trích rút những thông tin quan trọng về một vấn đề hoặc hiện tượng. Trong khi đó, **kỹ thuật dữ liệu (Data engineering)** là **hoạt động thiết kế và xây dựng** các hệ thống tổng hợp, lưu trữ và phân tích dữ liệu ở quy mô lớn. Các kỹ sư dữ liệu hỗ trợ các tổ chức xử lý thông tin chi tiết theo thời gian thực từ các tập dữ liệu lớn có thể được tạo bởi con người thông qua khảo sát, đọc nhật ký, v.v. hay dữ liệu từ máy móc gửi đến bao gồm dữ liệu thời tiết, hình ảnh đường xá, v.v ở nhiều định dạng khác nhau (văn bản, âm thanh, video, thực tế tăng cường hoặc thực tế ảo, v.v.).

Dựa trên các số liệu về phương tiện truyền thông xã hội hay tiếp thị đến thống kê hiệu suất nhân viên, dự báo xu hướng, các doanh nghiệp có tất cả dữ liệu cần thiết để tổng hợp cái nhìn toàn diện về hoạt động của mình. Nhiệm vụ của các kỹ sư dữ liệu là chuyển đổi lượng lớn dữ liệu thành những phát hiện chiến lược có giá trị. Với các kỹ thuật dữ liệu phù hợp, các bên liên quan trong toàn tổ chức bao gồm giám đốc điều hành, nhà phát triển, nhà khoa học dữ liệu và nhà phân tích trí tuệ kinh doanh (Business Intelligence) có thể truy cập các tập dữ liệu họ cần bất cứ lúc nào một cách đáng tin cậy, thuận tiện và an toàn. Mỗi bit dữ liệu đều có khả năng cung cấp thông tin cho một quyết định quan trọng. Các kỹ sư dữ liệu giúp quản lý việc quản lý dữ liệu để sử dụng cho các mục đích sau này, bao gồm phân tích, dự báo hoặc học máy. Nhiệm vụ chính của họ bao gồm tạo và triển khai các thuật toán, thiết lập đường ống dữ liệu (data pipeline) và quy trình làm việc để phân loại dữ liệu thô thành các tập dữ liệu mang tính khả dụng cao. Do đó, trong các doanh nghiệp, kỹ thuật dữ liệu là một thành phần không thể thiếu của nền tảng dữ liệu hiện đại, cho phép doanh nghiệp phân tích và áp dụng dữ liệu họ nhận được, bất kể nguồn dữ liệu hay định dạng nào, và từ đó đưa ra quyết định chính xác, cải thiện hiệu suất và tối ưu hóa các quy trình kinh doanh. Kỹ thuật dữ liệu cũng giúp doanh nghiệp quản lý và bảo vệ dữ liệu một cách hiệu quả, đảm bảo tính toàn vẹn và bảo mật trong suốt vòng đời của dữ liệu.

Tại sao khoa học dữ liệu lại quan trọng đến vậy vào thời điểm hiện nay? Tiến sĩ Tara Sinclair, nhà kinh tế trưởng của Indeed.com từ năm 2013 đã nói rằng: “Số lượng các tin tuyển dụng cho vị trí ‘kỹ sư dữ liệu’ đã tăng 57% mỗi năm trong quý đầu tiên của năm 2015.”. Tại sao cả ngành công nghiệp và học thuật lại gần đây gia tăng nhu cầu về khoa học dữ liệu và các nhà khoa học dữ liệu? Hiện nay, lượng dữ liệu đang ngày càng gia tăng và tiếp tục được tạo ra một cách liên tục. Việc phân tích dữ liệu một cách thông minh đòi hỏi sự tham gia của những người có năng lực và được đào tạo bài bản, và việc phân tích dữ liệu có thể cung cấp những thông tin quan trọng để đưa ra quyết định đúng đắn.

1.1.2. Kỹ thuật dữ liệu, khoa học dữ liệu và các lĩnh vực liên quan

Kỹ thuật dữ liệu và khoa học dữ liệu là hai lĩnh vực có sự giao thoa mạnh mẽ nhưng cũng có những đặc điểm riêng biệt. Kỹ thuật dữ liệu tập trung vào việc thu thập, xử lý và lưu trữ dữ liệu hiệu quả, giúp đảm bảo tính sẵn sàng và chất lượng của dữ liệu phục vụ cho các mục đích phân tích. Trong khi đó, khoa học dữ liệu không chỉ dừng lại ở việc xử lý dữ liệu mà còn khai thác, phân tích và rút ra những thông tin giá trị từ dữ liệu thông qua các phương pháp thống kê, học máy và các mô hình dự báo. Các lĩnh vực liên quan như trí tuệ nhân tạo (Artificial Intelligence), học máy (Machine Learning), và phân tích dữ liệu (data analytics) đóng vai trò quan trọng trong việc phát triển và ứng dụng các kỹ thuật dữ liệu, cung cấp nền tảng vững chắc để giải quyết những vấn đề phức tạp trong các ngành công nghiệp và nghiên cứu khoa học.

1.1.2.1. Khoa học dữ liệu và thống kê

Khoa học dữ liệu và thống kê là hai lĩnh vực có sự liên quan mật thiết, với thống kê đóng vai trò quan trọng trong nền tảng lý thuyết của khoa học dữ liệu, nhưng lại có sự phát triển và ứng dụng khác biệt trong bối cảnh hiện đại. Thống kê, từ lâu đã được biết đến như một công cụ để phân tích và giải thích dữ liệu trong các nghiên cứu khoa học, kinh tế, xã hội, v.v., chủ yếu tập trung vào việc rút ra các kết luận chính xác từ các mẫu dữ liệu, kiểm tra giả thuyết và ước lượng các tham số. Những phương pháp thống kê truyền thống, như kiểm định giả thuyết, phân tích hồi quy và phân tích phương sai, được sử dụng để phân tích và đưa ra kết luận về các mối quan hệ trong dữ liệu. Những kỹ thuật này vẫn còn vô cùng quan trọng trong khoa học dữ liệu ngày nay, đặc biệt là trong các lĩnh vực yêu cầu độ chính xác cao và tính khoa học trong phân tích.

Tuy nhiên, khoa học dữ liệu không chỉ dừng lại ở thống kê. Dưới sự phát triển mạnh mẽ của công nghệ máy tính và sự gia tăng khối lượng dữ liệu khổng lồ trong thời đại số, khoa học dữ liệu đã mở rộng ra ngoài phạm vi thống kê truyền thống, bao gồm cả việc thu thập, xử lý, lưu trữ, và trực quan hóa dữ liệu. Khoa học dữ liệu không chỉ quan tâm đến việc phân tích dữ liệu mà còn tập trung vào việc sử dụng các công cụ và kỹ thuật máy tính để làm sạch, sắp xếp và truy xuất dữ liệu từ các cơ sở dữ liệu lớn. Hơn nữa, khoa học dữ liệu hiện nay đòi hỏi kiến thức vững vàng về lập trình, đặc biệt là trong việc sử dụng các ngôn ngữ lập trình có khả năng phân tích dữ liệu như Python, R và SQL, để thực hiện các phép toán phức tạp trên các bộ dữ liệu khổng lồ.

Mối quan hệ giữa khoa học dữ liệu và thống kê còn thể hiện rõ trong các phương pháp học máy. Các thuật toán học máy như hồi quy tuyến tính, phân cụm, và các mô hình xác suất đều dựa trên các nguyên lý thống kê để xây dựng các mô hình dự báo và phân loại. Tuy nhiên, học máy không chỉ sử dụng các phương pháp thống kê mà còn kết

hợp với các kỹ thuật tính toán để cải thiện khả năng dự đoán, tìm kiếm mối quan hệ ẩn trong dữ liệu, và tối ưu hóa các mô hình thông qua việc học từ dữ liệu.

Andrew Gelman, một nhà thống kê nổi tiếng tại Đại học Columbia, đã từng chia sẻ rằng "thống kê có thể được xem là một phần của khoa học dữ liệu", nhưng theo ông, các kỹ thuật quản lý và xử lý dữ liệu, như thu thập, làm sạch, và lưu trữ dữ liệu, lại là các yếu tố quan trọng hơn đối với khoa học dữ liệu. Điều này cho thấy sự chuyển mình của khoa học dữ liệu từ một lĩnh vực phụ thuộc nhiều vào thống kê sang một lĩnh vực độc lập, có sự kết hợp mạnh mẽ giữa thống kê, lập trình và khả năng xử lý dữ liệu lớn.

1.1.2.2. Khoa học dữ liệu và kỹ thuật

Kỹ thuật trong các lĩnh vực khác nhau (hóa học, xây dựng, máy tính, cơ khí, v.v.) đã tạo ra nhu cầu lớn đối với các nhà khoa học dữ liệu và các phương pháp khoa học dữ liệu. Các kỹ sư luôn cần dữ liệu để giải quyết các vấn đề. Các nhà khoa học dữ liệu được yêu cầu phát triển các phương pháp và kỹ thuật để đáp ứng nhu cầu này. Tương tự, các kỹ sư hỗ trợ các nhà khoa học dữ liệu bằng cách giúp họ tận dụng tối đa các phần mềm và phần cứng mới, chẳng hạn như CPU (bộ vi xử lý trung tâm) và GPU (bộ xử lý đồ họa), qua đó giúp giảm đáng kể thời gian tính toán.

Trong những năm gần đây, công nghệ đã tạo ra những bước tiến lớn khi được ứng dụng trong nhiều ngành nghề khác nhau. Cụ thể, trong ngành xây dựng, xu hướng đã có sự thay đổi rõ rệt nhờ vào việc áp dụng công nghệ trong suốt vài thập kỷ qua. Hiện nay, nhờ vào việc thu thập và phân tích lượng lớn dữ liệu, các kỹ thuật xây dựng "thông minh" đã được triển khai một cách triệt để. Các thuật toán dự báo giúp ước tính chi phí xây dựng chỉ dựa trên giá của các thành phần cụ thể, như lan can, và các nhà thầu có thể đưa ra đấu thầu dựa trên các yếu tố như vị trí, thời gian trong năm, tổng giá trị dự án, chỉ số chi phí liên quan, v.v. Bên cạnh đó, các kỹ thuật xây dựng "thông minh" đã được giới thiệu thông qua việc sử dụng nhiều công nghệ khác nhau. Từ việc các mô hình in 3D có thể giúp dự đoán các điểm yếu trong kết cấu công trình xây dựng như các khu vực có khả năng bị nứt, lún, hoặc không đáp ứng đủ độ bền, cho đến việc sử dụng máy bay không người lái (drone) để giám sát công trường trong quá trình thi công thực tế. Tất cả các công nghệ này tạo ra một lượng lớn dữ liệu cần được phân tích để tối ưu hóa thiết kế và điều chỉnh các hoạt động xây dựng. Vì vậy, vai trò của khoa học dữ liệu trở nên thiết yếu khi việc sử dụng công nghệ trong thiết kế và ứng dụng kỹ thuật ngày càng gia tăng.

1.1.2.3. Khoa học dữ liệu và phân tích kinh doanh

Nhìn chung, mục tiêu chính của kinh doanh là tạo ra lợi nhuận ngay cả với các nguồn lực hạn chế thông qua các phương pháp sản xuất hiệu quả và bền vững, cùng với các mô

hình dịch vụ hiệu quả. Điều này đòi hỏi việc ra quyết định dựa trên đánh giá khách quan, và phân tích dữ liệu là yếu tố thiết yếu phục vụ cho mục tiêu này.

Dữ liệu kinh doanh (dù là từ các công ty hay khách hàng) ngày càng trở nên phổ biến và dễ dàng hơn trong việc thu thập, lưu trữ và xử lý do chi phí ngày càng giảm. Bên cạnh các loại dữ liệu truyền thống hiện nay đang được số hóa thông qua các quy trình tự động, các loại dữ liệu mới từ thiết bị di động, cảm biến đeo và các hệ thống nhúng đang cung cấp cho các doanh nghiệp rất nhiều thông tin phong phú. Các công nghệ mới đã ra đời giúp các công ty và tổ chức hiểu được lượng dữ liệu này, đặc biệt trong phân tích kinh doanh

Phân tích kinh doanh (Business Analytics – BA) đề cập đến các kỹ năng, công nghệ và phương pháp thực hành cho việc khám phá và nghiên cứu liên tục về hiệu suất kinh doanh trong quá khứ và hiện tại để thu được thông tin chi tiết và mang tính chiến lược. Phân tích kinh doanh tập trung vào việc phát triển các quan điểm mới và hiểu rõ về hiệu suất dựa trên dữ liệu và thống kê. Và đây chính là lúc vai trò của khoa học dữ liệu trở nên đặc biệt. Để đáp ứng các yêu cầu phân tích dữ liệu, các kỹ sư ngày càng đóng vai trò quan trọng trong mô hình hóa các hoạt động thống kê, bao gồm xây dựng các mô hình giải thích và dự đoán và quản lý dựa trên dữ liệu, giúp thúc đẩy việc ra quyết định thành công.

Phân tích kinh doanh được chia thành 4 nhóm chính, mỗi nhóm đều mang đến cơ hội cho các nhà khoa học dữ liệu trong phân tích:

- Phân tích quyết định: hỗ trợ ra quyết định với phân tích trực quan phản ánh lý luận.
- Phân tích mô tả: cung cấp cái nhìn từ dữ liệu trong quá khứ với các báo cáo hay phân nhóm, v.v.
- Phân tích dự đoán: sử dụng mô hình dự đoán thông qua các kỹ thuật thống kê và học máy.
- Phân tích đề xuất: đưa ra các quyết định bằng cách sử dụng tối ưu hóa, mô phỏng, v.v.

1.1.2.4. Khoa học dữ liệu và khoa học xã hội

Khoa học xã hội (social science), vốn ra đời gần bốn thế kỷ trước và chủ yếu quan tâm đến xã hội cùng mối quan hệ giữa các cá nhân lại có liên quan mật thiết đến khoa học dữ liệu. Đến thế kỷ XXI, không chỉ khoa học dữ liệu hỗ trợ khoa học xã hội, mà còn định hình lại, thậm chí tạo ra một nhánh mới gọi là khoa học xã hội tính toán (computational social science).

Kể từ khi ra đời, khoa học xã hội đã mở rộng ra rất nhiều lĩnh vực, không chỉ ở nhân chủng học, khảo cổ học, kinh tế học, ngôn ngữ học mà còn ở khoa học chính trị, tâm lý

học, y tế cộng đồng và xã hội học. Mỗi lĩnh vực này đã thiết lập các tiêu chuẩn, quy trình và phương thức thu thập dữ liệu riêng biệt trong suốt những năm qua. Tuy nhiên, việc kết nối các lý thuyết hoặc kết quả từ một lĩnh vực này sang lĩnh vực khác đang ngày càng trở nên khó khăn. Đây chính là yếu tố mà khoa học xã hội tính toán đã cách mạng hóa nghiên cứu khoa học xã hội trong vài thập kỷ qua. Với sự trợ giúp của khoa học dữ liệu, khoa học xã hội tính toán đã kết nối các kết quả từ nhiều lĩnh vực để khám phá câu hỏi quan trọng và cấp bách: cuộc cách mạng thông tin trong thời đại số sẽ biến đổi xã hội như thế nào?

Khoa học xã hội tính toán đã có những bước tiến vượt bậc trong việc tạo ra các dự án liên ngành, thường là sự hợp tác với các nhà khoa học máy tính, nhà thống kê, nhà toán học, và gần đây là các nhà khoa học dữ liệu. Một số dự án này bao gồm việc ứng dụng các công cụ và thuật toán dự báo và học máy để giúp giải quyết những vấn đề chính sách phức tạp. Trong khi các dự án khác áp dụng các tiến bộ gần đây trong nhận dạng hình ảnh, văn bản và giọng nói vào các vấn đề cổ điển trong khoa học xã hội. Những dự án này thường yêu cầu các bước đột phá về phương pháp luận, mở rộng các phương pháp đã chứng minh ở các cấp độ mới, cũng như thiết kế các chỉ số và giao diện mới làm cho kết quả nghiên cứu trở nên dễ hiểu đối với các học giả, nhà quản lý và các nhà hoạch định chính sách, những người có thể thiếu kỹ năng tính toán nhưng lại có chuyên môn sâu về chuyên ngành. Khoa học xã hội tính toán đặt ra những câu hỏi không thể tránh khỏi về chính trị và đạo đức thường được ẩn chứa trong nghiên cứu khoa học dữ liệu, đặc biệt khi dựa trên các vấn đề xã hội chính trị có ứng dụng thực tế và những hệ quả sâu rộng. Các chính sách của chính phủ, các mệnh lệnh của người dân trong các cuộc bầu cử, và chiến lược tuyển dụng trong khu vực tư nhân là những ví dụ điển hình cho những ứng dụng như vậy.

1.1.3. Một số vấn đề và xu hướng của dữ liệu

Nhiều vấn đề liên quan đến quyền riêng tư, thành kiến và đạo đức có thể bắt nguồn từ chính nguồn gốc của dữ liệu đang được thu thập. Các câu hỏi như dữ liệu được thu thập như thế nào, ở đâu và vì mục đích gì? Ai là người thu thập dữ liệu và họ dự định sử dụng dữ liệu đó ra sao? Và quan trọng hơn, nếu dữ liệu được thu thập từ con người, liệu những người này có nhận thức được rằng: (1) dữ liệu về họ đang được lấy đi; và (2) dữ liệu đó sẽ được sử dụng như thế nào? Thường thì những người thu thập dữ liệu lại nhầm lẫn việc dữ liệu có sẵn với quyền sử dụng dữ liệu đó. Ví dụ, sự sẵn có của dữ liệu trên các dịch vụ mạng xã hội như X (trước đây là Twitter) không đồng nghĩa với việc được phép thu thập

và kinh doanh dữ liệu đó mà không có sự chấp thuận của người sử dụng dịch vụ. Tháng 4 năm 2018, một vụ việc đã xảy ra khi một công ty phân tích dữ liệu, Cambridge Analytica, thu thập dữ liệu cá nhân của một số lượng lớn người dùng Facebook để sử dụng cho chiến dịch chính trị. Những người dùng Facebook này thậm chí không biết rằng: (1) dữ liệu về họ đã được Facebook thu thập và chia sẻ với các bên thứ ba; và (2) dữ liệu này đã được sử dụng để nhắm mục tiêu quảng cáo chính trị đến họ. Sự việc này đã làm sáng tỏ một vấn đề không hề mới; trong nhiều năm qua, các công ty như Facebook và Google đã thu thập một lượng dữ liệu khổng lồ từ hành vi người dùng, không chỉ để cải thiện và tiếp thị sản phẩm mà còn để chia sẻ và/hoặc bán dữ liệu đó cho các tổ chức khác để kiếm lợi nhuận, điều mà phần lớn công chúng đều không hề hay biết.

Trong thực tế, đã có vô số trường hợp dữ liệu người dùng bị tiết lộ hoặc chia sẻ—dù cố ý hay vô tình—gây ra những tổn hại đáng kể. Một vấn đề thường bị bỏ qua là: ngay cả những dữ liệu được thu thập tuân thủ nguyên tắc đạo đức vẫn tiềm ẩn sự đa dạng về ý kiến và quan điểm. Nếu các nhà khoa học dữ liệu thiếu thận trọng, những thành kiến (bias) này có thể len lỏi vào quá trình phân tích và làm sai lệch những hiểu biết (insights) được rút ra. Tuy nhiên, điều đáng khích lệ là nhiều công ty dữ liệu và công nghệ đang tích cực nỗ lực giải quyết những thách thức này, hướng tới một quy trình xử lý dữ liệu minh bạch và công bằng hơn.

Trong kỷ nguyên số, dữ liệu không chỉ bùng nổ về khối lượng mà còn thay đổi mạnh mẽ về cách thức ứng dụng và quản lý. Xu hướng nổi bật đầu tiên là sự trỗi dậy của Big Data, nơi dữ liệu không chỉ được thu thập nhanh chóng (tốc độ cao) mà còn tồn tại dưới nhiều dạng thức đa dạng (văn bản, hình ảnh, video), đòi hỏi các giải pháp lưu trữ và xử lý phân tán trên nền tảng điện toán đám mây (Cloud). Quan trọng hơn, dữ liệu đã trở thành "nhiên liệu" chính cho các ứng dụng Trí tuệ Nhân tạo (AI) và Học máy (ML), thúc đẩy khả năng dự đoán và cá nhân hóa trong mọi lĩnh vực. Tuy nhiên, sự phát triển này đi kèm với thách thức về đạo đức và quyền riêng tư, buộc các tổ chức phải tuân thủ nghiêm ngặt các quy định quốc tế và chủ động kiểm tra để loại bỏ thiên kiến khỏi các bộ dữ liệu, đảm bảo tính công bằng và minh bạch cho các thuật toán.

1.2. Dữ liệu

1.2.1. Giới thiệu chung

“Cây cối chính nguyên liệu thô để sản xuất giấy, dữ liệu cũng có thể được xem như nguyên liệu để trích xuất thông tin”. Để trình bày và giải thích thông tin một cách hiệu quả, quá trình thu thập và phân loại dữ liệu phải là bước khởi đầu. Theo đó, đối với bất kỳ hình thức phân tích dữ liệu nào, việc xác định các nguồn thông tin phù hợp luôn là ưu tiên hàng đầu.

Các ví dụ điển hình của dữ liệu bao gồm: dữ liệu đa phương tiện như một bức ảnh được chụp bằng điện thoại thông minh, hoặc dữ liệu dạng số và có cấu trúc như các số liệu chi tiết trong báo cáo bán hàng. Ngoài ra, còn tồn tại các bộ dữ liệu mở – cho phép mọi người truy cập không giới hạn và sử dụng miễn phí, nhằm khuyến khích sự chia sẻ thông tin và thúc đẩy nghiên cứu sáng tạo. Những bộ dữ liệu này thường được cung cấp bởi các tổ chức chính phủ, các tổ chức nghiên cứu, hoặc các nền tảng công nghệ, với mục tiêu hỗ trợ cộng đồng trong việc giải quyết các vấn đề xã hội, khoa học, hoặc công nghệ. Ngược lại, dữ liệu đóng là những bộ dữ liệu không công khai và bị hạn chế truy cập. Chúng thường được bảo vệ chặt chẽ bằng các chính sách bảo mật và quyền sở hữu trí tuệ. Các bộ dữ liệu đóng này có thể được thu thập bởi các công ty hoặc tổ chức và được sử dụng cho các mục đích nội bộ, chẳng hạn như phát triển sản phẩm hoặc phân tích kinh doanh, và thường không chia sẻ ra ngoài trừ khi có thỏa thuận hợp đồng rõ ràng.

1.2.2. Các kiểu dữ liệu

Một trong những phương pháp cơ bản nhất để hiểu về dữ liệu là xác định liệu dữ liệu đó có cấu trúc hay không. Điều này đặc biệt quan trọng đối với khoa học dữ liệu vì hầu hết các kỹ thuật xử lý đều phụ thuộc vào đặc tính vốn có của dữ liệu, có cấu trúc hoặc không có cấu trúc.

Dữ liệu có cấu trúc thường đề cập đến thông tin được tổ chức một cách có hệ thống, dễ dàng lưu trữ trong cơ sở dữ liệu và tìm kiếm thông qua các thao tác tìm kiếm đơn giản. Ngược lại, dữ liệu không có cấu trúc thiếu đi cấu trúc cơ bản khi so sánh với dữ liệu có cấu trúc. Trong dữ liệu có cấu trúc, các giá trị – cho dù là số hay bất kỳ loại giá trị nào khác – đều được gán nhãn, điều này không áp dụng với dữ liệu không có cấu trúc.

1.2.2.1. Dữ liệu có cấu trúc

Dữ liệu có cấu trúc là dữ liệu có định dạng chuẩn hóa để con người cũng như phần mềm có thể truy cập một cách hiệu quả. Loại dữ liệu này thường ở dạng bảng, bao gồm các hàng và cột xác định rõ ràng các thuộc tính dữ liệu. Với tính chất định lượng, dữ liệu có cấu trúc có thể được máy tính xử lý hiệu quả để thu thập thông tin chi tiết. Ví dụ: bảng dữ liệu có cấu trúc về khách hàng bao gồm các cột tên, địa chỉ và số điện thoại có thể cung cấp các thông tin chi tiết như tổng số khách hàng và khu vực có nhiều khách hàng nhất. Dưới đây là một bảng mẫu dữ liệu khách hàng chứa các dữ liệu có cấu trúc:

Bảng 2: Mẫu dữ liệu khách hàng

Mã khác	Giới tính	Có việc làm	Thu nhập	Tình trạng hôn nhân	Loại hình nhà ở	Số lượng	Tuổi	Bang sinh
---------	-----------	-------------	----------	---------------------	-----------------	----------	------	-----------

h hàng						xe		sống
2068	Nữ	Không	11300	Đã kết hôn	Chủ sở hữu	2	49	Michigan
2073	Nữ	Không	0	Đã kết hôn	Thuê	3	40	Florida
2848	Nam	Có	4500	Chưa kết hôn	Thuê	3	22	Georgia
5641	Nam	Có	20000	Chưa kết hôn	Đang ở, không phải trả tiền	0	22	New Mexico
6369	Nữ	Có	12000	Chưa kết hôn	Thuê	1	31	Florida

Trong đó:

- **Mã khách hàng:**
 - **Kiểu dữ liệu:** Số nguyên (Integer)
 - **Giải thích:** Đây là một mã định danh duy nhất cho mỗi khách hàng.
- **Giới tính:**
 - **Kiểu dữ liệu:** Chuỗi văn bản (String)
 - **Giải thích:** Giới tính của khách hàng, có thể là "Nam" hoặc "Nữ".
- **Có việc làm:**
 - **Kiểu dữ liệu:** Chuỗi văn bản (String)
 - **Giải thích:** Thể hiện tình trạng việc làm của khách hàng, có thể là "Có" hoặc "Không".
- **Thu nhập:**
 - **Kiểu dữ liệu:** Số thực
 - **Giải thích:** Thu nhập của khách hàng, biểu thị bằng giá trị số.
- **Tình trạng hôn nhân:**
 - **Kiểu dữ liệu:** Chuỗi văn bản (String)
 - **Giải thích:** Tình trạng hôn nhân của khách hàng, có thể là "Đã kết hôn" hoặc "Chưa kết hôn".
- **Loại hình nhà ở:**
 - **Kiểu dữ liệu:** Chuỗi văn bản (String)
 - **Giải thích:** Loại hình nhà ở của khách hàng, có thể là "Chủ sở hữu", "Thuê", hoặc "Đang ở, không phải trả tiền".
- **Số lượng xe:**
 - **Kiểu dữ liệu:** Số nguyên (Integer)

- **Giải thích:** Số lượng xe của khách hàng.
- **Tuổi:**
 - **Kiểu dữ liệu:** Số nguyên (Integer)
 - **Giải thích:** Tuổi của khách hàng.
- **Bang:**
 - **Kiểu dữ liệu:** Chuỗi văn bản (String)
 - **Giải thích:** Tên bang nơi khách hàng sinh sống (ví dụ: "Michigan", "Florida", "Georgia", "New Mexico").

Dữ liệu có cấu trúc là loại dữ liệu được định hình bởi các thuộc tính rõ ràng, cho phép việc xác định và quản lý trở nên dễ dàng. Mỗi bản ghi trong dữ liệu có cấu trúc đều có chung thuộc tính mô tả, chẳng hạn như tên sự kiện, ngày diễn ra sự kiện và số tiền đặt trước. Ngoài ra, một đặc điểm nổi bật khác của dữ liệu có cấu trúc là khả năng thiết lập mối quan hệ giữa các bảng thông qua các thuộc tính chung, ví dụ như liên kết dữ liệu khách hàng với dữ liệu đặt trước nhằm xác định ai là người đã đặt bàn trước. Vì có cấu trúc rõ ràng, loại dữ liệu này thường được lưu trữ hiệu quả trong các cơ sở dữ liệu quan hệ. Ngoài ra, dữ liệu có cấu trúc còn nổi bật với tính định lượng cao, cho phép áp dụng dễ dàng các phương pháp phân tích toán học như đếm, đo lường tần suất và thực hiện các phép tính. Việc lưu trữ dữ liệu có cấu trúc thường được thực hiện trong cơ sở dữ liệu quan hệ và quản lý bằng ngôn ngữ truy vấn SQL, cho phép định nghĩa lược đồ dữ liệu với các quy tắc về trường, định dạng và giá trị. Từ đó, dữ liệu có thể được lưu trữ và khai thác hiệu quả trong kho dữ liệu hoặc các công nghệ cơ sở dữ liệu quan hệ khác.

Dưới đây là một số ví dụ điển hình về nơi lưu trữ hoặc định dạng của dữ liệu có cấu trúc:

- **Bảng tính:** Các tệp như Microsoft Excel, Google Sheets, nơi dữ liệu được sắp xếp theo hàng và cột.
- **Cơ sở dữ liệu quan hệ (RDBMS):** Hệ thống được thiết kế để lưu trữ dữ liệu theo các bảng có mối quan hệ xác định (ví dụ: MySQL, PostgreSQL).
- **Dữ liệu Giao dịch:** Thông tin chi tiết từ hệ thống điểm bán hàng (POS) hoặc các giao dịch thương mại điện tử.
- **Biểu mẫu web:** Kết quả thu thập từ các biểu mẫu đăng ký, khảo sát trực tuyến, nơi các trường thông tin đã được xác định trước.

Việc sử dụng dữ liệu có cấu trúc mang lại nhiều lợi ích đáng kể cho doanh nghiệp. Đầu tiên, dữ liệu có cấu trúc rất dễ sử dụng, với khả năng truy cập và cập nhật nhanh chóng, giúp việc lưu trữ hiệu quả nhờ vào các đơn vị lưu trữ có độ dài cố định. Đáng chú ý, dữ liệu có cấu trúc cung cấp khả năng mở rộng vượt trội. Nhờ đó, ngay cả khi khối lượng dữ liệu tăng đột biến, việc tăng cường dung lượng lưu trữ và khả năng xử lý vẫn có

thể được thực hiện một cách hiệu quả và thuận lợi. Hệ thống hiện đại có thể xử lý dữ liệu có cấu trúc lên tới hàng nghìn terabyte. Bên cạnh đó, việc phân tích dữ liệu cũng trở nên dễ dàng nhờ các thuật toán học máy hay sử dụng ngôn ngữ truy vấn SQL, giúp doanh nghiệp có thể tạo báo cáo và khai thác dữ liệu hiệu quả. Dữ liệu có cấu trúc đặc biệt hữu ích khi phân tích dữ liệu lớn, cung cấp những mẫu chung giúp đưa ra quyết định kinh doanh chính xác hơn.

Tuy nhiên, việc sử dụng dữ liệu có cấu trúc cũng không thiếu những thách thức. Một trong những khó khăn lớn là giới hạn mức sử dụng của dữ liệu. Do cấu trúc đã được định sẵn, dữ liệu có cấu trúc chỉ có thể phục vụ cho những mục đích đã xác định trước. Điều này gây trở ngại lớn nếu muốn mở rộng phạm vi phân tích sang các mục tiêu mới mà không cần thay đổi cấu trúc dữ liệu cơ bản. Ví dụ: dữ liệu giao dịch hoặc yêu cầu đặt chỗ trước trong các hệ thống dịch vụ có thể cung cấp thông tin về tài chính và mức phổ biến của ứng dụng. Tuy nhiên, dữ liệu này không cho biết chiến dịch tiếp thị nào hiệu quả hơn trong việc mang lại nhiều lượt đặt hơn, trừ khi bổ sung thêm thông tin về các chiến dịch tiếp thị liên quan khác. Điều này gây khó khăn trong việc đánh giá hiệu quả của các chiến dịch tiếp thị mà không có dữ liệu bổ sung, vì dữ liệu đặt chỗ trước chỉ phản ánh kết quả cuối cùng mà không cung cấp thông tin chi tiết về các yếu tố thúc đẩy hành vi khách hàng. Một khó khăn khác của kiểu dữ liệu này là sự thiếu linh hoạt khi việc thay đổi thiết kế dữ liệu để đáp ứng yêu cầu mới có thể tốn kém nguồn lực, thời gian và gặp phải nhiều vấn đề về tài nguyên trong việc cập nhật các dữ liệu khác, đặc biệt trong trường hợp các mối quan hệ giữa các thực thể trong thực tế ngày càng phức tạp và yêu cầu thay đổi theo thời gian.

1.2.2.2. Dữ liệu không có cấu trúc

Dữ liệu không có cấu trúc (Unstructured data) không có định dạng cố định như dữ liệu có cấu trúc. Các bộ dữ liệu không có cấu trúc thường có dung lượng lớn (tính bằng terabyte hoặc petabyte) và chiếm 90% tổng lượng dữ liệu được tạo ra bởi các doanh nghiệp. Khối lượng lớn này là do sự xuất hiện của dữ liệu lớn (big data) — các bộ dữ liệu khổng lồ và phức tạp từ internet và các công nghệ kết nối khác.

Dữ liệu không có cấu trúc có thể chứa cả dữ liệu văn bản và phi văn bản, cũng như cả dữ liệu định tính (ví dụ: bình luận trên mạng xã hội) và dữ liệu định lượng (ví dụ: số liệu nhúng trong văn bản).

Các nguồn dữ liệu văn bản phi cấu trúc (Unstructured Text Data) rất đa dạng. Dưới đây là một số ví dụ điển hình:

- **Tài liệu và Thư tín:** Email, các loại Tài liệu văn bản (như Word, PDF) và các hồ sơ dạng văn bản khác.

- **Truyền thông Xã hội:** Bài viết và bình luận trên các nền tảng mạng xã hội.
- **Giao tiếp Nội bộ:** Tập tin tin nhắn từ các ứng dụng cộng tác (ví dụ: Microsoft Teams, Slack), cùng với Biên bản cuộc gọi hoặc ghi âm đã được chuyển ngữ.

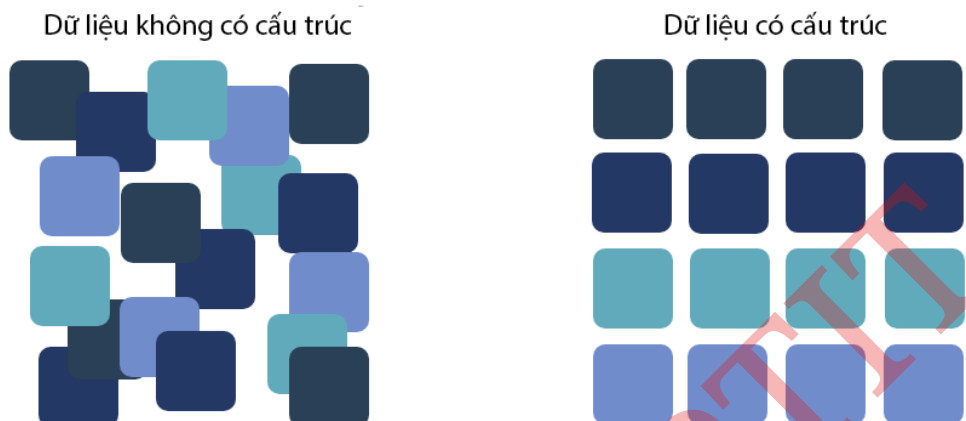
Các ví dụ về dữ liệu không có cấu trúc phi văn bản bao gồm:

- **Dữ liệu Đa phương tiện (Multimedia)** gồm có
 - **Hình ảnh:** Các tệp định dạng phổ biến như **JPEG, GIF, PNG** và các định dạng hình ảnh khác.
 - **Âm thanh:** Tệp âm thanh và ghi âm.
 - **Video:** Các tệp video và luồng phát trực tiếp.
- **Dữ liệu Máy móc và Hành vi** gồm có
 - **Dữ liệu Cảm biến:** Dữ liệu thu thập từ các thiết bị **Internet Vạn vật (IoT)**, chẳng hạn như nhiệt độ, vị trí, hoặc áp suất.
 - **Hoạt động Di động:** Dữ liệu theo dõi **hành vi người dùng** trên các ứng dụng và thiết bị di động (ví dụ: nhấp chuột, vị trí, thời gian sử dụng).

Dữ liệu không có cấu trúc mang lại nhiều lợi ích đáng chú ý, đặc biệt về tính linh hoạt, tốc độ thu thập và khả năng lưu trữ. Với tính linh hoạt cao, dữ liệu không có cấu trúc có thể được lưu trữ dưới dạng dữ liệu gốc mà không cần phải định nghĩa cấu trúc trước đó, từ đó cho phép các nhà khoa học dữ liệu hay kỹ sư dữ liệu sử dụng cho nhiều mục đích khác nhau mà không cần phải sửa thiết kế dữ liệu. Đáng chú ý, dữ liệu phi cấu trúc tăng trưởng nhanh gấp ba lần dữ liệu có cấu trúc, vì không đòi hỏi định nghĩa cấu trúc trước khi thu thập. Đây là lợi thế then chốt để phát triển các ứng dụng AI tạo sinh và tinh chỉnh mô hình ngôn ngữ lớn. Việc lưu trữ dữ liệu phi cấu trúc đã trở nên dễ dàng và tiết kiệm chi phí hơn rất nhiều nhờ sự ra đời của các hệ thống Hồ dữ liệu (Data Lake) và hệ thống tệp quy mô lớn. Các nền tảng này sử dụng mô hình giá trả theo mức sử dụng (pay-as-you-go), cho phép tổ chức giảm chi phí và mở rộng quy mô linh hoạt khi khối lượng dữ liệu gia tăng.

Tuy nhiên, việc quản lý dữ liệu không có cấu trúc cũng gặp phải không ít thách thức. Chính vì thiếu cấu trúc và định dạng rõ ràng, việc chuẩn bị và phân tích loại dữ liệu này rất phức tạp, buộc phải có sự can thiệp của các chuyên gia khoa học dữ liệu. Điều này tạo ra rào cản cho người dùng doanh nghiệp thiếu kinh nghiệm phân tích. Bên cạnh đó, các công cụ truyền thống như Excel không đủ khả năng để xử lý hiệu quả dữ liệu phi cấu trúc. Điều này dẫn đến sự hạn chế trong việc lựa chọn các công cụ quản lý chuyên dụng và phù hợp.

Cuối cùng, mức độ sạch sẽ của dữ liệu là một vấn đề then chốt. Do khối lượng lớn và cấu trúc không đồng nhất, dữ liệu dễ dẫn đến sự không nhất quán và sai sót; vì vậy, quá trình làm sạch dữ liệu là bắt buộc trước khi tiến hành xử lý và phân tích.



Hình 2: Dữ liệu có cấu trúc và dữ liệu không có cấu trúc

1.2.3. Thu thập dữ liệu

Thu thập dữ liệu là quá trình tập hợp và nhập các tệp dữ liệu từ nhiều nguồn khác nhau vào một cơ sở dữ liệu để lưu trữ, xử lý và phân tích. Mục tiêu của việc thu thập dữ liệu là làm sạch và lưu trữ dữ liệu trong một kho lưu trữ trung tâm để truy cập và nhất quán, nhằm chuẩn bị dữ liệu cho việc sử dụng trong tổ chức. Dưới đây là một số nguồn dữ liệu thường được sử dụng

1.2.3.1. Dữ liệu mở

Dữ liệu mở là dữ liệu có thể được bất kỳ ai sử dụng, tái sử dụng và phân phối lại một cách tự do - chỉ cần tuân thủ yêu cầu ghi rõ nguồn và chia sẻ tương tự. Ý tưởng đằng sau dữ liệu mở là một số dữ liệu được cung cấp tự do trong cộng đồng để bất cứ ai cũng có thể sử dụng theo ý muốn mà không bị hạn chế bởi bản quyền, bằng sáng chế hoặc các cơ chế kiểm soát khác. Các chính phủ địa phương và liên bang, các tổ chức phi chính phủ (NGO) và cộng đồng học thuật là những cơ quan đi đầu trong các sáng kiến dữ liệu mở. Điển hình như, công chúng có thể tự do truy cập vào các kho dữ liệu được tạo ra bởi Chính phủ Hoa Kỳ hoặc Thành phố Chicago ví dụ như <https://data.uspto.gov> (Dữ liệu về bằng sáng chế, nhãn hiệu và các quyết định liên quan) hay <https://data.nasa.gov> (Dữ liệu cơ quan hàng không và vũ trụ Mỹ)

Dữ liệu mở có các đặc điểm sau đây:

- **Tính công khai:** Các cơ quan công quyền phải áp dụng nguyên tắc ưu tiên mở dữ liệu trong phạm vi pháp luật cho phép và tuân thủ các hạn chế về quyền riêng tư, bảo mật, và các quy định hợp pháp khác.
- **Được phép truy cập:** Dữ liệu mở phải được cung cấp dưới các định dạng thuận tiện, có thể sửa đổi và truy xuất, tải xuống, lập chỉ mục và tìm kiếm. Các định dạng phải đọc được bằng máy (tức là dữ liệu được cấu trúc hợp lý để cho phép xử lý tự động). Cấu trúc dữ liệu mở không phân biệt đối xử với bất kỳ cá nhân hay nhóm người nào và nên được cung cấp cho càng nhiều người dùng và mục đích sử dụng càng tốt, thường bằng cách cung cấp dữ liệu dưới nhiều định dạng khác nhau để người dùng dễ sử dụng. Trong phạm vi pháp luật cho phép, các định dạng này phải là mã nguồn mở, có sẵn công khai và không được đặt bất kỳ hạn chế nào đối với việc sử dụng.
- **Có mô tả đầy đủ:** Dữ liệu mở phải được mô tả đầy đủ để người dùng có đủ thông tin để hiểu các điểm mạnh, điểm yếu, hạn chế, yêu cầu bảo mật và cách thức xử lý. Điều này bao gồm việc sử dụng siêu dữ liệu (metadata) chi tiết, tài liệu mô tả, từ điển dữ liệu và nếu có thể, mô tả thêm về mục đích thu thập dữ liệu, đối tượng nghiên cứu, đặc điểm của mẫu, và phương pháp thu thập dữ liệu.
- **Có khả năng tái sử dụng:** Dữ liệu mở phải được cung cấp theo giấy phép mở, không đặt bất kỳ hạn chế nào đối với việc sử dụng chúng.
- **Hoàn chỉnh:** Dữ liệu mở phải được công bố ở dạng gốc (tức là dữ liệu thu thập được từ nguồn), với mức độ chi tiết cao nhất có thể trong phạm vi. Dữ liệu mở đã qua xử lý hoặc tổng hợp cũng phải được công bố nhưng phải tham chiếu đến dữ liệu gốc.
- **Kịp thời:** Dữ liệu mở phải được công bố nhanh chóng để bảo toàn giá trị của dữ liệu. Tần suất công bố dữ liệu cần phải xem xét đối tượng và nhu cầu sử dụng của các bên liên quan.
- **Quản lý sau khi công bố:** Dữ liệu mở cần một số điểm hỗ trợ được chỉ định để trợ giúp trong quá trình sử dụng dữ liệu và giải quyết các khiếu nại về việc tuân thủ các yêu cầu dữ liệu này.

1.2.3.2. Dữ liệu truyền thông xã hội

Mạng xã hội đã trở thành một nguồn tài nguyên quan trọng để thu thập dữ liệu phục vụ cho nghiên cứu hoặc tiếp thị. Để hỗ trợ việc thu thập dữ liệu từ các nền tảng này, các mạng xã hội như Facebook và Twitter đã cung cấp Giao diện Lập trình Ứng dụng (API), cho phép các nhà nghiên cứu và nhà phát triển khai thác dữ liệu từ hệ thống của họ. Để dễ hình dung, có thể hiểu API như một tập hợp các quy tắc và phương pháp cho phép yêu

cầu và trao đổi dữ liệu giữa các hệ thống với nhau. Đối với các nhu cầu liên quan đến các loại dữ liệu khác nhau (ví dụ: truy xuất ảnh đại diện của người dùng), người dùng có thể gửi yêu cầu API đến một dịch vụ mạng xã hội tương ứng. Dịch vụ đó sau đó gửi phản hồi ở định dạng dữ liệu có cấu trúc, chẳng hạn như XML hay JSON, cho phép người dùng thực hiện thêm các thao tác sau khi đã thu thập được dữ liệu. Facebook Graph API là một ví dụ phổ biến về giao diện lập trình ứng dụng (API) được sử dụng để thu thập dữ liệu trên mạng xã hội. Các API này có thể được bất kỳ cá nhân hoặc tổ chức nào sử dụng để thu thập và sử dụng dữ liệu này nhằm thực hiện nhiều nhiệm vụ khác nhau, chẳng hạn như phát triển các ứng dụng mới có tác động xã hội, nghiên cứu về hành vi thông tin của con người, và theo dõi hậu quả của thiên tai, v.v. Hơn nữa, để khuyến khích nghiên cứu về các lĩnh vực chuyên biệt, các tập dữ liệu như vậy thường được chính nền tảng mạng xã hội phát hành. Ví dụ, Yelp—một nền tảng đánh giá cộng đồng nổi tiếng về các doanh nghiệp địa phương—đã phát hành các bộ dữ liệu được sử dụng rộng rãi trong nghiên cứu. Các chủ đề ứng dụng rất đa dạng, từ phân loại ảnh tự động và xử lý ngôn ngữ tự nhiên (NLP) của văn bản, cho đến phân tích cảm xúc và khai thác biểu đồ (graph mining).



Hình 3: Facebook Graph API cho phép truy cập dữ liệu người dùng.

1.2.3.3. Dữ liệu đa phương thức

Chúng ta đang sống trong một thế giới mà ngày càng nhiều thiết bị—từ bóng đèn đến ô tô—được kết nối Internet, tạo ra xu hướng Internet Vạn vật (IoT). Những thiết bị này không chỉ sinh ra lượng dữ liệu khổng lồ mà còn sử dụng dữ liệu sẵn có để phục vụ đời sống con người. Dữ liệu ngày nay không chỉ giới hạn ở các con số hay ký tự. Trong

nhiều bối cảnh khác nhau, việc thu thập và khám phá các hình thức dữ liệu đa phương thức và đa phương tiện (như hình ảnh, âm nhạc, âm thanh, cử chỉ hay tư thế cơ thể) đã trở thành một bài toán cấp thiết. Sau khi xác định các nguồn dữ liệu, bước tiếp theo cần xem xét chính là loại dữ liệu có thể được trích xuất. Dựa trên bản chất của thông tin thu thập, dữ liệu được phân thành hai loại chính: dữ liệu có cấu trúc và dữ liệu phi cấu trúc như đã đề cập ở phần trên

Một trong những ứng dụng nổi bật của dữ liệu đa phương tiện là phân tích chuỗi hình ảnh não bộ nhằm xác định các bệnh lý liên quan đến chức năng não. Quá trình này được thực hiện thông qua việc quét một loạt hình ảnh thu thập từ các cảm biến khác nhau. Bộ dữ liệu này bao gồm đầu ra của các kỹ thuật chẩn đoán hình ảnh y tế hiện đại như: EEG (Điện não đồ), EMG (Tâm não đồ), và fMRI (Chụp cộng hưởng từ chức năng). Các kỹ thuật này được áp dụng đồng thời trên cùng một đối tượng nghiên cứu trong một bối cảnh nhất định, tạo ra một nguồn dữ liệu toàn diện cho việc phân tích. Việc kết hợp các dữ liệu từ nhiều cảm biến này giúp các nhà nghiên cứu có cái nhìn sâu sắc và toàn diện về hoạt động của não bộ, từ đó hỗ trợ việc chẩn đoán các bệnh lý như rối loạn tâm thần, động kinh, và các vấn đề liên quan đến não bộ. Hơn nữa, các kỹ thuật phân tích dữ liệu đa phương thức này có thể xác định chính xác các mô hình hoạt động bất thường trong não. Qua đó mở ra cơ hội phát triển các phương thức trị liệu đặc hiệu và hiệu quả vượt trội hơn.

1.2.3.4. Lưu trữ và trình bày dữ liệu

Dữ liệu được lưu trữ với nhiều định dạng khác nhau, tùy thuộc vào bản chất của dữ liệu, trong đó dạng văn bản là dạng phổ biến nhất. Đối với dữ liệu có cấu trúc, chúng thường được phân tách bằng một ký tự đặc biệt và được lưu trữ để đảm bảo việc truy vấn và xử lý dễ dàng. Điều này có nghĩa là các trường và giá trị trong dữ liệu được phân tách bằng các dấu phân cách (delimiter), điển hình là dấu phẩy hoặc Tab. Việc này đã tạo ra hai định dạng lưu trữ dữ liệu văn bản phổ biến nhất hiện nay: giá trị phân cách bằng dấu phẩy (CSV) và giá trị phân cách bằng dấu Tab (TSV).

- CSV: CSV viết tắt của cụm từ 'Comma Separated Values', là định dạng tệp chứa dữ liệu văn bản trong đó các giá trị được phân tách bằng dấu phẩy. Tuy nhiên, cũng có một số trường hợp các giá trị được phân tách bằng dấu chấm phẩy. Được sử dụng rộng rãi trong việc trao đổi dữ liệu giữa các ứng dụng, CSV là công cụ phổ biến cho việc xuất và nhập dữ liệu phức tạp giữa các ứng dụng khác nhau. Cấu trúc một file CSV thường có ba thành phần chính, được sắp xếp theo trình tự
 - Hàng tiêu đề: Đại diện cho các tiêu đề cột trong bảng, chỉ ra tên của mỗi cột.

- Các hàng dữ liệu: Phần còn lại của tệp bao gồm các hàng dữ liệu (records). Mỗi hàng này đại diện cho một bản ghi hoặc một mục nhập (entry) cụ thể. Các hàng này có cấu trúc tương tự nhau và các giá trị phải tuân theo thứ tự tương ứng với các tiêu đề cột đã được định nghĩa ở phần đầu.
- Dấu phân cách: Các trường (giá trị) trong cùng một hàng được phân tách bằng dấu phẩy để duy trì cấu trúc bảng.

```
treat,before,after,diff
No Treatment,13,16,3
No Treatment,10,18,8
No Treatment,16,16,0
Placebo,16,13,-3
Placebo,14,12,-2
Placebo,19,12,-7
Seroxat (Paxil),17,15,-2
Seroxat (Paxil),14,19,5
Seroxat (Paxil),20,14,-6
Effexor,17,19,2
Effexor,20,12,-8
Effexor,13,10,-3
```

Hình 4: Ví dụ về 1 file CSV

Một lợi thế của định dạng CSV là định dạng này rất phổ biến và hữu ích khi chia sẻ với người khác. Lý do là bởi vì định dạng này không yêu cầu công cụ chuyên biệt để đọc hoặc xử lý. Bất kỳ chương trình bảng tính nào, chẳng hạn như Microsoft Excel hoặc Google Sheets, đều tương thích với định dạng CSV đều có thể mở và hiển thị dữ liệu. Tuy nhiên, định dạng này cũng có một số nhược điểm. Ví dụ, vì dấu phẩy được sử dụng để tách các trường, nếu dữ liệu thực tế chứa dấu phẩy trong các trường giá trị, sẽ dễ xảy ra xung đột với dấu phân cách, dẫn đến lỗi trong quá trình xử lý và phân tích. Do đó, vấn đề này có thể được giải quyết bằng cách thoát dấu phẩy (thường là thêm dấu gạch chéo ngược (\) trước dấu phẩy đó). Tuy nhiên, phương pháp này ít được khuyến nghị vì dễ dẫn đến sai sót khi người dùng không tuân theo chuẩn mực đã được quy định.

- TSV: Định dạng file TSV là viết tắt của Tab-Separated Values (các giá trị được phân tách bằng tab). Điều này có nghĩa là các tệp TSV chứa thông tin được sắp xếp trong một loạt các bảng dữ liệu, với mỗi trường dữ liệu được phân tách bằng ký tự Tab (t). Một ưu điểm của định dạng TSV là dấu phân cách (tab) không cần phải tránh vì rất hiếm khi tồn tại ký tự Tab trong một trường dữ liệu. Trong trường hợp ký tự Tab xuất hiện trong dữ liệu thực tế (bên ngoài vai trò phân

cách), kí tự này cần phải được loại bỏ để đảm bảo tính toàn vẹn của cấu trúc. Mặt khác, định dạng TSV ít phổ biến hơn so với các định dạng phân cách khác, chẳng hạn như CSV.

```
Name<TAB>Age<TAB>Address
Ryan<TAB>33<TAB>1115 W Franklin
Paul<TAB>25<TAB>Big Farm Way
Jim<TAB>45<TAB>W Main St
Samantha<TAB>32<TAB>28 George St
```

Hình 5: Ví dụ về 1 file TSV

- XML: XML là từ viết tắt của từ Extensible Markup Language là ngôn ngữ đánh dấu mở rộng được thiết kế để lưu trữ và truyền tải dữ liệu một cách có cấu trúc. Tác dụng chính của XML là đơn giản hóa việc chia sẻ dữ liệu giữa các nền tảng và các hệ thống được kết nối thông qua mạng Internet. Ưu điểm lớn nhất của XML (Extensible Markup Language) là tính độc lập nền tảng (platform independence). Vì XML được sử dụng để mô tả dữ liệu dưới dạng văn bản thuần túy, hầu hết các phần mềm hay chương trình đều có khả năng đọc và xử lý dễ dàng mà không cần phần mềm chuyên biệt. Do đó, XML đã nhanh chóng trở thành định dạng mặc định để chia sẻ dữ liệu giữa các hệ thống thông tin khác nhau. Đặc điểm nổi bật của XML là khả năng trao đổi dữ liệu cùng với siêu dữ liệu, giúp duy trì tính toàn vẹn cao. Nhờ thông tin mô tả này, người dùng có thể thực hiện nhiều thao tác quan trọng như: xác minh độ chính xác, tự động tùy chỉnh cách trình bày dữ liệu cho từng người dùng, hoặc lưu trữ dữ liệu nhất quán trên các nền tảng khác nhau.
Dưới đây là một ví dụ về một trang XML:


```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="information science" cover="hardcover">
    <title lang="en">Social Information Seeking</title>
    <author>Chirag Shah</author>
    <year>2017</year>
    <price>62.58</price>
  </book>
  <book category="data science" cover="paperback">
    <title lang="en">Hands-On Introduction to Data
    Science</title>
    <author>Chirag Shah</author>
    <year>2019</year>
    <price>50.00</price>
  </book>
</bookstore>
```

Hình 6: Ví dụ về định dạng XML

- RSS: RSS (Really Simple Syndication) là một định dạng chuẩn dùng để phân phối và chia sẻ nội dung của các trang web, blog, hoặc các nguồn tin tức khác. RSS cho phép người dùng theo dõi các cập nhật mới từ các trang web yêu thích mà không cần phải truy cập vào từng trang một cách thủ công. Thông qua RSS, nội dung mới (ví dụ như bài viết, tin tức, podcast) được cung cấp dưới dạng các "feed" (dòng thông tin), thường được hiển thị dưới dạng một danh sách các tiêu đề với tóm tắt. Người dùng có thể đăng ký "feed" này thông qua phần mềm đọc RSS (RSS reader), và khi có bài viết mới, phần mềm này sẽ tự động cập nhật. Ưu điểm của RSS là tiết kiệm thời gian và dễ dàng quản lý việc theo dõi nội dung từ nhiều nguồn khác nhau một cách hiệu quả.

Hãy cùng xem một ví dụ thực tế về việc sử dụng RSS. Giả sử bạn có một trang web cung cấp nhiều bản cập nhật thông tin (tin tức, chứng khoán, thời tiết) mỗi ngày. Để theo kịp những cập nhật này, và thậm chí chỉ đơn giản là kiểm tra xem có cập nhật nào không, người dùng sẽ phải liên tục quay lại trang web này suốt cả ngày. Điều này không chỉ tốn thời gian mà còn không hiệu quả, vì người dùng có thể kiểm tra quá thường xuyên mà không có bất kỳ cập nhật nào, hoặc ngược lại, kiểm tra không đủ thường xuyên và bỏ lỡ những thông tin quan trọng khi chúng được cập nhật. Người dùng có thể kiểm tra trang của bạn nhanh hơn bằng cách sử dụng một bộ tổng hợp RSS (một trang web hoặc chương trình thu thập và sắp xếp các feed RSS). Bộ tổng hợp này sẽ đảm bảo rằng thông tin được cập nhật ngay

khi trang web cung cấp, và sau đó thông tin đó sẽ được gửi đến người dùng – thường là thông qua thông báo.

Vì dữ liệu RSS có kích thước nhỏ nên có thể dễ dàng được sử dụng với các dịch vụ như điện thoại di động, máy trợ lý cá nhân (PDA), và đồng hồ thông minh. Ngoài ra, RSS rất hữu ích cho các trang web có cập nhật thường xuyên, chẳng hạn như:

- Các trang tin tức – Liệt kê tin tức với tiêu đề, ngày và mô tả.
- Sản phẩm của các công ty – Liệt kê tin tức và sản phẩm mới.
- Lịch – Liệt kê các sự kiện sắp tới và các ngày quan trọng.

Để xuất bản nội dung qua RSS, trước hết cần đăng ký nội dung với các bộ tổng hợp RSS. Đầu tiên, tạo một tài liệu RSS và lưu với phần mở rộng *.xml*. Sau đó, tải tệp này lên trang web. Cuối cùng, đăng ký trang web này với một bộ tổng hợp RSS. Mỗi ngày (hoặc theo tần suất chỉ định), bộ tổng hợp sẽ tìm kiếm các trang web đã đăng ký để tìm các tài liệu RSS, xác minh liên kết, và hiển thị thông tin về bản tin để khách hàng có thể xem tài liệu mà họ quan tâm thông qua các đường dẫn.

Dưới đây là một mẫu tài liệu RSS:

```
<?xml version="1.0" encoding="UTF-8" ?>
<rss version="2.0">
  <channel>
    <title>Dr. Chirag Shah's Home Page</title>
    <link>http://chiragshah.org/</link>
    <description>Chirag Shah's webhome
  </description>
    <item>
      <title>Awards and Honors</title>
      <link>http://chiragshah.org/awards
        .php</link>
      <description>Awards and Honors
        Dr. Shah received</description>
    </item>
  </channel>
</rss>
```

Hình 7: Mẫu một văn bản RSS

Ở đây, phần tử *<channel>* mô tả RSS bao gồm ba phần tử “con” bắt buộc: *<title>* xác định tiêu đề của kênh (ví dụ: Trang chủ của Dr. Chirag Shah); *<link>* xác định liên kết dẫn đến kênh (ví dụ: <http://chiragshah.org/>); và *<description>* mô tả kênh (ví dụ: Trang web của Chirag Shah). Phần tử *<channel>* thường chứa một hoặc nhiều phần tử *<item>*. Mỗi phần tử *<item>* xác định một bài viết hoặc “câu chuyện” trong feed RSS.

Dưới đây là các bước xuất bản RSS:

- Đặt tên cho tệp RSS. Lưu ý rằng tệp phải có phần mở rộng .xml.
- Xác thực tệp RSS vừa tạo tại FEED Validator để đảm bảo tệp RSS không có lỗi
- Tải tệp RSS lên máy chủ Web để người dùng truy cập được.
- Sao chép biểu tượng  RSS hoặc XML vào thư mục Web và đặt nút RSS hoặc XML trên trang phổ thông (ví dụ: trên trang chủ) để người dùng có thể dễ dàng tìm thấy và đăng ký theo dõi feed của bạn
- Thêm một liên kết đến nút "RSS" hoặc "XML" trên trang web, trỏ tới URL của tệp RSS đã tải lên máy chủ.
- Đăng ký feed với các công cụ tìm kiếm lớn như Google hoặc Bing
- Sau khi đăng ký feed RSS, cần thường xuyên cập nhật nội dung và đảm bảo rằng các tin tức RSS được cập nhật để người dùng luôn nhận được thông tin mới nhất.

Bây giờ, khi có thông tin mới trên trang web, tin tức này sẽ gửi đến người dùng đã đăng ký nhận tin thông qua thông báo.

- **JSON:** JSON là từ viết tắt của JavaScript Object Notation, một định dạng văn bản nhẹ, có cấu trúc, được sử dụng để lưu trữ và trao đổi dữ liệu giữa máy chủ và ứng dụng web hoặc giữa các thành phần khác nhau của một chương trình. JSON độc lập với ngôn ngữ lập trình, có thể được con người đọc và máy tính hiểu dễ dàng. JSON được xây dựng trên cấu trúc bao gồm Đối tượng (object) và Mảng (array) và sử dụng các cặp khóa-giá trị (key-value) và các dấu ngoặc {}, [], và dấu phẩy để tổ chức dữ liệu. Khóa luôn là một chuỗi ký tự trong dấu ngoặc kép, còn giá trị có thể là một chuỗi, số, giá trị logic, mảng, đối tượng, hoặc null. Đây là 1 cấu trúc dữ liệu phổ biến, hầu như tất cả các ngôn ngữ lập trình hiện đại đều hỗ trợ cấu trúc này.

```

1 {
2   "name": "Product",
3   "properties":
4   {
5     "id":
6     {
7       "type": "number",
8       "description": "Product identifier",
9       "required": true
10    },
11    "name":
12    {
13      "description": "Name of the product",
14      "type": "string",
15      "required": true
16    },
17    "price":
18    {
19      "type": "number",
20      "minimum": 0,
21      "required": true
22    },
23    "tags":
24    {
25      "type": "array",
26      "items":
27      {
28        "type": "string"
29      }
30    }
31  }
32 }
33 }
34 }

```

Hình 8: Kết quả truy vấn trả về dưới định dạng JSON, được mô tả qua cặp khóa-giá trị.

1.2.4. Tiền xử lý dữ liệu

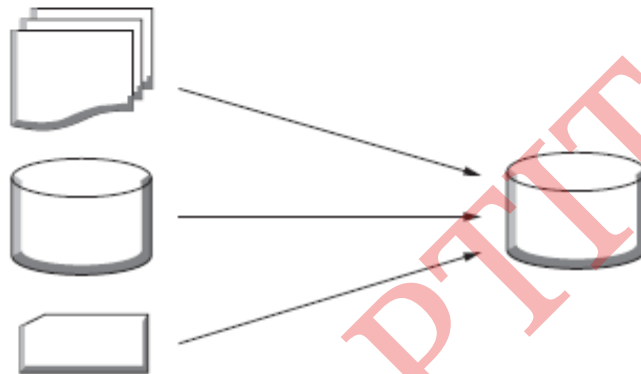
Dữ liệu trong thế giới thực thường là dữ liệu “bẩn”, do đó cần được làm sạch trước khi có thể được sử dụng cho một mục đích mong muốn. Quá trình này được gọi là tiền xử lý dữ liệu (data pre-processing). Điều gì làm cho dữ liệu trở nên "bẩn"? Dưới đây là một số yếu tố cho thấy dữ liệu không sạch hoặc chưa sẵn sàng để xử lý:

- Không đầy đủ (Incomplete): có thể kể đến một số giá trị thuộc tính bị thiếu hay một số thuộc tính quan trọng bị thiếu, hoặc các thuộc tính chỉ chứa dữ liệu tổng hợp.
- Nhiễu (Noisy): Khi dữ liệu chứa lỗi hoặc các giá trị ngoại lai (outliers). Ví dụ, một số điểm dữ liệu trong một tập dữ liệu có thể chứa các giá trị cực đoan có thể ảnh hưởng nghiêm trọng đến phạm vi của tập dữ liệu.
- Không nhất quán (Inconsistent): Dữ liệu chứa sự không đồng nhất về giá trị. Ví dụ, một số giá trị trong cột “Tên” trong hồ sơ đăng ký nhân viên không phải là chữ cái trong khi một số giá trị khác là chữ cái; hoặc nếu một số bản ghi không bắt đầu bằng chữ cái viết hoa, thì có sự không nhất quán.

Data Cleaning



Data Integration



Data Transformation $-17, 25, 39, 128, -39$ $\longrightarrow$ $0.17, 0.25, 0.39, 1.28, -0.39$

Data Reduction

	A1	A2	A3	...	A200
T1					
T2					
T3					
....					
T200					

	A1	A2	A3	...	A120
T1					
T2					
T3					
....					
T150					

Hình 9: Một số nhiệm vụ liên quan đến xử lý dữ liệu bao gồm làm sạch dữ liệu, tích hợp dữ liệu, biến đổi dữ liệu và loại bỏ dữ liệu không cần thiết

1.2.4.1. Làm sạch dữ liệu

Do có nhiều lý do khiến dữ liệu bị “bẩn” nên cần các phương pháp để “làm sạch”. Dưới đây là ba phương pháp chính đề cập đến quy trình làm sạch dữ liệu:

- Chuẩn bị dữ liệu: Đây là quá trình thu thập, làm sạch, cấu trúc hóa và chuyển đổi dữ liệu thô ban đầu không sử dụng được thành một định dạng sạch sẽ, dễ hiểu và có giá trị để phân tích và ra quyết định. Quá trình này giải quyết các vấn đề như dữ liệu lỗi, thiếu, trùng lặp, định dạng không nhất quán nhằm đảm bảo chất lượng dữ liệu và giúp các nhà phân tích, nhà khoa học dữ liệu tập trung vào việc trích

xuất thông tin hữu ích thay vì tốn thời gian chuẩn bị. Một ví dụ có thể kể đến ở đây:

Định dạng văn bản: “Thêm hai quả cà chua thái hạt lựu, ba tép tỏi, và một nhúm muối vào hỗn hợp.”

Từ văn bản trên có thể trình bày một cách dễ nhìn hơn dưới dạng bảng như sau

Bảng 3: Bảng mô hình hóa dữ liệu từ đoạn văn trên

Nguyên liệu	Số lượng	Đơn vị tính
Cà chua	2	Quả
Tỏi	3	Tép
Muối	1	Nhúm

- Xử lý dữ liệu bị thiếu: Đôi khi, dữ liệu có thể ở đúng định dạng nhưng lại thiếu một số giá trị. Hãy xét một tập dữ liệu khách hàng, trong đó một số số điện thoại nhà bị thiếu. Điều này có thể là do một số người không có điện thoại nhà – thay vào đó, họ sử dụng điện thoại di động làm số điện thoại chính. Vấn đề dữ liệu bị thiếu là một thách thức phổ biến, thường xuất phát từ hai nhóm nguyên nhân chính. Thứ nhất là lỗi kỹ thuật hoặc quy trình thu thập, điển hình là sự cố thiết bị (như cảm biến hoặc hệ thống ghi nhận bị lỗi) hoặc trục trặc trong quá trình truyền tải và xử lý dữ liệu. Thứ hai, cũng quan trọng không kém, là lỗi quy trình và nhận thức; cụ thể, tính đầy đủ của dữ liệu có thể không được coi là quan trọng hoặc không được ưu tiên tại thời điểm thu thập, dẫn đến việc người dùng hoặc hệ thống bỏ qua các trường thông tin không bắt buộc. Tuy nhiên, vấn đề phát sinh khi tổ chức quyết định mở rộng phạm vi hoạt động ra ngoài khu vực ban đầu. Lúc này, các số điện thoại mới đến từ nhiều vùng khác nhau, khiến cho cấu trúc dữ liệu cũ thiếu tính đầy đủ và khả năng mở rộng. Hơn nữa, dữ liệu có thể bị thất thoát hoặc hỏng trong quá trình lưu trữ và chuyển giao do lỗi hệ thống hoặc lỗi do con người. Khi gặp phải tình trạng dữ liệu bị thiếu (Missing Data) do các nguyên nhân này, chúng ta cần phải xử lý như thế nào để đảm bảo tính toàn vẹn và chất lượng của bộ dữ liệu? Trong trường hợp này, cần lựa chọn một chiến lược phù hợp tùy theo tình huống cụ thể. Các chiến lược xử lý dữ liệu thiếu có thể bao gồm bỏ qua bản ghi, sử dụng một giá trị hằng số toàn cục để thay thế các giá trị bị thiếu, ước lượng, hoặc áp dụng các giải pháp suy luận (như công thức Bayes hoặc cây quyết định).

- **Làm mượt dữ liệu bị nhiễu:** Không chỉ dữ liệu bị thiếu là vấn đề, mà ngay cả khi dữ liệu đầy đủ, chúng vẫn có thể bị hỏng (Corrupted) do một lỗi nào đó trong quá trình xử lý hoặc lưu trữ. Vấn đề dữ liệu hỏng thường được coi là nghiêm trọng hơn dữ liệu bị thiếu, bởi lẽ dữ liệu hỏng vẫn tồn tại trong tập dữ liệu nhưng mang lại thông tin sai lệch, gây ra các quyết định sai lầm mà không dễ dàng phát hiện như dữ liệu bị bỏ trống. Dữ liệu bị hỏng có thể xuất phát từ các công cụ thu thập dữ liệu bị lỗi hoặc gặp các vấn đề trong quá trình nhập liệu, hoặc bị giới hạn bởi công nghệ. Ví dụ, một nhiệt kế kỹ thuật số đo nhiệt độ đến một chữ số thập phân (ví dụ: 70.1°F), nhưng hệ thống lưu trữ lại bỏ qua các chữ số thập phân. Kết quả là, cả 70.1°F và 70.9°F đều được lưu trữ là 70°F. Mặc dù việc làm tròn có vẻ không đáng kể, nhưng trong các bài toán chăm sóc sức khỏe, độ chính xác của dữ liệu là tối quan trọng. Ví dụ: nhiệt độ 99.4°F có thể được coi là bình thường, trong khi 99.8°F có thể là dấu hiệu ban đầu của sốt. Nếu hệ thống lưu trữ y tế làm tròn và ghi nhận cả hai giá trị này thành 99°F, dữ liệu sẽ bị mất thông tin nghiêm trọng, dẫn đến việc không thể phân biệt được giữa người khỏe mạnh và người cần được theo dõi y tế!

Tương tự như việc áp dụng các kỹ thuật để xử lý dữ liệu thiếu (Missing Data), việc làm mượt dữ liệu bị nhiễu (Noisy Data) cũng là một thách thức không hề dễ dàng. Tuy nhiên, tình trạng này hoàn toàn có thể được cải thiện thông qua việc áp dụng một loạt các kỹ thuật và quy trình làm sạch dữ liệu phù hợp. Bước đầu tiên là phải xác định và loại bỏ các giá trị ngoại lệ. Ví dụ, trong một bảng điểm của các học viên tham gia kỳ thi, tất cả các học viên đều đạt từ 70 đến 90 điểm, ngoại trừ một học viên có điểm là 12. Trong trường hợp này, có thể giả định điểm số này là một ngoại lệ. Thứ hai, cần giải quyết các sự không nhất quán trong dữ liệu. Ví dụ, tất cả các mục tên khách hàng nên tuân theo quy tắc viết hoa tất cả các chữ cái, và cho phép chỉnh sửa nếu tên khách hàng không tuân thủ nguyên tắc này.

1.2.4.2. Tích hợp dữ liệu

Để đạt được hiệu quả tối ưu trong phân tích dữ liệu từ nhiều nguồn khác nhau, dữ liệu thường cần được tích hợp vào một nguồn duy nhất. Dưới đây là các bước mô tả cách tích hợp nhiều cơ sở dữ liệu hoặc tệp tin:

1. Kết hợp dữ liệu từ nhiều nguồn vào một nơi lưu trữ thống nhất (ví dụ: một tệp tin duy nhất hoặc một cơ sở dữ liệu).
2. Thực hiện tích hợp lược đồ, hay kết hợp các siêu dữ liệu từ các nguồn khác nhau.
3. Phát hiện và giải quyết các xung đột giá trị. Ví dụ:
 - a. Xung đột có thể xảy ra khi các thuộc tính và giá trị khác nhau từ các nguồn

khác nhau đối với cùng một thực thể trong thế giới thực.

b. Nguyên nhân của xung đột này có thể là sự khác biệt trong cách biểu diễn hoặc quy mô khác nhau; ví dụ, đơn vị mét so với đơn vị inches.

4. Giải quyết dữ liệu dư thừa trong quá trình tích hợp dữ liệu. Dữ liệu dư thừa thường xuất hiện trong quá trình tích hợp nhiều dữ liệu. Ví dụ:

a. Các thuộc tính khác nhau có cùng tính chất.

b. Thuộc tính đó có thể "được suy ra" từ một thuộc tính trong một bảng khác; ví dụ, doanh thu hàng năm.

c. Phân tích tương quan có thể phát hiện các trường hợp dữ liệu dư thừa.

1.2.4.3. Biến đổi dữ liệu

Dữ liệu cần được biến đổi để đảm bảo tính nhất quán. Dưới đây là quy trình có thể được sử dụng để chuyển đổi dữ liệu.

1. **Làm mượt (Smoothing):** Kỹ thuật khử nhiễu trong dữ liệu.

2. **Tổng hợp (Aggregation):** Quá trình tóm tắt và hợp nhất dữ liệu từ nhiều nguồn hoặc cấp độ chi tiết khác nhau.

3. **Tổng quát hóa (Generalization):** Kỹ thuật xây dựng cấu trúc phân cấp cho các khái niệm bằng cách thay thế dữ liệu cấp thấp bằng các khái niệm cấp cao hơn (ví dụ: thay thế địa chỉ cụ thể bằng tên thành phố).

4. **Chuẩn hóa (Normalization):** Quá trình **điều chỉnh dữ liệu** về một phạm vi nhỏ và xác định để loại bỏ sự thiên vị do khác biệt về thang đo. Một số kỹ thuật phổ biến bao gồm:

- **Chuẩn hóa Min-max:** Tái tỷ lệ dữ liệu về phạm vi $[0,1]$ hoặc $[a, b]$.

- **Chuẩn hóa Z-score:** Chuyển đổi dữ liệu về phân phối có mean=0 và standard deviation=1.

- **Chuẩn hóa bằng phân tỷ lệ thập phân (Decimal Scaling).**

5. **Xây dựng Thuộc tính hoặc Đặc trưng (Feature Construction/Engineering):**

Tạo các thuộc tính (đặc trưng) mới có ý nghĩa hơn từ những thuộc tính đã có.

(Ví dụ: Tính chỉ số BMI từ cân nặng và chiều cao).

1.2.4.4. Giảm dữ liệu

Giảm dữ liệu là một quá trình quan trọng, trong đó dữ liệu được loại bỏ hoặc tóm gọn sao cho kết quả phân tích vẫn đạt hiệu quả tương tự mà không bị mất mát quá nhiều thông tin. Một ví dụ dễ hiểu về kỹ thuật Giảm dữ liệu (Data Reduction) là việc sử dụng Khối dữ liệu (Data Cube). Khối dữ liệu là một cấu trúc dữ liệu đa chiều, thường được sử dụng trong kho dữ liệu (Data Warehouse) và phân tích dữ liệu, để đại diện cho dữ liệu

theo nhiều chiều khác nhau. Mỗi chiều trong khối dữ liệu đại diện cho một thuộc tính cụ thể, và các dữ liệu được tóm tắt hoặc tổng hợp tại các cấp độ khác nhau của mỗi chiều. Việc sử dụng Khối dữ liệu giúp tóm tắt và tổng hợp dữ liệu ở các cấp độ khác nhau, qua đó làm giảm kích thước dữ liệu mà vẫn giữ được thông tin cần thiết cho phân tích đa chiều. Ví dụ, trong một hệ thống quản lý bán hàng, các chiều có thể là "thời gian," "khu vực," và "sản phẩm." Khi làm việc với dữ liệu đa chiều, không phải tất cả các chiều này đều quan trọng trong mọi tình huống. Chẳng hạn, khi phân tích doanh thu theo thời gian, khu vực, và sản phẩm, có thể không cần thiết phải tính toán tất cả các đặc tính cho mọi trường hợp, và có thể giảm thông tin từ các chiều đó để có cái nhìn tổng quát và dễ quản lý hơn mà không làm mất đi quá nhiều giá trị. Điều này dẫn đến hai kỹ thuật giảm dữ liệu phổ biến:

1. Tổng hợp Khối Dữ liệu (Data Cube Aggregation): Đây là việc tổng hợp dữ liệu từ các chiều để chỉ giữ lại thông tin quan trọng nhất. Ví dụ, trong một báo cáo doanh thu theo tháng, có thể tổng hợp tất cả dữ liệu theo từng quý hoặc năm để đơn giản hóa phân tích mà vẫn giữ được thông tin quan trọng. Đây là cách giúp giảm độ phức tạp của dữ liệu khi chỉ muốn biết tổng doanh thu của cả năm thay vì mỗi tháng riêng lẻ.

2. Giảm chiều dữ liệu (Dimensionality Reduction): Phương pháp này khác với việc tổng hợp khối dữ liệu, vì không chỉ làm việc với dữ liệu trong mối liên hệ với nhiệm vụ cụ thể, mà còn tập trung vào đặc tính của dữ liệu. Ví dụ, trong phân tích dữ liệu khách hàng, các thuộc tính như "tuổi," "giới tính," "thói quen mua sắm," ... có thể không phải tất cả đều có ảnh hưởng mạnh mẽ đến kết quả. Vì vậy, việc loại bỏ các thuộc tính dư thừa hoặc hợp nhất chúng lại thành một đặc trưng chung có thể giúp dữ liệu dễ dàng xử lý hơn mà không làm mất đi thông tin cần thiết. Một ví dụ thực tế là khi phân tích hành vi của người tiêu dùng, thay vì theo dõi mọi chi tiết nhỏ, bạn có thể giảm bớt các đặc trưng và chỉ tập trung vào những yếu tố quan trọng nhất, chẳng hạn như "mức chi tiêu trung bình" và "tần suất mua hàng."

Trong đời sống thực tế, việc giảm dữ liệu có thể so sánh với việc chọn lọc thông tin quan trọng từ rất nhiều nguồn thông tin khổng lồ. Ví dụ, khi đọc một cuốn sách dày, không cần phải ghi nhớ mọi chi tiết mà chỉ cần nắm bắt những ý chính hoặc những điểm quan trọng nhất, giúp tiết kiệm thời gian và công sức mà vẫn có thể nắm bắt được nội dung cốt lõi.

1.2.4.5. Rời rạc hóa dữ liệu

Chúng ta thường xuyên làm việc với dữ liệu liên tục, được thu thập từ các quá trình diễn ra liên tục, chẳng hạn như nhiệt độ, cường độ ánh sáng môi trường hay giá cổ phiếu. Tuy nhiên, để đơn giản hóa quá trình phân tích hoặc làm cho dữ liệu dễ quản lý hơn, đôi

khi cần thiết phải rời rạc hóa, tức là chuyển đổi những giá trị liên tục này thành các khoảng rời rạc hoặc các phần dễ quản lý hơn. Quá trình này được gọi là *rời rạc hóa* (discretization). Rời rạc hóa là quá trình chuyển đổi dữ liệu liên tục (continuous data) thành dữ liệu rời rạc (discrete data). Điều này có nghĩa là các giá trị liên tục được chia thành một số lượng hạn chế các khoảng hoặc nhóm, mỗi nhóm tương ứng với một giá trị hoặc một phạm vi giá trị rời rạc. Khi thực hiện rời rạc hóa, số lượng giá trị duy nhất trong tập dữ liệu sẽ giảm đi đáng kể vì các giá trị liên tục được phân loại vào các khoảng đã xác định trước. Ví dụ, thay vì giữ nguyên tất cả các giá trị chính xác (như 2.35, 2.47, 2.89), ta có thể nhóm chúng vào các khoảng như 2.0-2.5, 2.5-3.0, v.v. Điều này giúp giảm độ phức tạp của dữ liệu và làm cho việc phân tích và mô hình hóa dữ liệu trở nên dễ dàng hơn, đồng thời giữ lại các đặc điểm quan trọng trong dữ liệu. Vì vậy, quá trình rời rạc hóa có thể được xem như một phương pháp giảm dữ liệu, đặc biệt quan trọng đối với các thuộc tính dữ liệu dạng số. Có ba loại thuộc tính liên quan đến rời rạc hóa dữ liệu:

- **Thuộc tính danh nghĩa (Nominal):** Các giá trị từ một tập hợp không có thứ tự
- **Thuộc tính thứ tự (Ordinal):** Các giá trị từ một tập hợp có thứ tự.
- **Thuộc tính liên tục (Continuous):** Các số thực.

Để thực hiện rời rạc hóa, phạm vi của các thuộc tính liên tục sẽ được chia thành các khoảng. Ví dụ, có thể chia phạm vi nhiệt độ thành các nhóm "lạnh", "vừa phải", và "nóng", hoặc phân loại giá cổ phiếu của công ty thành "cao hơn" hoặc "thấp hơn" so với giá trị thị trường.

Trong thực tế, quá trình rời rạc hóa có thể được liên hệ với việc phân loại các yếu tố hay hiện tượng trong cuộc sống thành các nhóm đơn giản hơn để dễ dàng xử lý và phân tích. Ví dụ, khi dự báo thời tiết, thay vì sử dụng giá trị nhiệt độ chính xác (ví dụ 22.7°C), có thể phân loại thành các nhóm như "nhiệt độ cao", "nhiệt độ thấp" hoặc "mát mẻ" để thuận tiện hơn cho việc truyền tải thông tin đến công chúng. Hay trong hệ thống giáo dục, việc đánh giá học sinh không chỉ dừng lại ở điểm số chính xác mà có thể được phân loại vào các nhóm như "giỏi", "khá", "trung bình" để dễ dàng đánh giá và đưa ra phản hồi phù hợp.

Ví dụ về quy trình tiền xử lý dữ liệu:

Ví dụ này sử dụng một phiên bản đã được chỉnh sửa của bộ dữ liệu về số lượng ca tử vong do tiêu thụ rượu quá mức. Bộ dữ liệu trên Bảng 4 này bao gồm các thông tin sau:

- Tên quốc gia từ đó mẫu được lấy
- Mức tiêu thụ rượu được đo bằng số lít / người
- Số lượng ca tử vong do tiêu thụ rượu, tính trên 100.000 người
- Số lượng ca tử vong do bệnh tim, tính trên 100.000 người

- Số lượng ca tử vong do bệnh gan, cũng tính trên 100.000 người

Dưới đây là cách thức thực hiện:

1. **Làm sạch dữ liệu (Data Cleaning).** Ở giai đoạn này, chúng ta sẽ thực hiện các bước tiền xử lý sau:

- **Làm mượt dữ liệu nhiều:** Có thể thấy rằng giá trị tiêu thụ rượu bình quân đầu người của Iceland là -0.800000012 . Tuy nhiên, giá trị tiêu thụ rượu bình quân đầu người không thể là số âm. Do đó, đây chắc chắn là một lỗi nhập liệu và cần thay đổi giá trị tiêu thụ rượu của Iceland thành 0.800000012 . Tương tự, số ca tử vong của Israel nên được chuyển từ -834 thành 834 .
- **Xử lý thiếu hụt dữ liệu:** Như đã thấy trong bộ dữ liệu, các giá trị thiếu (được biểu thị bằng NA) đối với số ca bệnh tim của Canada và số ca bệnh tim và bệnh gan của Tây Ban Nha. Một cách đơn giản để giải quyết vấn đề này là thay thế tất cả các giá trị NA bằng một giá trị chung, chẳng hạn như số 0 hoặc giá trị trung bình của tất cả các giá trị của thuộc tính đó. Trường hợp này sử dụng giá trị trung bình của thuộc tính để xử lý các giá trị bị thiếu. Vì vậy, đối với cả Canada và Tây Ban Nha, giá trị 185 sẽ được sử dụng làm số ca bệnh tim, Chỉ số này được tính bằng giá trị trung bình của số ca tử vong do bệnh tim khi xem xét dữ liệu ở tất cả các quốc gia được thu thập. Tương tự, số ca bệnh gan của Tây Ban Nha sẽ được thay thế bằng 20.27. Cần lưu ý rằng: tùy thuộc vào tính chất của vấn đề, việc thay thế tất cả các giá trị NA bằng cùng một giá trị chưa phải là một ý tưởng tốt nhất. Một trong những giải pháp tốt hơn và chính xác hơn để xử lý giá trị bị thiếu là suy luận giá trị của thuộc tính bị thiếu đó từ các giá trị của các thuộc tính khác (các thuộc tính đã có dữ liệu đầy đủ) trong cùng bản ghi. Kỹ thuật này, được gọi là Điền khuyết dựa trên Mô hình (Model-Based Imputation), sử dụng các mối quan hệ giữa các biến để đưa ra ước tính chính xác hơn so với việc chỉ dùng giá trị trung bình hay trung vị.

Bảng 4: Dữ liệu về tiêu thụ rượu quá mức và tỷ lệ tử vong

#	Quốc gia	Tiêu thụ rượu (Lít/người)	Số ca tử vong do tiêu thụ rượu	Số ca tử vong do bệnh tim	Số ca tử vong do bệnh gan
1	Australia	2.5	785	211	15.3000019
2	Áo	3.000000095	863	167	45.5999847
3	Bỉ và Luxembourg	2.900000095	883	131	20.7000076
4	Canada	2.400000095	793	NA	16.3999962
5	Đan Mạch	2.900000095	971	220	23.8999962
6	Phần Lan	0.800000012	743	297	19
7	Pháp	9.100000381	751	111	37.9000153

8	Iceland	-0.800000012	743	211	11.1999991
9	Ireland	0.699999988	1000	300	6.5
10	Israel	0.600000024	-834	183	13.69999981
11	Ý	27.900000095	775	107	42.2000076
12	Nhật Bản	1.5	680	36	23.2000076
13	Hà Lan	1.799999952	773	116	9.19999973
14	New Zealand	1.899999976	916	266	7.699999809
15	Na Uy	0.800000012	836	227	12.19999981
16	Tây Ban Nha	6.5	724	NA	NA
17	Thụy Điển	1.600000024	743	207	11.1999991
18	Thụy Sĩ	1.600000012	693	111	20.29999924
19	Vương Quốc Anh (UK)	1.299999952	941	248	10.300001
20	Hoa Kỳ (US)	1.200000048	926	199	22.1000038
21	Tây Đức	2.700000048	861	172	36.7000067

- **Làm sạch dữ liệu.** Như đã được đề cập, làm sạch dữ liệu là quá trình chuyển đổi hoặc ánh xạ dữ liệu từ dạng "thô" sang một dạng khác. Ví dụ, đối với một quốc gia, số ca tử vong được tính theo 10.000 người, thay vì theo 100.000 người như các quốc gia khác. Trong trường hợp đó, cần phải chuyển đổi giá trị số ca tử vong của quốc gia đó thành số ca tử vong tính theo 100.000 người, hoặc chuyển tất cả các quốc gia khác thành 10.000. May mắn thay, bộ dữ liệu này không yêu cầu bất kỳ bước làm sạch dữ liệu nào.

2. Tích hợp dữ liệu

Giả sử bây giờ có một bộ dữ liệu được thu thập từ một nguồn khác, liên quan đến việc tiêu thụ rượu và số ca tử vong ở các bang khác nhau của Ấn Độ, được trình bày ở Bảng 5

Bảng 5: Dữ liệu về tiêu thụ rượu và sức khỏe từ các bang khác nhau của Ấn Độ

#	Tên bang	Tiêu thụ rượu (Lít/người)	Số ca tử vong do bệnh tim
1	Quần đảo Andaman và Nicobar	1.73	20,312
2	Andhra Pradesh	2.05	16,723
3	Arunachal Pradesh	1.98	13,109
4	Assam	0.91	8532
5	Bihar	3.21	12,372
6	Chhattisgarh	2.03	28,501

7	Goa	5.79	19,932
---	-----	------	--------

Bộ dữ liệu này bao gồm các thông tin như sau

- Tên bang.
- Lượng rượu tiêu thụ tính theo lít/người.
- Số ca tử vong do bệnh tim, tính trên quy mô 10.000.000 người.

Bây giờ, có thể sử dụng bộ dữ liệu này để tích hợp các thuộc tính cho Ấn Độ vào bộ dữ liệu ban đầu. Để làm điều này, ta tính toán tổng lượng rượu tiêu thụ cho Ấn Độ là giá trị trung bình của mức tiêu thụ rượu ở tất cả các bang, kết quả là 2.52. Tương tự, có thể tính toán số ca tử vong do bệnh tim trên 10.000.000 người cho Ấn Độ là 171 (làm tròn đến giá trị nguyên gần nhất). Vì không có nguồn dữ liệu về số ca tử vong tổng cộng hoặc số ca tử vong do bệnh gan cho Ấn Độ, ta sẽ xử lý các giá trị này giống như cách đã xử lý các giá trị thiếu trước đó.

Bảng 6: Dữ liệu về tiêu thụ rượu quá mức và tỷ lệ tử vong (sau khi đã làm sạch)

#	Quốc gia	Tiêu thụ rượu (Lít/người)	Số ca tử vong do rượu	Số ca tử vong do bệnh tim	Số ca tử vong do bệnh gan
1	Australia	2.5	785	211	15.3000019
2	Áo	3.000000095	863	167	45.5999847
3	Bỉ và Luxembourg	2.900000095	883	131	20.7000076
4	Canada	2.400000095	793	185	16.3999962
5	Đan Mạch	2.900000095	971	220	23.89999962
6	Phần Lan	0.800000012	970	297	19
7	Pháp	9.100000381	751	11	37.9000153
8	Iceland	0.800000012	743	211	11.1999991
9	Ireland	0.699999988	1000	300	6.5
10	Israel	0.600000024	834	183	13.69999981
11	Ý	27.900000095	775	107	42.2000076
12	Nhật Bản	1.5	680	36	23.2000076
13	Hà Lan	1.799999952	773	167	9.19999973
14	New Zealand	1.899999976	916	266	7.699999809
15	Na Uy	0.080000012	806	227	12.19999981
16	Tây Ban Nha	6.5	724	185	20.27
17	Thụy Điển	1.600000024	743	207	11.1999991
18	Thụy Sĩ	5.800000191	693	115	20.29999924
19	Vương Quốc Anh (UK)	1.299999952	941	285	10.300001

20	Hoa Kỳ (US)	1.200000048	926	199	22.1000038
21	Tây Đức	2.700000048	861	172	36.7000067
22	Ấn Độ	2.520000000	830	171	20.27

3. Biến đổi dữ liệu

Như đã đề cập, quá trình chuyển đổi dữ liệu bao gồm các bước như làm mượt, loại bỏ nhiễu, tóm tắt, tổng quát hóa và chuẩn hóa. Trong ví dụ này, chúng ta sẽ sử dụng làm mượt, vì cách này đơn giản hơn so với tóm tắt và chuẩn hóa. Như đã thấy, trong dữ liệu của chúng ta, mức tiêu thụ rượu bình quân đầu người ở Ý là quá cao (27), trong khi mức tiêu thụ ở Na Uy lại quá thấp. Vì vậy, có thể đây là các giá trị ngoại lệ. Trong trường hợp này, chúng ta sẽ thay thế giá trị tiêu thụ rượu của Ý bằng 7.900000095. Tương tự, đối với Na Uy, chúng ta sẽ sử dụng giá trị 0.800000012 thay cho 0.0800000012. Chúng ta đang coi cả hai giá trị này là "lỗi thiết bị" hoặc "lỗi nhập liệu", dẫn đến việc thêm một chữ số thừa cho cả hai quốc gia này (chữ số “2” thừa ở trước cho Ý và chữ số “0” thừa sau dấu thập phân cho Na Uy). Đây là một giả định hợp lý dựa trên bối cảnh hạn chế mà chúng ta có về bộ dữ liệu. Một cách tiếp cận thực tế hơn là tìm các giá trị gần nhất từ các khu vực địa lý có dữ liệu sẵn có và sử dụng những giá trị đó để dự đoán cho các quốc gia có giá trị nhập liệu sai.

Bảng 7: Dữ liệu về tiêu thụ rượu quá mức và tỷ lệ tử vong sau khi biến đổi dữ liệu

#	Quốc gia	Tiêu thụ rượu (Lít/người)	Số ca tử vong	Số ca tử vong do bệnh tim	Số ca tử vong do bệnh gan
1	Australia	2.5	785	211	15.3000019
2	Áo	3.000000095	863	167	45.5999847
3	Bỉ và Luxembourg	2.900000095	883	131	20.7000076
4	Canada	2.400000095	793	185	16.3999962
5	Đan Mạch	2.900000095	971	220	23.89999962
6	Phần Lan	0.800000012	970	297	19
7	Pháp	9.100000381	751	11	37.9000153
8	Iceland	0.800000012	743	211	11.1999991
9	Ireland	0.699999988	1000	300	6.5
10	Israel	0.600000024	834	183	13.69999981
11	Ý	7.900000095	775	107	42.2000076
12	Nhật Bản	1.5	680	36	23.2000076
13	Hà Lan	1.799999952	773	167	9.19999973
14	New Zealand	1.899999976	916	266	7.699999809
15	Na Uy	0.800000012	806	227	12.19999981
16	Tây Ban Nha	6.5	724	185	20.27

17	Thụy Điển	1.600000024	743	207	11.1999991
18	Thụy Sĩ	5.800000191	693	115	20.29999924
19	Vương quốc Anh (UK)	1.299999952	941	285	10.300001
20	Hoa Kỳ (US)	1.200000048	926	199	22.1000038
21	Tây Đức	2.700000048	861	172	36.7000067
22	Ấn Độ	2.52	830	171	20.27

4. Giảm dữ liệu

Giảm dữ liệu là một quá trình quan trọng, trong đó dữ liệu được loại bỏ hoặc tóm gọn sao cho kết quả phân tích vẫn đạt hiệu quả tương tự mà không bị mất mát quá nhiều thông tin. Trong ví dụ thì mẫu dữ liệu chỉ có 22 dòng. Giả sử cần thu thập dữ liệu của 196 quốc gia khác nhau. Trong trường hợp đó, số lượng dòng là rất lớn, và tùy thuộc vào dung lượng xử lý và lưu trữ có sẵn, có thể sẽ hợp lý hơn nếu làm tròn mức tiêu thụ rượu bình quân đầu người đến hai chữ số thập phân. Mỗi chữ số thập phân bổ sung cho mỗi điểm dữ liệu trong bộ dữ liệu lớn như vậy sẽ cần một lượng lớn dung lượng lưu trữ. Do đó, việc giảm cột bệnh gan xuống một chữ số thập phân và cột tiêu thụ rượu xuống hai chữ số thập phân sẽ tạo ra bộ dữ liệu tương đương như trong Bảng 8.

Lưu ý rằng giảm dữ liệu không chỉ có nghĩa là giảm kích thước của các thuộc tính mà còn có thể bao gồm việc loại bỏ một số thuộc tính, điều này được gọi là chọn lựa không gian đặc trưng. Ví dụ, nếu chỉ quan tâm đến mối quan hệ giữa lượng rượu tiêu thụ và số ca tử vong do bệnh tim, có thể lựa chọn loại bỏ thuộc tính "số ca bệnh gan" trong trường hợp nếu không có giả thuyết nào cho thấy mối quan hệ giữa bệnh tim và bệnh gan (ví dụ, có thể không có lý do khoa học hoặc thống kê chứng minh rằng số ca tử vong do bệnh tim có liên quan đến số ca tử vong do bệnh gan), thì có thể loại bỏ thuộc tính "số ca bệnh gan" khỏi phân tích. Điều này giúp đơn giản hóa mô hình phân tích và giảm bớt sự nhiễu loạn, đồng thời tiết kiệm tài nguyên tính toán và lưu trữ.

Bảng 8: Dữ liệu về tiêu thụ rượu quá mức và tỷ lệ tử vong sau khi giảm dữ liệu

#	Quốc gia	Tiêu thụ rượu (Lít/người)	Số ca tử vong do rượu	Số ca tử vong do bệnh tim	Số ca tử vong do bệnh gan
1	Australia	2.50	785	211	15.3
2	Áo	3.00	863	167	45.6
3	Bỉ và Luxembourg	2.90	883	131	20.7
4	Canada	2.40	793	185	16.4
5	Đan Mạch	2.90	971	220	23.9
6	Phân Lan	0.80	970	297	19

7	Pháp	9.10	751	11	37.9
8	Iceland	0.80	743	211	11.2
9	Ireland	0.70	1000	300	6.5
10	Israel	0.60	834	183	13.7
11	Ý	7.90	775	107	42.2
12	Nhật Bản	1.50	680	36	23.2
13	Hà Lan	1.80	773	167	9.2
14	New Zealand	1.90	916	266	7.7
15	Na Uy	0.80	806	227	12.2
16	Tây Ban Nha	6.50	724	185	20.3
17	Thụy Điển	1.60	743	207	11.2
18	Thụy Sĩ	5.80	693	115	20.3
19	Vương quốc Anh (UK)	1.30	941	285	10.3
20	Hoa Kỳ (US)	1.20	926	199	22.1
21	Tây Đức	2.70	861	172	36.7
22	Ấn Độ	2.52	830	171	20.3

5. **Rời rạc hóa dữ liệu:** Như đã thấy, tất cả các thuộc tính trong bộ dữ liệu đều có kiểu dữ liệu liên tục (giá trị là số thực). Tuy nhiên, tùy thuộc vào mô hình muốn xây dựng, có thể cần phải rời rạc hóa các giá trị thuộc tính thành các kiểu nhị phân hoặc phân loại. Ví dụ, có thể muốn rời rạc hóa mức tiêu thụ rượu bình quân đầu người thành bốn loại – nhỏ hơn hoặc bằng 1.00 lít/người (được biểu thị bằng 0), lớn hơn 1.00 nhưng nhỏ hơn hoặc bằng 2.00 lít/người (1), lớn hơn 2.00 và nhỏ hơn hoặc bằng 5.00 lít/người (2), và lớn hơn 5.00 lít/người (3). Kết quả nhận được được trình bày như sau

Bảng 9: Dữ liệu về tiêu thụ rượu quá mức và tỷ lệ tử vong sau khi làm sạch dữ liệu

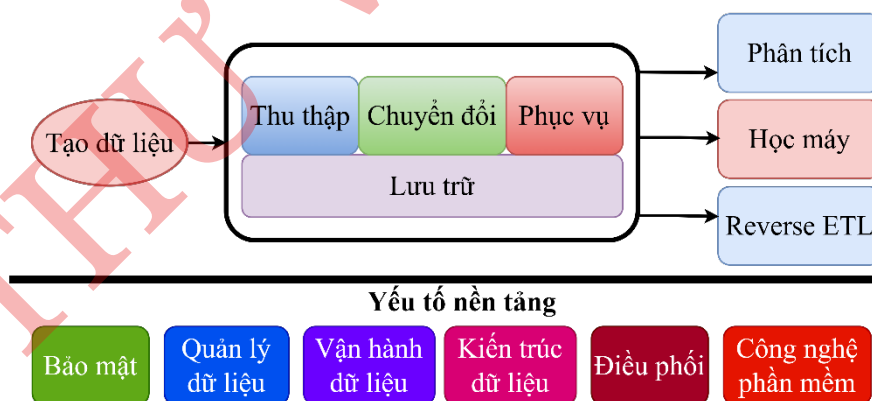
#	Quốc gia	Tiêu thụ rượu (Lít/người)	Số ca tử vong	Số ca tử vong do bệnh tim	Số ca tử vong do bệnh gan
1	Australia	2	785	211	15.3
2	Áo	2	863	167	45.6
3	Bỉ và Luxembourg	2	883	131	20.7
4	Canada	2	793	185	16.4
5	Đan Mạch	2	971	220	23.9
6	Phần Lan	0	970	297	19
7	Pháp	3	751	11	37.9
8	Iceland	0	743	211	11.2
9	Ireland	0	1000	300	6.5
10	Israel	0	834	183	13.7
11	Ý	3	775	107	42.2
12	Nhật Bản	1	680	36	23.2
13	Hà Lan	1	773	167	9.2
14	New Zealand	1	916	266	7.7

15	Na Uy	0	806	227	12.2
16	Tây Ban Nha	3	724	185	20.3
17	Thụy Điển	1	743	207	11.2
18	Thụy Sĩ	3	693	115	20.3
19	Vương quốc Anh (UK)	1	941	285	10.3
20	Hoa Kỳ (US)	1	926	199	22.1
21	Tây Đức	2	861	172	36.7
22	Ấn Độ	2	830	171	20.3

1.3. Các kĩ thuật cơ bản

1.3.1. Giới thiệu chung

Trong thời đại bùng nổ dữ liệu, các tổ chức phải đối mặt với thách thức không chỉ ở việc thu thập lượng dữ liệu khổng lồ, mà còn ở khả năng biến chúng thành thông tin có giá trị một cách hiệu quả và bền vững. Kỹ thuật dữ liệu (Data Engineering) xuất hiện như một lĩnh vực then chốt, cung cấp nền tảng kỹ thuật cho mọi hoạt động phân tích, học máy và ra quyết định dựa trên dữ liệu. Trọng tâm của kỹ thuật dữ liệu là một khung khái niệm toàn diện: Vòng đời kỹ thuật dữ liệu (Data Engineering Lifecycle), như thể hiện trên Hình 10. Đây không phải là một quy trình tuyến tính cứng nhắc, mà là một mô hình động, lặp lại, mô tả hành trình “từ nơi đến đâu” của dữ liệu trong một tổ chức, từ điểm phát sinh cho đến khi tạo ra giá trị kinh doanh. Vòng đời này cung cấp một lăng kính để các kỹ sư dữ liệu vượt ra khỏi những công cụ công nghệ cụ thể, thay vào đó tập trung vào các nguyên tắc nền tảng và mục tiêu cuối cùng của dữ liệu.



Hình 10. Vòng đời kỹ thuật dữ liệu

Vòng đời có thể được hình dung qua năm giai đoạn chính. Đầu tiên là Tạo dữ liệu (Generation). Đây là bước khởi nguyên, nơi dữ liệu được sinh ra từ các hệ thống nguồn đa dạng như cơ sở dữ liệu giao dịch (OLTP), tệp log, cảm biến IoT, API của bên thứ ba, hay thậm chí từ các hoạt động thủ công. Hiểu rõ nguồn dữ liệu – bao gồm tần suất, vận

tốc, cấu trúc (schema), độ nhất quán và các đặc tính nghiệp vụ – là điều kiện tiên quyết. Một thách thức lớn ở giai đoạn này là sự thay đổi schema từ phía nguồn, đòi hỏi các pipeline phải có khả năng phục hồi và thích ứng.

Khi dữ liệu được tạo ra, nó cần một nơi để cư trú, dẫn đến giai đoạn Lưu trữ (Storage). Việc lựa chọn hệ thống lưu trữ (kho dữ liệu, data lake, data lakehouse, object storage) là một quyết định kiến trúc quan trọng, ảnh hưởng sâu sắc đến hiệu suất, chi phí và tính linh hoạt của toàn bộ hệ thống. Các nguyên tắc như tách biệt tính toán và lưu trữ (separation of compute and storage), phân tầng dữ liệu theo "nhiệt độ" (hot, warm, cold), và quản lý vòng đời dữ liệu (data retention) trở nên thiết yếu. Lưu trữ không tồn tại biệt lập mà đan xen trong suốt vòng đời, đóng vai trò là nền tảng cho các giai đoạn tiếp theo.

Dữ liệu từ nguồn phải được di chuyển đến hệ thống lưu trữ, đó là nhiệm vụ của giai đoạn Thu thập (Ingestion). Giai đoạn này thường là nút thắt cổ chai của hệ thống, chịu ảnh hưởng bởi độ tin cậy của nguồn và hạ tầng mạng. Có hai mô hình chính: Batch (theo lô) và Streaming (theo luồng). Batch xử lý dữ liệu theo từng khối lớn tại một thời điểm cố định (ví dụ: mỗi đêm), phù hợp với báo cáo lịch sử. Streaming xử lý dữ liệu liên tục, gần thời gian thực, phục vụ các use-case như giám sát hệ thống hay đề xuất trực tiếp. Lựa chọn mô hình nào phụ thuộc vào yêu cầu về độ trễ (latency) và tính phức tạp nghiệp vụ.

Dữ liệu sau khi được thu thập thường ở dạng thô, chưa sẵn sàng để sử dụng. Giai đoạn Chuyển đổi (Transformation) là trái tim của việc tạo ra giá trị. Tại đây, dữ liệu được làm sạch, chuẩn hóa, hợp nhất, áp dụng logic nghiệp vụ và mô hình hóa để trở nên có ý nghĩa. Quá trình này có thể diễn ra trong batch (sử dụng Spark, dbt) hoặc trong luồng (sử dụng Flink, Beam). Các hoạt động chuyển đổi biến một bản ghi giao dịch thô thành một chỉ số kinh doanh, hoặc biến dữ liệu người dùng thô thành các đặc trưng (features) cho mô hình học máy.

Cuối cùng, mục đích tối thượng của mọi nỗ lực kỹ thuật dữ liệu là giai đoạn Phục vụ (Serving). Đây là lúc dữ liệu được trao đến tay người dùng cuối. Có ba nhóm use-case chính: Phân tích (Analytics) qua các báo cáo, dashboard BI và truy vấn trực tiếp; Học máy (Machine Learning) cung cấp dữ liệu đã được xử lý đặc trưng cho training và inference; và Reverse ETL, một mô hình mới nổi, đưa dữ liệu insights từ kho trung tâm ngược trở lại các hệ thống vận hành (như CRM, ERP) để thúc đẩy hành động.

Xuyên suốt và là nền tảng cho cả năm giai đoạn trên là sáu yếu tố nền tảng (Undercurrents). Chúng là các nguyên tắc và thực hành phải được tích hợp vào mọi khía cạnh của vòng đời:

1. Bảo mật (*Security*) & Quyền riêng tư (*Privacy*): Tuân thủ nguyên tắc đặc quyền tối thiểu, mã hóa dữ liệu, và các quy định như GDPR.
2. Quản lý Dữ liệu (*Data Management*): Bao gồm quản trị dữ liệu (governance), chất lượng dữ liệu (quality), quản lý metadata và theo dõi nguồn gốc dữ liệu (lineage).
3. Vận hành Dữ liệu (*DataOps*): Áp dụng văn hóa và tự động hóa của DevOps vào pipeline dữ liệu, nhấn mạnh vào giám sát (monitoring), quan sát (observability) và phản hồi sự cố (incident response).
4. Kiến trúc Dữ liệu (*Data Architecture*): Thiết kế hệ thống linh hoạt, có thể mở rộng, với các quyết định có thể đảo ngược (reversible decisions) và ghép nối lỏng lẻo (loose coupling).
5. Điều phối (*Orchestration*): Sử dụng các công cụ như Apache Airflow, Dagster để quản lý, lập lịch và theo dõi các luồng công việc phức tạp.
6. Kỹ thuật Phần mềm (*Software Engineering*): Áp dụng các phương pháp viết mã sạch, kiểm thử, quản lý phiên bản và Infrastructure as Code vào phát triển pipeline.

Vòng đời kỹ thuật dữ liệu cung cấp một bản đồ chiến lược để điều hướng sự phức tạp ngày càng tăng của hệ sinh thái dữ liệu. Nó nhấn mạnh rằng, công nghệ chỉ là phương tiện; mục tiêu cuối cùng là xây dựng các hệ thống mạnh mẽ, an toàn và hiệu quả có thể biến dữ liệu thô thành một tài sản chiến lược, thúc đẩy sự đổi mới và tăng trưởng bền vững cho tổ chức.

1.3.2. Truyền thông và lưu trữ dữ liệu

Truyền thông và lưu trữ dữ liệu là hai trụ cột vật lý của bất kỳ hệ thống kỹ thuật dữ liệu nào, đóng vai trò là hệ tuần hoàn và kho dự trữ của toàn bộ vòng đời dữ liệu. Nếu việc truyền thông kém hiệu quả sẽ gây tắc nghẽn, khiến dữ liệu không thể di chuyển đến nơi cần thiết, thì việc lưu trữ kém phù hợp sẽ biến dữ liệu thành một gánh nặng chi phí

khổng lồ, khó khai thác. Hai yếu tố này liên kết chặt chẽ với nhau, quyết định trực tiếp đến hiệu năng, độ tin cậy, chi phí và tính linh hoạt của kiến trúc dữ liệu tổng thể.

A. Truyền thông dữ liệu

Truyền thông dữ liệu là quá trình vận chuyển dữ liệu từ điểm phát sinh (source systems) đến các hệ thống đích (storage, processing, serving), đảm bảo tính toàn vẹn, kịp thời và an toàn. Trong bối cảnh hiện đại, nó không còn là các tác vụ ETL (Extract, Transform, Load) đơn lẻ, mà là một mạng lưới phức tạp của các luồng dữ liệu đồng thời, liên tục.

1. Các mô hình và cơ chế chính

- **Batch (Theo mẻ/lô):** Đây là mô hình truyền thống, nơi dữ liệu được tích lũy trong một khoảng thời gian (ví dụ: 24 giờ) và được chuyển đi một lần. Phù hợp cho các tác vụ không yêu cầu thời gian thực như báo cáo tổng kết cuối ngày, xử lý lịch sử. Các công cụ phổ biến bao gồm các scheduler (cron, Airflow) kết hợp với scripts để kéo dữ liệu từ cơ sở dữ liệu hoặc API.
- **Streaming (Theo luồng/thời gian thực):** Dữ liệu được truyền đi ngay lập tức khi nó được tạo ra. Mô hình này là nền tảng cho kiến trúc hướng sự kiện (Event-Driven Architecture), cho phép phản ứng tức thì. Các nền tảng như **Apache Kafka**, **Apache Pulsar**, **AWS Kinesis** hoạt động như "đường cao tốc dữ liệu", đóng vai trò trung gian đệm và phân phối các message/event đến nhiều consumer một cách tin cậy và có thể mở rộng.
- **Change Data Capture (CDC):** Đây là một kỹ thuật tinh vi để theo dõi mọi thay đổi (INSERT, UPDATE, DELETE) trong cơ sở dữ liệu nguồn (thường là OLTP) và truyền tải chúng. Thay vì truy vấn toàn bộ bảng (gây tải nặng), CDC đọc trực tiếp từ transaction logs (như MySQL binlog, PostgreSQL WAL), đảm bảo độ trễ thấp và giảm thiểu ảnh hưởng đến hiệu năng của hệ thống nguồn. Công cụ như **Debezium** là mã nguồn mở phổ biến cho mục đích này.
- **API:** Là giao diện tiêu chuẩn để các hệ thống trao đổi dữ liệu. **RESTful API** với định dạng JSON là phổ biến nhất cho sự đơn giản và khả năng tương tác rộng rãi. **GraphQL** cho phép client linh hoạt yêu cầu chính xác dữ liệu cần thiết, giảm

thiếu việc truyền tải dư thừa. APIs là cổng kết nối chính với dữ liệu từ các dịch vụ SaaS bên ngoài (Salesforce, Stripe) hoặc giữa các microservices nội bộ.

2. Các nguyên tắc thiết kế quan trọng

- **Độ tin cậy (Reliability):** Hệ thống truyền thông phải đảm bảo "ít nhất một lần" (at-least-once) hoặc "chính xác một lần" (exactly-once) delivery, tránh mất mát hoặc trùng lặp dữ liệu. Các message queue như Kafka sử dụng cơ chế xác nhận (acknowledgment) và lưu trữ bền vững (persistent storage) để đạt được điều này.
- **Khả năng mở rộng (Scalability):** Hệ thống phải có khả năng xử lý sự gia tăng đột biến về lưu lượng (ví dụ: trong các chiến dịch marketing) thông qua kiến trúc phân tán.
- **Tính lỏng lẻo (Loose Coupling):** Các thành phần trong hệ thống nên giao tiếp thông qua các giao diện được xác định rõ ràng (như message schema). Điều này cho phép nguồn (producer) và đích (consumer) hoạt động độc lập, có thể được nâng cấp hoặc thay thế mà không ảnh hưởng đến toàn hệ thống.

B. Lưu trữ dữ liệu

Lựa chọn và thiết kế hệ thống lưu trữ là một trong những quyết định kiến trúc quan trọng nhất, vì nó ảnh hưởng lâu dài đến mọi thứ từ tốc độ truy vấn đến chi phí vận hành. Các công nghệ lưu trữ hiện đại tồn tại trên một quang phổ, từ có cấu trúc chặt chẽ đến phi cấu trúc.

1. Các mô hình lưu trữ chính

- **Data Warehouse (Kho dữ liệu):** Được tối ưu hóa cho phân tích truy vấn phức tạp (OLAP). Dữ liệu được lưu trữ theo mô hình có cấu trúc cao (thường là dạng cột - columnar) và được tổ chức theo các mô hình hình sao (star schema) hoặc bông tuyết (snowflake schema). **Snowflake, Google BigQuery, Amazon Redshift** là những data warehouse cloud tiêu biểu, nổi bật với khả năng tách biệt hoàn toàn tài nguyên tính toán và lưu trữ, cho phép scale độc lập và chi trả theo mức sử dụng.
- **Data Lake (Hồ dữ liệu):** Là kho lưu trữ tập trung, chi phí thấp, lưu giữ dữ liệu thô ở nguyên bản định dạng của nó (cấu trúc, bán cấu trúc, phi cấu trúc). Nền tảng

thường là **Amazon S3**, **Azure Blob Storage**, hoặc **Google Cloud Storage**. Ưu điểm lớn nhất là tính linh hoạt: lưu mọi thứ trước, phân tích sau. Tuy nhiên, nếu không có sự quản trị chặt chẽ (metadata, catalog, chất lượng), nó dễ dàng trở thành "đầm lầy dữ liệu" (data swamp) - nơi dữ liệu tồn tại nhưng không ai có thể tìm thấy hoặc tin tưởng để sử dụng.

- **Data Lakehouse:** Là thể hệ kế thừa, kết hợp ưu điểm của cả Data Lake và Data Warehouse. Nó cung cấp các tính năng quản lý của kho dữ liệu (ACID transactions, governance, hiệu năng SQL) trực tiếp trên dữ liệu được lưu trữ dưới dạng object trong data lake. **Databricks Delta Lake**, **Apache Iceberg**, và **Apache Hudi** là các công nghệ định dạng table mở (open table formats) cho phép xây dựng kiến trúc lakehouse.
- **Cơ sở dữ liệu Chuyên biệt:** Tùy use-case, các hệ thống chuyên biệt được sử dụng:
 - **Cơ sở dữ liệu Quan hệ (RDBMS/OLTP):** PostgreSQL, MySQL cho các giao dịch và ứng dụng nghiệp vụ.
 - **Cơ sở dữ liệu NoSQL:** MongoDB (tài liệu), Cassandra (cột rộng), Neo4j (đồ thị) cho các mô hình dữ liệu và mẫu truy cập đặc thù.
 - **Cơ sở dữ liệu Chuỗi Thời Gian (Time-Series):** InfluxDB, TimescaleDB tối ưu cho dữ liệu cảm biến và giám sát.

2. Các nguyên tắc và xu hướng hiện đại

- **Tách biệt tính toán và lưu trữ (Separation of Compute and Storage):** Đây là nguyên tắc thiết kế nền tảng của kiến trúc cloud-native. Nó cho phép scale hai tài nguyên này độc lập, giảm chi phí (vì chỉ trả tiền lưu trữ khi không chạy truy vấn) và tăng tính linh hoạt (nhiều cụm compute khác nhau có thể truy cập cùng một dataset).
- **Phân tầng dữ liệu (Data Tiering/Lifecycle):** Không phải tất cả dữ liệu đều có giá trị truy cập như nhau. Dữ liệu được phân loại theo "nhiệt độ":
 - **Hot:** Truy cập vài lần/giây, cần lưu trên SSD hoặc memory (ví dụ: bảng chiều trong data mart).

- **Warm:** Truy cập hàng tuần/tháng, có thể lưu trên HDD hoặc object storage tiêu chuẩn.
- **Cold:** Hầu như không truy cập, chỉ lưu cho mục đích tuân thủ, lưu trữ trên object storage archive giá rẻ (như AWS Glacier).
- **FinOps cho Lưu trữ:** Quản lý chi phí lưu trữ trở thành một kỹ năng bắt buộc. Điều này bao gồm việc giám sát sát sao dung lượng, thiết lập chính sách xóa vòng đời tự động, lựa chọn lớp lưu trữ phù hợp và tối ưu hoá định dạng dữ liệu (ví dụ: chuyển sang định dạng columnar như Parquet để vừa tiết kiệm dung lượng vừa tăng tốc độ truy vấn).

Truyền thông và lưu trữ dữ liệu không phải là hai hoạt động biệt lập, mà là một thể thống nhất. Một pipeline dữ liệu hiệu quả yêu cầu sự phối hợp nhịp nhàng giữa việc lựa chọn cơ chế truyền tải phù hợp (streaming cho thời gian thực, batch cho xử lý khối) với việc lựa chọn mô hình lưu trữ tối ưu (warehouse cho BI, lakehouse cho AI/ML).

1.3.3. Phân tích dữ liệu

Phân tích dữ liệu (Data Analysis) trong kỹ thuật dữ liệu không chỉ đơn thuần là hoạt động khám phá hoặc tạo báo cáo cuối cùng. Nó là một tập hợp các quy trình kỹ thuật có hệ thống, được vận hành ở quy mô sản xuất, nhằm chuyển hóa dữ liệu thô thành **ngữ cảnh (context)**, **thông tin (information)**, và cuối cùng là **trí tuệ (intelligence)** có thể hành động được. Đây chính là giai đoạn "tạo ra giá trị" trong vòng đời kỹ thuật dữ liệu, nơi mà các khoản đầu tư vào hạ tầng lưu trữ và pipeline được đền đáp. Phân tích dữ liệu kỹ thuật bao gồm bốn trụ cột chính: Truy vấn, Mô hình hóa, Chuyển đổi, và Phân tích phục vụ Học máy.

1. Truy vấn dữ liệu - Ngôn ngữ của sự Tương tác

Truy vấn là phương thức cơ bản để khai thác và kiểm chứng dữ liệu. Trong môi trường hiện đại, **SQL** vẫn giữ vị thế là *lingua franca* không thể thay thế, nhờ vào tính khai báo (declarative), mạnh mẽ và được hỗ trợ gần như phổ quát.

- **SQL hiện đại và hiệu năng cao:** Các hệ thống như Google BigQuery, Snowflake, và Spark SQL đã đưa SQL lên một tầm cao mới, cho phép xử lý truy vấn phân tán trên petabytes dữ liệu với cú pháp quen thuộc. Chúng hỗ trợ các cấu trúc dữ liệu phức tạp (JSON, mảng) và các hàm cửa sổ (window functions) cho

phân tích nâng cao.

- **Tối ưu hóa truy vấn (Query Optimization):** Đây là nhiệm vụ trọng tâm của kỹ sư dữ liệu để đảm bảo hiệu suất và kiểm soát chi phí. Các kỹ thuật bao gồm:
 - **Thiết kế vật lý:** Sử dụng **Phân vùng (Partitioning)** (theo ngày, mã khách hàng) để loại bỏ việc quét toàn bộ dữ liệu; **Cụm (Clustering)** để sắp xếp dữ liệu trong phân vùng, tối ưu cho các bộ lọc phổ biến.
 - **Lựa chọn định dạng lưu trữ:** Sử dụng định dạng cột (như Parquet, ORC) giúp truy vấn chỉ đọc những cột cần thiết, giảm đáng kể I/O.
 - **Điều chỉnh cấu hình hệ thống:** Cấu hình bộ nhớ, tính toán song song phù hợp với khối lượng công việc.
- **Truy vấn liên hợp (Query Federation):** Một xu hướng quan trọng là khả năng truy vấn xuyên suốt nhiều nguồn dữ liệu (data warehouse, data lake, cơ sở dữ liệu quan hệ) mà không cần di chuyển dữ liệu trung tâm. Công cụ như **Presto/Trino** tạo ra một lớp trừu tượng, cho phép người dùng viết một truy vấn SQL duy nhất để kết hợp dữ liệu từ S3, MySQL, và Cassandra.

2. Mô hình hóa dữ liệu - Kiến trúc của Ngũ nghĩa

Mô hình hóa dữ liệu là nghệ thuật và khoa học tổ chức dữ liệu để phản ánh chính xác các quy tắc nghiệp vụ, tối ưu cho hiệu suất truy vấn và đảm bảo tính nhất quán. Nó là bản thiết kế quyết định cách dữ liệu "sống" trong hệ thống phân tích.

- Các trường phái chính:
 - **Kimball (Dimensional Modeling):** Hướng đến người dùng cuối (end-user), tập trung vào việc xây dựng các **Data Mart** chuyên biệt theo từng quy trình nghiệp vụ (bán hàng, marketing). Mô hình hình sao (Star Schema) với một bảng **Fact** (sự kiện, số đo) ở trung tâm, xung quanh là các bảng **Dimension** (đối tượng, ngữ cảnh) là tiêu chuẩn. Ưu điểm là đơn giản, truy vấn nhanh và dễ hiểu.
 - **Inmon (Enterprise Data Warehouse):** Hướng đến việc xây dựng một **kho dữ liệu trung tâm (EDW)** được chuẩn hóa cao (3NF), đảm bảo

tính toàn vẹn, không dư thừa và là "single source of truth". Các Data Mart sau đó được xây dựng như những lượt xử lý (views) từ EDW. Ưu điểm là tính nhất quán tuyệt đối, phù hợp với tổ chức lớn, phức tạp.

- **Data Vault 2.0:** Là một mô hình linh hoạt, được thiết kế cho tính **agile** và khả năng lưu trữ lịch sử thay đổi. Nó tách biệt cấu trúc lưu trữ (Hubs, Links, Satellites) khỏi logic nghiệp vụ, cho phép tích hợp nguồn dữ liệu mới một cách dễ dàng và hỗ trợ kiến trúc dữ liệu có thể kiểm toán.
- **Mô hình hóa cho luồng dữ liệu:** Với dữ liệu sự kiện, mô hình hóa thường tập trung vào việc định nghĩa **schema của message** (sử dụng Avro, Protobuf), thiết kế các **aggregation tables** theo cửa sổ thời gian (tumbling, sliding windows), và xây dựng **feature stores** cho học máy thời gian thực.

3. Chuyển đổi Dữ liệu - Động cơ của Pipeline

Chuyển đổi là quá trình thực thi logic nghiệp vụ lên dữ liệu, biến dữ liệu từ dạng "có" sang dạng "hữu ích". Đây là trái tim của mọi pipeline dữ liệu.

- **Chuyển đổi theo mẻ/lô (Batch Transformation):** Xử lý khối lượng lớn dữ liệu theo chu kỳ. Các framework như **Apache Spark** cung cấp sức mạnh xử lý phân tán, trong khi các công cụ như **dbt (Data Build Tool)** đang trở thành tiêu chuẩn cho việc quản lý quy trình chuyển đổi bằng SQL một cách có kiểm soát, hỗ trợ testing, tài liệu hóa và version control cho logic nghiệp vụ.
- **Chuyển đổi luồng (Streaming Transformation):** Xử lý dữ liệu liên tục với độ trễ thấp. Các framework như **Apache Flink** và **Apache Beam** (mô hình Dataflow) cung cấp mô hình lập trình thống nhất cho cả batch và streaming, với các khái niệm mạnh mẽ về **windowing**, **watermark** (xử lý dữ liệu đến trễ), và **state management**.
- **Logic chuyển đổi cốt lõi** bao gồm: làm sạch (xử lý null, chuẩn hóa định dạng), ghép nối (joins), hợp nhất (aggregations), tính toán các chỉ số KPI mới, và áp dụng các quy tắc nghiệp vụ phức tạp.

4. Phân tích phục vụ học máy

Kỹ thuật dữ liệu và học máy gặp nhau ở giai đoạn phân tích. Kỹ sư dữ liệu xây dựng nền tảng dữ liệu mà trên đó các mô hình được đào tạo và vận hành.

- **Kỹ thuật đặc trưng (Feature Engineering):** Đây là bước then chốt để biến dữ liệu thô thành đầu vào có ý nghĩa cho mô hình. Kỹ sư dữ liệu tự động hóa các quy trình như tạo đặc trưng tổng hợp (ví dụ: tổng chi tiêu 30 ngày của khách hàng), mã hóa biến phân loại (categorical encoding), và chuẩn hóa dữ liệu (scaling).
- **Kho đặc trưng (Feature Store):** Là một thành phần kiến trúc quan trọng trong MLOps, đóng vai trò như một "kho dữ liệu cho ML". Nó quản lý vòng đời của đặc trưng: lưu trữ, versioning, chia sẻ giữa các nhóm, và đảm bảo tính nhất quán tuyệt đối giữa đặc trưng dùng để huấn luyện (training) và phục vụ dự đoán (serving). Công cụ như **Feast**, **Tecton** giúp tự động hóa quy trình này.
- **Phân tích dữ liệu đào tạo mô hình:** Kỹ sư dữ liệu hỗ trợ việc tạo và quản lý các **tập dữ liệu đào tạo (training datasets)** chất lượng cao, theo dõi sự trôi dạt dữ liệu (data drift) có thể làm giảm hiệu suất mô hình, và đảm bảo pipeline cung cấp dữ liệu kịp thời cho cả batch inference và real-time inference.

Phân tích dữ liệu trong kỹ thuật dữ liệu là một kỷ luật kỹ thuật nghiêm ngặt, kết nối sâu sắc giữa hiểu biết nghiệp vụ và năng lực công nghệ. Nó chuyển hóa dữ liệu từ tài nguyên thô thành một hệ thống thông tin có cấu trúc, được tối ưu hóa, sẵn sàng không chỉ cho việc quan sát quá khứ (BI) mà còn cho việc dự đoán tương lai (AI).

1.3.4. Trực quan hoá dữ liệu

Trực quan hóa dữ liệu không đơn giản là bước trang trí cuối cùng của một pipeline dữ liệu. Nó là điểm giao tiếp tối hậu giữa hệ thống dữ liệu phức tạp và trí tuệ con người, là cơ chế chuyển đổi thông tin thành sự thấu hiểu và hành động. Trong bối cảnh kỹ thuật dữ liệu, trực quan hóa được xem như một dịch vụ dữ liệu (data serving) cấp cao, yêu cầu sự kết hợp nhuần nhuyễn giữa tối ưu hóa kỹ thuật (hiệu năng, bảo mật), nguyên tắc thiết kế (nhận thức, mỹ thuật), và hiểu biết sâu sắc về nghiệp vụ. Một hệ thống trực quan hóa được thiết kế tốt biến kho dữ liệu từ một tài sản tiềm năng trở thành một nền tảng tương tác, dân chủ hóa việc truy cập thông tin và thúc đẩy văn hóa dựa trên dữ liệu.

1. Vai trò trung tâm và các cấp độ trực quan hóa

Trực quan hóa đáp ứng các nhu cầu chiến lược khác nhau trong tổ chức, tương ứng với các cấp độ người dùng và yêu cầu kỹ thuật riêng biệt:

- **Phân tích kinh doanh (Business Intelligence - BI) và Báo cáo:** Đây là hình thức phổ biến nhất, tập trung vào việc **mô tả** quá khứ và hiện tại. Các **dashboard** và **báo cáo được lập lịch** (ví dụ: Doanh thu theo Quý, Tỷ lệ Giữ chân Khách hàng) cho phép nhà quản lý và điều hành theo dõi sức khỏe doanh nghiệp. Yêu cầu kỹ thuật chính là tính **ổn định, nhất quán** và khả năng xử lý **truy vấn tổng hợp (aggregate) trên khối lượng dữ liệu lớn**. Công cụ như Tableau, Power BI, Looker thống trị phân khúc này.
- **Phân tích vận hành (Operational Analytics):** Khác với BI truyền thống, phân tích vận hành nhắm đến **giám sát và hành động tức thì**. Nó hiển thị dữ liệu **theo thời gian thực hoặc gần thời gian thực** từ các luồng sự kiện, chẳng hạn như lượt truy cập website, chỉ số hiệu năng ứng dụng, hoặc tình trạng đơn hàng. Dashboard ở đây đóng vai trò như một "bảng điều khiển", yêu cầu **độ trễ cực thấp** và khả năng kết nối trực tiếp với các **nguồn streaming (Kafka)** hoặc cơ sở dữ liệu chuỗi thời gian (InfluxDB).
- **Phân tích nhúng (Embedded Analytics):** Đây là một thách thức kỹ thuật và thiết kế phức tạp, nơi các biểu đồ và báo cáo được **tích hợp trực tiếp vào sản phẩm hoặc ứng dụng SaaS** dành cho khách hàng bên ngoài. Ví dụ: bảng thống kê hiệu suất cá nhân trong một ứng dụng thể dục, hoặc báo cáo chỉ tiêu trong phần mềm kế toán. Yêu cầu then chốt bao gồm:
 - **Bảo mật và Đa tenancy mạnh mẽ:** Phải đảm bảo tuyệt đối rằng người dùng A không thể thấy dữ liệu của người dùng B, thường thông qua **row-level security (RLS)** phức tạp.
 - **Khả năng mở rộng (Scalability):** Hệ thống phải xử lý được hàng chục ngàn đến hàng triệu người dùng đồng thời.
 - **Khả năng tùy biến và Thương hiệu:** Giao diện phải phù hợp với thương hiệu của sản phẩm chính.

2. Kiến trúc và kỹ thuật nền tảng

Để phục vụ hiệu quả các cấp độ trên, kỹ sư dữ liệu phải xây dựng một nền tảng trực quan hóa vững chắc:

- **Kiến trúc phân tầng:** Một kiến trúc điển hình bao gồm: (1) **Lớp dữ liệu** (Data Warehouse/Lakehouse đã được chuyển đổi), (2) **Lớp ngữ nghĩa/mô hình Chỉ số (Semantic/Metrics Layer)** – nơi định nghĩa các chỉ số KPI, quan hệ giữa các bảng (như LookML trong Looker hoặc Cube.js), (3) **Lớp trực quan hóa** (Công cụ BI), và (4) **Lớp phân phối** (Nhúng qua iframe, API hoặc ứng dụng).
- **Tối ưu hóa hiệu năng:** Truy vấn dashboard chậm là kẻ hủy diệt trải nghiệm người dùng. Các kỹ thuật bao gồm:
 - **Tạo vật liệu hóa trước (Pre-aggregation/Persisted Derived Tables):** Tính toán trước các tổng hợp phức tạp vào các bảng vật lý để truy vấn nhanh.
 - **Lập chỉ mục (Indexing) và Bộ nhớ đệm (Caching):** Sử dụng cache ở nhiều cấp độ (trong công cụ BI, tại lớp ngữ nghĩa, hoặc tại cơ sở dữ liệu).
 - **Thiết kế truy vấn Hiệu quả:** Đảm bảo logic dashboard tạo ra các truy vấn SQL được tối ưu, tránh các hàm phức tạp ở cấp độ trình bày.
- **Quản lý Metadata và tính nhất quán:** Một thách thức lớn là đảm bảo "một nguồn chân lý duy nhất" (single source of truth). Một lớp ngữ nghĩa tập trung là then chốt để quản lý định nghĩa chỉ số, logic nghiệp vụ, và từ điển dữ liệu, đảm bảo mọi dashboard đều sử dụng cùng một công thức tính "Doanh thu" hay "Khách hàng hoạt động".

3. Nguyên tắc thiết kế hướng đến con người

Công cụ mạnh mẽ phải đi kèm với thiết kế xuất sắc để tạo ra thấu hiểu, không chỉ đơn thuần là hiển thị dữ liệu.

- **Lựa chọn biểu đồ phù hợp:** Tuân theo các nguyên tắc nhận thức: biểu đồ đường cho xu hướng thời gian, biểu đồ cột cho so sánh, biểu đồ phân tán cho mối quan hệ. Tránh các biểu đồ "lạ" gây khó hiểu như biểu đồ bánh 3D.

- **Tính rõ ràng và giảm nhiễu:** Loại bỏ mọi yếu tố trang trí không cần thiết (đường kẻ dày, nền chấm, hiệu ứng 3D). Tối đa hóa tỷ lệ "mực dữ liệu" - mỗi pixel trên màn hình đều nên truyền tải thông tin.
- **Tương tác và phân tầng thông tin:** Cho phép người dùng lọc (filter), phóng to (zoom), và "khoan sâu" (drill-down) từ tổng quan vào chi tiết. Điều này biến một báo cáo tĩnh thành một công cụ khám phá linh hoạt.
- **Kể chuyện bằng dữ liệu (Data Storytelling):** Đây là cấp độ cao nhất của trực quan hóa. Một dashboard được "kể chuyện" tốt sẽ dẫn dắt người xem qua một luận điểm rõ ràng: đặt bối cảnh, trình bày dữ liệu chính, giải thích nguyên nhân, và gợi ý hành động. Nó kết hợp các yếu tố như bố cục, chú thích, và trình tự logic.

4. Các thách thức và xu hướng hiện đại

- **Tự phục vụ (Self-Service) vs. Kiểm soát (Governance):** Cân bằng giữa việc trao quyền cho người dùng doanh nghiệp tự tạo báo cáo với việc duy trì tính nhất quán, chất lượng và bảo mật dữ liệu. Giải pháp nằm ở các công cụ có lớp ngữ nghĩa mạnh và cơ chế phê duyệt (governed self-service).
- **Trực quan hóa cho dữ liệu thời gian thực và lớn:** Yêu cầu các kỹ thuật như lấy mẫu thông minh (smart sampling), xấp xỉ (approximation), và sử dụng các engine xử lý streaming để cập nhật visualization liên tục mà không làm quá tải hệ thống.
- **Automation và cá nhân hóa:** Tự động tạo và phân phối báo cáo qua email hoặc Slack dựa trên sự kiện hoặc lịch trình. Hơn nữa, xu hướng cá nhân hóa dashboard dựa trên vai trò người dùng đang gia tăng.

Trực quan hóa dữ liệu trong kỹ thuật dữ liệu vượt xa khỏi việc vẽ biểu đồ, nó là một sản phẩm kỹ thuật đầy đủ, đòi hỏi kiến trúc cẩn trọng, kỹ thuật tối ưu hóa, và tư duy thiết kế lấy con người làm trung tâm. Một hệ sinh thái trực quan hóa thành công không chỉ cung cấp cái nhìn sâu sắc mà còn xây dựng niềm tin vào dữ liệu, biến thông tin thành một động lực hữu hình cho sự phát triển của tổ chức. Nó là cầu nối không thể thiếu, biến sức mạnh tiềm ẩn của các pipeline và kho dữ liệu thành giá trị kinh doanh hữu hình và có thể hành động được.

1.4. Câu hỏi và bài tập Chương 1

Bài 1: (*Câu hỏi lý thuyết*) Trình bày khái niệm kỹ thuật dữ liệu (Data Engineering) và phân biệt với khoa học dữ liệu (Data Science).

Bài 2: (*Bài tập tình huống*) Hãy liệt kê 3 ví dụ thực tế về ứng dụng của kỹ thuật dữ liệu trong lĩnh vực viễn thông hoặc IoT.

Bài 3: (*Câu hỏi thảo luận*) Tại sao dữ liệu lại được coi là “nguyên liệu” của cuộc cách mạng công nghiệp 4.0?

Bài 4: (*Bài tập nhận diện*) Cho các kiểu dữ liệu sau: hình ảnh, file log, bảng Excel, video stream. Hãy phân loại chúng thành dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc.

Bài 5: (*Câu hỏi tự luận*) Mô tả quy trình thu thập và tiền xử lý dữ liệu trong một hệ thống giám sát giao thông thông minh.

Bài 6: (*Bài tập thực hành*) Sử dụng công cụ miễn phí (như Google Dataset Search), tìm một bộ dữ liệu về chất lượng không khí Hà Nội và phân tích sơ bộ các trường dữ liệu.

Bài 7: (*Câu hỏi so sánh*) So sánh ưu nhược điểm giữa lưu trữ dữ liệu trên đám mây và lưu trữ cục bộ.

Bài 8: (*Bài tập nhóm*) Thiết kế sơ đồ luồng dữ liệu (data pipeline) đơn giản cho ứng dụng thu thập và phân tích ý kiến người dùng từ mạng xã hội.

Bài 9: (*Câu hỏi trắc nghiệm*) Công cụ nào sau đây thường được dùng để trực quan hóa dữ liệu?

- a) MySQL,
- b) Tableau,
- c) Hadoop,
- d) Linux.

Bài 10: (*Bài tập tư duy*) Nêu 3 thách thức lớn nhất trong việc xử lý dữ liệu lớn (Big Data) hiện nay và đề xuất hướng giải quyết.

Bài 11: (*Bài tập nhóm*) Phân tích vai trò của kỹ thuật dữ liệu trong hệ sinh thái AI - phân tích mối quan hệ giữa Data Engineer, Data Scientist và Data Analyst.

Bài 12: (*Bài tập nhóm*) Nghiên cứu case study về thất bại do thiếu hệ thống kỹ thuật dữ liệu (ví dụ: hệ thống y tế, ngân hàng).

Bài 13: (*Bài tập nhóm*) Thiết kế kiến trúc data pipeline cho startup công nghệ tài chính (FinTech) ở giai đoạn đầu.

Bài 14: (*Bài tập nhóm*) So sánh các mô hình xử lý dữ liệu: Batch Processing vs. Stream Processing.

Bài 15: (*Bài tập nhóm*) Phân tích đạo đức trong thu thập dữ liệu cá nhân theo Luật An

ninh mạng Việt Nam và GDPR.

Bài 16: (*Bài tập nhóm*) Nghiên cứu ứng dụng kỹ thuật dữ liệu trong thành phố thông minh (Smart City).

Bài 17: (*Bài tập nhóm*) Xây dựng chiến lược quản lý vòng đời dữ liệu cho doanh nghiệp SME tại Việt Nam.

Bài 18: (*Bài tập nhóm*) Phân tích xu hướng Edge Computing và tác động đến kiến trúc dữ liệu phân tán.

Bài 19: (*Bài tập nhóm*) Thiết kế hệ thống dữ liệu cho ứng dụng giám sát môi trường sử dụng cảm biến IoT.

Bài 20: (*Bài tập nhóm*) Nghiên cứu các framework và tiêu chuẩn chất lượng dữ liệu (Data Quality Frameworks).

CHƯƠNG II: CÔNG CỤ VÀ NỀN TẢNG

1.5. Ngôn ngữ lập trình Python

1.5.1. Giới thiệu chung

Python là một ngôn ngữ lập trình vừa đơn giản và cũng rất mạnh mẽ. Ngôn ngữ này cho phép chúng ta giải quyết các bài toán dữ liệu ở nhiều quy mô và mức độ phức tạp khác nhau. Python hiện cũng là công cụ được sử dụng nhiều nhất trong lĩnh vực khoa học dữ liệu và thường xuyên xuất hiện trong yêu cầu tuyển dụng việc làm về khoa học dữ liệu. Python rất thân thiện và dễ học, vì vậy ngôn ngữ này đặc biệt phù hợp cho người mới bắt đầu. Python được đánh giá cao nhờ sự mạnh mẽ, linh hoạt và khả năng mở rộng cao. Những ưu điểm trên khiến ngôn ngữ này trở thành một công cụ lý tưởng để giải quyết những nhu cầu phức tạp và nâng cao trong lĩnh vực Khoa học Dữ liệu.

Trong phần này, chúng ta sẽ bắt đầu bằng phần giới thiệu về Python cũng như cấu trúc điều khiển của nó, sau đó đi sâu vào việc sử dụng ngôn ngữ này để xử lý nhiều bài toán dữ liệu khác nhau bằng cách áp dụng các phương pháp xử lý thống kê và học máy.

1.5.2. Cấu trúc điều khiển

Để ra quyết định dựa trên một (hoặc nhiều) điều kiện, chúng ta có thể sử dụng câu lệnh “if”. Đây là đoạn code kiểm tra xem năm 2020 có phải là năm nhuận hay không:

```
year = 2020
if (year % 4 == 0):
    print("Leap year")
else:
    print("Not a leap year")
```

Ở đây, toán tử chia lấy dư (%) thực hiện phép chia 2020 cho 4 và trả về phần dư của phép tính đó. Nếu phần dư bằng 0, chương trình sẽ in ra “Leap year” (năm nhuận), ngược lại sẽ in “Not a leap year” (không phải năm nhuận).

Vậy trong trường hợp có nhiều điều kiện cần kiểm tra thì sao? Rất đơn giản, chỉ cần sử dụng chuỗi các câu lệnh “if” và “elif” (viết tắt của “else if”).

Ví dụ dưới đây kiểm tra giá trị của một biến (collegeYear) và gán nhãn tương ứng:

```
collegeYear = 3
if (collegeYear == 1):
    print("Freshman")
elif (collegeYear == 2):
```

```
print("Sophomore")
elif (collegeYear == 3):
    print("Junior")
elif (collegeYear == 4):
    print("Senior")
else:
    print("Super-senior or not in college!")
```

Một dạng cấu trúc điều khiển khác là vòng lặp. Có hai loại vòng lặp chính: “while” và “for”. Vòng lặp **while** cho phép thực hiện một khối lệnh lặp đi lặp lại miễn là điều kiện được chỉ định vẫn còn đúng. Hành động lặp lại này sẽ tiếp tục cho đến khi điều kiện đó không còn được thỏa mãn nữa

Đây là ví dụ khi sử dụng vòng lặp “while” để in ra 5 số đầu tiên:

```
a, b = 1, 5
while (a <= b):
    print(a)
    a += 1
```

Với vòng lặp “for”, ta có thể viết lại ngắn gọn hơn:

```
for x in range(1, 6):
    print(x)
```

Hãy cùng xem một loạt ví dụ khác và xem các cấu trúc điều khiển này hoạt động như thế nào. Dưới đây là cấu trúc tổng thể của mã nguồn kiểm tra điều kiện:

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
else:
    statement(s)
```

Trong các ví dụ trước, chúng ta đã xem xét các điều kiện liên quan đến các biến số. Bây giờ, hãy thử áp dụng một điều kiện tương tự cho các biến ký tự (Character variables). Tưởng tượng một tình huống trong bảng câu hỏi trắc nghiệm: một câu hỏi trắc nghiệm có bốn lựa chọn là A, B, C, và D. Trong tình huống này (khi A và D là các lựa chọn đúng và các lựa chọn còn lại là sai), biểu thức logic để xác định một đáp án đúng sẽ được thiết lập như thế nào? Để kiểm tra xem câu trả lời mà người dùng đã chọn

có phải là đáp án đúng hay không, mã nguồn sẽ cần được viết sử dụng biểu thức điều kiện sau:

```
if ans == 'A' or ans == 'D':  
    print("Correct answer")  
else:  
    print("Wrong answer")
```

Tiếp theo, hãy cùng xem xét cách giải cùng một bài toán bằng cách sử dụng cấu trúc vòng lặp while:

```
ans = input('Guess the right answer: ')  
  
while (ans != 'A') and (ans != 'D'):  
  
    print("Wrong answer")  
  
    ans = input('Guess the right answer: ')
```

Đoạn code trên sẽ yêu cầu người dùng nhập lại cho đến khi câu trả lời đúng (A hoặc D) được nhập vào.

Cấu trúc tổng quát của vòng lặp while như sau:

```
while condition:  
    statement(s)
```

Các câu lệnh trong vòng lặp while sẽ được thực thi lặp đi lặp lại miễn là điều kiện còn đúng.

Cùng mục tiêu hoặc bài toán đó cũng có thể được giải quyết một cách hiệu quả bằng cách sử dụng cấu trúc vòng lặp for:

```
correctAns = ["A", "D"]  
for ans in correctAns:  
    print(ans)
```

Đoạn code trên sẽ in ra các đáp án đúng cho câu hỏi.

Bài tập thực hành: Cấu trúc điều khiển

Chọn bất kỳ số nào trong khoảng từ 99 đến 199 và kiểm tra xem số đó có chia hết cho 7 hay không bằng mã Python. Nếu số đó chia hết, hãy in ra dòng sau: "số đó chia hết cho 7"; nếu không, hãy in ra số gần nhất với số bạn đã chọn mà chia hết cho

7. Sử dụng vòng lặp while, hãy in ra tất cả các số chia hết cho 7 trong cùng một khoảng.

1.5.3. Thống kê với Python

Trong phần này, chúng ta sẽ khám phá cách Python và các thư viện mạnh mẽ của ngôn ngữ này cho phép đo lường và thể hiện các yếu tố thống kê cơ bản. Để nắm vững các kỹ thuật này, bạn nên tìm hiểu hoặc ôn lại các khái niệm thống kê bằng cách sử dụng các tài nguyên bên ngoài. Hãy bắt đầu với phân phối số. Trong Python, có thể biểu diễn phân phối này một cách hiệu quả thông qua một mảng (array) — vốn là một tập hợp có cấu trúc của các phần tử (trong trường hợp này là các số). Khả năng làm việc với mảng là nền tảng giúp Python trở thành công cụ hàng đầu trong việc tính toán thống kê và khoa học dữ liệu.

Ví dụ: khi xây dựng một cây phả hệ gia đình (family tree), sau khi nhập các dữ liệu liên quan (tức là các nhánh và lá của cây) vào hệ thống, mục tiêu tiếp theo có thể là thực hiện một số phân tích thống kê. Cụ thể, chúng ta có thể muốn xem xét và tính toán các chỉ số thống kê trên độ tuổi của các thành viên trong cây phả hệ đó.

Trước khi thực hiện bất kỳ quá trình xử lý nào, chúng ta cần biểu diễn nó như sau:

```
data1=[85,62,78,64,25,12,74,96,63,45,78,20,5,30,45,78,45, 96,65,45,74,12,78,23,8]
```

Nếu muốn, bạn có thể gọi đây là một tập dữ liệu. Để thực hiện các phân tích, chúng ta sử dụng thư viện Python rất phổ biến có tên là NumPy (Numerical Python). Vì vậy, hãy tiến hành nhập (import) và định nghĩa thư viện này (thường là với biệt danh np) để bắt đầu làm việc:

```
import numpy as np
```

Python đã được yêu cầu nhập thư viện tên là NumPy, và thư viện đó khi sử dụng sẽ được gọi bằng biệt danh là **np**. Gói cụ thể này cực kỳ hữu ích cho việc tính toán dữ liệu số sẽ thấy trong các bước tiếp theo. Việc thấy dòng lệnh trên này xuất hiện ở đầu nhiều chương trình Python là điều rất phổ biến. Bây giờ, hãy bắt đầu đặt câu hỏi để kiểm chứng khả năng của thư viện này

1. Giá trị lớn nhất và nhỏ nhất trong số các giá trị này là bao nhiêu?

```
max=np.max(data1)
print("Max: {0:d}".format(max)) # In ra: 96
min=np.min(data1)
print("Min: {0:d}".format(min)) # In ra: 5
```

2. Độ tuổi trung bình là bao nhiêu?

```
mean=np.mean(data1)
print("Mean: {0:8.4f}".format(mean)) # In ra: 52.24
```

3. Các giá trị độ tuổi được phân bố như thế nào trên phân phối này? Numpy cung cấp các hàm để tính phương sai (**var**) và độ lệch chuẩn (**std**) để thực hiện tính toán.

```
variance=np.var(data1)
print("Variance: {0:8.4f}".format(variance))
standarddev=np.std(data1)
print("STD: {0:8.4f}".format(standarddev))
```

4. Giá trị trung vị của độ tuổi là bao nhiêu? Numpy cung cấp các hàm để tính trung vị (**median**) để thực hiện tính toán.

```
median=np.median(data1)
print("Median: {0:8.4f}".format(median))
```

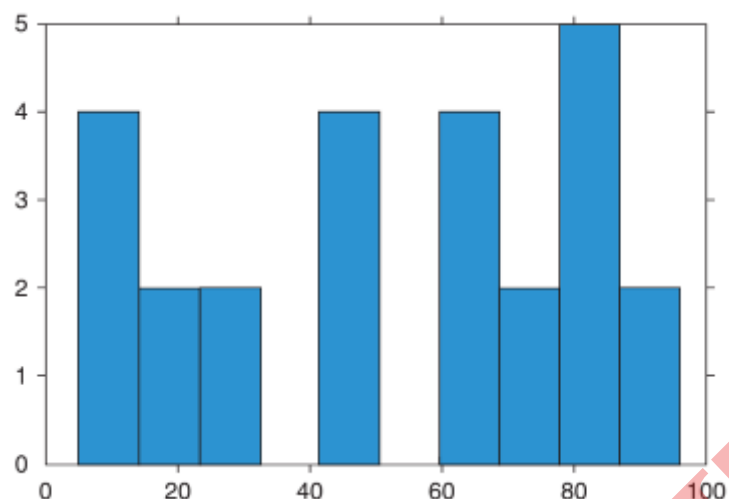
Cuối cùng, cũng có thể vẽ toàn bộ phân phối (dưới dạng biểu đồ histogram) bằng cách sử dụng một thư viện phù hợp. Trước tiên, hãy nhập (import) thư viện đó

```
import matplotlib.pyplot as plt
```

Một lần nữa, gói cần được nhập có tên là **matplotlib.pyplot**, và biệt danh **plt** được gán cho phiên làm việc hiện tại. Bây giờ, hãy chạy các câu lệnh sau:

```
plt.figure()
hist1, edges1 = np.histogram(data1)
plt.bar(edges1[:-1], hist1, width=edges1[1:]-edges1[:-1])
```

Ở đây, **plt.figure()** tạo một môi trường để vẽ đồ thị. Sau đó, chúng ta lấy dữ liệu để tạo biểu đồ histogram bằng dòng thứ hai. Dữ liệu này được truyền đến hàm **plt.bar()** cùng với một số tham số cho các trục để tạo ra biểu đồ histogram mà chúng ta thấy trong Hình 10.



Hình 10: Biểu đồ histogram thể hiện sự phân bố độ tuổi

Trong trường hợp gặp một số cảnh báo với hàm **plt.figure()**, hãy bỏ qua và tiếp tục chạy các lệnh còn lại. Điều này không làm cản trở hoạt động của chương trình

Nếu muốn tránh việc nhập thủ công một loạt giá trị để tạo tập dữ liệu mẫu, hàm khởi tạo số ngẫu nhiên (**randn**) của thư viện NumPy là một công cụ lý tưởng để sử dụng, ví dụ như trong đoạn mã sau:

```
data2 = np.random.randn(1000)
```

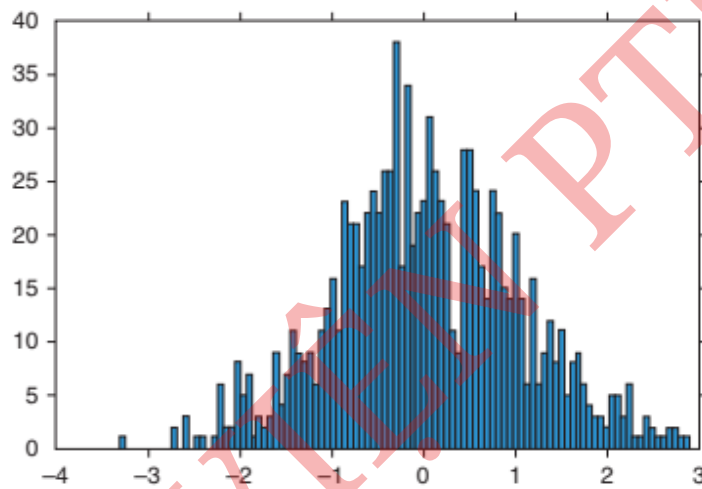
Bài tập thực hành: Thống kê cơ bản 1

Tạo một tập dữ liệu nhân tạo với 1000 số ngẫu nhiên. Chạy tất cả các phân tích chúng ta đã thực hiện trước đó với tập dữ liệu mới. Nhiệm vụ cần làm bao gồm tìm các giá trị trung bình, phương sai, và tạo biểu đồ

Nếu bạn đã thực hiện bài tập này, bạn sẽ nhận thấy kết quả là một biểu đồ histogram với một số lượng cột (bins) mặc định. Nhưng nếu bạn muốn trực quan hóa phân phối với một số lượng thanh khác thì sao? Ở đây có 1000 điểm dữ liệu, vì thế có thể yêu cầu hiển thị lên đến 1000 cột. Việc hiển thị biểu đồ với quá nhiều cột là quá nhiều và không hiệu quả, khiến dữ liệu trở nên khó đọc và làm mất đi ý nghĩa của việc trực quan hóa. Để khắc phục điều này và đạt được sự cân bằng tối ưu, cần chỉ định số lượng cột (bins) hợp lý thay vì sử dụng cài đặt mặc định. Ví dụ: nếu muốn hiển thị 100 cột, đoạn mã được chỉnh sửa như sau:


```
plt.figure()
hist2, edges2 = np.histogram(data2, bins=100)
plt.bar(edges2[:-1], hist2, width=edges2[1:]-edges2[:-1])
```

Kết quả được hiển thị trong Hình 11. Lưu ý rằng biểu đồ khi bạn chạy có thể trông hơi khác một chút vì bộ dữ liệu khi được tạo có thể khác với phạm vi cuốn sách này. Tại sao? Bởi vì chúng trong phạm vi cuốn sách này đang lấy các điểm dữ liệu này bằng cách sử dụng trình tạo số ngẫu nhiên. Trên thực tế, bạn có thể thấy các biểu đồ khác nhau mỗi lần chạy mã của bạn, bắt đầu bằng việc khởi tạo dữ liệu một cách ngẫu nhiên.



Hình 11: Biểu đồ thể hiện sự phân bố sử dụng 100 cột

Tập dữ liệu để thực hành tính toán giá trị tối thiểu, tối đa, phạm vi và trung bình cho tất cả các thuộc tính. Biểu diễn dữ liệu theo từng thuộc tính trên biểu đồ thanh để trực quan hóa phân phối.

1.5.3.1. Nhập dữ liệu

Để xử lý dữ liệu, trước tiên, chúng ta cần nhập dữ liệu vào Python. Để làm điều này, chúng ta sẽ sử dụng thư viện Pandas. Pandas thực sự là một thành phần cốt lõi và không thể thiếu trong hệ sinh thái phần mềm Khoa học Dữ liệu. Pandas DataFrame rất tiện dụng vì cung cấp thông tin hữu ích, chẳng hạn như tên cột được đọc từ nguồn dữ liệu để người dùng có thể hiểu và thao tác dữ liệu được nhập dễ dàng hơn. Giả sử dữ liệu nằm trong tệp "data.csv" trong thư mục hiện tại. Dòng sau sẽ tải dữ liệu đó vào biến "CSV_data".

```
from pandas import read_csv
CSV_data = read_csv('data.csv')
```

Một cách khác để sử dụng các chức năng của Pandas là cách làm việc với DataFrame. Đầu tiên, hãy nạp thư viện Pandas và sau đó gọi các hàm tương ứng như sau:

```
import pandas as pd
df = pd.read_csv('data.csv')
```

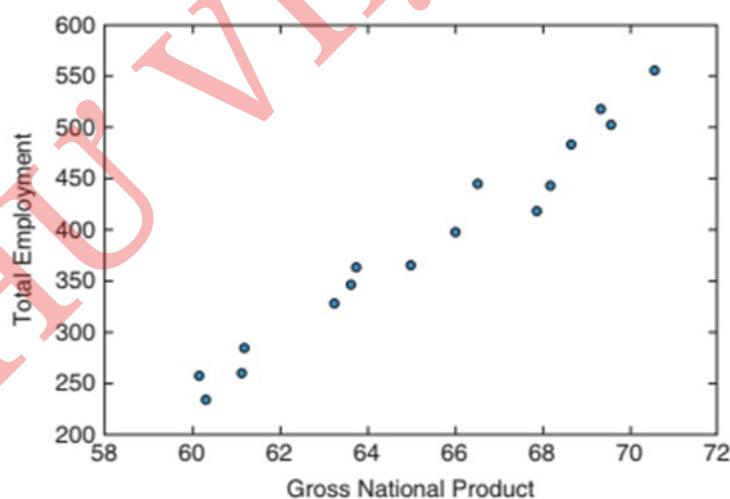
Điều này đặc biệt hữu ích nếu chúng ta cần sử dụng các chức năng của Pandas nhiều lần trong mã nguồn.

1.5.3.2. Vẽ biểu đồ

Một trong những đặc điểm nổi bật của Python, với sự trợ giúp của các thư viện mạnh mẽ, là ngôn ngữ này sở hữu các chức năng rất dễ sử dụng và hiệu quả trong việc trực quan hóa dữ liệu. Tất cả những gì cần làm là nạp thư viện **matplotlib.pyplot** và sử dụng hàm thích hợp để vẽ biểu đồ tương ứng. Dưới đây là mã nguồn tạo một biểu đồ phân tán để trực quan hóa mối quan hệ giữa hai biến "Employed" và "GNP".

```
import matplotlib.pyplot as plt

plt.scatter(df.Employed, df.GNP)
```



Hình 12: Biểu đồ phân tán để hình dung mối quan hệ giữa GNP và Việc làm

Hình 12 đã hiển thị biểu đồ và dường như hai biến này có mối liên hệ nào đó. Để khám phá thêm mối liên hệ này, trước tiên cần tìm độ mạnh của mối quan hệ giữa chúng bằng hàm tương quan, và sau đó thực hiện hồi quy.

1.5.3.3. Các mối tương quan

Một trong những bài toán phổ biến nhất khi giải các bài toán dựa trên dữ liệu là kiểm tra hai biến có liên quan hay không. Để thực hiện điều này, chúng ta có thể thống kê về mối tương quan.

Giả sử dữ liệu trước đó đã được nhập vào hệ thống. Để kiểm tra xem trường "Employed" và trường "GNP" có tương quan với nhau hay không, ta có thể sử dụng hàm **corrcoef** của thư viện NumPy. Hàm này sẽ tìm hệ số tương quan, cung cấp cho ta biết độ mạnh của mối tương quan giữa hai biến này. Mã nguồn đó được viết như sau:

```
np.corrcoef(df.Employed,df.GNP)[0,1]
```

Cần lưu ý rằng hệ số tương quan này là số dương, nghĩa là cả hai biến đều biến động cùng chiều (khi biến này tăng, biến kia cũng tăng). Nếu hệ số tương quan là số âm (gần -1), ta vẫn có một mối tương quan mạnh, nhưng theo chiều ngược lại (khi biến này tăng, biến kia sẽ giảm). Nói cách khác, trong trường hợp này, việc biết một biến sẽ cung cấp đủ hiểu biết về biến còn lại. Điều này đặt ra câu hỏi: Nếu biết giá trị của một biến (được gọi là biến độc lập hoặc biến dự báo), liệu có thể dự đoán giá trị của biến kia (được gọi là biến phụ thuộc hoặc biến phản hồi) một cách chính xác không?

Để làm được điều đó, chúng ta cần thực hiện phân tích hồi quy.

1.5.3.4. Hồi quy tuyến tính

Sau khi tìm hiểu về mối liên hệ giữa hai biến, câu hỏi tiếp theo được đặt ra: Nếu tồn tại một mối liên hệ, liệu một biến có thể dự đoán biến kia hay không và bằng phương pháp nào? Việc này thường được giải quyết thông qua kỹ thuật hồi quy (regression), cho phép thiết lập một mô hình toán học để dự đoán giá trị của biến phụ thuộc dựa trên giá trị của biến độc lập. Cụ thể, chúng ta muốn xem biến X ảnh hưởng đến biến y như thế nào. Ở đây, X được gọi là biến độc lập hoặc biến dự báo; và y được gọi là biến phụ thuộc hoặc kết quả.

Việc tìm ra mối quan hệ giữa biến độc lập (X) và biến phụ thuộc (y) có thể được xem như quá trình xây dựng hoặc điều chỉnh một mô hình bằng kỹ thuật Hồi quy Tuyến tính (Linear Regression). Có nhiều phương pháp để thực hiện kỹ thuật này, nhưng có lẽ phổ biến nhất là phương pháp bình phương tối thiểu thông thường (Ordinary Least Square - OLS).

Để thực hiện hồi quy tuyến tính bằng Python, chúng ta có thể sử dụng các hàm API của thư viện statsmodels như sau:

```
import statsmodels.api as sm

lr_model = sm.OLS(y, X).fit()
```

Ở đây, `lr_model` là mô hình được xây dựng bằng hồi quy tuyến tính sử dụng phương pháp OLS.

Làm thế nào để biết mô hình hồi quy này được xây dựng hiệu quả? Để kiểm chứng, cần phải kiểm tra kết quả của mô hình bằng cách chạy lệnh sau để đánh giá hiệu suất:

```
print(lr_model.summary())
```

Trong kết quả đầu ra, chúng ta có thể tìm thấy các giá trị hệ số: một cho const (hằng số) và một cho GNP. Phương trình hồi quy được diễn đạt lại như sau:

$$\text{Employed} = \text{coeff} * \text{GNP} + \text{const}$$

Bây giờ, để thực hiện dự đoán, chỉ cần thay thế giá trị của biến GNP vào mô hình hồi quy tuyến tính, sử dụng hệ số và hằng số tương ứng. Hãy so sánh giá trị dự đoán được cho biến "Employed" với dữ liệu thực tế. Mặc dù có thể có một số khác biệt (sai số), nhưng sự chênh lệch sẽ không đáng kể, nhờ mối tương quan mạnh. Tiếp theo, hãy cùng xem một ví dụ từ tập dữ liệu, chúng ta biết rằng vào năm 1960, giá trị GNP là 502,601. Chúng ta sẽ sử dụng phương trình hồi quy để tính giá trị của Employed. Thay các giá trị $\text{GNP} = 502,601$, hệ số $= 0.0348$, và hằng số $= 51.8436$ vào phương trình trên, ta được:

$$\text{Employed} = 0.0348 * 502,601 + 51.8436 = 69.334$$

Bây giờ, hãy tra cứu giá trị thực tế của biến Employed tại năm 1960. Con số đó là 69,564. Điều đó có nghĩa là sự sai lệch chỉ có 0.23. Do đó, con số đã được dự đoán đã gần sát với con số thực tế. Quan trọng hơn, giờ đây có một mô hình (phương trình đường thẳng) cho phép nội suy và ngoại suy. Nói cách khác, thậm chí có thể đưa vào một giá trị GNP chưa biết nào đó và tìm ra giá trị gần đúng tương ứng của biến "Employed".

Vậy tại sao không thực hiện việc này một cách có hệ thống hơn? Cụ thể, hãy đưa ra tất cả các loại giá trị cho biến độc lập, sau đó tìm các giá trị tương ứng cho biến phụ thuộc bằng phương trình hồi quy đã tìm được ở trên, và cuối cùng, biểu diễn tất cả các điểm dự đoán này trên biểu đồ phân tán cùng với dữ liệu gốc. Việc trực quan hóa này sẽ được xem xét chi tiết trong ví dụ đầy đủ bên dưới.

Ví dụ thực hành: Hồi quy tuyến tính

Dưới đây là mã nguồn đầy đủ hiển thị tất cả những điều đã thảo luận trong phần này: từ việc nhập dữ liệu đến thực hiện các phân tích thống kê và vẽ các đồ thị. Cần chú ý

rằng bất kỳ dòng nào hoặc phần nào của dòng bắt đầu bằng ký tự thăng (#) trong Python đều được coi là chú thích và sẽ bị bỏ qua hoàn toàn trong quá trình chương trình chạy. Kết quả cuối cùng của đoạn mã nguồn này là một biểu đồ phân tán hiển thị các điểm dữ liệu cùng với đường hồi quy đã được mô hình hóa. Dữ liệu được tải tại: <http://vincentarelbundock.github.io/Rdatasets/csv/datasets/longley.csv>

```
# Nạp các thư viện – numpy, pandas, pyplot, và statsmodels.api
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import statsmodels.api as sm
```

```
# Nạp bộ dữ liệu Longley vào Pandas DataFrame – cột đầu tiên (năm) được sử dụng làm # nhãn
```

```
df = pd.read_csv('longley.csv', index_col=0)
```

```
# Tìm mối tương quan giữa trường Employed và GNP
```

```
print("Correlation coefficient = ", np.corrcoef(df.Employed, df.GNP)[0,1])
```

```
# Chuẩn bị X và y cho mô hình hồi quy
```

```
y = df.Employed          # Biến phụ thuộc
```

```
X = df.GNP               # Biến độc lập
```

```

X = sm.add_constant(X)      # Thêm hằng số vào bộ dự đoán

# Xây dựng mô hình hồi quy sử dụng kỹ thuật OLS

lr_model = sm.OLS(y, X).fit()

print(lr_model.summary())

# Chọn 100 điểm nằm trên biến độc lập (X) được chia đều (equally spaced) từ giá trị
# nhỏ nhất (Min) đến giá trị lớn nhất (Max) của dữ liệu gốc
X_prime = np.linspace(X.GNP.min(), X.GNP.max(), 100)

X_prime = sm.add_constant(X_prime)      # Thêm hằng số

# Tiến hành dự đoán giá trị
y_hat = lr_model.predict(X_prime)

plt.scatter(X.GNP, y) # Plot the raw data

plt.xlabel("Gross National Product")

plt.ylabel("Total Employment")

# Thêm đường hồi quy được đánh dấu màu đỏ
plt.plot(X_prime[:, 1], y_hat, 'red', alpha=0.9)

```

(*) Nếu nhận thấy có điều gì đó bất thường trong biểu đồ được tạo từ đoạn mã trên, rất có thể là do môi trường vẽ biểu đồ đang sử dụng gặp lỗi hoặc xung đột cấu hình. Do đó cần sửa lại như sau:

```
plt.figure(1)
```

```
plt.subplot(211)
```

```
plt.scatter(df.Employed, df.GNP)
```

```
plt.subplot(212)
```

```
plt.scatter(X.GNP, y) # Plot the raw data
```

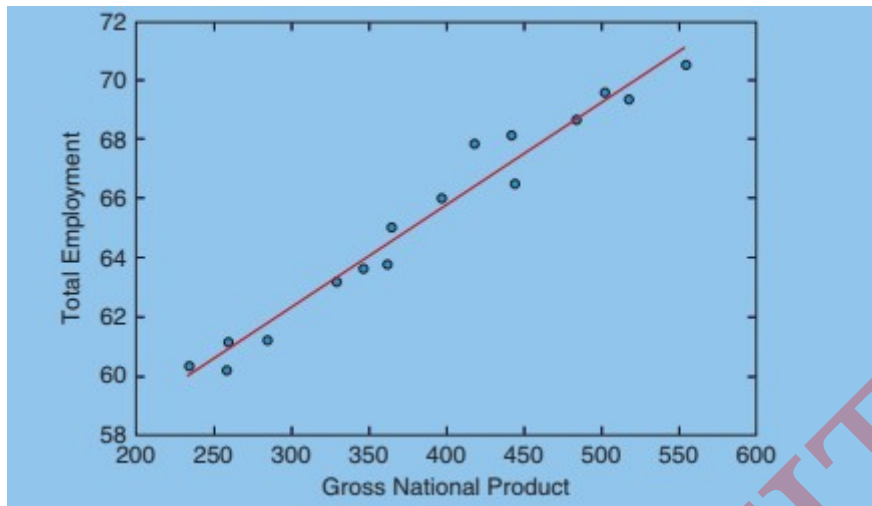
```
plt.xlabel("Gross National Product")
```

```
plt.ylabel("Total Employment")
```

```
# Thêm đường hồi quy được đánh dấu màu đỏ
```

```
plt.plot(X_prime[:, 1], y_hat, 'red', alpha=0.9)
```

Kết quả cuối cùng được hiển thị như trên Hình 13



Hình 13: Biểu đồ phân tán của GNP so với mức sử dụng được phủ lên đường hồi quy

1.5.3.5. Hồi quy tuyến tính đa biến

Những gì đã thấy cho đến nay là một biến duy nhất (biến dự báo) giúp dự đoán một biến khác (biến phản hồi hoặc biến phụ thuộc). Tuy nhiên, trong thực tế cuộc sống, có rất nhiều tình huống mà kết quả không được quyết định bởi một yếu tố duy nhất. Vì lý do đó, cần phải xem xét đồng thời nhiều yếu tố hoặc biến số khác nhau (sử dụng kỹ thuật Hồi quy đa biến) để xây dựng một mô hình dự đoán chính xác hơn.

Đây là lúc chúng ta sử dụng hồi quy tuyến tính đa biến. Đúng như tên gọi, đây chính là Hồi quy Đa biến — một phương pháp tính toán sử dụng nhiều yếu tố dự báo (biến độc lập) để dự đoán một biến phản ứng hoặc biến kết quả (biến phụ thuộc) duy nhất.

Ví dụ thực hành: Hồi quy tuyến tính đa biến

Chúng ta sẽ bắt đầu bằng cách lấy một tập dữ liệu chứa thông tin về ngân sách quảng cáo cho TV và radio, cũng như các số liệu bán hàng tương ứng. Điều cần tìm hiểu ở đây là mức độ ảnh hưởng của những ngân sách đến doanh số bán hàng là như thế nào.

Đầu tiên, hãy tải tập dữ liệu này vào môi trường Python của chúng ta.

```
# Nạp các thư viện cần thiết – numpy, pandas, pyplot, and
```

```
statsmodels.api
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import statsmodels.api as sm
```

```
# Tải tập dữ liệu Advertising.csv vào DataFrame
```

```
df = pd.read_csv('Advertising.csv', index_col=0)
```

Chúng ta bắt đầu phân tích bằng cách thực hiện hồi quy tuyến tính, như đã làm trước đó, có thể sử dụng biến "TV" để dự đoán "Doanh số" như thế nào.

```
y = df.Sales
```

```
X = df.TV
```

```
X = sm.add_constant(X)
```

```
lr_model = sm.OLS(y,X).fit()
```

```
print(lr_model.summary())
```

```
print(lr_model.params)
```

Trong kết quả này, điều cần chú ý là **giá trị R-squared (R^2)**. Giá trị này rơi khoảng 0.61, có nghĩa là khoảng **61%** của sự **phương sai (variance)** trong mối quan hệ giữa

biến TV và Sales có thể được giải thích bằng mô hình hồi quy đã xây dựng.

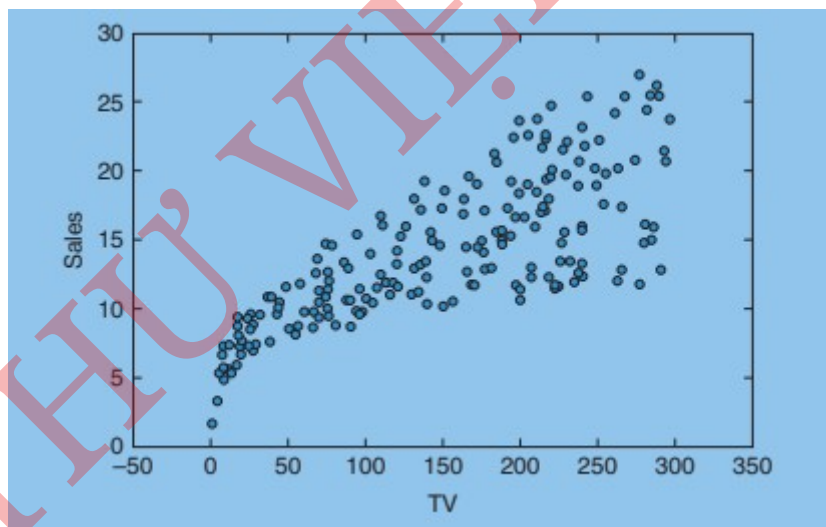
```
plt.figure()
```

```
plt.scatter(df.TV,df.Sales)
```

```
plt.xlabel('TV')
```

```
plt.ylabel('Sales')
```

Biểu đồ phân tán thể hiện sự tương quan giữa TV và Doanh số được thể hiện trong Hình 14.



Hình 14: Biểu đồ phân tán thể hiện sự tương quan giữa TV và Doanh số

Chúng ta hãy lặp lại quá trình này đối với radio.

```
y = df.Sales
```

```

X = df.Radio

X = sm.add_constant(X)

lr_model = sm.OLS(y,X).fit()

print(lr_model.summary())

print(lr_model.params)

plt.figure()

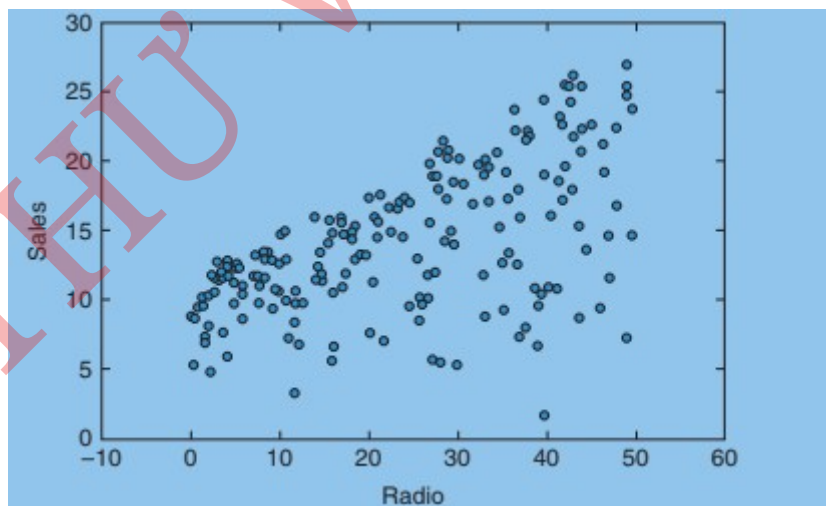
plt.scatter(df.Radio,df.Sales)

plt.xlabel('Radio')

plt.ylabel('Sales')

```

Biểu đồ phân tán thể hiện sự tương quan giữa Radio và Doanh số được thể hiện trong Hình 15.



Hình 15: Biểu đồ phân tán của Đài phát thanh so với Doanh số từ dữ liệu quảng cáo

Mô hình này cho chúng ta giá trị $R^2 = 0.33$, thậm chí còn tệ hơn so với kết quả chúng ta

thu được với TV. Bây giờ, hãy xem điều gì sẽ xảy ra nếu chúng ta kết hợp cả hai biến độc lập này (TV và radio) để dự đoán doanh số:

```
y = df['Sales']
```

```
X = df[['TV', 'Radio']]
```

```
X = sm.add_constant(X)
```

```
lr_model = sm.OLS(y,X).fit()
```

```
print(lr_model.summary())
```

Kết quả cho ra R^2 gần 90%. Kết quả này tốt hơn rất nhiều so với hai kết quả chúng ta tính toán sử dụng duy nhất một biến phụ thuộc đơn. Việc sử dụng nhiều biến độc lập để giải thích cho sự biến động của biến phụ thuộc chính là minh họa cho sức mạnh của phương pháp hồi quy tuyến tính đa biến.

Và đây là mã để vẽ hồi quy này trong không gian ba chiều (3D), với kết quả được hiển thị trong. Hãy coi đây là phần tùy chọn hoặc phần bổ sung.

```
from mpl_toolkits.mplot3d import Axes3D
```

```
# Dự đoán giá trị trên các trục tung và trục hoành sử dụng các khoảng TV và Radio
```

```
X_axis, Y_axis = np.meshgrid(np.linspace(X.TV.min(), X.TV.
```

```
max(), 100), np.linspace(X.Radio.min(), X.Radio.max(), 100))
```

```
# Vẽ mặt phẳng hồi quy bằng cách tính toán trực Z (Doanh số) tương ứng.
```

```
Z_axis = lr_model.params[0] + lr_model.params[1] * X_axis +  
lr_model.params[2] * Y_axis
```

```
# Tạo biểu đồ 3 chiều
```

```
fig = plt.figure(figsize=(12, 8)) # figsize refers to width and  
height of the figure
```

```
ax = Axes3D(fig, azimuth=-100)
```

```
# Vẽ biểu đồ
```

```
ax.plot_surface(X_axis, Y_axis, Z_axis, cmap=plt.cm.cool-  
warm, alpha=0.5, linewidth=0)
```

```
# Vẽ các điểm trên biểu đồ vừa dựng
```

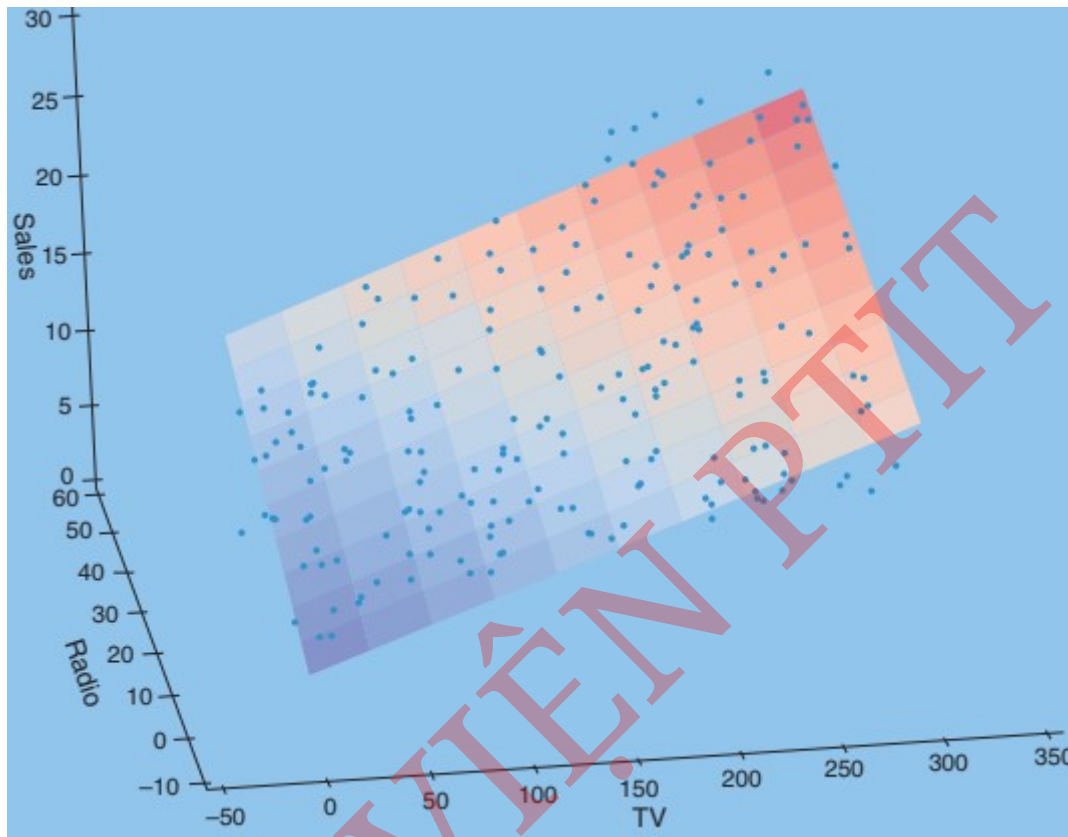
```
ax.scatter(X.TV, X.Radio, y)
```

```
# Đặt nhãn các trục tung, trục hoành và trục ca
```

```
ax.set_xlabel('TV')
```

```
ax.set_ylabel('Radio')
```

```
ax.set_zlabel('Sales')
```



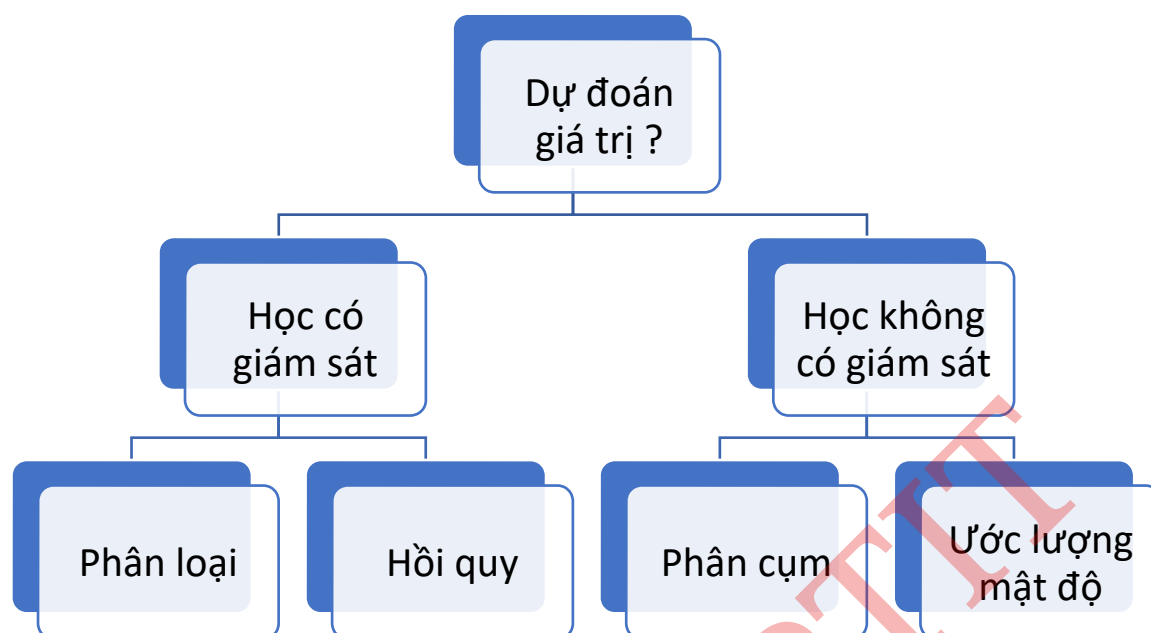
Hình 16: Biểu đồ phân tán ba chiều hiển thị các biến TV, Radio và Doanh số bán hàng

1.5.4. Học máy với Python

Phần này sẽ đề cập đến học máy và ứng dụng của học máy sử dụng Python. Việc thử sức với một lĩnh vực mới như Học máy và xem xét các vấn đề có thể giải quyết từ Học máy cũng rất đáng giá.

1.5.4.1. Học máy là gì?

Học máy (Machine Learning – ML) là một lĩnh vực nghiên cứu và ứng dụng, đồng thời là một trong những kỹ năng quan trọng nhất mà những người làm khoa học dữ liệu cần liệt kê trong sơ yếu lý lịch của mình.



Hình 17: Tổng quan các vấn đề học máy giải quyết

Khoa học máy tính và thống kê, cùng các lĩnh vực liên quan khác như kỹ thuật được sử dụng trong hầu hết mọi lĩnh vực liên quan đến xử lý dữ liệu, bao gồm kinh doanh, tin sinh học, dự báo thời tiết và tình báo (ví dụ như NSA và CIA).

Học máy là việc cho phép máy tính và các hệ thống nhân tạo khác học hỏi mà không cần lập trình một cách rõ ràng. Đây là quá trình các hệ thống “nhìn” thấy một tập dữ liệu, học hỏi từ nó, và sau đó sử dụng kiến thức đã học để suy ra hoặc dự đoán những điều mới từ dữ liệu khác. Tại sao lại là học máy? Bởi vì học máy có thể giúp bạn biến dữ liệu thành thông tin; có thể cho phép bạn dịch một tập hợp các điểm dữ liệu có vẻ nhàm chán thành các mẫu có ý nghĩa có thể hỗ trợ việc ra quyết định quan trọng. Đồng thời còn cho phép khai thác sức mạnh thực sự khi có rất nhiều dữ liệu trong tay.

Trong ví dụ phần trước (Hồi quy), một mô hình học máy đã được áp dụng khi dự đoán các giá trị dựa trên mối quan hệ đã học giữa các yếu tố dự báo và một kết quả.

Cốt lõi của Học máy có thể được giải thích một cách trực quan bằng cách sử dụng cây quyết định. Thật ngẫu nhiên, đây cũng là tên của một trong những thuật toán học máy phổ biến nhất. Cây quyết định giúp phân loại các vấn đề dữ liệu thành các nhánh logic, từ đó xác định phương pháp Học máy phù hợp. Nếu mục tiêu là dự đoán một giá trị, biến độc lập và biến phụ thuộc liên quan đến nhau, thì đó chính là Học có Giám sát. Cũng trong học máy có giám sát, nếu biến phụ thuộc là liên tục, bài toán trở thành hồi quy. Ví dụ: dự đoán thu nhập khi biết tuổi và nghề nghiệp của người đó. Mặt khác, nếu biến phụ thuộc là rời rạc (có một vài giá trị hoặc nhãn có thể), thì đây sẽ trở thành một bài

toán phân loại. Ví dụ: dự đoán mức thu nhập của một người là cao, trung bình hay thấp dựa trên tuổi và nghề nghiệp. Đây cũng chính là ví dụ của bài toán phân loại

Những bài toán này đòi hỏi phải biết trước đáp án (hay còn gọi là dữ liệu được gán nhãn). Ví dụ, để biết được mức thu nhập của một người dựa trên tuổi tác và nghề nghiệp, chúng ta cần biết giá trị thực sự của tầng lớp đó – họ thuộc nhóm thu nhập cao, thu nhập trung bình hay thu nhập thấp. Nhưng đôi khi, dữ liệu được cung cấp không có nhãn rõ ràng hoặc giá trị thực (ground truth). Tuy nhiên, vẫn cần phải khám phá và giải thích dữ liệu đó. Trong trường hợp này, ta đang xử lý Học không Giám sát (Unsupervised Learning).

Các vấn đề mà trong đó mục tiêu là sắp xếp dữ liệu thành các nhóm khác nhau chính là bài toán phân cụm (Clustering). Tương tự nhưng không giống với bài toán Phân loại (Classification), ta không biết trước có bao nhiêu nhóm (cụm) và các cụm đó là gì.

Mặt khác, nếu cố gắng giải thích dữ liệu bằng cách ước tính các quy trình cơ bản có thể chịu trách nhiệm cho cách dữ liệu được phân phối, thì đây chính là bài toán Ước lượng Mật độ (Density Estimation)

1.5.4.2. Phân loại (Học có giám sát)

Phân loại có thể được hiểu: Cho một tập hợp các điểm dữ liệu và nhãn tương ứng, hãy tìm hiểu cách thức phân loại. Khi một điểm dữ liệu mới xuất hiện, có thể xếp nó vào đúng lớp tương ứng. Có nhiều phương pháp và thuật toán để xây dựng bộ phân loại, một trong số đó là thuật toán k-láng giềng gần nhất (k-Nearest Neighbor - kNN).

kNN hoạt động theo nguyên tắc sau:

1. Tương tự với các bài toán phân loại đã có một tập hợp các điểm dữ liệu mà nhãn lớp chính xác đã được biết
2. Khi có một điểm dữ liệu mới, tiến hành so sánh điểm dữ liệu đó với từng điểm dữ liệu hiện có và tìm ra sự tương đồng (khoảng cách) giữa chúng.
3. Lấy k điểm dữ liệu giống nhau nhất (k lân cận gần nhất).
4. Từ k điểm dữ liệu này, thực hiện xác định nhãn chiếm phiếu bầu đa số. Nhãn chiến thắng sẽ là nhãn hoặc lớp được gán cho điểm dữ liệu mới.

Thông thường k là một số nhỏ từ 2 đến 20. Số lượng các lân cận gần nhất (giá trị k) càng nhiều thì thời gian xử lý càng lâu. Việc tìm kiếm sự tương đồng giữa các điểm dữ liệu cũng là một điều rất quan trọng. Để dễ hình dung, các điểm dữ liệu được đặt trên một mặt phẳng (hai hoặc ba chiều). Dựa vào vị trí này, việc tính toán khoảng cách giữa các điểm dữ liệu bằng kiến thức đại số tuyến tính (thường là sử dụng Khoảng cách Euclidean) sẽ trở nên dễ dàng và chính xác hơn. Khoảng cách này chính là thước đo cho sự tương đồng giữa các điểm.

Ví dụ thực hành: Phân loại

Trong bài toán này, một tập dữ liệu về rượu vang được sử dụng để tiến hành phân loại. Dữ liệu này chứa thông tin về các thuộc tính khác nhau của các loại rượu vang và chất lượng tương ứng của các loại rượu đó. Cụ thể, rượu vang được phân loại thành rượu chất lượng cao hay không. Chúng ta sẽ coi những thông tin này là nhãn lớp và xây dựng một bộ phân loại để học (dựa trên các thuộc tính khác) cách một loại rượu vang được phân loại vào một trong hai lớp này.

Chúng ta sẽ bắt đầu bằng cách nhập các thư viện cần thiết. Lưu ý ở đây sử dụng thư viện mới: sklearn (<http://scikit-learn.org/stable/index.html>) là một thư viện phổ biến để thực hiện các ứng dụng học máy bằng Python. Ví dụ này nhập dữ liệu, tiến hành huấn luyện với 70% tổng dữ liệu và kiểm tra trên 30% còn lại đồng thời tính toán độ chính xác của thuật toán được sử dụng

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier

from sklearn.cross_validation import train_test_split

df = pd.read_csv("wine.csv")

# Sử dụng 70% số lượng dữ liệu để huấn luyện và 30% số lượng dữ liệu để kiểm tra

# Sử dụng các đặc trưng như 'density', 'sulfates', and 'residual_sugar'

# để huấn luyện nhận biết nhãn 'high_quality'
```

```

X_train, X_test, y_train, y_test = train_test_split(df
[['density', 'sulfates', 'residual_sugar']], df['high_quality'], test_size=.3)

# Sử dụng thuật toán phân loại k-láng giềng gần nhất (kNN), xác định số lượng hàng
xóm lân cận là 3

classifier = KNeighborsClassifier(n_neighbors=3)

# Tiến hành huấn luyện

classifier.fit(X_train, y_train)

# Dự đoán nhãn trên các tập dữ liệu chưa được thấy

prediction = classifier.predict(X_test)

# Đếm số lượng nhãn dự đoán đúng

correct = np.where(prediction==y_test, 1, 0).sum()

print (correct)

# Tính toán độ chính xác của thuật toán phân loại

accuracy = correct/len(y_test)

print (accuracy)

```

Lưu ý rằng ví dụ trên sử dụng $k = 3$ (kiểm tra ba láng giềng lân cận gần nhất khi thực hiện so sánh). Độ chính xác thu được khoảng 76% (con số này có thể khác do khi chia tập dữ liệu, tập dữ liệu khi huấn luyện và kiểm tra khi chạy mỗi lần khác nhau). Nhưng điều gì sẽ xảy ra nếu giá trị k khác nhau? Hãy thử xây dựng và kiểm tra thuật toán bằng cách sử dụng các hệ số k khác nhau và vẽ biểu đồ độ chính xác tương ứng với mỗi hệ số k tương ứng.

```
# Khai báo mảng lưu trữ với kết quả (hệ số  $k$  và độ chính xác tương ứng với mỗi  $k$ )
```

```
results = []
```

```
# Giá trị của  $k$  trong khoảng từ 1 đến 51, mỗi lần thử tăng 2 đơn vị
```

```
for k in range(1, 51, 2):
```

```
    classifier = KNeighborsClassifier(n_neighbors=k)
```

```
    classifier.fit(X_train, y_train)
```

```
    prediction = classifier.predict(X_test)
```

```
    accuracy = np.where(prediction==y_test, 1, 0).sum() / (len(y_test))
```

```
    print ("k=",k,"Accuracy=", accuracy)
```

```
# Lưu trữ dưới dạng tuple  $k, accuracy$  vào mảng
```

```
    results.append([k, accuracy])
```

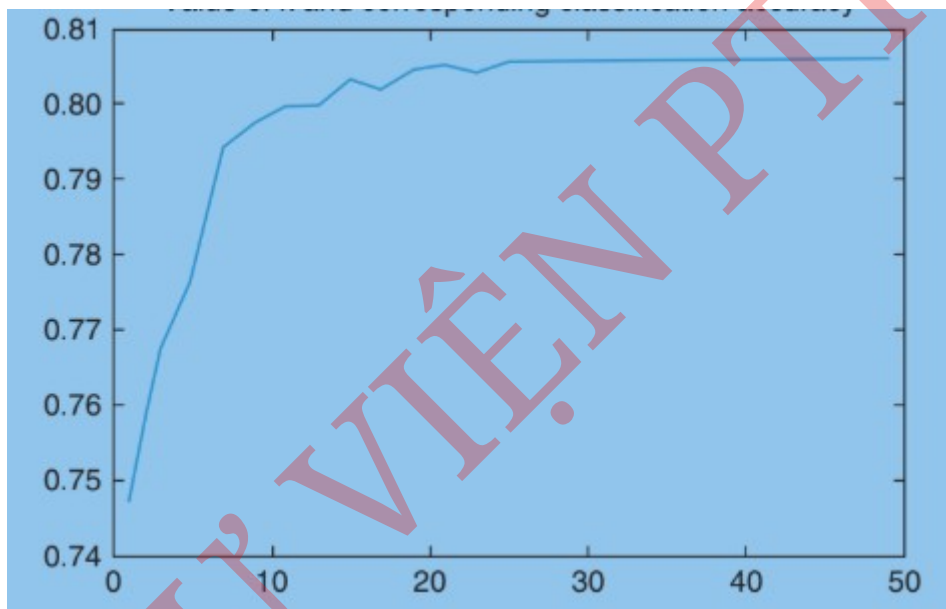
```
# Biến đổi tuple đó thành dạng DataFrame để phục vụ vẽ biểu đồ
```

```
results = pd.DataFrame(results, columns=["k", "accuracy"])
```

```
plt.plot(results.k, results.accuracy)

plt.title("Value of k and corresponding classification
accuracy")

plt.show()
```



Hình 18: Sơ đồ thể hiện cách các giá trị k khác nhau ảnh hưởng đến độ chính xác của mô hình kNN

Kết quả vẽ đồ thị được hiển thị trong Hình 18. Lưu ý rằng, mỗi lần chạy chương trình này, kết quả (và đồ thị) sẽ khác nhau. Tại hình này, mỗi khi giá trị hệ số k tăng lên (trong khoảng 20-30) thì sự cải thiện về độ chính xác hầu như không đáng kể (80.5-81%), hay đạt được điểm bão hòa.

1.5.4.3. Phân cụm (Học không có giám sát)

Bây giờ hãy xem xét một nhánh khác của Học máy.

Trong ví dụ vừa đề cập, ta biết rằng có hai lớp và nhiệm vụ là gán một điểm dữ liệu mới cho một trong những nhãn lớp hiện có. Nhưng điều gì sẽ xảy ra nếu các nhãn thực không được đánh nhãn hoặc các lớp chưa được xác định?

Đó là lúc ta cần áp dụng Học không Giám sát với sự trợ giúp của kỹ thuật Phân cụm (Clustering). Phân cụm giúp khám phá và sắp xếp dữ liệu thành các nhóm tự nhiên mà không cần thông tin nhãn lớp ban đầu.

Ví dụ thực hành: Phân cụm

Hãy cùng xem một ví dụ. Tương tự như các phần trước, cần nhập một số thư viện cần thiết trước khi bắt đầu làm việc:

```
import numpy as np

import matplotlib.pyplot as plt

# Nhập lớp style từ thư viện matplotlib và áp dụng tùy chỉnh giao diện

from matplotlib import style

style.use("ggplot")
```

Bây giờ, chúng ta sẽ lấy một loạt điểm dữ liệu và phân cụm chúng. Vậy làm thế nào để thực hiện được điều này? Ta sử dụng một thuật toán tuyệt vời gọi là K-Means. Thuật toán này cho phép khám phá cấu trúc nhóm trong dữ liệu mà không cần nhãn lớp có sẵn. Cách thức hoạt động của thuật toán này như sau.

1. Đầu tiên, chúng ta đoán hoặc xác định số cụm k mong muốn. Nếu các điểm dữ liệu của chúng ta ở dạng n chiều, tâm của các cụm này cũng sẽ là các điểm n chiều. Nói cách khác, sẽ có tổng cộng k điểm trong không gian n chiều, mà ta gọi là Tâm điểm (Centroid). Những điểm này đại diện cho trung tâm của mỗi cụm. Về cơ bản, đây chỉ là một số điểm ngẫu nhiên trong không gian n chiều đó.

2. Bây giờ, tiến hành gán mỗi điểm dữ liệu thực tế cho một trong k tâm điểm này dựa

trên khoảng cách của chúng đến các tâm điểm đó. Cụ thể, mỗi điểm sẽ được gán vào cụm có tâm điểm gần nhất. Sau bước này, mỗi điểm dữ liệu sẽ được gán cho một trong k cụm.

3. Bây giờ, chúng ta tính toán lại tâm điểm của mỗi cụm. Vậy là chúng ta lại có k trọng tâm, nhưng giờ đây đã được điều chỉnh để phản ánh cách phân bố các điểm dữ liệu. Bước 2 và 3 sẽ được thực hiện lặp đi lặp lại cho đến khi hội tụ. Nói cách khác, ta dừng quá trình lặp này khi trọng tâm của k cụm không còn thay đổi nữa. Tại thời điểm đó, ta đã có k cụm "thực", với mỗi điểm dữ liệu thuộc về một trong số các cụm này. May mắn thay, trong Python đã có các thư viện hỗ trợ nên không cần phải làm lại từ đầu. Thư viện đó là `sklearn.cluster`, thư viện này chứa các triển khai cho nhiều thuật toán phân cụm khác nhau bao gồm cả k-means. Sử dụng thư viện này như sau

```
# Nạp thư viện KMeans từ thư viện sklearn.cluster
```

```
from sklearn.cluster import KMeans
```

Đối với bài tập này, ta sẽ tạo một số điểm dữ liệu trong không gian hai chiều (2D) (để có thể dễ dàng hình dung):

```
# Định nghĩa các điểm trên tọa độ 2 chiều
```

```
X = np.array([[1, 2],[5, 8],[1.5, 1.8],[8, 8],[1, 0.6],[9, 11]])
```

Bây giờ, sẽ tiến hành phân cụm cũng như trực quan hóa các cụm mà thuật toán k-means tạo ra:

```
# Thực hiện phân cụm
```

```
kmeans = KMeans(n_clusters=2)
```

```
kmeans.fit(X)
```

```
# Trích xuất thông tin từ các cụm được biểu diễn thông qua các tâm điểm cùng với các  
nhãn tương ứng
```

```
centroids = kmeans.cluster_centers
```

```
labels = kmeans.labels
```

```
print(centroids)
```

```
print(labels)
```

```
# Định nghĩa mảng màu sử dụng trong biểu đồ
```

```
colors = ["g.", "r.", "c.", "y."]
```

```
# Lặp thông qua các điểm được vẽ trên biểu đồ với một màu được lấy từ danh sách ở
```

```
# trên, mỗi cụm một màu
```

```
for i in range(len(X)):
```

```
    print("Coordinate:", X[i], "Label:", labels[i])
```

```
    plt.plot(X[i][0], X[i][1], colors[labels[i]], markersize = 10)
```

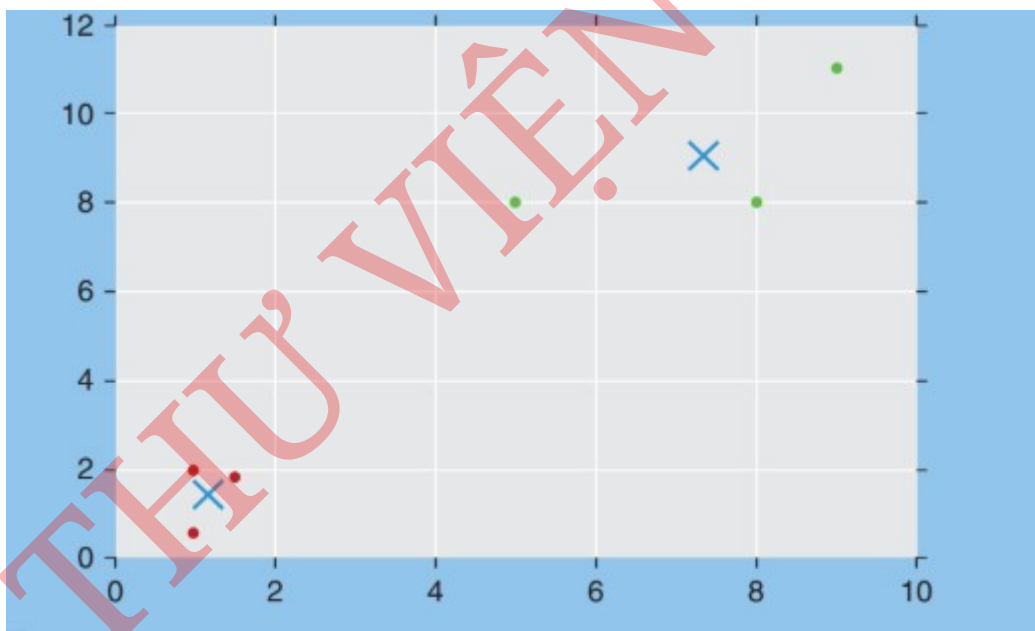


```
# Vẽ tâm điểm của cụm sử dụng kí hiệu “x”
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], marker="x", s=150, linewidths=2,  
zorder=10)
```

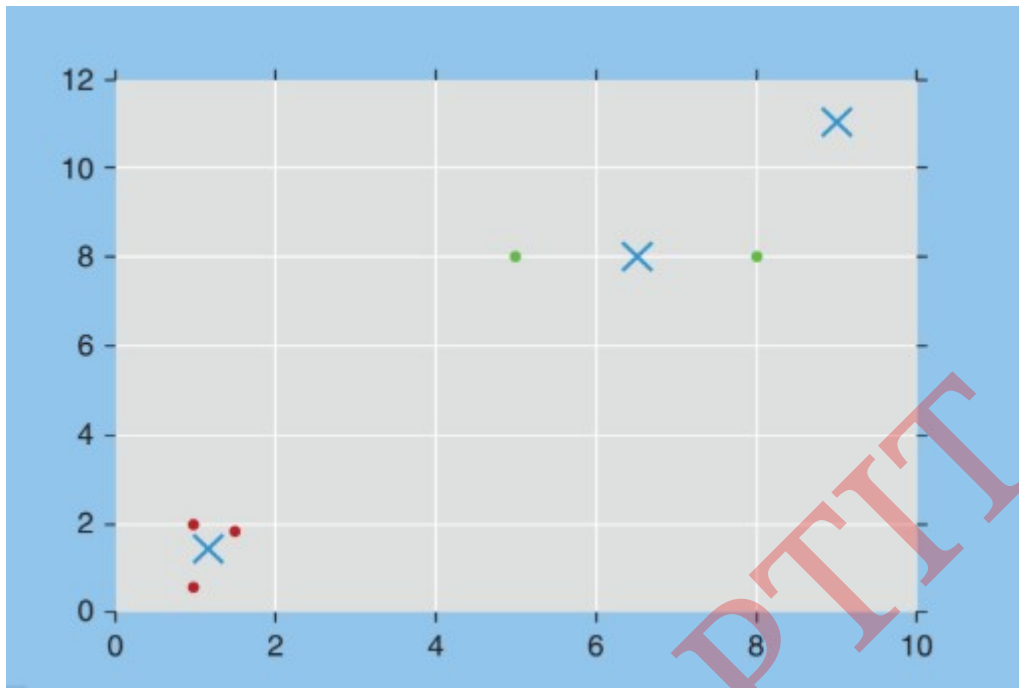
```
plt.show()
```

Hình 19 hiển thị phân cụm với số cụm là 2. Có thể thấy rằng có sáu điểm được vẽ xung quanh hai tâm điểm. Để dễ hình dung, rằng nếu chúng ta tìm kiếm hai cụm, có thể có một nhóm ở góc dưới bên trái (biểu thị bằng màu đỏ) và một nhóm khác ở góc trên bên phải (biểu thị bằng màu xanh lá cây). Ở đây đã sử dụng thuật toán k-means, nhằm mục đích tìm các cụm k trong đó tâm điểm của mỗi cụm là giá trị trung bình của các giá trị trong cụm đó. Các cụm này được biểu diễn bằng ký hiệu "X" màu xanh lam.



Hình 19: Mô hình hóa cụm k-means với số cụm là 2.

Bây giờ hãy xem điều gì xảy ra tách thành 3 cụm. Bằng việc thay đổi đối số `n_clusters` (đầu vào hoặc tham số) trong hàm `KMeans` thành 3. Kết quả trong hình Hình 20 cho thấy thuật toán này có thể cung cấp ba cụm với cùng sáu điểm dữ liệu.



Hình 20: Mô hình hóa cụm k -means với số cụm là 3

Bạn thậm chí có thể thử với số cụm là 4.

Đây là học không giám sát vì **nhãn** của điểm dữ liệu không thể được xếp vào bất kì lớp nào cho trước. Trên thực tế, thậm chí không cần biết có bao nhiêu nhãn hoặc lớp trong dữ liệu. Do đó, có thể áp đặt (hay lựa chọn) gần như bao nhiêu nhãn hoặc lớp (K cụm) tùy thích vào **mô hình**, tùy thuộc vào mục tiêu phân tích cụ thể.

1.5.4.4. Ước lượng Mật độ (Học không có giám sát)

Một cách để hình dung về Phân cụm khi chưa biết nên có bao nhiêu cụm là thiết lập một quy trình tìm kiếm các vùng dữ liệu dày đặc với nhau, và sử dụng thông tin về mật độ đó để tự động tạo thành một cụm. Kỹ thuật này được gọi là MeanShift.

Trước tiên, hãy tìm hiểu thông tin mật độ hoặc hàm là gì. Lấy ví dụ về khả năng tìm thấy một cửa hàng Starbucks trong một khu vực bất kì. Nếu trong trung tâm thương mại hoặc khu mua sắm, thì khả năng có thể tìm thấy một cửa hàng Starbucks (thậm chí hai hoặc ba) và không phổ biến bằng ở khu vực ít dân cư. Nói cách khác, Starbucks có mật độ xuất hiện cao hơn ở các thành phố và khu mua sắm so với các khu vực ít dân cư hoặc ít người ghé thăm. Hàm mật độ là một hàm (nghĩ về một đường cong trên đồ thị) biểu

diễn khả năng tương đối của một biến (ví dụ: sự tồn tại của Starbucks) và nhận một giá trị nhất định.

Quay lại với kỹ thuật MeanShift. Đây là một thuật toán xác định giá trị cực đại của một hàm mật độ dựa trên một tập hợp các điểm dữ liệu cho trước phù hợp với hàm đó. Vì vậy, nói một cách đại khái là nếu có các điểm dữ liệu tương ứng với vị trí của Starbucks, MeanShift cho phép xác định vị trí có khả năng tìm thấy Starbucks; hoặc ngược lại, cho trước một vị trí, xác suất tìm thấy một Starbucks là bao nhiêu.

Bài tập thực hành: Ước lượng mật độ

Để chứng minh, ta cần định nghĩa một hàm mật độ, hàm này sẽ tạo ra một loạt các điểm dữ liệu phù hợp với hàm đó. Sau đó, tiến hành cố gắng xác định trung tâm của các điểm dữ liệu này bằng cách sử dụng kỹ thuật Ước lượng Mật độ. Kỹ thuật này cho phép theo dõi cách phân cụm không giám sát hoạt động khi chúng ta thậm chí không biết cần có bao nhiêu cụm.

```
import numpy as np

from sklearn.cluster import MeanShift

from sklearn.datasets.samples_generator import make_blobs

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

# Nhập lớp style để định dạng biểu đồ

ggplot styling

from matplotlib import style
```

```

style.use("ggplot")

# Tạo các điểm xung quanh 3 trung tâm trên không gian 3 chiều.

# Biến X chứa các điểm đó và có thể bỏ qua biến y

centers = [[1,1,1],[5,5,5],[3,10,10]]

X, y = make_blobs(n_samples = 100, centers = centers, cluster_std = 2)

# Tiến hành phân cụm sử dụng kỹ thuật MeanShift

ms = MeanShift()

ms.fit(X)

# Trích xuất thông tin về các cụm được biểu diễn qua trung tâm cùng với nhãn tương ứng

centroids = ms.cluster_centers

labels = ms.labels

print(centroids)

print(labels)

# Xác định số cụm được tạo ra

```

```

n_clusters_ = len(np.unique(labels))

print("Number of estimated clusters:", n_clusters_)

# Định nghĩa mảng màu

colors = ['r', 'g', 'b', 'c', 'k', 'y', 'm']

# Vẽ biểu đồ 3D

fig = plt.figure()

ax = fig.add_subplot(111, projection='3d')

# Thực hiện lặp lại cho mỗi điểm và vẽ trên biểu đồ 3 chiều sử dụng
# màu đã định nghĩa ở trên, mỗi màu xác định một cụm

for i in range(len(X)):

    print("Coordinate:", X[i], "Label:", labels[i])

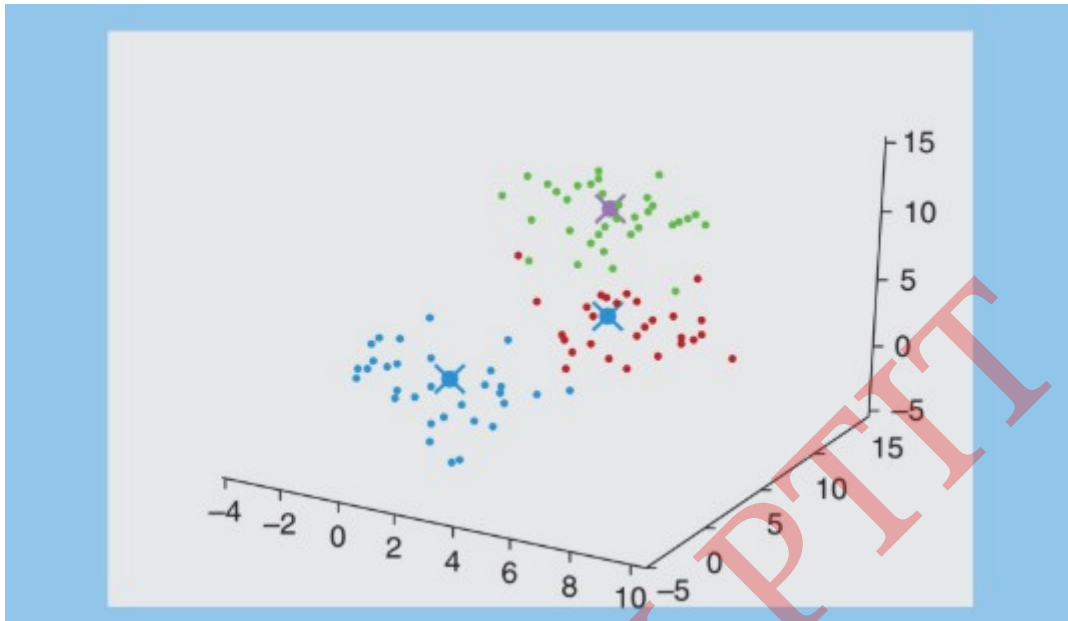
    ax.scatter(X[i][0], X[i][1], X[i][2], c=colors[labels
[i]], marker='o')

ax.scatter(centroids[:,0], centroids[:,1], centroids
[:,2], marker="x", s=150, linewidths=5, zorder=10)

plt.show()

```

Kết quả hiển thị như trên Hình 21



Hình 21: Biểu đồ ước tính mật độ với ba cụm được xác định.

Biểu đồ này hiển thị ba cụm, nhưng hãy thử chạy chương trình nhiều lần và bạn có thể thấy số lượng cụm hiển thị khác nhau.

1.6. Ngôn ngữ R

1.6.1. Giới thiệu chung

Mặc dù một ngôn ngữ lập trình đa năng như Python có thể cung cấp một khung làm việc hiệu quả để xử lý dữ liệu và logic, nhưng các nhà phân tích thường muốn tập trung sâu hơn vào công việc phân tích dữ liệu cốt lõi, thay vì dành thời gian xây dựng các cấu trúc lập trình tổng quát. Nói cách khác, ta có thể sử dụng một môi trường lập trình được thiết kế chỉ để xử lý dữ liệu và không quá quan tâm đến lập trình. Có một số môi trường hoặc gói như vậy – SPSS, Stata và Matlab. Nhưng không gì có thể vượt qua R về mặt một nền tảng phân tích dữ liệu miễn phí, mã nguồn mở và rất mạnh mẽ. Và chỉ vì R miễn phí, đừng nghĩ rằng nó kém hơn. R có thể làm được tất cả – từ các thao tác toán học đơn giản đến trực quan hóa nâng cao. Trên thực tế, R đã trở thành một trong những công cụ được sử dụng nhiều nhất trong khoa học dữ liệu, không chỉ vì giá của nó. Chương này giới thiệu về ngôn ngữ R với cú pháp, một vài ví dụ và cách tích hợp nó với Python. Python và R có lẽ là hai công cụ quan trọng nhất trong khoa học dữ liệu.

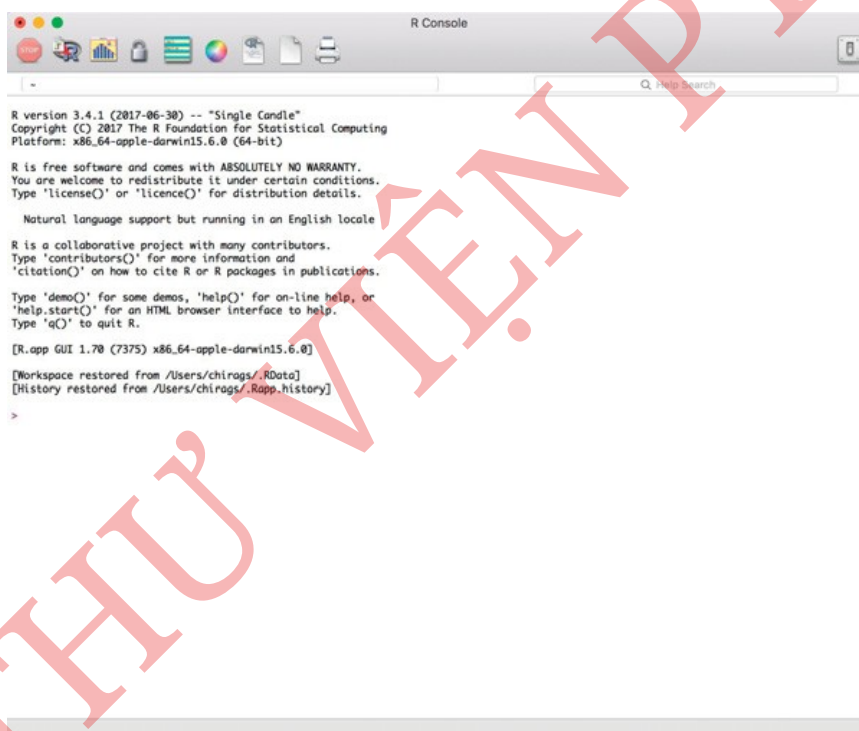
1.6.2. Truy cập vào R

R là một phần mềm mã nguồn mở dành cho tính toán thống kê, được cung cấp miễn phí trên tất cả các nền tảng chính. Nếu bạn từng sử dụng hoặc nghe nói về Matlab, SPSS, SAS, v.v., thì R có thể thực hiện tất cả các tác vụ tương tự: thống kê, thao tác dữ liệu, trực quan hóa, và chạy các thuật toán khai thác dữ liệu (data mining) và học máy (machine learning).

Tuy nhiên, R hoàn toàn miễn phí và đây cũng là công cụ tuyệt vời, luôn được mở rộng liên tục. Ngoài ra, R còn có một cộng đồng tích cực xây dựng và hỗ trợ các công cụ này. R mang đến tất cả sức mạnh của thống kê và đồ họa cần thiết. Do đó, R đang trở thành một tiêu chuẩn công nghiệp cho tính toán khoa học.

Bạn có thể tải xuống R từ trang web của R (<https://www.r-project.org/>). Trang này có tất cả các thứ cần thiết để tìm hiểu thêm về R và các dự án liên quan, và tìm thấy tất cả sự trợ giúp cần thiết để bắt đầu và làm những điều tuyệt vời.

Sau khi tải xuống và cài đặt, hãy khởi động chương trình R. **Bảng điều khiển R (R console)** xuất hiện. Để thoát khỏi chương trình, hãy nhập lệnh `q()`.



```
R Console

R version 3.4.1 (2017-06-30) -- "Single Candle"
Copyright (C) 2017 The R Foundation for Statistical Computing
Platform: x86_64-apple-darwin15.6.0 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

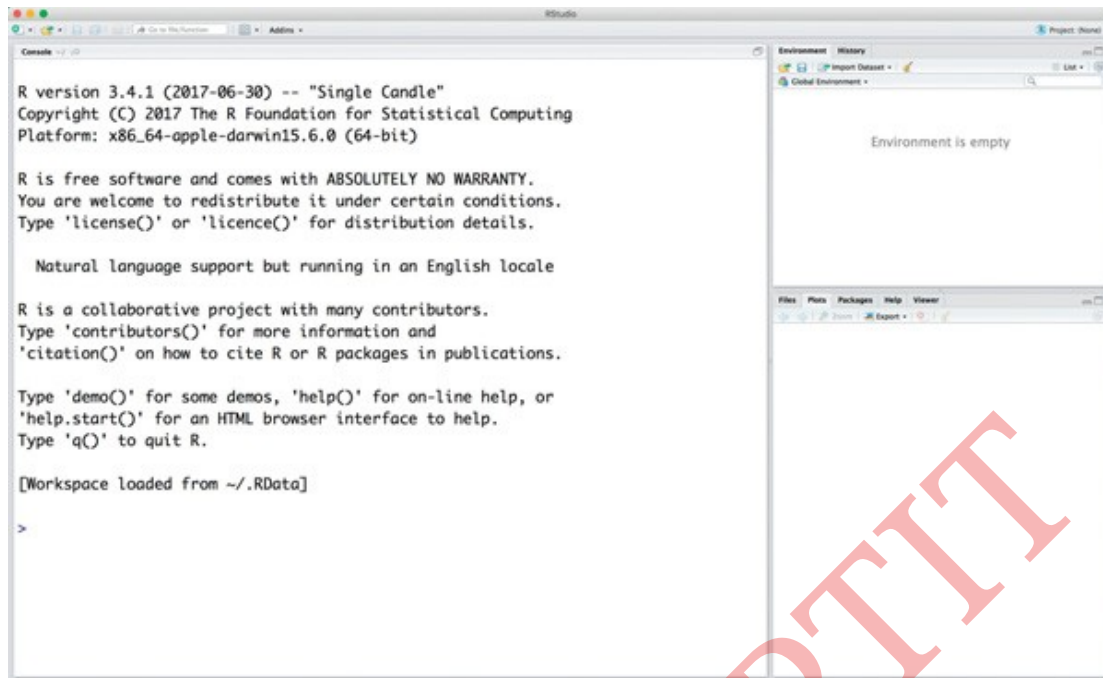
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

[R.app GUI 1.70 (7375) x86_64-apple-darwin15.6.0]
[Workspace restored from /Users/chirags/.RData]
[History restored from /Users/chirags/.Rapp.history]

>
```

Hình 22: R console



Hình 23: R studio

1.6.3. Bắt đầu với R

1.6.3.1. Tính toán cơ bản với R

Hãy bắt đầu với những thao tác tương tự như những gì đã làm trên Python. Mọi thứ thấy bên dưới bắt đầu bằng ký tự `>` là những gì bạn sẽ nhập tại dấu nhắc lệnh (`>`) trong R (hoặc RStudio). Các dòng có số trong ngoặc vuông (ví dụ: `[1]`) hiển thị kết quả đầu ra khi chạy các lệnh đó.

```
> 2+2
[1] 4
> x=2
> y=2
> z=x+y
> z
[1] 4
```

R bắt đầu bằng nhập một biểu thức toán học đơn giản (`2+2`) và sẽ thực thi biểu thức này và trả về câu trả lời. Tiếp theo, các biến `x` và `y` được định nghĩa và gán giá trị. Tương tự như các ngôn ngữ kịch bản khác như Python, khi lập trình với R không cần phải lo lắng về kiểu dữ liệu của một biến. Ngoài ra, cũng như có thể gán một giá trị trực tiếp cho một biến (ví dụ: `x=2`), bạn cũng có thể gán một biểu thức toán học cho một biến (như

trên, $z=x+y$). Trong trường hợp này, R sẽ tính toán biểu thức đó và lưu trữ kết quả vào biến. Nếu muốn xem giá trị trong một biến, chỉ cần nhập tên biến đó tại dấu nhắc lệnh.

Lưu ý về Gán Giá trị: Bạn cũng có thể gán giá trị cho một biến bằng cách sử dụng toán tử gán `<-` (mũi tên hướng về bên trái) như sau:

```
R
a <- 7
```

Trong tài liệu này sử dụng cả 2 dấu “=” và dấu “<-” để gán giá trị và viết biểu thức.

Tiếp theo, hãy làm việc với **các phép toán logic**. Các toán tử logic phổ biến nhất rất quen thuộc bao gồm các toán tử lớn hơn (>), nhỏ hơn (<), lớn hơn hoặc bằng (>=), và nhỏ hơn hoặc bằng (<=). Ngoài ra còn các toán tử khác như so sánh bằng (==) và so sánh không bằng (!=)

Hãy thực hành một số phép toán sau đây:

```
> 2>3
[1] FALSE
> 2==2
[1] TRUE
```

Có thể thấy, kết quả của một phép toán logic là một **giá trị Boolean** — **TRUE** (Đúng) hoặc **FALSE** (Sai).

Bây giờ, hãy viết một chương trình nhỏ ngay trên R

```
> year = 2020
> if (year%%4 == 0)
+ print ("Leap year")
[1] "Leap year"
```

Trong ví dụ này, giá trị 2020 được gán cho biến year. Sau đó, kiểm tra xem biến này có chia hết cho 4 hay không bằng cách sử dụng toán tử chia dư (%% - modulo). Nếu điều kiện là đúng (kết quả của phép chia lấy dư bằng 0), chương trình sẽ thông báo đó là năm nhuận. Vấn đề này đã được giải quyết trước đó (trong Python), nhưng bây giờ sẽ được giải quyết bằng R

Bây giờ, hãy đặt đoạn mã này vào một tập tin riêng.

1. Trong **RStudio**, chọn "**File > New File > R Script.**" Thao tác này sẽ mở một cửa sổ soạn thảo (editor) để nhập mã.

2. Nhập đoạn mã sau vào cửa sổ soạn thảo:

```
year = 2020
if (year%%4 == 0) {
  print ("Leap year")
} else {
  print ("Not a leap year")
}
```

3. **Lưu tập tin.** Các tập tin R script thường có phần mở rộng là **.r**.

Cách Chạy mã nguồn

- **Chạy từng dòng:** Đặt con trỏ vào dòng muốn chạy và nhấn nút "**Run**" trên thanh công cụ của trình soạn thảo.
- **Chạy toàn bộ script:** Chọn tất cả mã (Ctrl+A trên PC hoặc Cmd+A trên Mac) và nhấn vào "**Run**".

Kết quả sẽ được hiển thị trên bảng điều khiển. Từ đây, bạn có thể bắt đầu tận dụng các mã R có sẵn và thậm chí tự tạo ra các mã nguồn của riêng mình.

1.6.3.2. Cấu trúc điều khiển

Tương tự như Python, R hỗ trợ một số **cấu trúc điều khiển cơ bản**. Nhìn chung, các cấu trúc điều khiển được chia thành hai loại: **cấu trúc điều khiển quyết định** và **cấu trúc điều khiển lặp**.

- **Cấu trúc điều khiển quyết định (Decision control structure):** Đúng như tên gọi, nếu cần **quyết định** xem một câu lệnh hoặc một tập hợp các câu lệnh có được thực thi hay không, dựa trên một điều kiện nào đó, cấu trúc này sẽ được sử dụng (ví dụ: khối lệnh **if-else**).
- **Cấu trúc điều khiển lặp (Loop control structure):** Nếu cần một tập hợp các câu lệnh được thực thi **lặp đi lặp lại** miễn là điều kiện quyết định vẫn còn đúng, cấu trúc lặp sẽ được sử dụng (ví dụ: vòng lặp **for**, vòng lặp **do-while**, v.v.).

Hãy xem xét một vài ví dụ. Giả sử bạn muốn quyết định xem thời tiết có **phù hợp cho một chuyến đi xe đạp** hay không dựa trên phần trăm độ ẩm hiện tại, và bạn muốn viết mã để kiểm tra điều này. Đoạn mã sẽ được viết như sau:

```

humidity = 45
if (humidity < 40) {
  print (“Độ ẩm phù hợp với chuyến đi”)
} else if (humidity > 70) {
  print (“Độ ẩm không phù hợp với chuyến đi”)
} else {
  print (“Độ ẩm có thể phù hợp với chuyến đi”)
}

```

Như được thể hiện trong các mã nguồn trên, ba điều kiện dựa trên đó quyết định được đưa ra được xác định là: "**độ ẩm dưới 40%**", hoặc "**trên 70%**", hoặc **các trường hợp còn lại**.

Bây giờ sẽ làm việc với một ví dụ khác để thực hành **cấu trúc điều khiển lặp**. Điều gì sẽ xảy ra nếu bạn có các dự đoán chính xác về độ ẩm trong bảy ngày tới? Sẽ rất tốt nếu bạn có thể đưa ra quyết định cho cả bảy ngày đó. Dưới đây là cách thực hiện:

Cách 1. Sử dụng Vòng lặp while

Vòng lặp while sẽ lặp lại việc kiểm tra điều kiện độ ẩm cho từng ngày.

```

# Khai báo biến dùng để lưu trữ phần trăm độ ẩm trong một véc-tơ (mảng)
humidity <- c(20, 30, 60, 70, 65, 40, 35)
count <- 1
while (count <= 7) {
  cat (“Weather for day ”, count, “:”)
  if (humidity[count] < 40) {
    print (“Perfect for a trip”)
  } else if (humidity[count] > 70) {
    print (“Not suitable for a trip!”)
  } else {
    print (“May or may not be suitable for trip”)
  }
  count = count + 1 # Tăng biến đếm lên 1 sau mỗi lần lặp
}

```

Cách 2. Sử dụng Vòng lặp for

Cùng một mục tiêu có thể đạt được bằng cách sử dụng **vòng lặp for**. Dưới đây là cách minh họa:

```
# Dùng để lưu trữ phần trăm độ ẩm trong một vector (mảng)
humidity <- c(20, 30, 60, 70, 65, 40, 35)
for (count in 1:7){
  cat ("Weather for day ", count, ":")
  if (humidity[count] < 40) {
    print ("Perfect for a trip")
  } else if (humidity[count] > 70) {
    print ("Not suitable for a trip!")
  } else {
    print ("May or may not be suitable for trip")
  }
}
```

Hãy dừng lại một chút và đảm bảo rằng bạn hiểu ý nghĩa của toàn bộ mã nguồn trên. Tất nhiên, cách tốt nhất để đảm bảo điều này là thực sự tự mình thử và thực hiện một số thay đổi để xem logic có hoạt động đúng hay không.

1.6.3.3. Hàm

Như đã biết, các hàm cho phép lưu trữ một quy trình, một khối tính toán hoặc một khối logic để có thể tái sử dụng.

Hãy bắt đầu bằng cách tạo một hàm có thể tái sử dụng. Dưới đây là một hàm có tên Square (Bình phương). Hàm này nhận vào một giá trị và trả về bình phương của giá trị đó:

```
Square = function(x) {
  return (x * x)
}
```

Hãy viết đoạn mã này vào một tập tin và lưu lại. Bây giờ, chạy toàn bộ mã nguồn. Sẽ không có gì xảy ra trên bảng điều khiển.

Tuy nhiên, khi bạn quay lại bảng điều khiển (console) và chạy một lệnh như

Square(4), kết quả hiển thị là 16.

Điều gì đang xảy ra? Một hàm được tạo tên là Square và đã sẵn sàng để được sử dụng bất cứ lúc nào cần đến chức năng này. Trên thực tế, bạn có thể thấy trong khung Môi trường (Environment) của RStudio rằng hàm cụ thể này hiện đã khả dụng để sử dụng.

1.6.3.4. Nhập dữ liệu

Bây giờ đến phần hữu ích nhất của R dành cho khoa học dữ liệu. Hầu như không thể giải quyết bất kỳ bài toán thực tế nào nếu không có dữ liệu sẵn sàng trong R. Hãy cùng xem cách nhập dữ liệu vào R. Hiện tại, dữ liệu định dạng CSV vẫn là dữ liệu được thực hiện chủ yếu, tương tự như trong Python.

R có hàm **file.choose()** cho phép chọn một tệp từ máy tính và hàm **read.table()** để đọc tệp đó dưới dạng bảng. Hai hàm này hoạt động rất tốt với dữ liệu định dạng CSV.

Trong ví dụ này, ta sẽ sử dụng tệp dữ liệu IQ (iqsize.csv). File này được tải tại <https://www2.math.binghamton.edu/lib/exe/fetch.php/people/kargin/math448/iqsize.csv>. Nhập đoạn mã sau từng dòng một, hoặc lưu nó dưới dạng R script và chạy:

```
df = read.table(file.choose(), header = TRUE, sep = ",")
brain = df["Brain"]
print(summary(brain))
```

Chạy dòng lệnh đầu tiên sẽ hiển thị một hộp thoại chọn tệp. Điều hướng đến thư mục chứa tệp **iqsize.csv** và chọn nó. Đây chính là kết quả của hàm **file.choose()**, được dùng làm đối số (tham số, đầu vào) đầu tiên trong hàm **read.table()**.

- **Tùy chọn:** Bạn cũng có thể đặt tên tệp (cùng với đường dẫn đầy đủ) trong dấu ngoặc kép thay cho **file.choose()** cho đối số đầu tiên.
- **header=TRUE:** Đối số thứ hai có nghĩa là ta đang mong đợi thấy **nhân cột** ở hàng đầu tiên (header).
- **sep=",":** Đối số thứ ba chỉ ra rằng các cột được **phân tách bằng dấu phẩy** (comma-separated).

Sau khi dữ liệu được tải vào biến **dataframe** (df), ta có thể xử lý tiếp như sau.

- Ở dòng thứ hai, ta đang **chọn dữ liệu** được lưu trữ trong cột **Brain**.
- Và sau đó, ở dòng thứ ba, ta sử dụng hàm **summary()** để có được một số **đặc điểm thống kê cơ bản** của cột dữ liệu này và in chúng ra, như thấy bên dưới:

```
Brain
Min. : 79.06
```

1st Qu.: 85.48
Median : 90.54
Mean : 90.68
3rd Qu.: 94.95
Max. :107.95

Những đại lượng này có quen thuộc không? Chắc chắn rồi, nếu bạn đã nắm vững kiến thức thống kê cơ bản hoặc đã xem lại phần **thống kê mô tả** được đề cập ở các chương trước của cuốn sách này!

1.6.4. Đồ họa và trực quan hóa dữ liệu

Một trong những lợi ích quan trọng nhất của R chính là khả năng trực quan hóa dữ liệu chỉ với rất ít thao tác. Điều này có được nhờ sự hỗ trợ tích hợp sẵn của ngôn ngữ, cùng với kho thư viện, gói và hàm phong phú do cộng đồng các nhà phát triển trên toàn thế giới đóng góp. Phần này sẽ làm rõ ứng dụng ngôn ngữ R trong trực quan hóa dữ liệu.

1.6.4.1. Cài đặt ggplot2

Trước khi bắt đầu làm việc với đồ họa và biểu đồ, cần đảm bảo rằng các thư viện cần thiết đã được cài đặt sẵn. Trong RStudio, chọn **Công cụ > Cài đặt Gói**. Trong hộp thoại hiện ra, hãy kiểm tra để chắc chắn rằng kho lưu trữ CRAN được chọn làm nguồn cài đặt. Tiếp theo, nhập ggplot2 vào ô tìm kiếm gói, đồng thời đảm bảo tùy chọn Cài đặt phụ thuộc được tích chọn. Cuối cùng, nhấn Cài đặt. Gói ggplot2 sẽ được tự động tải xuống và cài đặt.

Về cơ bản, khung dữ liệu trong R cũng có cấu trúc tương tự, tức là một dạng mảng hoặc ma trận gồm các hàng và cột. Tuy nhiên, giữa hai ngôn ngữ vẫn tồn tại điểm khác biệt quan trọng như:

- Trong ngôn ngữ R, khung dữ liệu là một cấu trúc dữ liệu có sẵn, không cần cài đặt thêm bất kỳ gói bên ngoài nào, khác với Python khi chúng ta phải sử dụng thư viện Pandas. Điểm khác biệt quan trọng thứ hai nằm ở cách R xử lý các phần tử trong khung dữ liệu. Để minh họa, chúng ta sẽ xem xét một ví dụ cụ thể dưới đây.

Ví dụ này sẽ sử dụng một trong những khung dữ liệu tích hợp có tên là "mtcars". Khung dữ liệu này có danh sách các mẫu xe theo hàng và các thuộc tính về xe dưới dạng cột.

- Nếu muốn tìm mpg của một mẫu xe Fiat 128
> mtcars['Fiat 128','mpg']
- Nếu bạn muốn có toàn bộ hồ sơ về chiếc xe đó, bạn có thể nhập
> mtcars['Fiat 128',]

Chúng ta đang tham chiếu đến một hàng cụ thể cùng với tất cả các cột tương ứng thông qua cách định địa chỉ như trên. Tất nhiên, cũng có thể truy cập trực tiếp một phần tử trong khung dữ liệu bằng cách sử dụng chỉ mục, chẳng hạn như `mtcars[12, 1]`. Tuy nhiên, việc định địa chỉ hàng, cột hoặc một phần tử cụ thể bằng tên sẽ giúp câu lệnh trở nên dễ hiểu và dễ đọc hơn đáng kể.

1.6.4.2. Tải dữ liệu

Trong các ví dụ của phần này, chúng ta sẽ sử dụng dữ liệu khách hàng liên quan đến bảo hiểm y tế. Bộ dữ liệu này được lưu trong tệp `custdata.tsv`. Xem chi tiết hơn tại <https://github.com/WinVector/zmPDSwR/blob/master/Custdata/custdata.tsv>. Phần mở rộng `tsv` là viết tắt của *tab-separated values*, nghĩa là các trường dữ liệu được phân tách bằng ký tự tab thay vì dấu phẩy. Vì vậy, lệnh để tải dữ liệu sẽ được viết như sau:

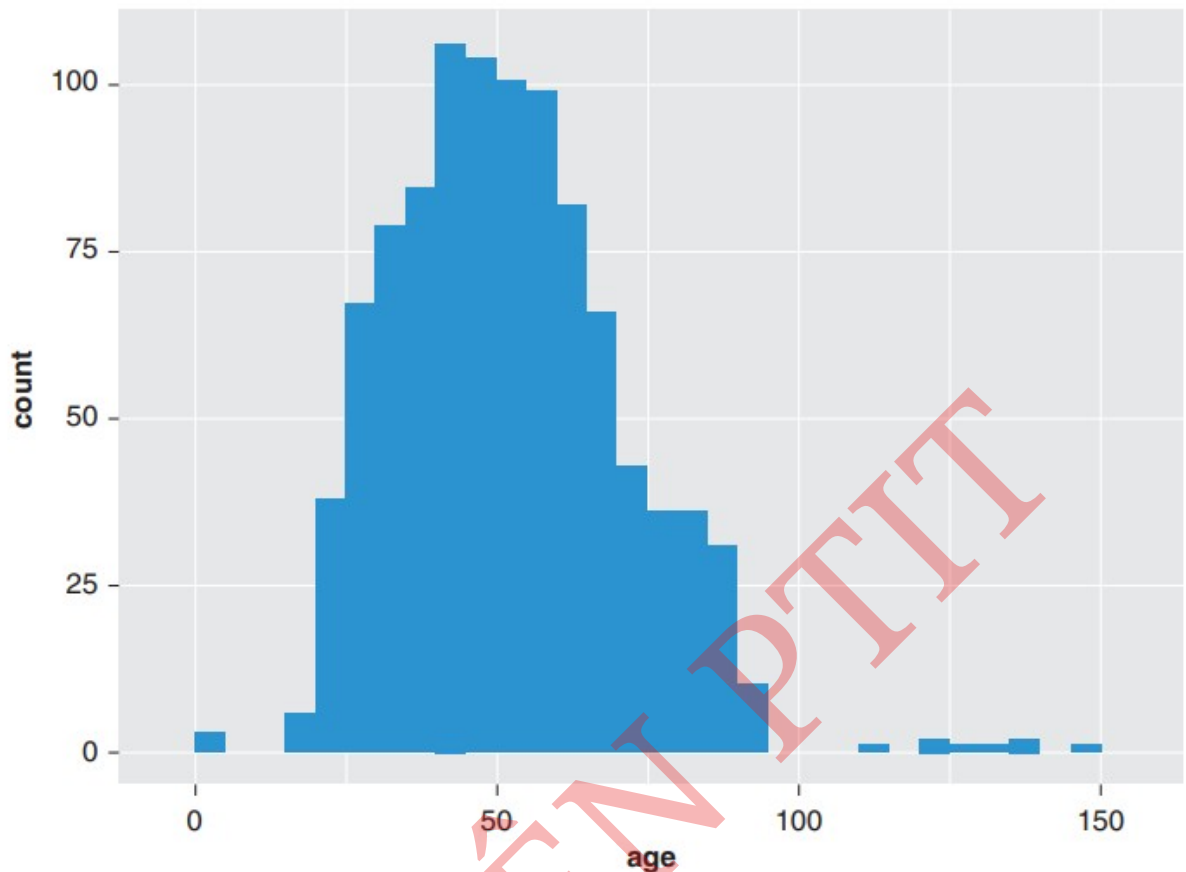
```
custdata = read.table('custdata.tsv', header=T, sep='\t')
```

Ký hiệu “\t” đại diện cho ký tự tab và lệnh trên giả định rằng tệp `custdata.tsv` đang được lưu trong thư mục làm việc hiện tại. Trong trường hợp không chắc chắn về điều đó, có thể thay thế tên tệp bằng hàm `file.choose()`. Khi kết hợp với hàm `read.table()`, một hộp thoại sẽ xuất hiện, cho phép người dùng lựa chọn trực tiếp tệp dữ liệu từ máy tính. Câu lệnh khi đó sẽ có dạng như sau:

```
custdata = read.table(file.choose(), header=T, sep='\t')
```

1.6.4.3. Vẽ biểu đồ dữ liệu

Vẫn tiếp tục sử dụng bộ dữ liệu ở trên, chúng ta sẽ bắt đầu với một biểu đồ histogram đơn giản về độ tuổi của khách hàng. Trước tiên, chúng ta cần tải thư viện "ggplot2" và sử dụng hàm `histogram` như sau:



Hình 24: Biểu đồ thể hiện sự phân bố số lượng khách hàng có độ tuổi nhất định

```
library(ggplot2)
ggplot(custdata) + geom_histogram(aes(x=age), binwidth=5, fill="blue")
```

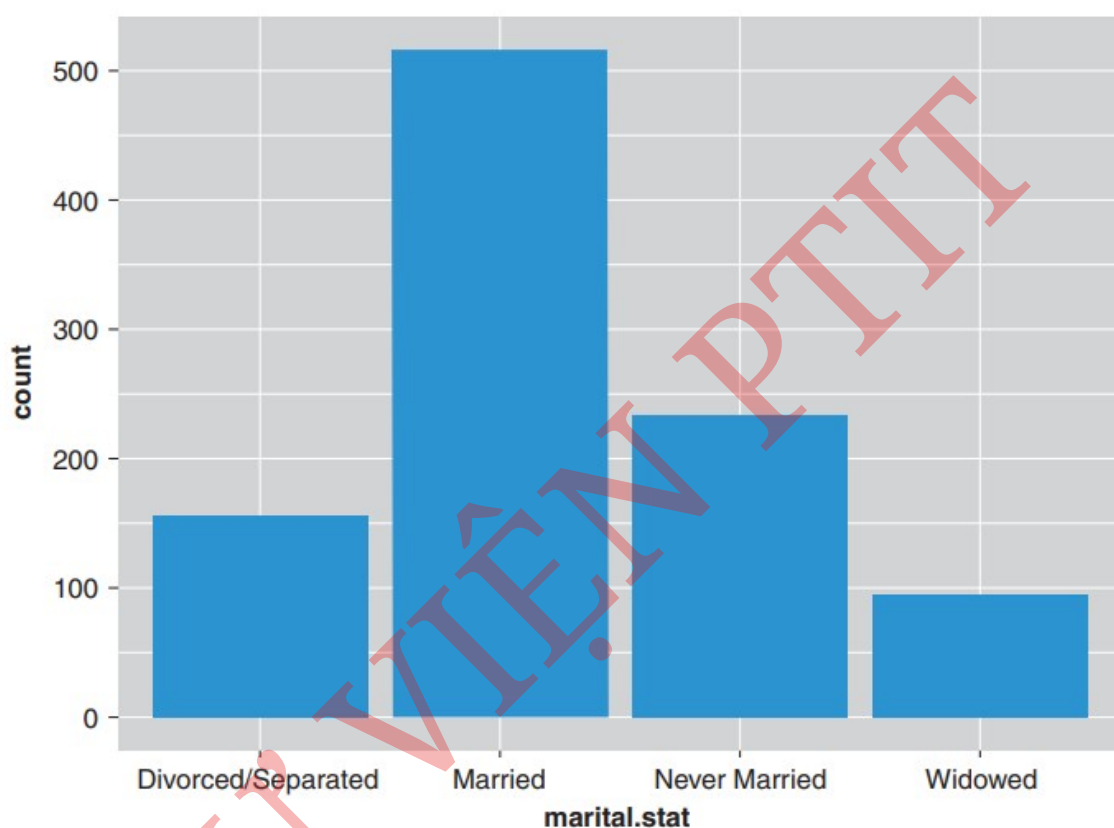
Câu lệnh trên sẽ tạo ra một biểu đồ histogram như minh họa trong Hình 24. Trong đoạn mã, tham số *binwidth* xác định độ rộng của khoảng giá trị mà mỗi cột biểu đồ bao phủ trên trục hoành. Trong ví dụ này, giá trị *binwidth* được đặt bằng 5, nghĩa là các khoảng được chia thành 0–5, 6–10, v.v. Mỗi cột trong biểu đồ thể hiện số lượng đối tượng hoặc thành viên nằm trong khoảng giá trị tương ứng. Có thể thấy rằng, nếu tăng độ rộng của khoảng (*binwidth*), mỗi cột sẽ bao gồm nhiều đối tượng hơn và biểu đồ tổng thể sẽ trở nên "phẳng" hơn. Ngược lại, nếu giảm độ rộng khoảng, số đối tượng trong mỗi cột sẽ ít hơn và biểu đồ sẽ hiển thị nhiều biến động hơn.

Như vậy, chỉ với một dòng lệnh đã có thể tạo ra một biểu đồ như trên trong khi ở các phần mềm bảng tính, việc tạo biểu đồ thường được thực hiện thông qua thao tác trỏ và nhấp chuột, thì cách tiếp cận bằng lệnh trong R không hề phức tạp. Hơn nữa, với phương pháp này, việc kiểm soát các đặc điểm hiển thị của biểu đồ trở nên linh hoạt và kết quả

thường mang tính chuyên nghiệp hơn so với các biểu đồ được tạo từ chương trình bảng tính.

Trong ví dụ vừa trình bày, hàm tạo biểu đồ histogram có tham số xác định biến được hiển thị trên trục hoành và số lượng điểm dữ liệu được gộp vào mỗi khoảng (*bin*).

Tiếp theo, xét đến một trường dữ liệu có giá trị phân loại. Biến *marital.stat* được sử dụng để minh họa. Dữ liệu này có thể được trực quan hóa bằng biểu đồ cột (Hình 25).



Hình 25: Biểu đồ thanh thể hiện sự phân bố tình trạng hôn nhân trong dữ liệu khách hàng.

```
ggplot(custdata) + geom_bar(aes(x=marital.stat), fill="blue")
```

Biểu đồ Histogram không phải là loại biểu đồ duy nhất có sẵn trong R. Ngôn ngữ này hỗ trợ nhiều loại biểu đồ khác nhau. Chẳng hạn, có thể sử dụng biểu đồ hình tròn để biểu diễn sự phân bố của các loại nhà ở. Tuy nhiên, trong tài liệu hướng dẫn chính thức của R, biểu đồ hình tròn thường không được khuyến nghị và các tính năng hỗ trợ sẵn cũng còn hạn chế.

Để xây dựng biểu đồ hình tròn, trước tiên cần tạo một bảng tần suất (contingency table) nhằm tổng hợp số lượng quan sát ở từng cấp độ.

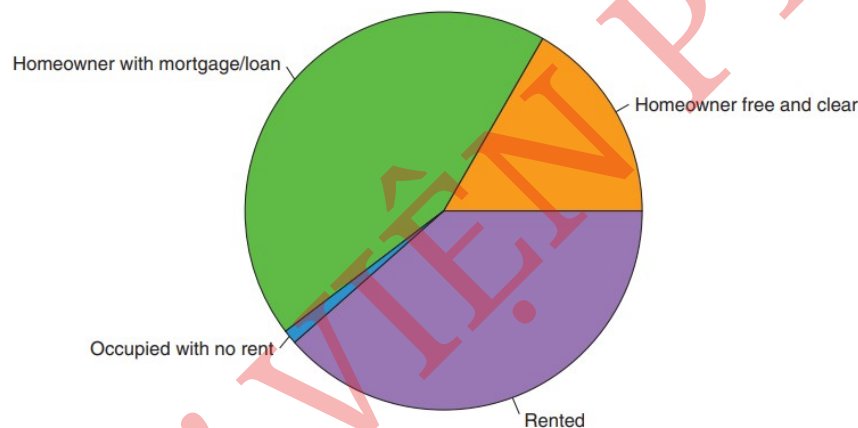
```
contingencyTable <- table(custdata$housing.type)
```

Bây giờ, có thể sử dụng hàm `pie()` trong R để vẽ biểu đồ hình tròn về các loại nhà như Hình 26

```
pie(contingencyTable, main="Pie Chart of Housing Types")
```

Nếu bảng màu mặc định của hàm `pie()` trong R không phù hợp, có thể lựa chọn các biến thể màu khác từ những tùy chọn có sẵn. Khi thực hiện, cần lưu ý rằng số lượng màu được chỉ định phải tương ứng với số lượng lát cắt trong biểu đồ hình tròn. Cách thực hiện được minh họa như sau.

```
pie(contingencyTable, main="Pie Chart of Housing Types", col =  
rainbow(length(contingencyTable)))
```



Hình 26: Biểu đồ hình tròn về các loại nhà ở.

Ví dụ thực hành: Vẽ đồ thị

Trong hai hình minh họa cuối, một đặc điểm dễ nhận thấy là tính rời rạc của các điểm dữ liệu. Tuy nhiên, trong trường hợp các điểm dữ liệu thuộc cùng một chuỗi và cần duy trì tính tuần tự khi trực quan hóa, một giải pháp thích hợp là sử dụng biểu đồ đường.

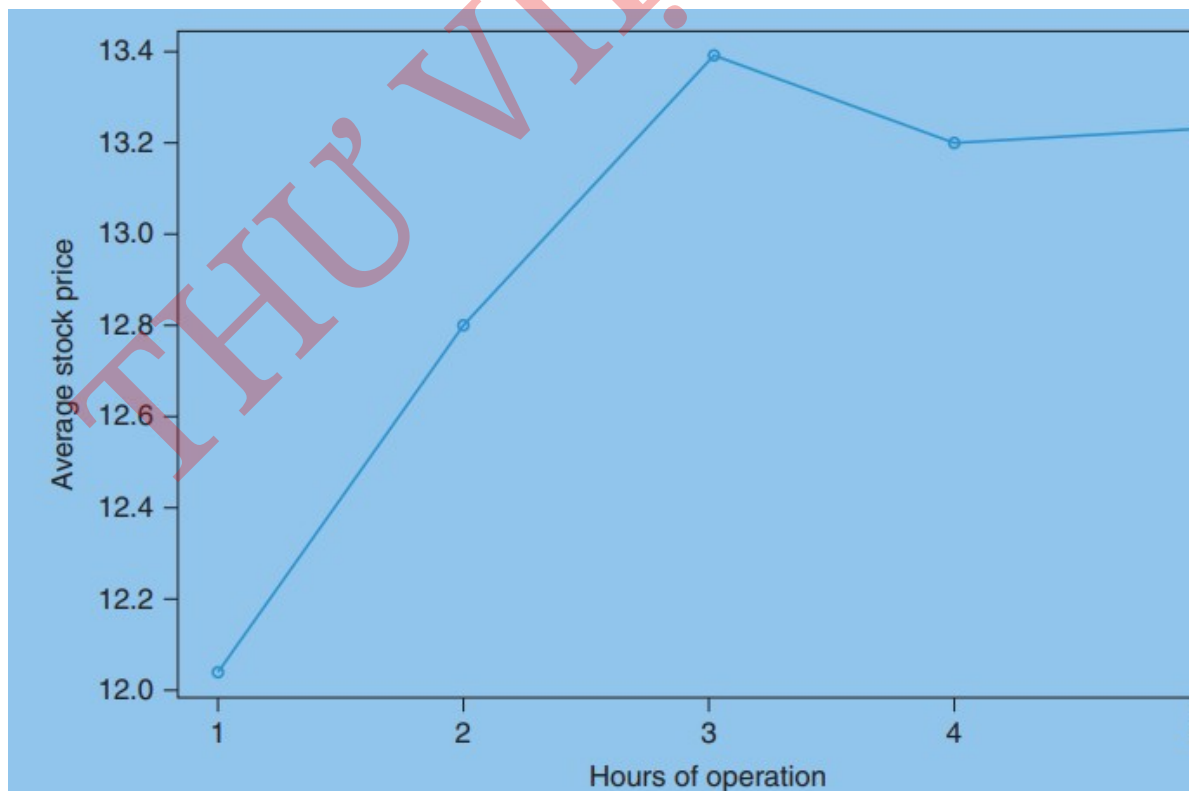
Hình 27 trình bày tập dữ liệu về giá cổ phiếu trung bình của một công ty X trong năm giờ giao dịch đầu tiên trên sàn chứng khoán. Để theo dõi diễn biến giá cổ phiếu của công ty, biểu đồ đường là công cụ trực quan hóa phù hợp. Đoạn mã dưới đây minh họa cách thực hiện.

Hour of operation	Average stock price in USD
1	12.04
2	12.80
3	13.39
4	13.20
5	13.23

Hình 27: Giá cổ phiếu trung bình (tính bằng USD) của công ty X trong năm giờ giao dịch đầu tiên.

```
stock <- read.csv('stocks.csv', header = TRUE, sep = ",")
plot(stock$Average.stock.price.in.USD., type = "o", col = "red", xlab = "Hours of operation", ylab = "Average stock price")
```

Các dòng trên sẽ tạo ra biểu đồ đường trong Hình 28.



Hình 28: Biểu đồ đường biểu diễn giá cổ phiếu trung bình theo giờ hoạt động

R cung cấp nhiều loại biểu đồ khác nhau cũng như các biến thể của từng loại biểu đồ. Tuy nhiên, trong phạm vi này chỉ xem xét bốn loại biểu đồ cơ bản, trước khi chuyển sang một số kỹ thuật khác có thể áp dụng để trực quan hóa dữ liệu hiện có.

Bước tiếp theo là tải một tập dữ liệu về độ tuổi và thu nhập của khách hàng:

```
custdata <- read.csv('custdata.tsv', header = TRUE, sep = "\t")
```

Tiếp theo, cần xác định mối tương quan giữa độ tuổi và thu nhập nhằm đánh giá mức độ cũng như cách thức liên hệ giữa hai biến này. Tác vụ này có thể thực hiện bằng một lệnh đơn giản như sau:

```
cor(custdata$age, custdata$income)
```

Kết quả cho thấy hệ số tương quan ở mức thấp, chỉ đạt 0,027. Điều này có nghĩa là hai biến này không có mối liên hệ đáng kể nào về mặt thống kê. Qua quá trình kiểm tra dữ liệu chi tiết, nhận thấy rằng tồn tại một số giá trị null, tức là có những trường hợp độ tuổi và thu nhập được ghi nhận bằng 0. Đây là kết quả không hợp lý và nhiều khả năng phản ánh các giá trị bị thiếu. Do đó, cần tính toán lại hệ số tương quan, lần này chỉ xem xét những giá trị khác không. Cách tạo tập dữ liệu con thỏa mãn điều kiện này được minh họa như sau:

```
custdata2 <- subset(custdata, (custdata$age > 0 & custdata$age < 100 &  
custdata$income > 0))
```

Hàm subset() được sử dụng để trích xuất mẫu dữ liệu dựa trên các điều kiện đã chỉ định. Trong trường hợp này, điều kiện được áp dụng là giá trị biến *tuổi* lớn hơn 0 và nhỏ hơn 100. Sau khi tạo tập dữ liệu con thỏa mãn điều kiện, phép tính hệ số tương quan được thực hiện lại để đảm bảo kết quả phản ánh chính xác hơn mối quan hệ giữa các biến.

```
cor(custdata2$age, custdata2$income)
```

Lần này, kết quả thu được là -0,022 và con số này vẫn ở mức rất thấp, nhưng có thể nhận thấy dấu hiệu đã thay đổi so với trước. Khi tiến hành các hình thức phân tích dữ liệu khác (dù với R hay bất kỳ công cụ nào), cần lưu ý đến bản chất của dữ liệu. Dữ liệu không phải lúc nào cũng ở trạng thái sạch hoặc chính xác, và nếu không xử lý cẩn thận, kết quả phân tích có thể thiếu hợp lý, thậm chí dẫn đến sai lệch hoàn toàn.

1.6.5. Thống kê và học máy

Trong mục 2.1, các khái niệm thống kê đã được xem xét, đồng thời ở chương trước cũng đã minh họa cách áp dụng cùng một số khái niệm khác bằng Python. Phần này sẽ trình bày cách sử dụng R để thực hiện các thao tác tương tự. Trước khi bắt đầu, cần đảm bảo rằng các khái niệm thống kê cơ bản đã được ôn tập và nội dung tiếp theo cũng sẽ đề cập đến một số kỹ thuật học máy cơ bản trong R có thể hỗ trợ giải quyết các bài toán dữ liệu. Về học máy, cần tham khảo lại phần giới thiệu đã trình bày trong chương trước.

1.6.5.1. Thống kê cơ bản

Phần này sẽ bắt đầu với một số thống kê mô tả. Tập dữ liệu *size.csv* được sử dụng, bao gồm 38 bản ghi về chiều cao và cân nặng của các cá nhân khác nhau. Việc tải dữ liệu được thực hiện như sau:

```
size = read.table('size.csv',header=T,sep=',')
```

Một lần nữa, tiếp tục giả định rằng tệp dữ liệu đã nằm trong thư mục hiện tại. Ngoài ra, có thể thay thế '*size.csv*' bằng *file.choose()* để lựa chọn tệp từ ổ đĩa khi chạy lệnh. Bên cạnh đó, mặc dù có thể chạy từng dòng lệnh trực tiếp trên bảng điều khiển, việc nhập các dòng và lưu dưới dạng tệp *.r* sẽ không chỉ chạy được từng dòng mà còn lưu trữ được tập lệnh cho các lần thực thi về sau.

Sau khi dữ liệu được tải thành công, R có thể cung cấp một số thống kê cơ bản về tập dữ liệu bằng lệnh *summary* như sau:

```
summary(size)
```

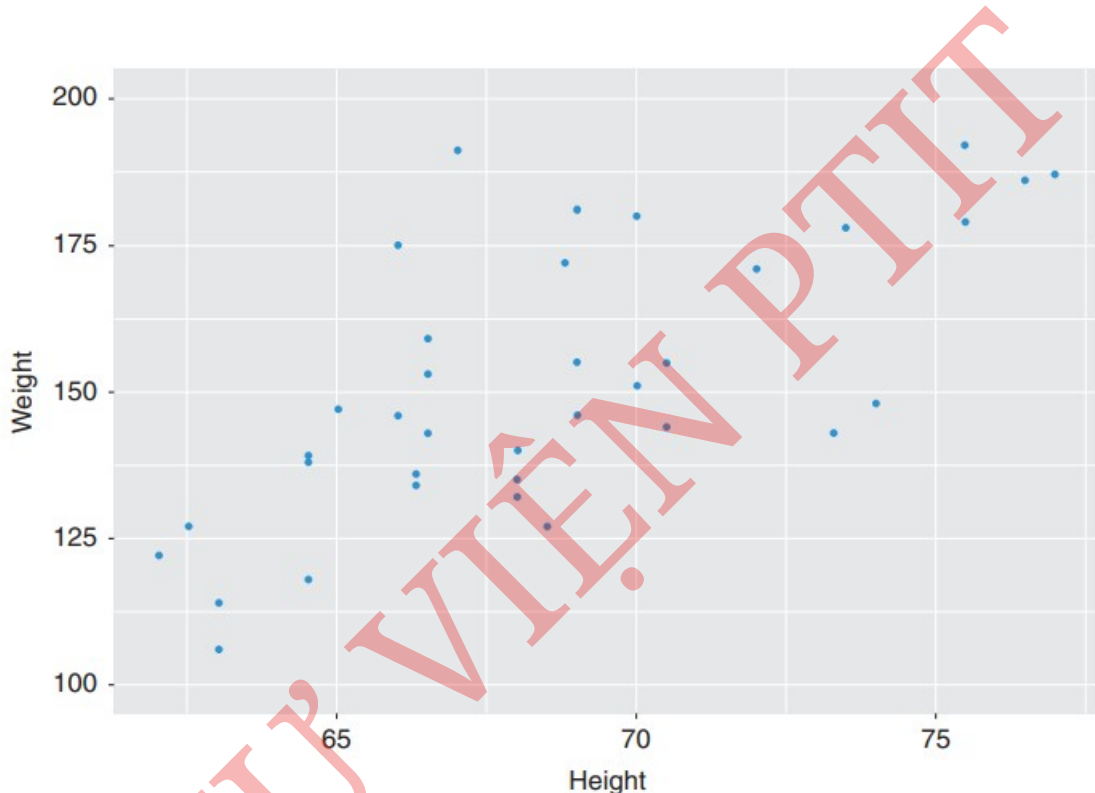
Height	Weight
Min. :62.00	Min. :106.0
1st Qu.:66.00	1st Qu.:135.2
Median :68.00	Median :146.5
Mean :68.42	Mean :151.1
3rd Qu.:70.38	3rd Qu.:172.0
Max. :77.00	Max. :192.0

Kết quả đầu ra ở trên đã cung cấp các số liệu thống kê mô tả cho hai biến trong tập dữ liệu: *Chiều cao* (Height) và *Cân nặng* (Weight). Tiếp theo, có thể trực quan hóa dữ

liệu bằng biểu đồ phân tán. Trong dòng lệnh minh họa, tham số *ylim* được sử dụng để chỉ định giá trị tối thiểu và tối đa cho trục tung (trục y).

```
library(ggplot2)
ggplot(size, aes(x=Height,y=Weight)) + geom_point() + ylim(100,200)
```

Kết quả được hiển thị trong Hình 2.16 dưới đây và có thể thấy rằng R hỗ trợ việc tạo ra những hình ảnh trực quan có tính chuyên nghiệp một cách đơn giản và nhanh chóng.



Hình 29: Biểu đồ phân tán của Chiều cao so với Cân nặng

1.6.5.2. Hồi quy

Với biểu đồ phân tán đã có, có thể bắt đầu đặt ra một số câu hỏi phân tích. Một câu hỏi cơ bản là: mối quan hệ giữa hai biến được thể hiện trong biểu đồ là gì? Trong R, vẫn có thể giữ nguyên thông tin từ biểu đồ hiện tại và chỉ cần bổ sung thêm một hàm để xác định và hiển thị đường biểu diễn mối quan hệ với câu lệnh như sau:

```
ggplot(size, aes(x=Height,y=Weight)) + geom_point()+ stat_smooth(method="lm")
+ ylim(100,200)
```

So sánh lệnh này với lệnh đã sử dụng trước đó để tạo biểu đồ phân tán, có thể thấy rằng toàn bộ biểu đồ ban đầu được giữ nguyên, chỉ bổ sung thêm một đường thẳng chồng lên biểu đồ. Đây chính là cách đơn giản để thực hiện hồi quy tuyến tính cơ bản trong R, một phương pháp thuộc nhóm học có giám sát. Trong đó, hàm *lm* đại diện cho mô hình tuyến tính và kết quả hiển thị trong Hình 30 cho thấy đường màu xanh lam chính là đường hồi quy. Đường này thể hiện mối quan hệ giữa hai biến *Chiều cao* và *Cân nặng*. Điều này có nghĩa là, khi biết giá trị của *Chiều cao*, có thể ước lượng giá trị *Cân nặng* tương ứng tại bất kỳ điểm nào trên đường thẳng.

Để xem phương trình của đường thẳng hồi quy, có thể sử dụng hàm *lm* để trích xuất các hệ số với câu lệnh dưới đây:

```
lm(Weight ~ Height, size)
```

Và đây là kết quả.

Call:

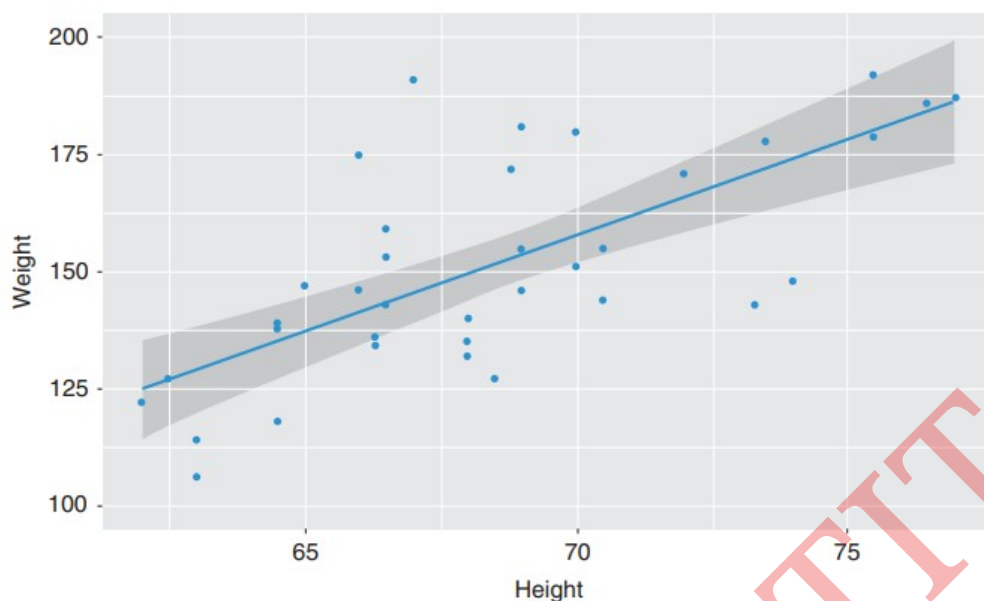
```
lm(formula = Weight ~ Height, data = size)
```

Coefficients:

(Intercept)	Height
-130.354	4.113

Kết quả hiển thị hai hệ số: hệ số của biến độc lập (Chiều cao) và hệ số chặn (Intercept). Từ đó, phương trình đường hồi quy tuyến tính có dạng dưới đây:

```
Weight = -130.354 + 4.113*Height
```

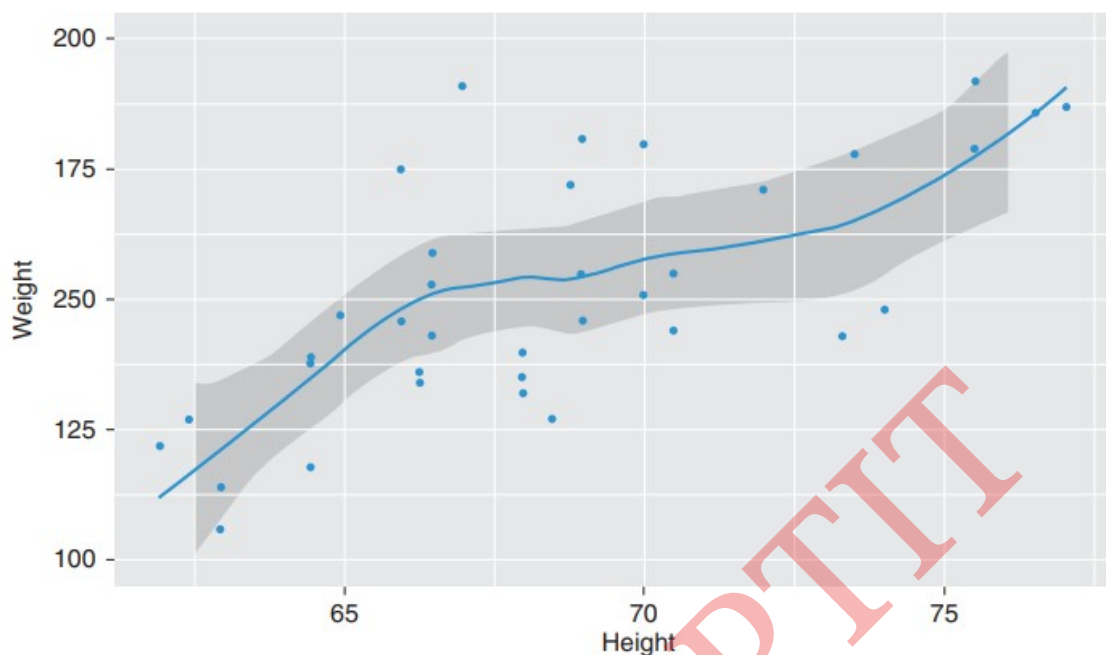


Hình 30: Hồi quy tuyến tính kết nối Chiều cao với Cân nặng

Có thể thử thay các giá trị khác nhau của biến *Chiều cao* vào phương trình trên để quan sát giá trị *Cân nặng* được dự đoán, đồng thời so sánh mức độ gần gũi của các giá trị dự đoán này với dữ liệu thực tế. Trong hồi quy tuyến tính, một đường thẳng được vẽ để mô tả mối quan hệ giữa hai biến. Tuy nhiên, mối quan hệ giữa *Chiều cao* và *Cân nặng* trong thực tế có thể không hoàn toàn tuyến tính. Do đó, cần xem xét việc loại bỏ ràng buộc của mô hình tuyến tính để áp dụng các mô hình linh hoạt hơn.

```
ggplot(size, aes(x=Height,y=Weight)) + geom_point() + geom_smooth() +
ylim(100,200)
```

Kết quả đầu ra được minh họa trong Hình 31 cho thấy dữ liệu phù hợp với đường cong tốt hơn so với đường thẳng. Việc sử dụng đường cong mang lại sự khớp tốt hơn so với cách tiếp cận tuyến tính đơn thuần. Tuy nhiên, cần lưu ý rằng việc áp dụng mô hình cong có thể dẫn đến hiện tượng quá khớp (overfitting). Điều này đồng nghĩa với việc mô hình có thể biểu diễn dữ liệu hiện có rất tốt, nhưng lại làm suy giảm khả năng tổng quát hóa đối với dữ liệu mới. Vấn đề này sẽ được thảo luận chi tiết hơn trong các chương về học máy. Tạm thời, có thể chấp nhận rằng hồi quy tuyến tính bằng đường thẳng là một lựa chọn hợp lý, và trong phạm vi cuốn sách này, khi đề cập đến hồi quy, mặc định đó là hồi quy tuyến tính.



Hình 31: Hồi quy không tuyến tính

1.6.5.3. Phân loại

Thực hiện một số bài toán phân loại trên R cũng tương tự như trên Python. Để dễ minh họa, phương pháp phân loại k-láng giềng gần nhất (kNN) được sử dụng. Như đã đề cập, kNN là một ví dụ điển hình của học có giám sát, trong đó dữ liệu huấn luyện đi kèm với nhãn thực được sử dụng để xây dựng mô hình (trình phân loại), từ đó hỗ trợ phân loại các dữ liệu mới chưa được biết đến. Trước khi áp dụng phân loại trong R, cần đảm bảo rằng gói “class” tương ứng đã được cài đặt. Trong RStudio, các gói có sẵn có thể được tìm thấy trong tab *Packages* (thông thường nằm ở góc dưới bên phải, cùng khu vực hiển thị biểu đồ). Nếu gói “class” đã có trong danh sách, hãy chắc chắn rằng gói này đã được chọn để sử dụng. Trong trường hợp chưa có, có thể cài đặt gói này bằng cách sử dụng phương pháp tương tự như khi cài đặt gói *ggplot2*.

Ví dụ thực hành: Phân loại bằng kNN

Ví dụ này sử dụng lại dữ liệu wine đã được áp dụng ở phần phân loại trước đó. Lệnh dưới đây sẽ mở một hộp thoại cho phép chọn tệp wine.csv từ máy tính:

```
wine = read.table(file.choose(), header = TRUE, sep = ",")
```

Trong khung dữ liệu *wine*, ba biến phụ thuộc (X) được lựa chọn là **density**, **sulfates**,

và **residual_sugar**. Biến độc lập (y) là **high_quality**. Các lệnh sau được sử dụng để trích xuất dữ liệu:

```
X_wine <- wine[, c("density", "sulfates", "residual_sugar")]
```

```
y_wine <- wine[, c("high_quality")]
```

Tiếp theo, dữ liệu được chia thành tập huấn luyện và tập kiểm tra bằng cách sử dụng gói “caret” (*classification and regression training*). Nếu gói này chưa có sẵn, có thể cài đặt bằng phương pháp thông thường trong RStudio. Sau khi cài đặt, tải gói và sử dụng hàm `createDataPartition()` để ngẫu nhiên chọn 70% dữ liệu làm tập huấn luyện:

```
library(caret)
```

```
set.seed(123)
```

```
inTrain <- createDataPartition(y = y_wine, p = 0.7, list = FALSE)
```

```
X_train <- X_wine[inTrain, ]
```

```
X_test <- X_wine[-inTrain, ]
```

```
y_train <- y_wine[inTrain]
```

```
y_test <- y_wine[-inTrain]
```

Trong đó, `X_wine[inTrain, ]` lấy ra các hàng được chỉ định trong *inTrain* cùng toàn bộ các cột, tức là tập huấn luyện. Các hàng còn lại thuộc tập kiểm tra. Tương tự, `y_wine[inTrain]` và `y_wine[-inTrain]` phân chia biến độc lập thành hai phần tương ứng.

Sau khi chuẩn bị dữ liệu, tiến hành phân loại kNN bằng gói “class”. Sử dụng `X_train` và `y_train` để xây dựng mô hình, sau đó dự đoán nhãn lớp cho dữ liệu trong `X_test`:

```
library(class)
```

```
wine_pred <- knn(train = X_train, test = X_test, cl = y_train, k = 3)
```

Cuối cùng, để đánh giá độ chính xác của mô hình, sử dụng gói “gmodels” và hàm CrossTable():

```
library(gmodels)
```

```
CrossTable(x = y_test, y = wine_pred, prop.chisq = FALSE)
```

Kết quả thu được là một bảng hiển thị số lượng quan sát trong từng trường hợp như sau

Total Observations in Table: 1949

y_test	wine_pred		Row Total
	0	1	
0	1363	198	1561
	0.873	0.127	0.801
	0.836	0.623	
	0.699	0.102	
1	268	120	388
	0.691	0.309	0.199
	0.164	0.377	
	0.138	0.062	
Column Total	1631	318	1949
	0.837	0.163	

Hình 32: Bảng hiển thị kết quả

Trong số 1.949 trường hợp được dùng để kiểm tra (tương ứng với 30% dữ liệu), có thể dễ dàng quan sát được số lần mô hình dự đoán đúng và sai. Cụ thể, có những trường hợp mô hình dự đoán là 0 và kết quả thực tế cũng là 0, hoặc dự đoán là 1 và kết quả thực tế là 1 (cả hai đều được xem là dự đoán đúng). Ngược lại, cũng tồn tại những

trường hợp mô hình dự đoán là 0 nhưng kết quả thực tế là 1 (âm tính giả), hoặc dự đoán là 1 nhưng kết quả thực tế là 0 (dương tính giả).

Cần lưu ý rằng tham số cuối cùng "prop.chisq" cho biết có nên hiển thị đóng góp của từng ô (cell) vào thống kê chi-square (χ^2) hay không. Thống kê χ^2 được tính bằng tổng các giá trị đóng góp từ từng ô trong bảng, và được sử dụng để đánh giá liệu sự khác biệt giữa giá trị quan sát và giá trị kỳ vọng có ý nghĩa thống kê hay không.

1.6.5.4. Phân cụm

Tiếp theo là phần nhánh học không giám sát của học máy. Khác với học có giám sát, trong nhánh này chúng ta không có nhãn thực đi kèm dữ liệu huấn luyện. Nói cách khác, sẽ không biết trước điểm dữ liệu nào thuộc vào lớp nào. Thay vào đó, mục tiêu chính của học không giám sát là mô tả và khám phá cấu trúc tiềm ẩn trong dữ liệu. Thông qua đó có thể tìm ra những nhóm (lớp) hoặc mẫu đặc trưng nào đó trong dữ liệu mà trước đó chưa được gán nhãn. Quá trình này thường mang tính chất khám phá, giúp chúng ta hiểu dữ liệu sâu hơn. Trong số các phương pháp học không giám sát, phân cụm (clustering) là kỹ thuật được sử dụng phổ biến nhất. Dưới đây sẽ giải thích sâu hơn phân cụm thông qua một ví dụ thực hành cụ thể.

Ví dụ thực hành: Phân cụm với dữ liệu Iris

Bộ dữ liệu **Iris** thường được sử dụng để minh họa các kỹ thuật phân cụm. Dữ liệu này nằm trong thư viện **datasets** của R và có thể được truy cập như sau:

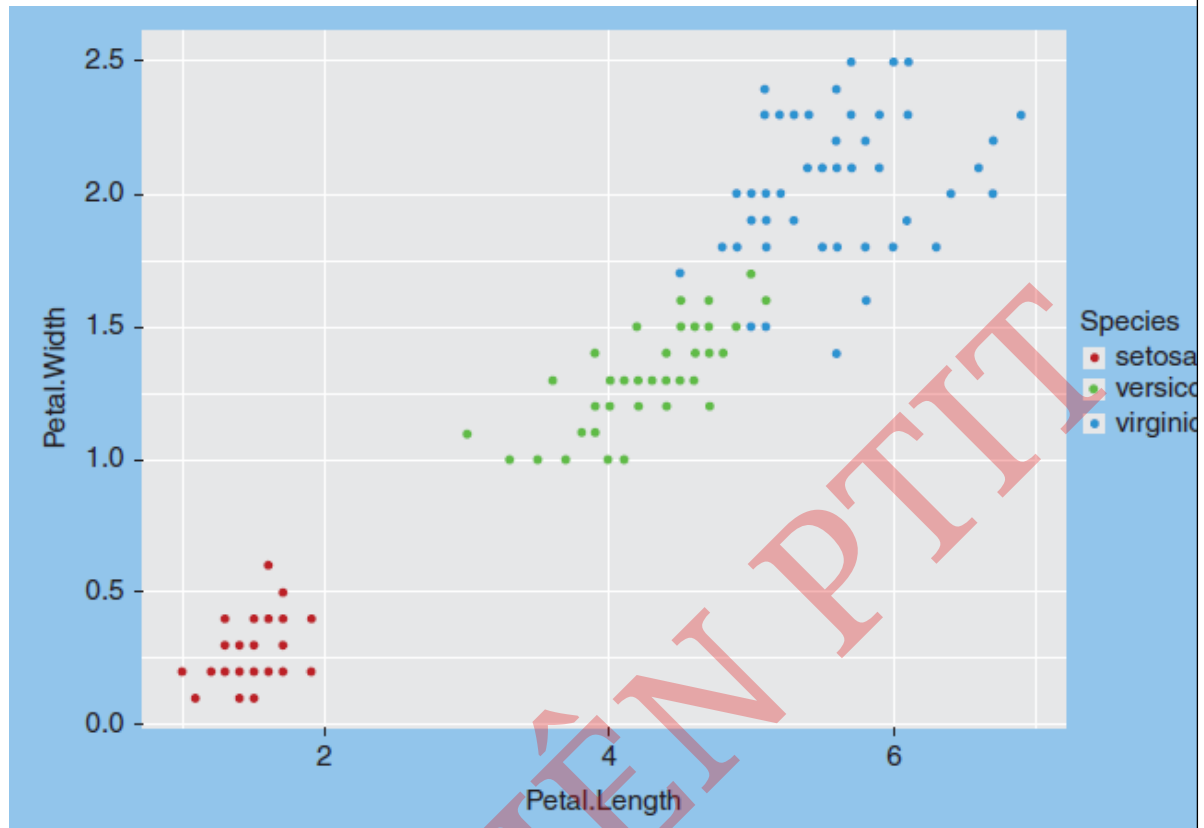
```
library(datasets)
head(iris)
```

Dữ liệu bao gồm các thông tin về ba loài hoa diên vĩ (Iris), với các đặc trưng như chiều dài và chiều rộng của cánh hoa (Petal) và đài hoa (Sepal).

Để trực quan hóa dữ liệu, có thể sử dụng thư viện **ggplot2**. Đoạn mã dưới đây vẽ biểu đồ phân tán giữa chiều dài và chiều rộng cánh hoa, với màu sắc biểu thị các màu hoa:

```
library(ggplot2)
```

```
ggplot(iris, aes(Petal.Length, Petal.Width, color = Species)) + geom_point()
```



Hình 33: Hình ảnh hóa dữ liệu Iris theo loài.

Kết quả hiển thị trên Hình 33 cho thấy các điểm dữ liệu tạo thành ba nhóm khá rõ rệt bao gồm Hoa diên vĩ đỏ (Setosa), Hoa diên vĩ xanh lá (Versicolor) và Hoa diên vĩ xanh dương (Virginica)

Phân cụm bằng k-means

Thuật toán **k-means** sẽ được sử dụng để phân cụm dữ liệu, với số cụm được đặt là 3:

```
set.seed(20) # Tạo số giả ngẫu nhiên
```

```
irisCluster <- kmeans(iris[, 3:4], 3, nstart = 20)
```

```
irisCluster
```

Kết quả trả về cung cấp thông tin tóm tắt về các cụm được tạo. Để so sánh các cụm này

với nhãn thực tế, có thể sử dụng bảng sau:

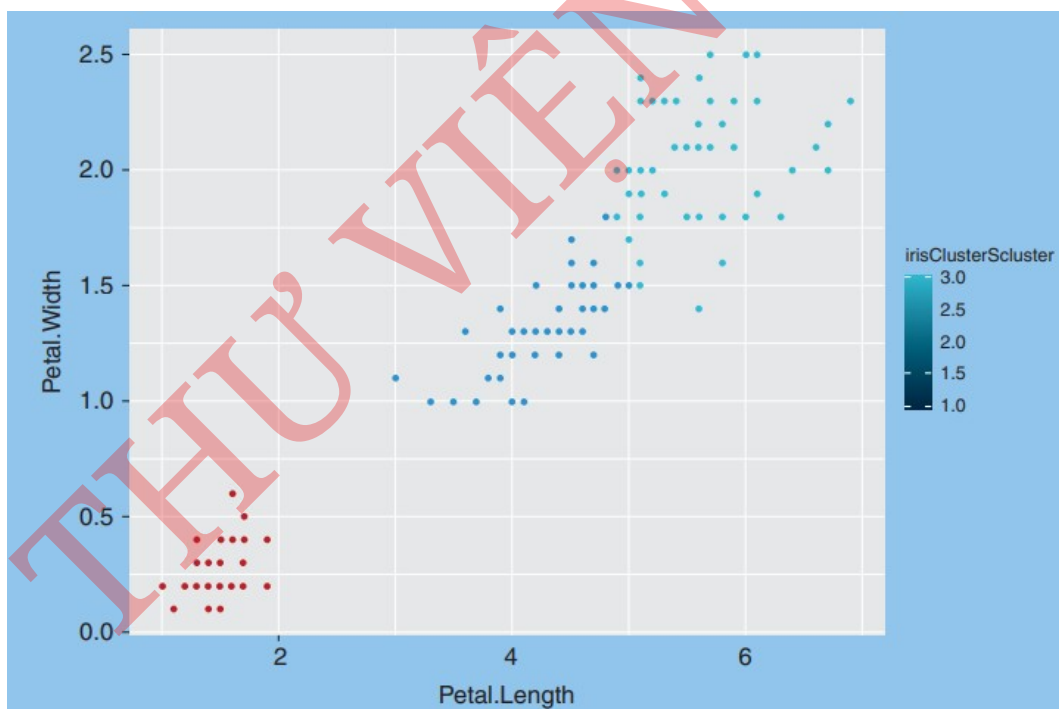
```
table(irisCluster$cluster, iris$Species)
```

Trực quan hóa cụm

Biểu đồ phân tán có thể được vẽ lại với màu sắc thể hiện nhãn cụm thay vì loài thực tế:

```
ggplot(iris, aes(Petal.Length, Petal.Width, color = irisCluster$cluster)) +  
geom_point()
```

Trong Hình 34 dưới đây có thể thấy thông tin phân cụm gần như khớp với các lớp thực tế của các điểm dữ liệu, nhưng không hoàn toàn chính xác và có một vài điểm nằm sai lớp.



Hình 34: Hình ảnh hóa dữ liệu Iris bằng cách sử dụng các cụm dự đoán.

1.7. Cơ sở dữ liệu

1.7.1. Giới thiệu chung

Cơ sở dữ liệu là một bộ sưu tập rất lớn về các loại dữ liệu tác nghiệp, bao gồm các loại dữ liệu âm thanh, tiếng nói, chữ viết, văn bản, đồ họa, hình ảnh tĩnh hay hình ảnh động, v.v. được mã hoá dưới dạng các chuỗi bit và được lưu trữ dưới dạng File dữ liệu trong các bộ nhớ của máy tính. Cấu trúc lưu trữ dữ liệu tuân theo các quy tắc dựa trên lý thuyết toán học. Cơ sở dữ liệu phản ánh trung thực thế giới dữ liệu hiện thực khách quan. Cơ sở dữ liệu là tài nguyên thông tin dùng chung cho nhiều người: Cơ sở dữ liệu (CSDL) là tài nguyên thông tin chung cho nhiều người cùng sử dụng. Bất kỳ người sử dụng nào trên mạng máy tính, tại các thiết bị đầu cuối, về nguyên tắc có quyền truy nhập khai thác toàn bộ hay một phần dữ liệu theo chế độ trực tuyến hay tương tác mà không phụ thuộc vào vị trí địa lý của người sử dụng với các tài nguyên đó. Cơ sở dữ liệu được các hệ ứng dụng khai thác bằng ngôn ngữ con dữ liệu hoặc bằng các chương trình ứng dụng để xử lý, tìm kiếm, tra cứu, sửa đổi, bổ sung hay loại bỏ dữ liệu. Tìm kiếm và tra cứu thông tin là một trong những chức năng quan trọng và phổ biến nhất của các dịch vụ cơ sở dữ liệu.

Hệ quản trị CSDL – HQTCSDL (Database Management System - DBMS) là phần mềm điều khiển các chiến lược truy nhập CSDL. Khi người sử dụng đưa ra yêu cầu truy nhập bằng một ngôn ngữ con dữ liệu nào đó, HQTCSDL tiếp nhận và thực hiện các thao tác trên CSDL lưu trữ. Đối tượng nghiên cứu của CSDL là các thực thể và mối quan hệ giữa các thực thể. Thực thể và mối quan hệ giữa các thực thể là hai đối tượng khác nhau về căn bản. Mối quan hệ giữa các thực thể cũng là một loại thực thể đặc biệt. Trong cách tiếp cận CSDL quan hệ, người ta dựa trên cơ sở lý thuyết đại số quan hệ để xây dựng các quan hệ chuẩn, khi kết nối không tổn thất thông tin và khi biểu diễn dữ liệu là duy nhất. Dữ liệu được lưu trữ trong bộ nhớ của máy tính không những phải tính đến yếu tố về tối ưu không gian lưu trữ, mà phải đảm bảo tính khách quan, trung thực của dữ liệu hiện thực. Nghĩa là phải đảm bảo tính nhất quán của dữ liệu và giữ được sự toàn vẹn của dữ liệu.

1.7.2. Phân loại cơ sở dữ liệu

Trên thực tế có rất nhiều cách thức để phân loại cơ sở dữ liệu, dưới đây là một số cách phân loại hay gặp:

Phân loại theo loại dữ liệu:

- Cơ sở dữ liệu có cấu trúc (structured database): có nghĩa là cơ sở dữ liệu dữ liệu được định hình theo một cấu trúc xác định từ trước. Chúng ta có thể hình dung như một thư viện với các tủ hồ sơ được đánh nhãn, trong mỗi tủ được phân ngăn rõ ràng. Cơ sở dữ liệu có cấu trúc được xây dựng sẽ dễ dàng quản lý và truy cập thông tin.

- Cơ sở dữ liệu phi cấu trúc (unstructured database): là cơ sở dữ liệu không được xác định cấu trúc thông tin từ trước. Thường là tập hợp các dữ liệu thô, hỗn tạp và không đồng nhất. Các thành phần của cơ sở dữ liệu không có đặc điểm chung. Để quản lý, dữ liệu phi cấu trúc cần được chuyển đổi thành dữ liệu có cấu trúc qua quá trình chuẩn hóa.
- Cơ sở dữ liệu bán cấu trúc (semi-structured database): thường là dữ liệu có cấu trúc nhưng không đồng nhất. Cấu trúc của dữ liệu phụ thuộc vào chính nội dung của dữ liệu ấy. Đây là cơ sở dữ liệu có nhiều ưu điểm do lưu trữ được hầu hết các loại dữ liệu khác nhau nên cơ sở dữ liệu bán cấu trúc là hướng mới trong nghiên cứu và ứng dụng và được sử dụng thông dụng trên mạng Internet.

Phân loại theo hình thức lưu trữ, mô hình tổ chức

- Cơ sở dữ liệu dạng tệp (file database) dữ liệu được lưu trữ dưới dạng các file có thể là văn bản, ảnh, thông tin nhị phân, hoặc phát triển hơn là tệp cơ sở dữ liệu nhỏ gọn của các phần mềm quản lý dữ liệu.
- Cơ sở dữ liệu quan hệ (relational database): dữ liệu được lưu trữ trong các bảng dữ liệu gọi là các thực thể, giữa các thực thể này có mối liên hệ với nhau gọi là các quan hệ, mỗi quan hệ có các thuộc tính, trong đó có một thuộc tính là khóa chính. Các hệ quản trị hỗ trợ cơ sở dữ liệu quan hệ như: MS SQL server, Oracle, MySQL... là đặc trưng thể hiện của các cơ sở dữ liệu này.
- Cơ sở dữ liệu phân cấp (herarchical database): Một mô hình cơ sở dữ liệu phân cấp là một mô hình dữ liệu trong đó các dữ liệu được tổ chức thành một cây cấu trúc. Các dữ liệu được lưu trữ như các hồ sơ đó được kết nối với nhau thông qua các liên kết. Một thực thể là một tập hợp của các thực thể con, mỗi thực thể con cuối cùng (gọi là lá) chỉ chưa giá trị.

Phân loại theo đặc tính sử dụng

- Cơ sở dữ liệu hoạt động (operational databases): Trong hoạt động của mỗi cơ quan, đơn vị hay doanh nghiệp luôn sản sinh ra một lượng lớn thông tin. Các thông tin này lại là đầu vào cho quá trình thực hiện một nghiệp vụ khác. Các thông tin này được đưa vào các cơ sở dữ liệu để quản lý và truy xuất bởi các đối tượng khác nhau. Đây chính là hình thức của các cơ sở dữ liệu hoạt động.
- Cơ sở dữ liệu kho (data warehouse): Các tổ chức được yêu cầu phải giữ tất cả các dữ liệu có liên quan trong nhiều năm. Những thông tin này cũng là một nguồn quan trọng của thông tin để phân tích và so sánh các dữ liệu năm nay với các năm trước đó cũng làm cho nó dễ dàng hơn để xác định xu hướng chính phát triển của các tổ chức đang diễn ra. Tất cả các dữ liệu này từ năm trước đó được lưu trữ trong một kho dữ liệu. Kể từ khi dữ liệu được lưu trữ đã trải qua tất cả các loại sàng lọc, chỉnh sửa và tích hợp.

- Cơ sở dữ liệu ngữ nghĩa (semantic database): Đây là cơ sở dữ liệu mới thường được nhắc đến trong thời đại Internet. Là một cơ sở dữ liệu mềm dẻo và linh hoạt lưu trữ ngữ nghĩa của thông tin như như các sự kiện của các đối tượng. Cơ sở dữ liệu ngữ nghĩa được thiết kế để đại diện cho thế giới thực một cách chính xác nhất có thể trong tập hợp dữ liệu. Ký hiệu dữ liệu được tổ chức tuyến tính và phân cấp để cung cấp cho những ý nghĩa nhất định như một trong những mô tả ở trên. Bảng đại diện cho thế giới thực trong bộ dữ liệu, dữ liệu ngữ nghĩa cho phép các máy để tương tác với thông tin của thế gian mà không cần giải thích bởi con người.

Phân loại theo mô hình triển khai

- Cơ sở dữ liệu tập trung (centralized database): Cơ sở dữ liệu tập trung là một cơ sở dữ liệu được đặt, lưu trữ, và duy trì trong một địa điểm duy nhất. Đây là vị trí thường xuyên nhất thiết đặt một hệ thống máy tính hoặc cơ sở dữ liệu hệ thống trung tâm, ví dụ một máy tính chủ, hoặc một hệ thống máy tính máy tính lớn. Thông thường, một cơ sở dữ liệu tập trung sẽ được duy trì và quản lý bởi một đầu mối, một tổ chức hoặc một cơ quan. Người dùng truy cập vào một cơ sở dữ liệu tập trung thông qua hệ thống mạng nội bộ hoặc mạng diện rộng, internet để truy cập vào các cơ sở dữ liệu để cập nhật hoặc khai thác trung tâm CPU, do đó duy trì cơ sở dữ liệu riêng của mình

- Cơ sở dữ liệu phân tán (distributed database) là cơ sở dữ liệu không được lưu trữ và xử lý bởi nhiều máy tính, nhiều hệ thống thông tin và thường được đặt ở nhiều vị trí khác nhau. Các vị trí được kết nối với nhau bằng hệ thống mạng (có thể không thường xuyên trực tuyến). cơ sở dữ liệu phân tán có thể được quản lý bởi nhiều cơ quan, đơn vị và tổ chức khác nhau nhưng cần có một chính sách thống nhất để các cơ sở dữ liệu tương hợp và trao đổi thông tin với nhau.

- Cơ sở dữ liệu tập trung được trang bị bản sao: Các cơ sở dữ liệu tập trung và phân tán thường có những ưu điểm và nhược điểm của mình, do đó quyết định sử dụng loại nào còn phụ thuộc vào yếu tố quản lý và kỹ thuật như sự tham gia của các cơ quan, chính sách quản lý và khai thác dữ liệu, sự ổn định và tốc độ kết nối mạng, mức độ đáp ứng người sử dụng, v.v. Cơ sở dữ liệu tập trung được trang bị bản sao là cơ sở dữ liệu được lai giữa hai loại tập trung và phân tán trong đó một cơ sở dữ liệu tập trung lưu toàn bộ dữ liệu theo phạm vi quản lý, đồng thời triển khai thêm các bản sao theo từng phạm vi dữ liệu và kết nối đồng bộ đến cơ sở dữ liệu trung tâm. Điều này giúp khôi phục cơ sở dữ liệu một cách nhanh chóng bằng cách khôi phục bản sao về cơ sở dữ liệu gốc.

Ưu điểm

Trước đây người ta chỉ quản lý dữ liệu bằng các file riêng biệt, cách làm này chỉ phù hợp trong điều kiện quản lý dữ liệu ở quy mô nhỏ, còn đối với những trường quản lý dữ liệu số lượng lớn thì đòi hỏi phải sử dụng đến cơ sở dữ liệu.

- Việc quản lý dữ liệu bằng cơ sở dữ liệu mang đến lợi ích tích cực, giúp cho việc lưu trữ, truy xuất, sử dụng dữ liệu đạt hiệu suất tốt hơn.
- Khi cả công ty dùng chung một cơ sở dữ liệu, các phòng ban khác có thể dễ dàng truy cập vào cơ sở dữ liệu và lấy thông tin khi cần, điều này giúp tránh được việc trùng lặp dữ liệu, gây lãng phí tài nguyên.
- Khi đã dùng chung cơ sở dữ liệu, thông tin được cập nhật trên cơ sở dữ liệu thì khi người dùng lấy dữ liệu, dữ liệu sẽ được đảm bảo đồng bộ hoàn toàn.
- Việc tìm kiếm thông tin cũng vô vùng dễ dàng. Ngoài ra, chúng ta có thể thêm các bộ lọc, tìm kiếm, gom nhóm các thông tin lại với nhau bằng cách dùng câu truy vấn.
- Thông tin được quản lý ở một nơi duy nhất, đảm bảo việc bảo mật dữ liệu, tránh rò rỉ thông tin. Ngoài ra có thể phân quyền cho người dùng, ai có thể truy cập dữ liệu, ai không được.

Nhược điểm

Việc sử dụng cơ sở dữ liệu cũng có một số nhược điểm sau:

- Dùng chung cơ sở dữ liệu khi xảy ra sự cố, toàn bộ dữ liệu sẽ bị mất hoàn toàn. Tuy nhiên nhược điểm này có thể dễ dàng khắc phục bằng cách sao lưu dữ liệu thường xuyên.
- Việc dùng chung cơ sở dữ liệu khi có quá nhiều truy vấn sẽ gây nghẽn, sập hệ thống. Tuy nhiên, trong thực tế, lượng truy vấn đến cơ sở dữ liệu đã được ước tính và đầu tư vào cấu hình máy chủ cho phù hợp.

1.7.3. Cơ sở dữ liệu có cấu trúc

Cơ sở dữ liệu SQL hoặc cơ sở dữ liệu quan hệ được sử dụng rộng rãi để lưu trữ, quản lý và tổ chức dữ liệu có cấu trúc ở định dạng bảng. Các cơ sở dữ liệu này lưu trữ dữ liệu trong các bảng bao gồm các hàng và cột. SQL là ngôn ngữ lập trình tiêu chuẩn được sử dụng để tương tác với các cơ sở dữ liệu này. Nó cho phép người dùng tạo, cập nhật, truy xuất và quản lý dữ liệu một cách hiệu quả.

Tại sao cơ sở dữ liệu SQL tồn tại?

- **Xử lý dữ liệu hiệu quả:** Cơ sở dữ liệu SQL được thiết kế để lưu trữ, truy xuất và thao tác dữ liệu có cấu trúc một cách hiệu quả.
- **Độ tin cậy:** Chúng đảm bảo tính nhất quán và toàn vẹn của dữ liệu, khiến chúng trở nên lý tưởng cho các ứng dụng có mức độ ưu tiên cao.

- **Mối quan hệ phức tạp:** Cơ sở dữ liệu SQL có thể mô hình hóa các mối quan hệ dữ liệu phức tạp, cần thiết cho các hệ thống giao dịch và doanh nghiệp.
- **Hỗ trợ giao dịch:** Chúng cung cấp khả năng xử lý giao dịch đáng tin cậy, đảm bảo tính nhất quán ngay cả trong các tình huống có nhu cầu cao.
- **Khả năng mở rộng:** Cơ sở dữ liệu SQL mở rộng quy mô hiệu quả để xử lý khối lượng dữ liệu ngày càng tăng trong khi vẫn duy trì hiệu suất.
- **Tuân thủ ACID:** Cơ sở dữ liệu SQL đảm bảo độ chính xác và độ tin cậy của dữ liệu thông qua các thuộc tính ACID (Độ nguyên tử, Tính nhất quán, Cách ly, Độ bền).
- **Áp dụng phổ biến:** Là tiêu chuẩn cho dữ liệu quan hệ, cơ sở dữ liệu SQL được sử dụng rộng rãi trong các ngành, từ doanh nghiệp đến chính phủ.

Cơ sở dữ liệu SQL hoạt động như thế nào

Cơ sở dữ liệu SQL hoạt động bằng cách sử dụng mô hình máy khách-máy chủ. Máy chủ cơ sở dữ liệu lưu trữ tất cả dữ liệu, trong khi máy khách tương tác với máy chủ bằng cách gửi truy vấn để truy xuất hoặc cập nhật dữ liệu.



Hình 35: Quy trình truy vấn cơ sở dữ liệu

- **Lưu trữ dữ liệu:** Dữ liệu được lưu trữ trong các bảng, mỗi bảng chứa các hàng và cột. Mỗi cột xác định một thuộc tính cụ thể và mỗi hàng đại diện cho một thực thể hoặc bản ghi duy nhất.
- **Xử lý truy vấn:** Khi người dùng gửi truy vấn, máy chủ cơ sở dữ liệu sẽ xử lý truy vấn đó bằng cách phân tích cú pháp lệnh SQL, tối ưu hóa truy vấn về hiệu suất và thực thi truy vấn đó để truy xuất hoặc thao tác dữ liệu.
- **Truy xuất dữ liệu:** Truy vấn SQL có thể truy xuất dữ liệu cụ thể bằng cách sử dụng các câu lệnh và lọc kết quả bằng cách sử dụng mệnh đề. Máy chủ trả về dữ liệu

được truy vấn ở định dạng có cấu trúc để các ứng dụng sử dụng truy vấn SELECT ...
FROM...WHERE

Quản lý cơ sở dữ liệu SQL

Phần này giải thích các quy trình chính liên quan đến việc tạo, chọn, đổi tên và xóa cơ sở dữ liệu SQL. Hiểu các lệnh này là điều cần thiết để quản lý cơ sở dữ liệu của bạn một cách hiệu quả.

1. TẠO cơ sở dữ liệu

Lệnh CREATE DATABASE được sử dụng để khởi tạo cơ sở dữ liệu mới. Câu lệnh này tạo ra một cơ sở dữ liệu trống hoạt động như một vùng chứa cho các bảng của bạn và các đối tượng cơ sở dữ liệu khác. Câu lệnh này cho phép bạn sắp xếp và quản lý dữ liệu một cách dễ dàng.

```
CREATE DATABASE test_db;
```

2. LỰA CHỌN cơ sở dữ liệu

Lệnh USE hoặc SELECT DATABASE cho phép bạn lựa chọn một cơ sở dữ liệu cụ thể cho phiên hiện tại. Sau khi được chọn, cơ sở dữ liệu sẽ trở thành cơ sở dữ liệu đang hoạt động và tất cả các truy vấn tiếp theo sẽ được thực hiện trên chính cơ sở dữ liệu này. Lệnh này rất cần thiết khi làm việc với nhiều cơ sở dữ liệu.

```
USE test_db;
```

3. ĐỔI TÊN cơ sở dữ liệu

Truy vấn RENAME DATABASE có thể được sử dụng khi bạn cần đổi tên cơ sở dữ liệu hiện có.

```
RENAME DATABASE test_db TO new_test_db;
```

4. XÓA cơ sở dữ liệu

Lệnh DROP DATABASE được sử dụng để xóa vĩnh viễn cơ sở dữ liệu, cùng với tất cả các bảng, dữ liệu và đối tượng của nó. Hành động này là không thể đảo ngược, vì vậy nó nên được sử dụng cẩn thận. Nó rất hữu ích khi bạn không còn cần cơ sở dữ liệu và muốn giải phóng tài nguyên hệ thống.

```
DROP DATABASE test_db;
```

1.7.4. Cơ sở dữ liệu bán cấu trúc

Dữ liệu bán cấu trúc là dữ liệu không nằm trong cơ sở dữ liệu quan hệ truyền thống (như SQL) nhưng vẫn có một số thuộc tính tổ chức, chẳng hạn như thẻ hoặc điểm đánh dấu, giúp phân tích dễ dàng hơn so với dữ liệu hoàn toàn phi cấu trúc.

Dữ liệu bán cấu trúc không tuân theo một cấu trúc nhất định như dữ liệu có cấu trúc, nhưng vẫn chứa các yếu tố như nhãn hoặc khóa giúp dữ liệu có thể nhận dạng và tìm kiếm được.

Đặc điểm của dữ liệu bán cấu trúc

1. **Lược đồ linh hoạt:** Dữ liệu bán cấu trúc có thể không giống nhau trên tất cả các bản ghi. Ví dụ: một đối tượng JSON có thể có năm trường dữ liệu trong khi một đối tượng khác chỉ có ba.
2. **Định dạng con người có thể đọc được:** Nhiều kiểu dữ liệu như XML hoặc JSON rất dễ hiểu đối với con người và máy móc.
3. **Có thể mở rộng:** Dễ dàng xử lý bởi cơ sở dữ liệu phi quan hệ hiện đại, điều này khi lưu trữ dữ liệu lớn (Big Data).
4. **Siêu dữ liệu phong phú:** Thẻ và thuộc tính cung cấp ngữ cảnh giúp sắp xếp và phân tích.

Tầm quan trọng của dữ liệu bán cấu trúc

Khi dữ liệu trở nên phức tạp và đa dạng hơn, các định dạng bán cấu trúc mang lại sự cân bằng giữa tính linh hoạt và khả năng quản lý. Các tổ chức lưu trữ và xử lý các loại dữ liệu này cho nhiều mục đích khác nhau. Ngoài ra, dữ liệu bán cấu trúc cho phép thích ứng nhanh với các nguồn dữ liệu mới mà không cần thiết kế lại cơ sở dữ liệu hiện có. Tính linh hoạt này hỗ trợ phân tích và tích hợp dữ liệu hiệu quả hơn, đặc biệt là khi kết hợp dữ liệu có cấu trúc và phi cấu trúc, khiến nó trở thành một tài sản có giá trị trong môi trường dựa trên dữ liệu hiện đại.

Ví dụ về dữ liệu bán cấu trúc bao gồm:

- JSON (Là định dạng văn bản dựa trên ngôn ngữ JavaScript)
- XML (Ngôn ngữ đánh dấu có thể mở rộng)
- Tập CSV có hàng không nhất quán
- Email (tiêu đề có cấu trúc và nội dung không có cấu trúc)
- Dữ liệu cảm biến từ các thiết bị IoT
- Trang web HTML

Trích xuất thông tin từ dữ liệu bán cấu trúc

Dữ liệu bán cấu trúc có cấu trúc khác nhau do tính không đồng nhất của các nguồn dữ liệu khi nhập. Các nguồn dữ liệu này có cấu trúc khác nhau, do đó khi lưu trữ cần phải

thống nhất dữ liệu giữa các nguồn. Điều này gây khó khăn cho việc gắn thẻ và lập chỉ mục. Vì vậy, trong khi trích xuất thông tin từ họ là một công việc khó khăn. Dưới đây là một số giải pháp

- Các mô hình dựa trên đồ thị (ví dụ: OEM) có thể được sử dụng để lập chỉ mục dữ liệu bán cấu trúc
- Kỹ thuật mô hình hóa dữ liệu trong OEM cho phép dữ liệu được lưu trữ trong mô hình dựa trên đồ thị. Dữ liệu trong mô hình dựa trên đồ thị dễ tìm kiếm và lập chỉ mục hơn.
- Ngôn ngữ đánh dấu XML cho phép dữ liệu được sắp xếp theo thứ tự phân cấp cho phép dữ liệu được lập chỉ mục và tìm kiếm
- Sử dụng các công cụ khai thác dữ liệu khác nhau

Quản lý dữ liệu bán cấu trúc

Không giống như dữ liệu có cấu trúc, dữ liệu bán cấu trúc được quản lý tốt nhất bằng cách sử dụng các hệ quản trị cơ sở dữ liệu NoSQL. Các công nghệ phổ biến bao gồm:

- **MongoDB:** Cơ sở dữ liệu NoSQL dựa trên tài liệu hoạt động tốt với các định dạng lưu trữ tương tự như JSON.
- **Cassandra:** Xử lý dữ liệu họ cột với thiết kế lược đồ bán cấu trúc.
- **Elasticsearch:** Có thể lập chỉ mục và tìm kiếm thông qua các tệp nhật ký và tài liệu bán cấu trúc.
- **Lưu trữ đám mây (ví dụ: AWS S3, Azure Blob):** Được sử dụng để lưu trữ khối lượng lớn dữ liệu bán cấu trúc như nhật ký, email và dữ liệu đo từ xa.

Ứng dụng

Dữ liệu bán cấu trúc được sử dụng trong nhiều ngành khác nhau:

- **Thương mại điện tử:** Danh mục sản phẩm được lưu trữ ở định dạng JSON, cho phép linh hoạt trong các thuộc tính mặt hàng.
- **Chăm sóc sức khỏe:** Biểu mẫu và báo cáo bệnh nhân được lưu trữ trong XML với các trường biến đổi.
- **IoT và thiết bị thông minh:** Dữ liệu cảm biến được thu thập ở định dạng khóa-giá trị.
- **Phát triển web:** HTML và JSON được sử dụng để hiển thị nội dung động trên các trang web.
- **Nền tảng truyền thông xã hội:** Hoạt động của người dùng và tin nhắn được đăng nhập trong nhật ký bán cấu trúc.

Thách thức

Mặc dù tính linh hoạt, dữ liệu bán cấu trúc đi kèm với một số thách thức:

- **Truy vấn phức tạp:** Không đơn giản như truy vấn SQL trên dữ liệu có cấu trúc.
- **Làm sạch dữ liệu:** Cấu trúc không đều có thể dẫn đến sự không nhất quán và tích hợp khó khăn hơn.
- **Khả năng tương thích của công cụ:** Không phải tất cả các công cụ phân tích đều hỗ trợ các định dạng bán cấu trúc ngay lập tức.

1.7.5. Cơ sở dữ liệu phi cấu trúc

Dữ liệu Phi Cấu Trúc là loại dữ liệu không lồ và phát triển nhanh nhất hiện nay, được định nghĩa là thông tin không có định dạng hoặc mô hình dữ liệu được xác định trước (predefined schema). Loại dữ liệu này không thể được sắp xếp gọn gàng vào các bảng, hàng và cột của cơ sở dữ liệu quan hệ truyền thống.

Dữ liệu phi cấu trúc tồn tại ở dạng thô, tự nhiên, và bao gồm phần lớn nội dung được tạo ra hàng ngày. Mặc dù khó phân tích hơn so với dữ liệu có cấu trúc (như bảng tính), chúng lại chứa đựng những thông tin ngữ cảnh vô cùng phong phú và có giá trị.

Dữ liệu phi cấu trúc bao gồm mọi thứ từ các tài liệu văn bản tự do đến các tệp đa phương tiện bao gồm

- Văn bản: Nội dung email, tài liệu Word, PDF, ghi chú, bài đăng trên mạng xã hội.
- Đa phương tiện: Hình ảnh, video, tệp âm thanh (ví dụ: ghi âm cuộc gọi của khách hàng).
- Dữ liệu Khác: Dữ liệu cảm biến thô, nhật ký máy chủ (server logs).

Do thiếu cấu trúc cố định, việc lưu trữ và xử lý Dữ liệu Phi Cấu Trúc thường được thực hiện thông qua các nền tảng linh hoạt như Hồ Dữ liệu (Data Lakes) hoặc hệ thống quản lý tệp phân tán. Để trích xuất giá trị từ chúng, các tổ chức phải dựa vào các công nghệ tiên tiến như Trí tuệ Nhân tạo (AI), Xử lý Ngôn ngữ Tự nhiên (NLP), và Thị giác Máy tính (Computer Vision).

Vì không có cấu trúc cố định, dữ liệu phi cấu trúc thường không được lưu trữ trong các Cơ sở dữ liệu Quan hệ (RDBMS) truyền thống, mà thay vào đó được lưu trữ trong các hệ thống linh hoạt và có khả năng mở rộng cao:

- **Hồ Dữ liệu (Data Lakes):** Nơi dữ liệu thô được lưu trữ ở định dạng gốc.
- **Hệ thống tệp phân tán (Distributed File Systems):** Ví dụ: Hadoop HDFS.
- **Cơ sở dữ liệu NoSQL:** Đôi khi được sử dụng để lưu trữ các tài liệu hoặc các tệp lớn có chứa dữ liệu phi cấu trúc.

1.8. Hệ điều hành

1.8.1. Giới thiệu chung

Có nhiều cách định nghĩa khác nhau về hệ điều hành, nhưng thông thường, hệ điều hành được định nghĩa thông qua mục đích, vai trò, và chức năng trong hệ thống máy tính. Hệ điều hành là hệ thống phần mềm đóng vai trò trung gian giữa người sử dụng và phần cứng máy tính nhằm tạo ra môi trường giúp thực hiện các chương trình một cách thuận tiện. Ngoài ra, hệ điều hành còn quản lý và đảm bảo cho việc sử dụng phần cứng của máy tính một cách hiệu quả.

Để hoàn thành vai trò của mình, hệ điều hành cần thực hiện hai chức năng cơ bản là quản lý tài nguyên và quản lý việc thực hiện các chương trình. Ta sẽ xem xét kỹ hai chức năng này của hệ điều hành.

Quản lý tài nguyên

Quản lý tài nguyên nhằm đảm bảo cho tài nguyên hệ thống được sử dụng một cách có ích và hiệu quả. Nhờ có hệ điều hành, tài nguyên được quản lý và sử dụng hợp lý hơn. Các tài nguyên phần cứng chủ yếu của máy tính gồm có bộ xử lý (CPU), bộ nhớ chính, bộ nhớ thứ cấp, các thiết bị vào ra.

CPU là thành phần trung tâm của hệ thống, có chức năng xử lý dữ liệu và điều khiển toàn hệ thống. Bộ nhớ chính là nơi lưu trữ chương trình và dữ liệu trong quá trình xử lý. Bộ nhớ thứ cấp, hay bộ nhớ ngoài, bao gồm các đĩa từ, đĩa quang học, đĩa quang từ, băng từ, thẻ nhớ và các thiết bị nhớ khác có vai trò lưu trữ chương trình, dữ liệu trong thời gian dài với dung lượng lớn. Thiết bị vào ra cho phép máy tính trao đổi thông tin với thế giới bên ngoài.

Quản lý tài nguyên trước hết là phân phối tài nguyên tới các ứng dụng một cách hiệu quả. Để thực hiện được, các chương trình cần tài nguyên phần cứng như không gian bộ nhớ, thiết bị ngoại vi. Yêu cầu tài nguyên được hệ điều hành thu nhận và đáp ứng bằng cách cung cấp cho chương trình các tài nguyên tương ứng. Muốn cấp phát tài nguyên, hệ điều hành cần lưu trữ tình trạng tài nguyên để biết hiện tại tài nguyên nào còn trống, tài nguyên nào đang được sử dụng. Một ví dụ điển hình là trường hợp lưu trữ thông tin lên đĩa. Hệ điều hành cần biết những vùng nào trên đĩa chưa được sử dụng để ghi thông tin lên những vùng này. Việc ghi thông tin lên vùng trống cũng cần được tính toán sao cho quá trình truy cập tới thông tin khi cần có thể thực hiện nhanh nhất. Yêu cầu về phần cứng của các chương trình này có thể mâu thuẫn nhau. Chẳng hạn, hai chương trình cùng có yêu cầu ghi ra đĩa một lúc. Trong trường hợp xuất hiện các yêu cầu mâu thuẫn

khác nhau về phần cứng như ví dụ này, hệ điều hành sẽ quyết định thứ tự và thời gian cung cấp tài nguyên cho các chương trình nhằm đạt được mục tiêu tính toán của hệ thống đồng thời tối ưu hoá một số tiêu chí nào đó, chẳng hạn giảm thời gian các chương trình phải tạm ngừng để chờ đợi lẫn nhau.v.v.

Quản lý tài nguyên còn có nghĩa là đảm bảo sao cho chương trình không xâm phạm tài nguyên đã cấp cho chương trình khác. Ví dụ, nếu hai chương trình được cấp hai vùng bộ nhớ khác nhau, thì việc chương trình này truy cập và thay đổi vùng bộ nhớ của chương trình khác sẽ làm cho chương trình đó hoạt động không bình thường. Hệ điều hành cần thể hiện chức năng quản lý tài nguyên của mình qua việc ngăn ngừa những vi phạm kiểu này.

Quản lý việc thực hiện các chương trình

Nhiệm vụ quan trọng nhất của máy tính là thực hiện các chương trình. Một chương trình đang trong quá trình thực hiện được gọi là tiến trình. Chương trình cần được quản lý để có thể thực hiện một cách thuận lợi và không gặp lỗi, đồng thời đảm bảo môi trường để việc xây dựng và thực hiện chương trình được thuận lợi. Nhiệm vụ của hệ điều hành giúp việc chạy chương trình một cách dễ dàng hơn. Để chạy chương trình cần thực hiện một số thao tác nhất định, nhờ có hệ điều hành, người dùng không phải thực hiện các thao tác này. Hệ điều hành cũng cung cấp giao diện giúp người dùng dễ dàng chạy hoặc kết thúc các chương trình.

Để tạo môi trường thuận lợi cho chương trình, hệ điều hành tạo ra các máy ảo. Máy ảo là các máy logic với những tài nguyên ảo có sẵn các tính chất và khả năng khác so với tài nguyên thực: dễ sử dụng hơn, dễ lập trình hơn, số lượng nhiều hơn tài nguyên thực, khả năng có thể vượt quá khả năng tài nguyên thực. Tài nguyên ảo là bản mô phỏng của tài nguyên thực được thực hiện bằng phần mềm. Tài nguyên ảo giống tài nguyên thực ở chỗ tài nguyên ảo cung cấp các dịch vụ cơ bản như tài nguyên thực. Chẳng hạn, bộ vi xử lý ảo cung cấp khả năng thực hiện các lệnh hay bộ nhớ ảo cung cấp khả năng lưu trữ thông tin, thiết bị vào/ra ảo cho phép chương trình đọc ghi dữ liệu. Tài nguyên ảo khác tài nguyên thực ở chỗ dễ sử dụng hơn. Các tài nguyên thực đều rất khó lập trình trực tiếp. Lấy ví dụ việc ghi thông tin ra đĩa cứng. Các đĩa cứng thường được lập trình bằng cách ghi một số lệnh ra các thanh ghi điều khiển. Các thanh ghi khác làm nhiệm vụ chứa thông tin cần trao đổi và trạng thái đĩa. Để thực hiện việc đọc ghi thông tin, ta cần xác định chuỗi lệnh khởi động (làm đĩa quay nếu đĩa đang ở trạng thái dừng), kiểm tra xem đĩa đã đạt được tốc độ chưa, sau đó chuyển đầu đọc tới vị trí cần thiết, ghi thông tin ra các thanh ghi dữ liệu và đưa các lệnh tiến hành ghi thông tin ra các thanh ghi

điều khiển. Việc lập trình điều khiển đĩa như vậy đòi hỏi rất nhiều thời gian cùng những hiểu biết về giao diện phần cứng.

1.8.2. Unix

Trong khi nhiều ngôn ngữ lập trình mạnh mẽ được sử dụng để xử lý các vấn đề khoa học dữ liệu, người ta thường quên đi công cụ đơn giản nhất nhưng lại vô cùng hiệu quả. Đó chính là UNIX. Tuy nhiên, mặc dù UNIX thực sự là một trong những nền tảng điện toán lâu đời nhất, nhưng hệ điều hành này ẩn chứa sức mạnh đáng kinh ngạc vì khả năng xử lý hầu hết mọi loại vấn đề tính toán và dữ liệu đáng nể. Trên thực tế, xét về nhiều mặt, UNIX vượt trội hơn hẳn các hệ điều hành khác, hệ điều hành này có thể làm được những điều mà các hệ điều hành khác chỉ có thể mơ ước!

Khi mọi người nghĩ đến các công cụ cho khoa học dữ liệu hoặc phân tích dữ liệu, UNIX không xuất hiện trong tâm trí họ. Hầu hết các sách về những chủ đề này đều không đề cập đến UNIX. Nhưng đây là một cơ hội bị bỏ lỡ, vì UNIX cho phép thực hiện nhiều tác vụ khoa học dữ liệu, bao gồm làm sạch dữ liệu, lọc, sắp xếp (sắp xếp), và thậm chí trực quan hóa, thường chỉ sử dụng các lệnh và tiện ích tích hợp sẵn. Điều này khiến nó trở nên hấp dẫn đối với những người chưa thành thạo ngôn ngữ lập trình hoặc công cụ thống kê. Trong chương này, chúng ta sẽ tìm hiểu một số kiến thức cơ bản về làm việc trong môi trường UNIX. Phần này đề cập các nội dung bao gồm cách chạy lệnh, chuyển hướng và chỉnh sửa tệp. Ngoài ra, một số phím tắt giúp làm việc trên UNIX cũng được đề cập giúp thao tác dễ dàng và nhanh chóng. Cuối cùng, tất nhiên, mục tiêu không phải là thành thạo UNIX, mà là giải quyết các vấn đề dựa trên dữ liệu, và vì vậy chúng ta sẽ thấy UNIX hữu ích như thế nào trong việc giải quyết nhiều vấn đề mà không cần viết bất kỳ mã nào.

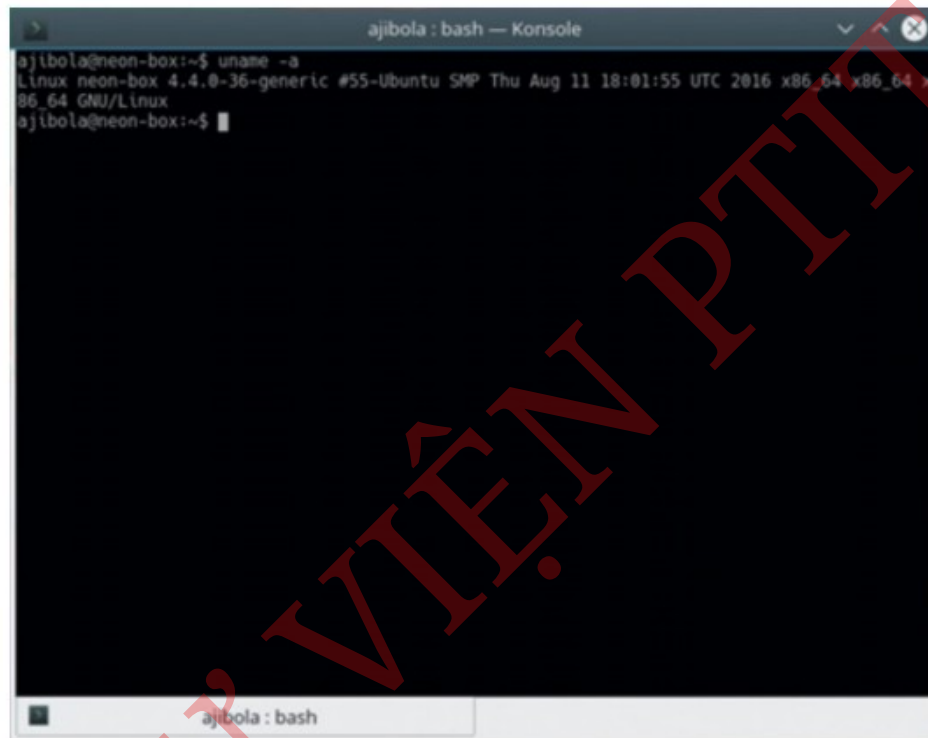
1.8.2.1. Truy cập UNIX

Hệ điều hành UNIX xuất hiện ở nhiều nơi trong các nền tảng máy tính hiện đại.

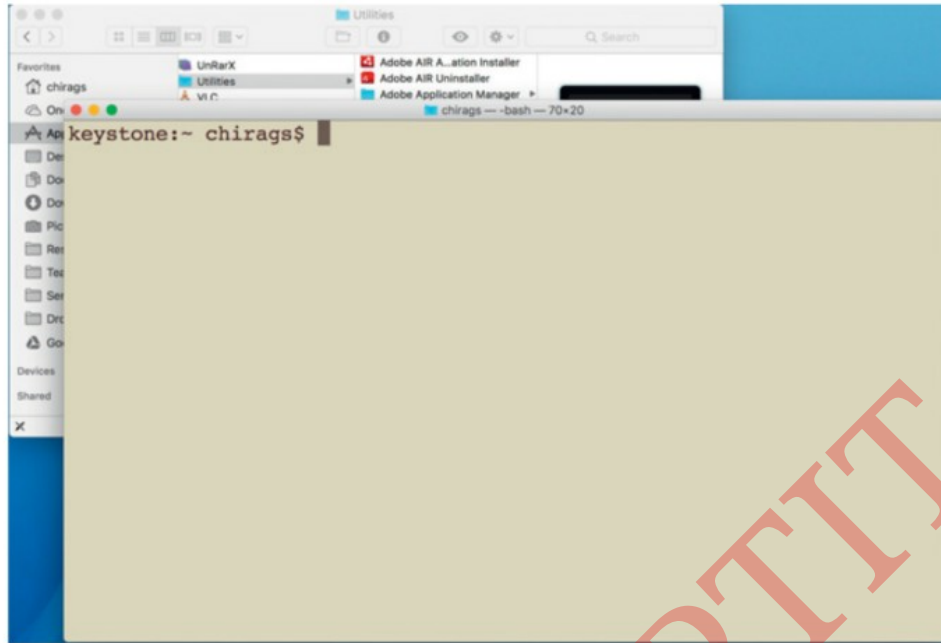
- **Linux:** Bản chất là một hệ điều hành dựa trên UNIX. Khi làm việc trên máy Linux, có thể mở *console* hoặc *terminal* để bắt đầu thao tác (Hình 36).
- **macOS:** Đây cũng là một nền tảng dựa trên UNIX. Để mở môi trường dòng lệnh, vào **Applications > Utilities** trong Finder và chọn ứng dụng **Terminal**. Một cửa sổ sẽ xuất hiện cùng với dấu nhắc lệnh (Hình 37).
- **Windows:** Việc sử dụng môi trường UNIX trên Windows phức tạp hơn, tuy nhiên có các giải pháp thay thế. Một lựa chọn phổ biến là **Cygwin**, một công cụ miễn phí cho phép tạo môi trường tương tự UNIX trên Windows (Hình 38). Trong trường hợp

không cài đặt Cygwin, có thể sử dụng các tiện ích cơ bản để kết nối trực tiếp tới máy chủ UNIX.

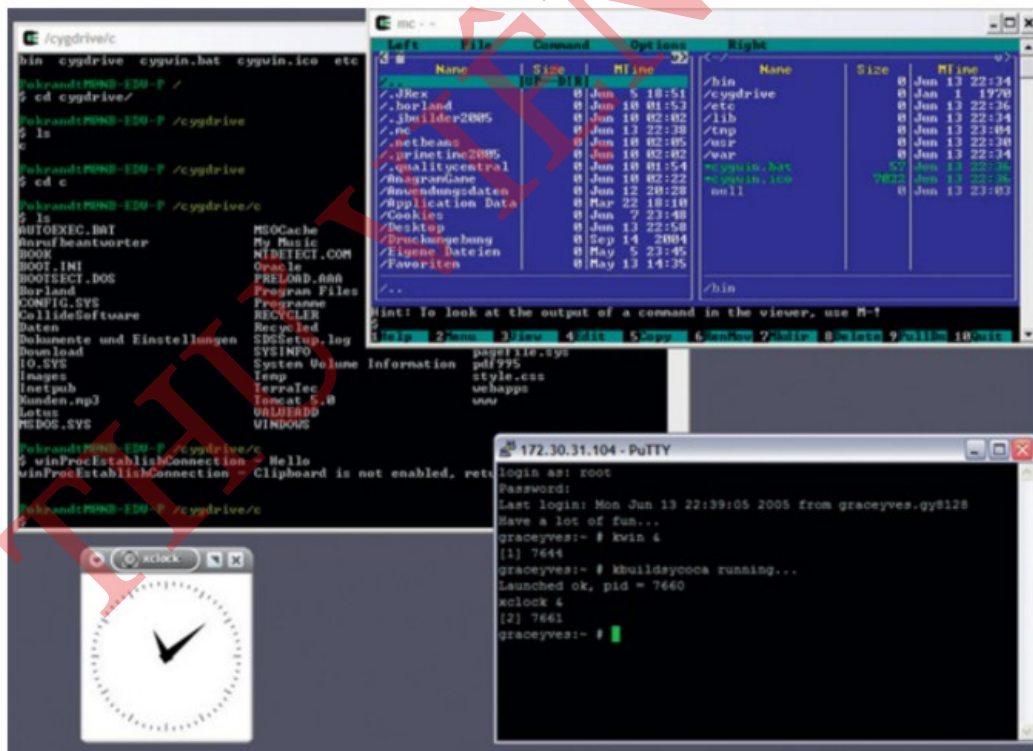
Trong phạm vi còn lại của chương, giả định việc thao tác được thực hiện trên một máy chủ UNIX, ngay cả khi hệ thống gốc là Linux hoặc macOS. Máy chủ này có thể được cung cấp bởi tổ chức giáo dục, cơ quan, hoặc nhà cung cấp dịch vụ web hosting. Ngoài ra, cũng tồn tại các dịch vụ máy chủ UNIX trực tuyến miễn phí. Đây là một phương án khác để truy cập và làm việc trong môi trường UNIX.



Hình 36: Cửa sổ bảng điều khiển trên màn hình nền Linux KDE.1



Hình 37: Ứng dụng Terminal trên máy Mac



Hình 38: Cygwin chạy trên máy tính để bàn Windows

Máy chủ UNIX có thể được cung cấp thông qua dịch vụ điện toán đám mây, chẳng hạn như **Amazon Web Services (AWS)** hoặc **Google Cloud Platform**.

Dù sử dụng môi trường nào – máy tính Linux, macOS, Cygwin trên Windows, hay kết nối đến máy chủ UNIX từ xa – các thao tác cơ bản như chạy lệnh, xử lý tệp và quản lý tiến trình đều hoạt động theo cùng một cách.

Lưu ý: Có nhiều cách hơn để truy cập UNIX

Trước khi bắt đầu làm việc, cần làm rõ khái niệm UNIX trong phạm vi sử dụng thực tế. UNIX là một hệ điều hành mạng mạnh mẽ, nhưng mục tiêu ở đây không phải là khai thác toàn bộ hệ thống mà chỉ là truy cập vào **shell** – lớp giao diện giúp kết nối với các ứng dụng và tiện ích sẵn có của UNIX.

Có nhiều cách để truy cập vào shell của UNIX. Cách đơn giản nhất là kiểm tra môi trường cục bộ. Nếu đang sử dụng hệ điều hành Linux hoặc macOS, một shell UNIX đã được tích hợp sẵn. Trong trường hợp sử dụng Windows, cần cân nhắc giữa việc cài đặt thêm phần mềm để mô phỏng môi trường UNIX (chẳng hạn Cygwin, WSL) và việc kết nối đến một máy chủ UNIX bên ngoài. Máy chủ như vậy thường có thể được cung cấp bởi trường học hoặc tổ chức làm việc.

Trong tình huống không có sẵn máy chủ tại chỗ, một lựa chọn khác là **thuê máy chủ thông qua dịch vụ đám mây**. Các nền tảng phổ biến như Google Cloud, Amazon Web Services hoặc Microsoft Azure cho phép tạo máy ảo chạy UNIX. Sau khi thiết lập, máy ảo có thể được đăng nhập và sử dụng tương tự như máy chủ vật lý.

Tùy vào nhu cầu, việc lựa chọn giải pháp cũng khác nhau. Nếu chỉ cần môi trường UNIX để thực hành cơ bản trong phạm vi chương này, có thể chọn phương án dễ thiết lập nhất. Trong trường hợp muốn tiếp tục nghiên cứu lâu dài và ổn định hơn, giải pháp phù hợp là sử dụng các dịch vụ đám mây, nhiều trong số đó có gói miễn phí cho người mới bắt đầu.

1.8.2.2. Kết nối với máy chủ UNIX

Trong trường hợp có quyền truy cập vào máy tính UNIX, Linux hoặc macOS, việc thiết lập môi trường làm việc trở nên đơn giản hơn so với môi trường Windows. Các hệ điều hành này vốn đã tích hợp sẵn nhiều công cụ miễn phí cần thiết cho quá trình thao tác. Trong trường hợp này, chỉ cần chuẩn bị hai công cụ sau:

1. **Công cụ kết nối tới máy chủ** – thường là các chương trình hỗ trợ giao thức SSH
2. **Công cụ truyền tệp giữa máy cục bộ và máy chủ** – phổ biến nhất là các tiện ích sử dụng giao thức SCP hoặc SFTP.

Với hai thành phần này, việc đăng nhập và trao đổi dữ liệu với máy chủ UNIX có thể được thực hiện một cách thuận tiện và hiệu quả.

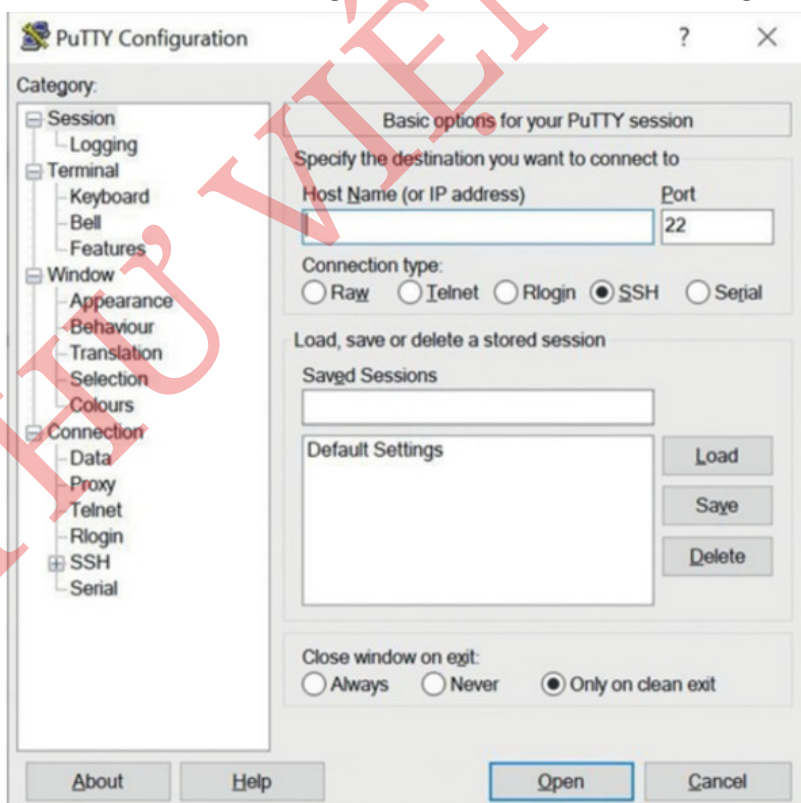
SSH

Trước khi thực hiện bất kỳ thao tác nào trên hệ thống UNIX, cần thiết lập kết nối với máy chủ. Phương pháp truyền thống là sử dụng dịch vụ Telnet, tuy nhiên Telnet thuần túy thường không an toàn nên nhiều máy chủ UNIX hiện nay không còn hỗ trợ công cụ này. Thay vào đó, phương thức phổ biến hơn là SSH (Secure Shell).

SSH bao gồm hai thành phần: phía máy chủ và phía máy khách. Phía máy chủ thường đã được cài đặt sẵn trên hệ thống UNIX khi cấp quyền truy cập. Phía máy cần chuẩn bị công cụ hoặc tiện ích chạy trên máy tính để tạo kết nối.

Hệ điều hành Linux hoặc macOS đã có công cụ sẵn, chỉ cần mở terminal hoặc console để sử dụng dịch vụ SSH đã được tích hợp. Đối với máy tính sử dụng Windows, cần cài đặt phần mềm có dịch vụ SSH client, chẳng hạn như WinSCP hoặc PuTTY (cả hai đều miễn phí).

Để thiết lập kết nối, cần chuẩn bị ba thông tin cơ bản: tên máy chủ (Host), tên người dùng (Username) và mật khẩu (Password). Trong đó, tên máy chủ có thể là dạng tên miền đầy đủ (ví dụ: example.organization.com) hoặc địa chỉ IP, ví dụ như 192.168.1.109. Giao diện kết nối bằng PuTTY được minh họa trong Hình 39.



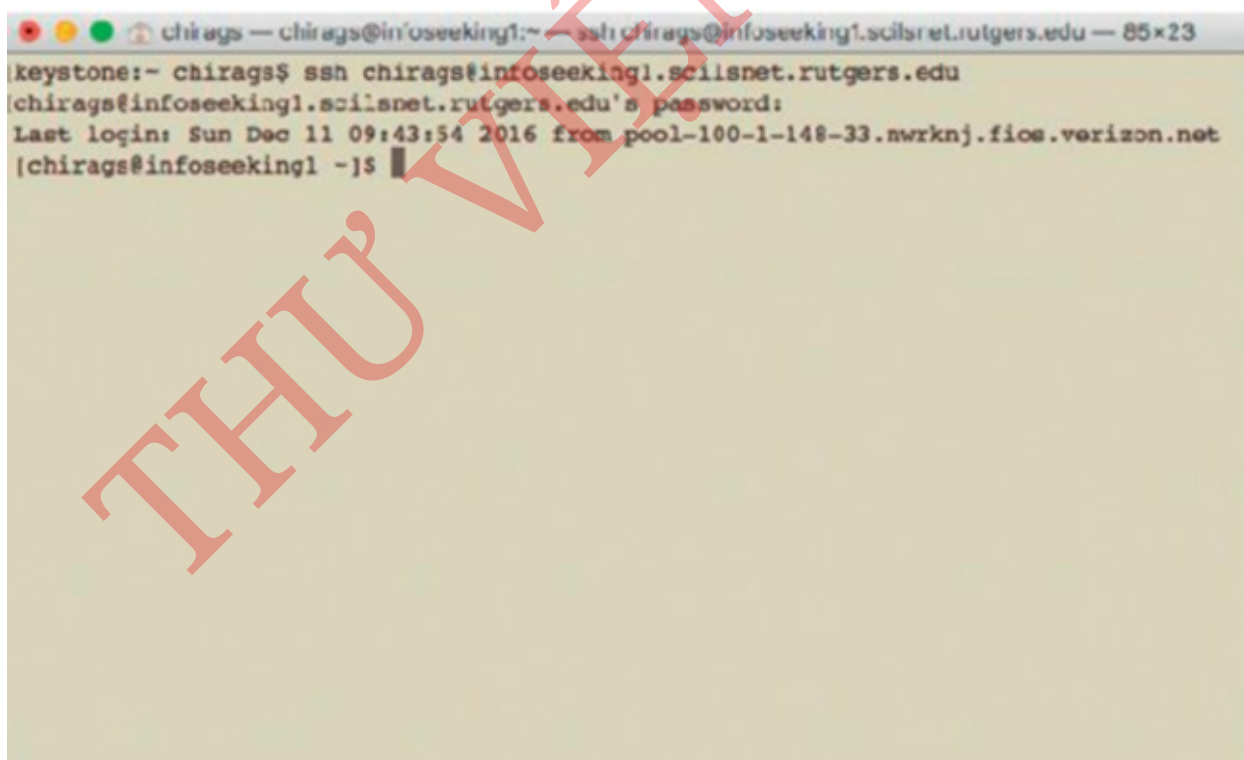
Hình 39: Ảnh chụp màn hình PuTTY

Để có được thông tin cần thiết cho việc kết nối, cần liên hệ với quản trị viên của máy chủ. Trong trường hợp sử dụng hệ điều hành UNIX như Linux hoặc macOS, có thể thực hiện kết nối bằng cách nhập và thực thi lệnh trong **terminal**.

```
ssh username@hostname
```

Sau đó tiến hành nhập mật khẩu đã được quản trị viên cung cấp để đăng nhập sử dụng SSH

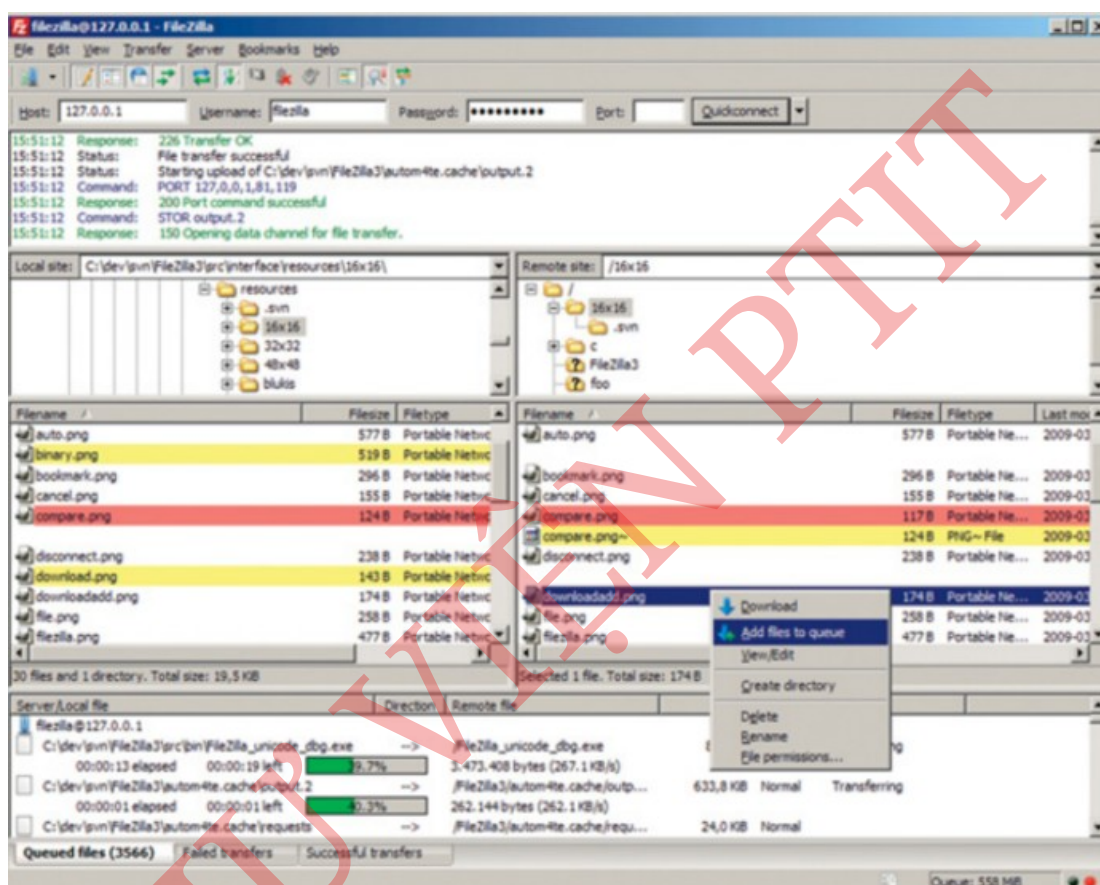
Khi sử dụng PC cùng với một trong các phần mềm đã nêu (PuTTY, WinSCP), cần mở công cụ tương ứng, nhập thông tin máy chủ hoặc địa chỉ IP, tên người dùng và mật khẩu vào các ô thích hợp, sau đó nhấn “Kết nối” (hoặc tùy chọn tương ứng). Sau khi kết nối thành công, màn hình sẽ hiển thị dấu nhắc lệnh (Dấu ~ : Đây là đường dẫn tại thư mục /home), tức là đã truy cập vào máy chủ. Hình 40 minh họa ví dụ về giao diện sau khi kết nối. Lưu ý rằng khi nhập mật khẩu, ký tự nhập vào sẽ không hiển thị trên màn hình, kể cả dấu “*”; do đó, cần nhập chính xác mật khẩu và nhấn **Enter** để tiến hành kết nối.



Hình 40: Ví dụ kết nối vào SSH thành công

FTP/SCP/SFTP

Một lý do quan trọng khác để kết nối với máy chủ là thực hiện truyền tệp giữa máy khách và máy chủ. Có ba giao thức phổ biến: FTP (File Transfer Protocol) - giao thức truyền tệp không bảo mật, hoặc SCP (Secure Copy) và SFTP (Secure FTP) – các lựa chọn bảo mật hơn. Trên hệ thống Linux hoặc Mac, các tiện ích này có thể được sử dụng trực tiếp thông qua dòng lệnh hoặc bảng điều khiển. Tuy nhiên, việc thao tác có thể trở nên phức tạp nếu chưa quen với hệ thống UNIX và các đường dẫn tệp.



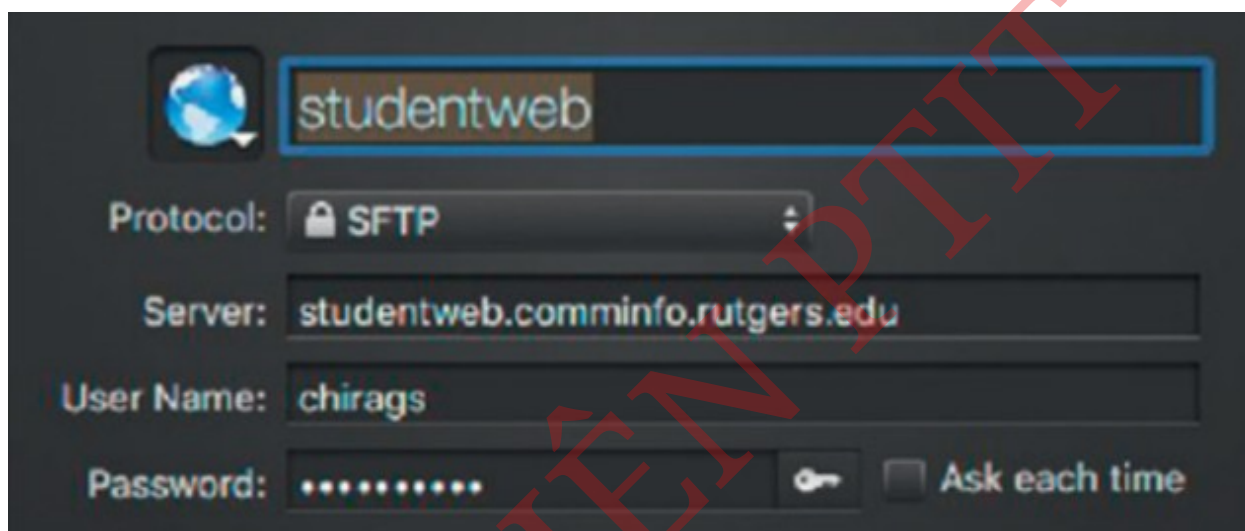
Hình 41: Truyền tệp bằng FileZilla trên Windows

Có thể đơn giản hóa quá trình truyền tệp bằng cách sử dụng các công cụ hỗ trợ trực quan thay vì thao tác hoàn toàn bằng dòng lệnh. Một phần mềm giúp đáp ứng nhu cầu trên là **FileZilla**, đây là một lựa chọn phổ biến, miễn phí và đa nền tảng. Ngoài ra, nhiều phần mềm tương tự khác cũng có sẵn trực tuyến, hầu hết đều cung cấp các chức năng gần như giống nhau.

Dù sử dụng công cụ nào, ba thông tin cần thiết để kết nối luôn bao gồm: **tên máy chủ**, **tên người dùng** và **mật khẩu**. Trên giao diện của FileZilla (Hình 41), các trường thông tin này thường nằm ở phía trên cùng với các nhãn “Máy chủ”, “Tên người dùng”,

“Mật khẩu” và “Cổng”. Trường “Cổng” có thể để trống, trừ khi quản trị viên hệ thống chỉ định cụ thể. Một ví dụ khác trong Hình 41 cho thấy giao diện của phần mềm FTP khác, tuy khác về thiết kế nhưng cũng yêu cầu cùng loại thông tin kết nối.

Sau khi nhập đúng thông tin và kết nối thành công, vị trí mặc định thường là thư mục gốc trên máy chủ. Đa số các ứng dụng truyền tệp cung cấp giao diện dạng hai ngăn: một ngăn hiển thị hệ thống tệp cục bộ và ngăn còn lại hiển thị hệ thống tệp của máy chủ. Từ đó, thao tác truyền tệp có thể thực hiện đơn giản bằng cách kéo và thả.



Hình 42: Kết nối với máy chủ FTP bằng ứng dụng Transmit trên máy Mac

1.8.2.3. Các lệnh cơ bản

Phần này tập trung vào một số lệnh UNIX thường dùng. Việc thực hành càng nhiều lệnh càng mang lại hiệu quả, đồng thời ghi nhớ những lệnh cơ bản sẽ giúp tiết kiệm đáng kể thời gian và công sức trong quá trình làm việc. Nội dung được trình bày dưới giả định rằng việc kết nối với máy chủ UNIX được thực hiện thông qua SSH. Ngoài ra, có thể lựa chọn cài đặt môi trường **Cygwin** trên PC Windows hoặc làm việc trực tiếp qua cửa sổ Terminal trên máy Linux và Mac. Đối với Linux hoặc Mac, chỉ cần mở **terminal** hoặc **console** để thao tác.

Các lệnh thao tác tệp và thư mục

Dưới đây là một số lệnh cơ bản thường dùng trong UNIX để thao tác với tệp và thư mục. Mỗi lệnh chỉ được giải thích ngắn gọn, việc thử nghiệm trực tiếp sẽ giúp hiểu rõ hơn. Trong các ví dụ, “filename” có thể được thay bằng tên tệp cụ thể, chẳng hạn như *test.txt*.

- **pwd**: Hiển thị thư mục làm việc hiện tại. Theo mặc định, sau khi đăng nhập vào máy chủ UNIX hoặc mở terminal, vị trí ban đầu là thư mục *home*. Từ đây có thể di chuyển đến thư mục khác bằng lệnh `cd`. Nếu không xác định được vị trí hiện tại, nhập `pwd` để nhận đường dẫn đầy đủ.
- **rm**: Xóa tệp (ví dụ: `rm filename`). Cần lưu ý rằng việc xóa này thường mang tính vĩnh viễn, không giống như thao tác “Thùng rác” trên máy tính cá nhân.
- **rmdir**: Xóa một thư mục rỗng (ví dụ: `rmdir myfolder`). Nếu thư mục không rỗng, lệnh sẽ không thực hiện được. Có thể sử dụng `rm -f myfolder` để buộc xóa, nhưng cần cẩn trọng.
- **cd**: Thay đổi thư mục (ví dụ: `cd data` để vào thư mục *data*). Nhập `cd` sẽ trở về thư mục *home*. Nhập `cd ..` sẽ di chuyển lên một cấp thư mục cha.
- **ls**: Liệt kê các tệp trong thư mục hiện tại. Sử dụng tùy chọn `-l` để hiển thị thêm chi tiết (ví dụ: `ls -l`).
- **du**: Kiểm tra dung lượng sử dụng của thư mục hoặc tệp. Lệnh `du` hiển thị dung lượng theo byte, trong khi `du -h` hiển thị theo đơn vị dễ đọc hơn như MB hoặc GB.
- **wc**: Thống kê kích thước tệp theo số dòng, số từ và số ký tự (ví dụ: `wc myfile.txt`).
- **cat**: Hiển thị nội dung tệp văn bản trên terminal (ví dụ: `cat myfile.txt`). Không nên dùng với tệp nhị phân vì có thể làm xuất hiện ký tự lạ, âm thanh bất thường hoặc gây treo hệ thống.
- **more**: Xem nội dung tệp văn bản theo từng trang màn hình (ví dụ: `more filename`). Sau khi hiển thị hết một màn hình, có thể nhấn Enter hoặc phím cách để tiếp tục.
- **head**: Hiển thị một số dòng đầu tiên của tệp (ví dụ: `head filename`). Có thể chỉ định số dòng cụ thể, chẳng hạn `head -3 filename` để xem ba dòng đầu.

Hình 43 cho thấy một số lệnh này đang chạy trong cửa sổ terminal.

```
keystone:data chirags$ rm edoc.txt-
keystone:data chirags$ mkdir myfolder
keystone:data chirags$ rm myfolder/
rm: myfolder/: is a directory
keystone:data chirags$ rmdir myfolder/
keystone:data chirags$ wc mydocument.txt
      16      40     207 mydocument.txt
keystone:data chirags$ cat mydocument.txt
Here is an example
of typing a document
in vi editor.

Currently I'm in 'insert' mode.
To save the file, type:

ESC (the escape key)
: (colon)
w (for write)

To quit, type:
ESC (the escape key)
: (colon)
q
```

Hình 43: Các lệnh mẫu chạy trên thiết bị đầu cuối.

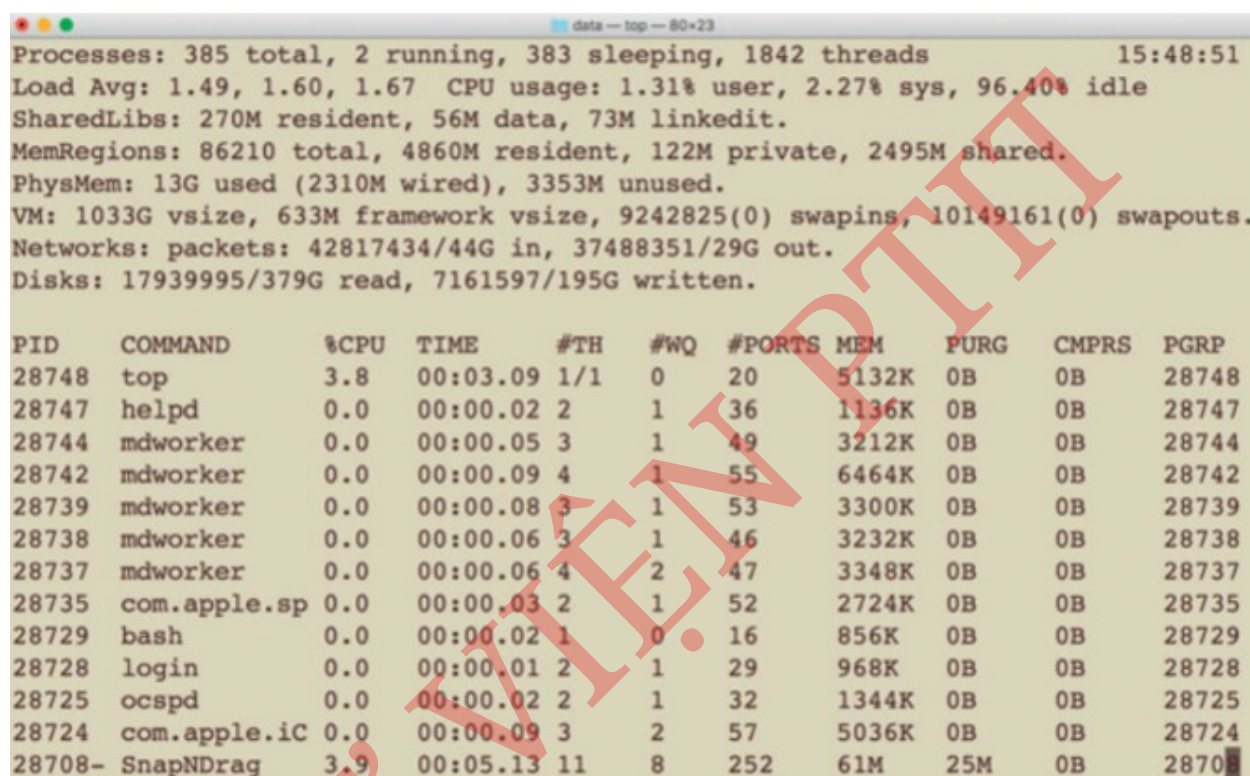
Các lệnh liên quan đến quy trình

Trong khi hầu hết các hệ điều hành che giấu các hoạt động nền phía sau giao diện đồ họa, UNIX cung cấp khả năng quan sát và điều khiển trực tiếp các quy trình. Dưới đây là một số lệnh cơ bản hữu ích để theo dõi và thao tác với các tiến trình:

- **Ctrl+c**: Dừng một quy trình đang chạy. Trong trường hợp một lệnh không trả lại dấu nhắc lệnh, tổ hợp phím này là cách nhanh nhất để thoát. Có thể cần nhấn nhiều lần nếu quy trình phản hồi chậm.
- **Ctrl+d**: Đăng xuất khỏi phiên làm việc hiện tại. Khi nhập tổ hợp phím này tại dấu nhắc lệnh, phiên sẽ kết thúc và trong nhiều trường hợp, cửa sổ terminal cũng sẽ đóng lại.
- **ps**: Liệt kê các quy trình đang chạy trong phiên làm việc hiện tại thông qua thiết bị đầu cuối.
- **ps aux**: Hiển thị danh sách tất cả các quy trình đang chạy trên hệ thống. Trong môi trường nhiều người dùng, lệnh này cho phép quan sát hoạt động của người dùng khác, đồng thời người khác cũng có thể theo dõi các tiến trình của mình.
- **ps aux | grep daffy**: Lọc danh sách quy trình theo người dùng, ví dụ với tên người dùng *daffy*. Tổ hợp này đặc biệt hữu ích để tìm các tiến trình liên quan khi số

lượng tiến trình toàn hệ thống quá lớn. Ký tự | (pipe) được sử dụng để chuyển kết quả từ lệnh đầu sang lệnh tiếp theo.

- **top**: Hiển thị danh sách tiến trình theo thời gian thực, sắp xếp theo mức độ tiêu thụ tài nguyên (Hình 44). Lệnh này hữu ích để xác định quy trình chiếm nhiều tài nguyên hệ thống. Trong kết quả, cần lưu ý đến cột **PID** (Process ID), biểu thị mã định danh của từng tiến trình.



```
Processes: 385 total, 2 running, 383 sleeping, 1842 threads      15:48:51
Load Avg: 1.49, 1.60, 1.67  CPU usage: 1.31% user, 2.27% sys, 96.40% idle
SharedLibs: 270M resident, 56M data, 73M linkedit.
MemRegions: 86210 total, 4860M resident, 122M private, 2495M shared.
PhysMem: 13G used (2310M wired), 3353M unused.
VM: 1033G vsize, 633M framework vsize, 9242825(0) swapins, 10149161(0) swapouts.
Networks: packets: 42817434/44G in, 37488351/29G out.
Disks: 17939995/379G read, 7161597/195G written.

PID    COMMAND      %CPU    TIME    #TH    #WQ    #PORTS  MEM    PURG    CMPRS    PGRP
28748  top           3.8     00:03.09 1/1     0      20      5132K  0B      0B      28748
28747  helpd        0.0     00:00.02 2        1      36      1136K  0B      0B      28747
28744  mdworker     0.0     00:00.05 3        1      49      3212K  0B      0B      28744
28742  mdworker     0.0     00:00.09 4        1      55      6464K  0B      0B      28742
28739  mdworker     0.0     00:00.08 3        1      53      3300K  0B      0B      28739
28738  mdworker     0.0     00:00.06 3        1      46      3232K  0B      0B      28738
28737  mdworker     0.0     00:00.06 4        2      47      3348K  0B      0B      28737
28735  com.apple.sp 0.0     00:00.03 2        1      52      2724K  0B      0B      28735
28729  bash         0.0     00:00.02 1        0      16      856K   0B      0B      28729
28728  login        0.0     00:00.01 2        1      29      968K   0B      0B      28728
28725  ocsdpd       0.0     00:00.02 2        1      32      1344K  0B      0B      28725
28724  com.apple.iC 0.0     00:00.09 3        2      57      5036K  0B      0B      28724
28708- SnapNDrag   3.9     00:05.13 11       8      252     61M    25M    0B      28708
```

Hình 44: Đầu ra của lệnh “top”.

Ở phía bên trái màn hình hiển thị của lệnh **top**, mỗi tiến trình được gán một mã định danh duy nhất gọi là **PID** (Process ID). PID đóng vai trò quan trọng khi cần chấm dứt một tiến trình cụ thể. Để thoát khỏi màn hình **top**, nhấn phím **q**.

- **kill**: Chấm dứt hoặc hủy bỏ một tiến trình đang chạy. Cú pháp: `kill -9 1234`

Trong đó:

- **-9**: tham số buộc tiến trình phải kết thúc ngay lập tức.
- **1234**: PID của tiến trình cần chấm dứt, có thể lấy từ cột thứ hai trong kết quả của lệnh **ps** hoặc cột đầu tiên trong kết quả của lệnh **top**.

Các lệnh hữu ích khác

- **man**: Truy cập trợ giúp và tài liệu hướng dẫn cho các lệnh UNIX. Ví dụ: `man pwd`. Lệnh này sẽ hiển thị trang hướng dẫn cho lệnh **pwd**, giúp tìm hiểu chi tiết về cú pháp, các tùy chọn và ví dụ minh họa. Tài liệu này rất phong phú và có thể cung cấp nhiều thông tin hơn bạn tưởng.
- **who**: Hiển thị danh sách người dùng đang đăng nhập vào hệ thống hoặc máy chủ tại thời điểm hiện tại. Thông tin này giúp nắm bắt hoạt động hiện tại trên máy chủ.

1.8.2.4. Phím tắt

Ngoài ra, UNIX còn cung cấp nhiều phím tắt hữu ích giúp thao tác trên terminal trở nên nhanh chóng và tiện lợi:

- **Tự động hoàn tất**: Khi nhập lệnh, tên tệp hoặc đường dẫn, chỉ cần nhập một phần và nhấn phím **Tab**. Hệ thống sẽ hoàn tất phần còn lại hoặc hiển thị các tùy chọn khả dĩ.
- **Gọi lại lệnh trước**: UNIX lưu lại lịch sử các lệnh đã sử dụng. Dùng phím mũi tên **lên** và **xuống** để duyệt và gọi lại các lệnh trước đó theo thứ tự thực thi.
- **Tìm kiếm và gọi lại lệnh**: Nhấn **Ctrl+R** và bắt đầu nhập một phần của lệnh. Hệ thống sẽ tìm kiếm trong lịch sử lệnh, và khi tìm thấy lệnh mong muốn, nhấn **Enter** để thực thi.
- **Thiết lập đường dẫn tự động**: Để chạy trực tiếp các chương trình hoặc tệp thường xuyên sử dụng mà không nhập đường dẫn đầy đủ, thực hiện các bước sau:
 - Chuyển đến thư mục **home** trên máy chủ.
 - Mở tệp `.bash_profile` trong trình soạn thảo (lưu ý dấu chấm trước tên tệp).
 - Giả sử chương trình muốn truy cập trực tiếp nằm tại `/home/user/daffy/MyProgram`. Thêm vào tệp dòng: `PATH=$PATH:$HOME/bin:/home/user/daffy/MyProgram`
 - Lưu tệp và thoát trình soạn thảo.
 - Trên dòng lệnh, chạy:


```
..bash_profile
```
- Thao tác này sẽ thiết lập môi trường đường dẫn cho phiên làm việc hiện tại. Trong các phiên làm việc sau, đường dẫn sẽ tự động được thiết lập khi đăng nhập. Khi đó, để chạy chương trình `/home/user/daffy/MyProgram`, chỉ cần nhập: `MyProgram`

Việc nắm vững các lệnh cơ bản trong UNIX có thể khó khăn nếu chưa từng sử dụng Linux hoặc UNIX trước đây. Không cần ghi nhớ tất cả ngay lập tức; thực hành và quay

lại các lệnh sẽ giúp ghi nhớ hiệu quả hơn. Việc thực hành các lệnh với mục tiêu cụ thể còn giúp hiểu cách các lệnh, quy trình và tổ hợp của chúng giải quyết các vấn đề dữ liệu.

1.8.3. Windows

Windows là phần mềm hệ điều hành của hãng Microsoft. Với giao diện đồ họa thông qua hệ thống thực đơn và các hộp hội thoại đa dạng, hệ điều hành Windows dễ sử dụng và tương đối dễ học. Các hệ thống máy vi tính hiện nay thường dùng hệ điều hành Windows (Việt Nam là một trường hợp). Đến nay, hãng Microsoft đã phát triển các hệ điều hành Windows3x, Windows9x, Windows 2000, Windows ME, Windows XP và Windows 2003.

Windows là hệ điều hành đa tác vụ, nghĩa là có thể thực hiện đồng thời nhiều công việc. Ví dụ, vừa nghe nhạc, vừa sử dụng phần mềm Excel để tính toán, hay phần mềm Winword để soạn thảo văn bản.

Ngoài ra hệ điều hành Windows còn có chức năng Plug and Play (cắm và chạy) tự động cài đặt các thiết bị được gắn thêm vào hệ thống, cũng như các tiện ích để nối kết mạng và Internet.

1.8.3.1. Khái niệm về ổ đĩa

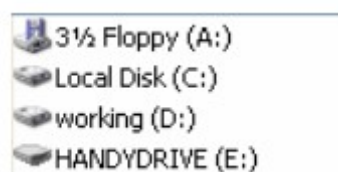
Để quản lý các ổ đĩa hệ thống máy tính thường đặt tên cho mỗi ổ đĩa bao gồm 1 ký tự chữ (bắt đầu từ ký tự “A”, và tiếp tục với các ký tự kế tiếp – thường do hệ thống tự đặt tên) và dấu hai chấm (:). Trong hệ điều hành Window ổ đĩa còn có nhãn (do người sử dụng hoặc máy tạo). Ổ đĩa trong Windows có 2 loại: Ổ đĩa cứng và ổ đĩa mềm. Để đọc các đĩa mềm, máy tính phải được gắn ổ đĩa tương ứng với loại đĩa. Một hệ thống máy tính có thể có tối đa 2 ổ đĩa mềm và dành 2 tên ổ đĩa A: và B: để đặt tên cho 2 ổ đĩa mềm này. Máy tính thường được gắn 1 đĩa cứng để tiện làm việc, tuy vậy trên các máy tính hiện nay có thể gắn được 4 đĩa cứng. Khi một đĩa cứng được gắn vào máy tính, nó sẽ được gán một tên gọi để phân biệt với các ổ đĩa khác. Ổ đĩa cứng được đặt tên từ ký tự C trở đi (C:), có thể có thêm các ổ đĩa D:, E:, ... Thường 1 máy tính PC có thể gắn 1-2 đĩa cứng vật lý, nhưng tên các ổ đĩa cứng có thể nhiều hơn vì trên 1 đĩa cứng chúng ta có thể phân thành nhiều vùng đĩa (partition), và mỗi vùng đĩa này được gọi là một đĩa “logical”, được đặt tên theo quy ước của ổ đĩa cứng.

Tương tự như ổ đĩa mềm, muốn đọc (và ghi) thông tin trên đĩa CD, máy tính phải có ổ đĩa tương ứng. Trên một máy tính, thường có thể gắn 1-2 ổ đĩa CD. Tên của ổ đĩa CD thường bắt đầu với chữ cái kế tiếp các chữ cái đặt tên cho đĩa cứng (và dấu :).

Ví dụ: Nếu tên các ổ đĩa cứng là C: hay D: thì tên của ổ đĩa CD thường sẽ là E:.

Ghi chú: Tên ổ đĩa, tên tập tin, tên thư mục trong hệ điều hành Windows hay DOS không phân biệt chữ thường và chữ hoa.

Ví dụ: Data (C:), Local Disk (D:), ...



Hình 45: Danh sách ổ đĩa

1.8.3.2. Khái niệm về tập tin

Tập tin là tập hợp các thông tin có liên quan với nhau, thường được lưu trữ trên bộ nhớ ngoài. Cụ thể tập tin chính là các chương trình, dữ liệu được lưu trữ trên đĩa. Để phân biệt giữa các tập tin với nhau, mỗi tập tin có một tên.

Tập tin có quy tắc đặt tên sau

$\langle \text{tên tập tin} \rangle \equiv \langle \text{tên chính} \rangle [\langle \text{mở rộng} \rangle]$

Trong đó phần tên chính (filename) là bắt buộc phải có, phần mở rộng (extension) có thể có hoặc không. Trong môi trường Windows, tên tập tin có thể dài đến 255 ký tự. Phần mở rộng, nếu có, bắt đầu từ dấu chấm cuối cùng tính từ trái qua phải. Tên tập tin không nên sử dụng các ký tự có dấu tiếng Việt, vì có thể không đọc được trên các máy tính khác. Lưu ý: Các ký tự không được dùng để đặt tên tập tin là / \ : * ? " | < >

Ví dụ: TONG KET HOC KY 1.DOC, BaoCaoThang012001.XLS

Phần mở rộng thường dùng để định loại tập tin (file type):

- Các tập tin có phần mở rộng là EXE, COM (trong Window thể hiện là kiểu Application) thường là các tập tin chương trình.
- Các tập tin có phần mở rộng là SYS thường là các tập tin hệ thống chứa các thông tin liên quan đến phần cứng, khai báo thiết bị.
- Các tập tin có phần mở rộng là DOC, TXT, HTM thường là các tập tin văn bản.
- Các tập tin có phần mở rộng là PAS, PRG, C, Java, ... là các tập tin chương trình nguồn của ngôn ngữ PASCAL, FOXPRO, C, Java, ...

Ghi chú: Trong môi trường DOS, phần tên chính chỉ có tối đa 8 ký tự và phần mở rộng tối đa là 3 ký tự đầu tiên. Ví dụ: BAITAP01.TXT, README.DOC,

CONFIG.SYS MS-DOS và Windows dành những tên sau để đặt tên cho một số thiết bị ngoại vi, không dùng những tên này đặt cho tên tập tin.

Tên dành riêng	Thiết bị
CON	Console (màn hình)
LPT1 (PRN)	Parallel Port 1 (Máy in song song 1)
LPT2, LPT3	Parallel Port 2,3 (Máy in song song 2, 3)
COM1 (AUX)	Serial Port 1 (Cổng nối tiếp 1)
COM2	Serial Port 2 (Cổng nối tiếp 2)
CLOCKS	Đồng hồ của máy
NUL	Thiết bị giả (Dummy device)

Hình 46: Các thiết bị trong Windows

Ký tự đặc biệt ? và *.

- Ký tự ?: là ký tự được hệ điều hành sử dụng để bao gồm tất cả các tập tin có tên tại vị trí ? là một ký tự bất kỳ, còn các vị trí khác trong tên tập tin phải đúng với yêu cầu.

Ví dụ: Có các tập tin BAITAP01.TXT, BAITAP02.TXT, BAITAP11.TXT,

BAIHOC01.TXT, BAIHOC02.DOC, BAIHOC20.DOC

Ký hiệu tập tin BAITAP0?.TXT sẽ bao gồm tập tin BAITAP01.TXT và BAITAP02.TXT.

Ký hiệu tập tin BAI???01.TXT sẽ bao gồm tập tin BAITAP01.TXT và BAIHOC01.TXT

- Ký tự * : là ký tự được hệ điều hành sử dụng để bao gồm tất cả các tập tin có tên bắt đầu tại vị trí dấu * trở về sau là các ký tự bất kỳ, còn các vị trí phía trước ký tự * trong tên tập tin phải đúng với yêu cầu.

Ví dụ : Với các tập tin trên, ký hiệu tập tin BAITAP*.TXT sẽ bao gồm các tập tin BAITAP01.TXT, BAITAP02.TXT, BAITAP11.TXT; ký hiệu tập tin BAI*.TXT sẽ bao gồm các tập tin BAITAP01.TXT, BAITAP02.TXT, BAITAP11.TXT, BAIHOC01.TXT; ký hiệu tập tin *.DOC sẽ bao gồm tất cả tập tin có phần mở rộng là DOC (BAIHOC02.DOC, BAIHOC20.DOC), và ký hiệu tập tin *.* sẽ bao gồm tất cả các tập tin trong thư mục tương ứng.

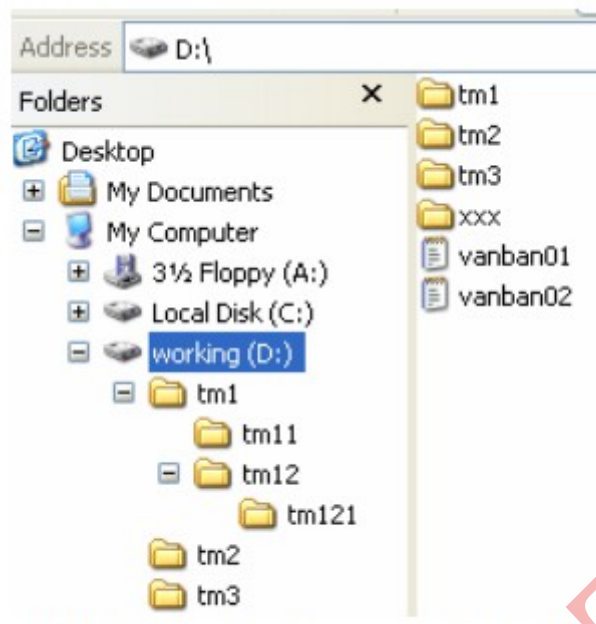
1.8.3.3. Khái niệm về thư mục

Thư mục là phân vùng hình thức trên đĩa để việc lưu trữ các tập tin được tổ chức một cách có hệ thống. Người sử dụng có thể phân một đĩa ra thành nhiều “ngăn” riêng biệt, mỗi ngăn là một thư mục. Trong một thư mục có thể chứa các tập tin hay / và thư mục con; các thư mục con hoặc mặc định theo hệ thống (ví dụ thư mục “COMMAND” trong thư mục WINDOWS”) hoặc tùy theo người sử dụng. Tên của thư mục (Folder/Directory Name) được đặt theo đúng quy luật đặt tên của tập tin, thường tên thư mục không đặt phần mở rộng.

Trong một thư mục, tên của các tập tin và thư mục con là duy nhất (không được giống nhau) Mỗi đĩa trên máy tương ứng với một thư mục và được gọi là thư mục gốc (Root Directory). Trên thư mục gốc có thể chứa các tập tin hay các thư mục con (Sub Directory). Các thư mục con này lại có thể chứa các tập tin và thư mục con khác, cấu trúc này được gọi là cây thư mục.

Trong cấu trúc cây thư mục của một hệ thống máy tính, có ba khái niệm cơ bản cần nắm rõ. Đầu tiên là **Thư mục Gốc (Root Directory)**, đây là thư mục **cao nhất** trong hệ thống phân cấp của một ổ đĩa hoặc phân vùng. Thư mục Gốc được tạo ra tự động trong quá trình định dạng đĩa và **không thể xóa bỏ**. Ký hiệu phổ biến của thư mục này là **C:** (hoặc **D:** v.v. đối với Windows) hay đơn giản là **/** (đối với Linux/macOS). Kế tiếp là **Thư mục Hiện Hành (Current Working Directory)**, là thư mục mà tại đó người dùng hoặc chương trình đang trực tiếp làm việc. Trong nhiều hệ thống dòng lệnh, thư mục hiện hành được ký hiệu bằng **một dấu chấm (.)**. Cuối cùng là **Thư mục Cha (Parent Directory)**, là thư mục nằm ở cấp ngay trên của thư mục hiện hành trong cấu trúc phân cấp. Thư mục Cha được ký hiệu bằng **hai dấu chấm (..)**, cho phép người dùng dễ dàng di chuyển lên một cấp trong hệ thống tệp.

Thư mục rỗng (Empty Directory) là thư mục trong đó không chứa tập tin hay thư mục con nào. Theo sơ đồ cấu trúc của cây thư mục (Hình 2.1), chúng ta thấy thư mục gốc D:\ có các thư mục tm1, tm2, tm3, xxx và tập tin VanBan01.txt, VanBan02.txt.

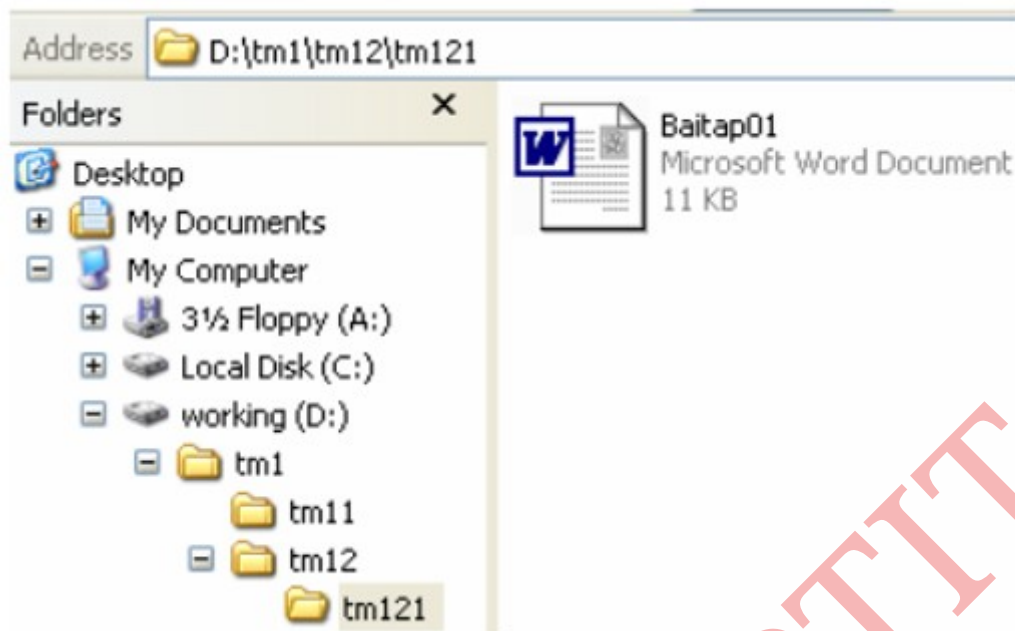


Hình 47: Cây thư mục trên ổ đĩa D

1.8.3.4. Khái niệm về đường dẫn

Chúng ta thường tổ chức các thư mục trên đĩa để việc quản lý các tập tin được dễ dàng hơn. Tuy nhiên khi đó người sử dụng muốn truy nhập đến một tập tin hay một thư mục con, không chỉ đơn giản là đưa ra tên của thư mục hay tập tin đó (vì có thể có nhiều tập tin hay thư mục con trùng tên trên các thư mục khác nhau) mà phải xác định thật rõ ràng vị trí của tập tin trên đĩa. Đường dẫn của một tập tin hay thư mục là vị trí của tập tin hoặc thư mục trong hệ thống ổ đĩa của máy tính đang sử dụng.

Ví dụ: Đường dẫn của tập tin BaiTap01.doc được xác định như sau (Hình 48):



Hình 48: Đường dẫn của tập tin BaiTap01.doc

Tên đường dẫn bao gồm có:

[<Tên ổ đĩa>:\][<Tên thư mục>\] <Tên tập tin>

Ví dụ: Tên tập tin có đường dẫn của tập tin **BaiTap01.doc** là :

D:\TM1\TM12\TM121\BaiTap01.doc

- Đường dẫn tuyệt đối (địa chỉ tuyệt đối): đường dẫn đầy đủ bao gồm từ tên ổ đĩa. Ví dụ **D:\TM1\TM12\TM121\BaiTap01.doc** là đường dẫn tuyệt đối của tập tin BaiTap01.doc.
- Đường dẫn tương đối (địa chỉ tương đối): đường dẫn của một đối tượng (thư mục hay tập tin) nhìn từ thư mục hiện hành. Ví dụ, nếu thư mục hiện hành là TM11, đường dẫn tương đối của tập tin BaiTap01.doc như sau:

..\TM12\TM121\BaiTap01.doc

1.9. Câu hỏi và bài tập Chương 2

Bài 1: Viết chương trình Python đọc file CSV chứa điểm sinh viên, tính điểm trung bình và xuất kết quả ra file mới.

Bài 2: (Câu hỏi lý thuyết) Trình bày sự khác biệt giữa ngôn ngữ Python và R trong phân tích dữ liệu.

Bài 3: Sử dụng ggplot2 trong R để vẽ biểu đồ cột thể hiện doanh thu theo tháng từ một dataset cho trước.

Bài 4: (Câu hỏi thực hành) Liệt kê 5 lệnh cơ bản trong Linux dùng để quản lý file và thư

mục.

Bài 5: (*Bài tập cơ sở dữ liệu*) Thiết kế schema cơ sở dữ liệu quan hệ cho một thư viện số, bao gồm các bảng: sách, tác giả, thể loại, mượn/trả.

Bài 6: (*Câu hỏi so sánh*) So sánh ưu điểm của cơ sở dữ liệu NoSQL (MongoDB) so với SQL (MySQL) trong lưu trữ dữ liệu phi cấu trúc.

Bài 7: (*Bài tập tích hợp*) Viết script Python kết nối với MySQL, thực hiện truy vấn SELECT và export kết quả ra file JSON.

Bài 8: (*Câu hỏi tình huống*) Khi nào nên sử dụng Windows Server thay vì Linux cho hệ thống lưu trữ dữ liệu doanh nghiệp?

Bài 9: Tìm hiểu và trình bày ngắn gọn về Docker và ứng dụng của nó trong triển khai các công cụ xử lý dữ liệu.

Bài 10: (*Câu hỏi tổng hợp*) Nếu phải xây dựng một hệ thống thu thập dữ liệu từ cảm biến IoT, bạn sẽ chọn công cụ nào trong chương 2? Giải thích.

Bài 11: (*Bài tập nhóm*) Xây dựng ứng dụng web đơn giản bằng Flask/Django để hiển thị dashboard phân tích dữ liệu.

Bài 12: (*Bài tập nhóm*) So sánh hiệu năng giữa Python (Pandas) và R (dplyr) trong xử lý dataset lớn (>1 triệu dòng).

Bài 13: (*Bài tập nhóm*) Thiết kế hệ thống cơ sở dữ liệu lai (Hybrid Database System) cho ứng dụng thương mại điện tử.

Bài 14: (*Bài tập nhóm*) Tự động hóa ETL pipeline sử dụng Apache Airflow hoặc Luigi.

Bài 15: (*Bài tập nhóm*) Nghiên cứu và triển khai container hóa ứng dụng dữ liệu với Docker và Docker Compose.

Bài 16: (*Bài tập nhóm*) Xây dựng RESTful API để truy vấn và cập nhật dữ liệu từ cơ sở dữ liệu.

Bài 17: (*Bài tập nhóm*) So sánh các hệ thống file phân tán: HDFS, Amazon S3, Google Cloud Storage.

Bài 18: (*Bài tập nhóm*) Triển khai hệ thống monitoring và logging cho data pipeline sử dụng ELK Stack.

Bài 19: (*Bài tập nhóm*) Nghiên cứu các công cụ Data Catalog và Metadata Management (Apache Atlas, Amundsen).

Bài 20: (*Bài tập nhóm*) Xây dựng CI/CD pipeline cho project khoa học dữ liệu sử dụng GitLab CI/GitHub Actions.

CHƯƠNG III: HỌC MÁY VÀ ỨNG DỤNG

1.10. Giới thiệu chung

Học máy được xác định là một phân nhánh của trí tuệ nhân tạo (Artificial Intelligence - AI) và được ứng dụng như một kỹ năng trọng tâm trong lĩnh vực khoa học dữ liệu. Mục tiêu cơ bản, theo định nghĩa của Arthur Samuel, là trang bị cho máy tính "khả năng học hỏi mà không cần được lập trình một cách tường minh." Tom Mitchell sau đó đã đưa ra một định nghĩa chính thức và có tính vận hành hơn: "Một chương trình máy tính được xem là học hỏi từ kinh nghiệm E đối với một lớp nhiệm vụ T và thước đo hiệu suất P nếu hiệu suất của nó tại các nhiệm vụ trong T, được đo bằng P, được cải thiện thông qua kinh nghiệm E."

Việc thiết lập bản chất của học máy dẫn đến các câu hỏi nền tảng về khái niệm học hỏi: Bản chất của sự học hỏi là gì? Và máy móc đang cố gắng học hỏi điều gì? Mặc dù đây là những vấn đề triết học sâu sắc, việc xem xét là cần thiết để làm rõ các giới hạn trong định nghĩa và ứng dụng thực tiễn của học máy. Theo *Từ điển New Oxford American Dictionary*, "học hỏi" bao gồm: tiếp thu kiến thức qua nghiên cứu/kinh nghiệm, trở nên nhận thức qua quan sát, ghi nhớ, hoặc nhận chỉ dẫn.

Tất cả những ý nghĩa này đều có những hạn chế khi gắn với máy tính hoặc máy móc.

Với hai ý nghĩa đầu tiên, gần như không thể kiểm chứng được việc học đã đạt được hay chưa. Làm sao có thể kiểm tra một cỗ máy đã thực sự thu nhận được kiến thức nào đó? Có lẽ không thể đặt câu hỏi cho nó; và ngay cả khi có thể, thì cũng chỉ đang kiểm tra khả năng trả lời câu hỏi của nó chứ không phải khả năng học tập. Làm sao bạn biết được máy tính đã trở nên nhận thức được điều gì nhất định? Vấn đề liệu máy tính có thể có ý thức hay không vẫn còn là một vấn đề triết học đang được quan tâm.

Đối với ba ý nghĩa còn lại, mặc dù có thể thấy rõ trong khía cạnh con người, thì chỉ đơn thuần ghi nhớ hay tiếp nhận chỉ dẫn dường như chưa đủ để gọi là học máy. Những điều này quá thụ động, và biết rằng các tác vụ này trở nên tầm thường đối với máy tính ngày nay. Thay vào đó, chúng ta quan tâm đến sự cải thiện trong hiệu suất, hoặc ít nhất là tiềm năng thực hiện trong các tình huống mới. Bạn có thể ghi nhớ hay được dạy dỗ theo kiểu học vẹt nhưng lại không thể áp dụng kiến thức đó vào tình huống mới. Nói cách khác, bạn có thể tiếp nhận chỉ dẫn mà không hề thu được lợi ích nào từ nó.

Vì vậy, điều quan trọng là phải đưa ra một định nghĩa vận hành mới về học tập trong bối cảnh của máy móc, điều này được diễn đạt như sau:

Máy (hoặc sự vật) được coi là học khi chúng thay đổi hành vi của mình nhằm thực hiện tốt hơn trong tương lai.

Điều này gắn khái niệm học tập với hiệu suất thay vì với tri thức. Ta có thể kiểm chứng việc học bằng cách quan sát hành vi hiện tại và so sánh nó với hành vi trong quá khứ. Đây là một định nghĩa mang tính khách quan hơn và phù hợp hơn với mục đích nghiên cứu. Dĩ nhiên, một định nghĩa toàn diện và chính thức hơn dựa trên ý tưởng này chính là định nghĩa mà Tom Mitchell đã đưa ra trước đó.

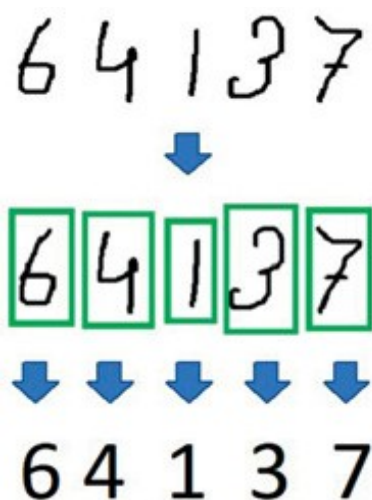
Trong bối cảnh định nghĩa này, học máy nghiên cứu việc sử dụng các thuật toán có khả năng học từ dữ liệu và vận dụng kiến thức đó để đưa ra dự đoán trên những dữ liệu chưa từng được thấy trước đây. Các thuật toán như vậy được thiết kế để vượt qua giới hạn của các chỉ dẫn lập trình tĩnh, bằng cách tạo ra các dự đoán hoặc quyết định dựa trên dữ liệu, thông qua việc xây dựng một mô hình từ những mẫu đầu vào. Mặc dù nhiều thuật toán học máy đã tồn tại từ lâu, nhưng khả năng tự động áp dụng các phép tính toán học phức tạp trên dữ liệu lớn (big data) một cách hiệu quả chỉ mới trở thành một bước tiến gần đây.

Một số ví dụ tiêu biểu và được truyền thông rộng rãi về ứng dụng của học máy có thể kể đến. Đầu tiên là **xe tự lái của Google** (nay được tái thương hiệu dưới tên **WAYMO**). Như minh họa trong Hình 49, chiếc xe này sử dụng dữ liệu hình ảnh thực tế của con đường để nhận diện các đối tượng và mô hình như bầu trời, biển báo giao thông hay các phương tiện đang di chuyển ở làn đường khác. Quá trình này vốn dĩ rất phức tạp đối với một cỗ máy. Rất nhiều vật thể có thể trông giống như một chiếc xe (ví dụ: khối màu xanh trong hình ảnh phía dưới chính là một chiếc xe), và việc xác định chính xác vị trí của một biển báo đường phố cũng không hề dễ dàng. Xe tự lái không chỉ cần thực hiện nhận diện đối tượng, mà còn phải ra quyết định điều hướng. Khối lượng các yếu tố bất định ở đây là quá lớn, khiến cho việc xây dựng một thuật toán (tập hợp các chỉ dẫn cố định) để chiếc xe thực hiện là bất khả thi. Thay vào đó, chiếc xe cần phải nắm được các quy tắc lái xe, đồng thời sở hữu khả năng nhận diện đối tượng và mô hình nhằm hỗ trợ ra quyết định trong môi trường phức tạp và biến động liên tục. Ngoài ra, còn phải luôn được cải tiến nhằm đảm bảo an toàn trong bất kì tình huống nào. Đây chính là vai trò của học máy được áp dụng.



Hình 49: Kỹ thuật học máy sử dụng trong xe tự lái

Một ví dụ kinh điển khác về học máy là nhận dạng ký tự quang học (OCR – Optical Character Recognition). Con người có khả năng nhận diện các ký tự viết tay khá tốt, nhưng máy tính thì không. Tại sao? Bởi vì có quá nhiều biến thể trong cách viết của cùng một ký tự, và không thể nào dạy máy tính tất cả những biến thể đó. Hơn nữa, còn có thể tồn tại nhiều – như một ký tự viết chưa hoàn chỉnh, các ký tự nối liền nhau, những chi tiết không liên quan ở nền, hay góc nhìn khi ký tự được quét, v.v. Vì vậy, một lần nữa, điều cần thiết là một tập hợp các quy tắc cơ bản để cho máy tính biết “A”, “a”, “5”, v.v. trông như thế nào, và sau đó để máy đưa ra quyết định dựa trên nhận dạng mẫu. Quá trình này diễn ra bằng cách cho máy tính xem nhiều phiên bản khác nhau của một ký tự, để nó “học” ký tự đó – tương tự như cách một đứa trẻ học thông qua lặp lại – và sau đó thực hiện quá trình nhận dạng Hình 50.



Hình 50: Vấn đề nhận biết kí tự

Hãy xét một ví dụ gần gũi hơn trong đời sống hàng ngày. Nếu đã từng sử dụng các dịch vụ trực tuyến, rất có thể bạn đã gặp phải các gợi ý đề xuất. Ví dụ điển hình là Amazon và Netflix. Làm sao các dịch vụ này biết được sản phẩm hay bộ phim nào nên gợi ý dựa theo sở thích của người dùng? Để hiểu rằng các dịch vụ này theo dõi hoạt động của người sử dụng, lưu trữ lịch sử đã xem hoặc đã mua, và từ đó đưa ra gợi ý. Nhưng chính xác thì bằng cách nào? Câu trả lời là chúng sử dụng một kỹ thuật có tên là lọc cộng tác (Collaborative Filtering – CF). Đây là một phương pháp sử dụng hành vi quá khứ và so sánh sự tương đồng với hành vi của những người dùng khác trong cộng đồng để dự đoán những gì bạn có thể sẽ thích trong tương lai.

Bảng 10: Bảng đánh giá phim của những người tham gia

Đánh giá	Sherlock	Avengers	Titani c	La La Land	Wall-E
Người số 1	4	5	3	4	2
Người số 2	3	2	3	4	4
Người số 3	4	3	4	5	3
Người số 4	3	4	4	5	2
Người số 5	4	?	4	?	4

Ví dụ, hãy quan sát Bảng 10, nơi lưu giữ dữ liệu về đánh giá phim của bốn người khác nhau. Nhiệm vụ của hệ thống trong trường hợp này là xác định xem Người số 5 có thích một bộ phim nào đó hay không, dựa trên dữ liệu này cũng như lịch sử sở thích

phim ảnh của cô ấy trong quá khứ. Nói cách khác, hệ thống đang cố gắng học xem Người 5 thích (hoặc không thích) những loại phim nào, những người tương đồng với Người 5 thích gì, và dùng kiến thức đó để tạo ra các gợi ý mới. Hơn nữa, khi Người 5 chấp nhận hoặc từ chối các gợi ý này, hệ thống sẽ tiếp tục mở rộng quá trình học của mình để bao gồm cả cách Người 5 phản hồi với các đề xuất, từ đó hiệu chỉnh lại mô hình.

Dưới đây là một vài ví dụ khác. Facebook sử dụng học máy để cá nhân hóa bảng tin (news feed) của từng thành viên. Hầu hết các tổ chức tài chính đều sử dụng các thuật toán học máy để phát hiện gian lận. Các cơ quan tình báo cũng áp dụng học máy để sàng lọc khối lượng lớn thông tin, nhằm tìm kiếm những mối đe dọa khủng bố có cơ sở.

Có rất nhiều ứng dụng khác mà chúng ta gặp trong đời sống hàng ngày, nơi học máy vận hành theo cách này hay cách khác. Thực tế, gần như không thể kết thúc một ngày mà không sử dụng đến một công nghệ được vận hành bởi học máy. Bạn có lướt web hoặc tìm kiếm trực tuyến hôm nay không? Bạn có đi siêu thị không? Bạn có sử dụng ứng dụng mạng xã hội trên điện thoại không? Nếu có, thì bạn đã sử dụng các ứng dụng của học máy.

Vậy, bạn đã bị thuyết phục rằng học máy là một lĩnh vực nghiên cứu vô cùng quan trọng chưa? Nếu câu trả lời là “có” và nếu đang băn khoăn điều gì cần có để xây dựng một hệ thống học máy tốt, thì danh sách các tiêu chí sau đây từ SAS có thể giúp ích:

- a. Khả năng chuẩn bị dữ liệu.
- b. Các thuật toán – cơ bản và nâng cao.
- c. Khả năng tự động hóa và quy trình lặp.
- d. Khả năng mở rộng (scalability).
- e. Mô hình tổ hợp (ensemble modeling).

Phần này sẽ tập trung chủ yếu vào tiêu chí thứ hai: thuật toán. Cụ thể hơn, phần này sẽ tìm hiểu một số kỹ thuật và thuật toán quan trọng nhất trong việc phát triển các ứng dụng học máy.

Cần lưu ý rằng, trong hầu hết các trường hợp, việc ứng dụng học máy gắn liền với việc ứng dụng phân tích thống kê. Do đó, cần nhớ rằng tồn tại sự khác biệt trong hệ thuật ngữ của hai lĩnh vực này:

- Trong học máy, *target* được gọi là *nhãn*.
- Trong thống kê, *target* được gọi là *biến phụ thuộc*.
- Một *biến* (*variable*) trong thống kê được gọi là *đặc trưng* (*feature*) trong học máy.
- Một *biến đổi* (*transformation*) trong thống kê được gọi là *feature creation* (tạo đặc trưng) trong học máy.

Các thuật toán học máy được phân loại thành các nhóm khác nhau, dựa trên kết quả mong muốn mà thuật toán cần đạt được. Các loại thuật toán phổ biến bao gồm:

a. **Học có giám sát (Supervised learning)**: khi biết nhãn thực của các mẫu huấn luyện được sử dụng để học đã được biết

b. **Học không giám sát (Unsupervised learning)**: khi nhãn thực (thậm chí không biết số lượng nhãn hoặc số lớp) từ các mẫu huấn luyện không được biết

c. **Học tăng cường (Reinforcement learning)**: khi muốn cung cấp phản hồi cho hệ thống dựa trên cách nó hoạt động với các mẫu huấn luyện.

Các phần tiếp theo sẽ lần lượt đi qua từng loại thuật toán, làm việc với các ví dụ cụ thể và áp dụng các công cụ, kỹ thuật khoa học dữ liệu.

1.11. Phân loại các kỹ thuật học máy

Học máy là một lĩnh vực cốt lõi của trí tuệ nhân tạo, trong đó các hệ thống máy tính học hỏi từ dữ liệu để cải thiện hiệu suất và đưa ra quyết định trong tương lai. Dựa trên cách tiếp cận và mục tiêu, các kỹ thuật học máy thường được phân loại thành ba nhóm chính: học có giám sát, học không giám sát và học tăng cường.

Học có giám sát là kỹ thuật phổ biến nhất, trong đó dữ liệu huấn luyện đã được gán nhãn thực sẵn. Mỗi mẫu dữ liệu đều đi kèm với một đầu ra đã được xác định, và nhiệm vụ của thuật toán là học mối quan hệ giữa đầu vào và đầu ra đó, từ đó dự đoán cho dữ liệu mới chưa từng thấy. Các thuật toán tiêu biểu của nhóm này bao gồm hồi quy tuyến tính, hồi quy logistic, cây quyết định, rừng ngẫu nhiên, máy vector hỗ trợ (SVM) và các mạng nơ-ron sâu (neural-deep learning). Ứng dụng của học có giám sát vô cùng đa dạng, chẳng hạn như dự đoán giá cổ phiếu, phân loại email spam hay chẩn đoán bệnh dựa trên dữ liệu y tế.

Học không giám sát được sử dụng khi dữ liệu không có nhãn. Trong trường hợp này, máy tính chỉ nhận được tập dữ liệu đầu vào và phải tự tìm ra các cấu trúc tiềm ẩn, chẳng hạn như nhóm, phân cụm hay các đặc trưng nổi bật. Các kỹ thuật điển hình bao gồm phân cụm K-means, phân cụm phân cấp, phân tích thành phần chính (PCA), t-SNE, và các mạng tự mã hóa (autoencoders). Học không giám sát thường được áp dụng trong phân khúc khách hàng, phát hiện bất thường trong giao dịch tài chính, hay nén dữ liệu hình ảnh và văn bản.

Thứ ba, học tăng cường lại dựa trên cơ chế học thông qua tương tác với môi trường. Ở đây, một tác nhân (agent) liên tục quan sát trạng thái, đưa ra hành động, và nhận về phần thưởng hoặc hình phạt từ môi trường. Mục tiêu là tối đa hóa phần thưởng tích lũy trong dài hạn. Các thuật toán phổ biến là Q-learning, Deep Q-Network (DQN), hay các phương pháp dựa trên chính sách (Policy Gradient, Actor-Critic). Nhờ đó, học tăng

cường đặc biệt phù hợp trong các ứng dụng yêu cầu ra quyết định tuần tự như xe tự lái, điều khiển robot, hệ thống trò chơi, hoặc tối ưu hóa hệ thống gợi ý.

Ngoài ba nhóm chính trên, còn có một số hướng tiếp cận mở rộng. Học bán giám sát (Semi-supervised Learning) kết hợp một lượng nhỏ dữ liệu có nhãn với lượng lớn dữ liệu không nhãn để đạt hiệu quả cao hơn trong bối cảnh dữ liệu nhãn khan hiếm. Học trực tuyến (Online Learning) cho phép mô hình liên tục cập nhật khi có dữ liệu mới, đặc biệt hữu ích trong môi trường dữ liệu lớn và biến động. Học chuyển giao (Transfer Learning) lại tập trung khai thác kiến thức từ một bài toán hoặc miền dữ liệu để áp dụng sang một bài toán khác, thường được sử dụng trong xử lý ngôn ngữ tự nhiên và thị giác máy tính.

Như vậy, sự đa dạng trong phân loại kỹ thuật học máy phản ánh sự đa dạng trong chính các bài toán thực tế. Nếu học có giám sát phù hợp cho các vấn đề dự đoán, học không giám sát thích hợp cho việc khám phá cấu trúc dữ liệu, thì học tăng cường lại mạnh trong môi trường động và tương tác. Các hướng lai ghép và mở rộng càng cho thấy sự linh hoạt của học máy trong việc giải quyết các thách thức của kỷ nguyên dữ liệu lớn và trí tuệ nhân tạo.

1.12. Học có giám sát

1.12.1. Giới thiệu chung

Học có giám sát là một nhánh cơ bản của Học máy, trong đó thuật toán học cách lập bản đồ (ánh xạ) từ dữ liệu đầu vào sang một đầu ra cụ thể, dựa trên các cặp ví dụ đầu vào-đầu ra đã được gán nhãn.

Cơ chế 'giám sát' diễn ra nhờ việc mô hình được huấn luyện bằng dữ liệu đã gán nhãn, nơi mỗi đầu vào đều đi kèm với kết quả đầu ra đúng đã biết trước, giúp thuật toán học được mối quan hệ chính xác. Các thuật toán học có giám sát sử dụng một tập dữ liệu huấn luyện gồm các hồ sơ quá khứ (ví dụ) để đưa ra dự đoán về tương lai. Ví dụ, giá của các xe ô tô hiện tại được sử dụng để dự đoán giá của các mẫu xe trong tương lai. Mỗi ví dụ được sử dụng để huấn luyện thuật toán đều được gán nhãn với giá trị mục tiêu (hay giá trị quan tâm) – trong trường hợp này là giá của xe ô tô. Mỗi thuật toán học có giám sát sẽ tìm kiếm các mẫu trong một dữ liệu huấn luyện. Các thuật toán này có thể sử dụng bất kỳ thông tin nào liên quan bao gồm doanh số bán hàng hiện tại của xe, các đề xuất tương tự từ các đối thủ hay nhận thức của người tiêu dùng về thương hiệu của nhà sản xuất. Mỗi thuật toán sẽ tìm kiếm các thông tin khác nhau, từ đó phát hiện ra các loại mẫu khác nhau. Sau khi thuật toán tìm ra các mẫu tốt nhất, các mẫu đó được sử dụng để đưa ra dự đoán cho dữ liệu thử nghiệm chưa được gán nhãn.

Học có giám sát được chia thành nhiều dạng. Trong đó, ba dạng thuật toán phổ biến nhất là hồi quy, phân loại và phát hiện bất thường. Chương này sẽ tập trung chủ yếu vào hồi quy và phân loại. Đúng như vậy, chúng ta đã đề cập đến hồi quy tuyến tính trong chương trước, nhưng đó là để dự đoán một biến liên tục như tuổi và thu nhập. Khi dự đoán các giá trị rời rạc, chúng ta cần sử dụng một dạng hồi quy khác – hồi quy logistic hoặc hồi quy softmax. Những phương pháp này thực chất là các dạng phân loại. Ngoài ra, chương này cũng đề cập các thuật toán phân loại phổ biến khác đang được sử dụng hiện nay.

1.12.2. Hồi quy Logistic

Một điều dễ nhận thấy từ các ví dụ trên về hồi quy tuyến tính chính là các biến kết quả ở dạng số. Từ đó, câu hỏi được đặt ra: Điều gì xảy ra khi biến kết quả không phải ở dưới dạng số? Ví dụ, một bộ dữ liệu thời tiết với các thuộc tính độ ẩm, nhiệt độ và tốc độ gió, mỗi thuộc tính này mô tả một khía cạnh của thời tiết trong ngày. Dựa trên những thông tin này, cần dự đoán liệu thời tiết trong ngày có phù hợp để chơi golf hay không. Trong trường hợp này, biến kết quả dự đoán ở dạng phân loại (“có” hoặc “không”). Để giải quyết vấn đề phân loại như vậy, một phương pháp được sử dụng là Hồi quy Logistic (Logistic Regression).

Hãy nghĩ về điều này theo cách chính thức. Trước đây, biến kết quả y của chúng ta là liên tục. Bây giờ, nó chỉ có hai giá trị có thể có (nhãn). Để đơn giản, chúng ta sẽ gọi những nhãn này là “1” và “0” (“có” và “không”). Nói cách khác:

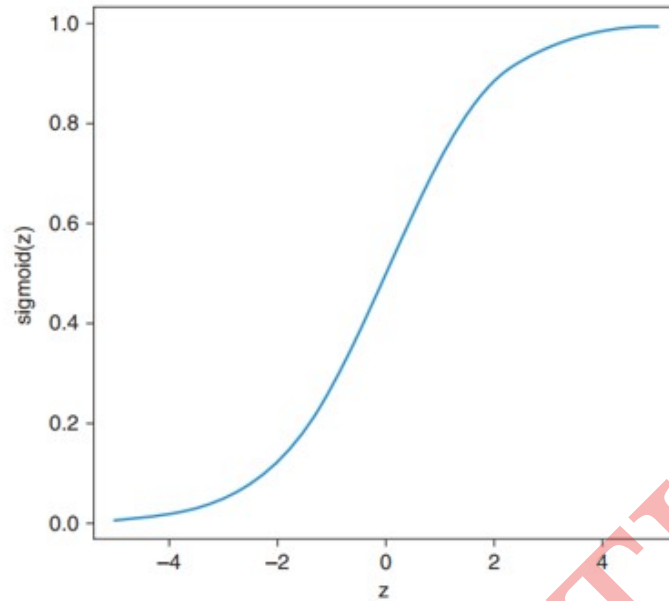
$$y \in \{0,1\}$$

Chúng ta vẫn sẽ có giá trị liên tục cho đầu vào, nhưng bây giờ chúng ta chỉ cần có hai giá trị có thể cho đầu ra. Làm thế nào để thực hiện điều này? Có một hàm tuyệt vời gọi là sigmoid, được định nghĩa trong biểu thức 9.1 như sau

$$g(z) = \frac{1}{1 + e^{-z}}$$

Như có thể thấy trong Hình 51, với bất kỳ đầu vào nào, đầu ra của hàm này luôn được giới hạn giữa 0 và 1, bao gồm cả 0 và 1.

$$h_{\theta}(x) \in [0,1]$$



Hình 51: Đồ thị hàm Sigmoid

Điều thú vị là hàm tuân theo các ràng buộc của một phân phối xác suất, nghĩa là giá trị của nó phải nằm trong khoảng $[0,1]$. Và nếu có thể tính toán một xác suất nằm trong khoảng $[0,1]$, chúng ta sẽ dễ dàng đặt một ngưỡng quyết định tại 0.5. Cụ thể, bất kỳ khi nào hàm giả thuyết h đưa ra giá trị lớn hơn ngưỡng này, ta sẽ phân loại kết quả đó vào lớp '1', nếu không sẽ được xếp vào lớp '0'

$$P(y = 1 | x; \theta) = h_{\theta}(x),$$

$$P(y = 0 | x; \theta) = 1 - h_{\theta}(x),$$

$$P(y | x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

Công thức cuối cùng là kết quả của việc kết hợp hai dòng đầu tiên để tạo thành một biểu thức duy nhất. Hãy xem liệu điều đó có hợp lý không. Thử thay $y = 1$ và $y = 0$ vào biểu thức 9.3 và xem liệu bạn có nhận được hai dòng trước đó không.

Bây giờ, làm thế nào để sử dụng biểu thức này cho phân loại? Về cơ bản cần đưa bất kỳ đặc trưng nào từ dữ liệu mà chúng ta có vào hàm giả thuyết (ở đây là một hàm sigmoid) và tìm ra giá trị đầu ra nằm trong khoảng từ 0 đến 1. Dựa trên việc giá trị đó nằm ở phía nào của ngưỡng quyết định (0.5), mô hình có thể xác định nhãn hoặc lớp phân loại phù hợp. Nhưng trước khi kiểm tra, chúng ta cần huấn luyện một mô hình. Để làm điều này cần một số dữ liệu. Một cách để xây dựng một mô hình từ dữ liệu là giả định một cấu trúc mô hình và đánh giá xem mô hình đó có thể giải thích hoặc phân loại dữ liệu huấn luyện với mức độ chính xác ra sao. Nói cách khác, chúng ta đang đo lường chất lượng của mô hình dựa trên dữ liệu sẵn có.

Để đánh giá độ 'tốt' của một mô hình (được đại diện bởi vector tham số θ), ta có thể tính toán xác suất mà dữ liệu hiện có được sinh ra bởi mô hình đã cho. Khái niệm này được gọi là Khả năng của Mô hình (Likelihood) và được ký hiệu là $L(\theta)$. Hãy mở rộng hàm khả năng này:

$$\begin{aligned} L(\theta) &= P(y = 1 | X; \theta) \\ &= \prod_{i=1}^m P(y^{(i)} | x^{(i)}; \theta) \\ &= \prod_{i=1}^m (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}} \end{aligned}$$

Để đạt được một mô hình tốt hơn so với mô hình chúng ta đã đoán, chúng ta cần tăng giá trị của $L(\theta)$. Nhưng, hàm trên có cả các phép nhân và lũy thừa. Vì vậy, để làm việc với hàm này dễ dàng hơn, chúng ta sẽ lấy logarit của hàm này. Tính chất của hàm logarit cho biết khi một hàm gọi là hàm tăng (increasing function), điều này có nghĩa khi x tăng, $\log(x)$ cũng tăng. Sử dụng logarit sẽ cho biểu thức như sau:

$$\begin{aligned} l(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

Một lần nữa, để đạt được mô hình tốt nhất, ta cần tối đa hóa hàm Khả năng này. Để thực hiện điều đó, cần tính đạo hàm riêng phần theo từng tham số. Để đơn giản hơn, chúng ta sẽ chỉ xem xét một mẫu dữ liệu tại một thời điểm.

$$\begin{aligned} \frac{\partial}{\partial \theta_j} l(\theta) &= \left(y \frac{1}{h_{\theta}(x)} - (1 - y) \frac{1}{1 - h_{\theta}(x)} \right) \frac{\partial}{\partial \theta_j} h_{\theta}(x) \\ &= [y(1 - h_{\theta}(x)) - (1 - y)h_{\theta}(x)] x_j \\ &= (y - h_{\theta}(x)) x_j \end{aligned}$$

Dòng thứ hai ở biểu thức trên cho biết đối với một hàm sigmoid $g(z)$, đạo hàm có thể được biểu diễn dưới dạng $g'(z) = g(z)(1 - g(z))$.

Xem xét tất cả các mẫu huấn luyện, chúng ta có:

$$\frac{\partial}{\partial \theta_j} l(\theta) = \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

Biểu thức trên cung cấp biểu thức sau:

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

Chú ý cách chúng ta đang cập nhật θ lần này. Chúng ta đang di chuyển lên theo độ dốc thay vì di chuyển xuống. Và đó là lý do tại sao đây được gọi là thuật toán tăng độ dốc (gradient ascent). Thuật toán này có vẻ tương đồng với thuật toán giảm độ dốc (Gradient Descent); tuy nhiên, điểm khác biệt then chốt nằm ở bản chất của hàm giả thuyết. Trước đây, hàm gradient tăng là một hàm tuyến tính. Bây giờ hàm này được gọi là hàm sigmoid hoặc logistic. Và vì lý do đó, hồi quy này được gọi là hồi quy logistic.

Ví dụ hồi quy Logistic.

Bộ dữ liệu Titanic có nhiều phiên bản khác nhau, trong đó, một số bộ này được cung cấp miễn phí trên mạng. Trong bài tập này, chúng ta sẽ cố gắng dự đoán khả năng sống sót của các hành khách trên tàu Titanic. Bộ dữ liệu này được tải tại đây: <https://www.kaggle.com/datasets/yasserh/titanic-dataset>

Sau khi có được bộ dữ liệu, đầu tiên hãy nhập bộ dữ liệu huấn luyện vào một dataframe trong R:

```
> titanic.data <- read.csv("train.csv", header = TRUE, sep = ",")
> View(titanic.data)
```

Sau đó cần kiểm tra có bao nhiêu giá trị thiếu và bao nhiêu giá trị duy nhất với từng trường dữ liệu, sử dụng hàm sapply()

Đầu tiên kiểm tra giá trị bị thiếu ở mỗi trường dữ liệu

```
> sapply(titanic.data, function(x) sum(is.na(x)))
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	0	0	177	0	0	0	0	0	0	0

Sau đó kiểm tra số lượng dữ liệu duy nhất ở mỗi trường

```
> sapply(titanic.data, function(x) length(unique(x)))
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
891	2	3	891	2	89	7	7	681	248	148	4

Xuất sang Trang tính

Một cách khác để ước lượng giá trị bị thiếu là sử dụng gói trực quan hóa: gói "Amelia" có một hàm vẽ đồ thị có tên missmap() phục vụ cho mục đích này. Gói này sẽ vẽ bộ dữ liệu và làm nổi bật các giá trị bị thiếu.

```
> library(Amelia)
```

Loading required package: Rcpp

##

Amelia II: Multiple Imputation

(Version 1.7.4, built: 2015-12-05)

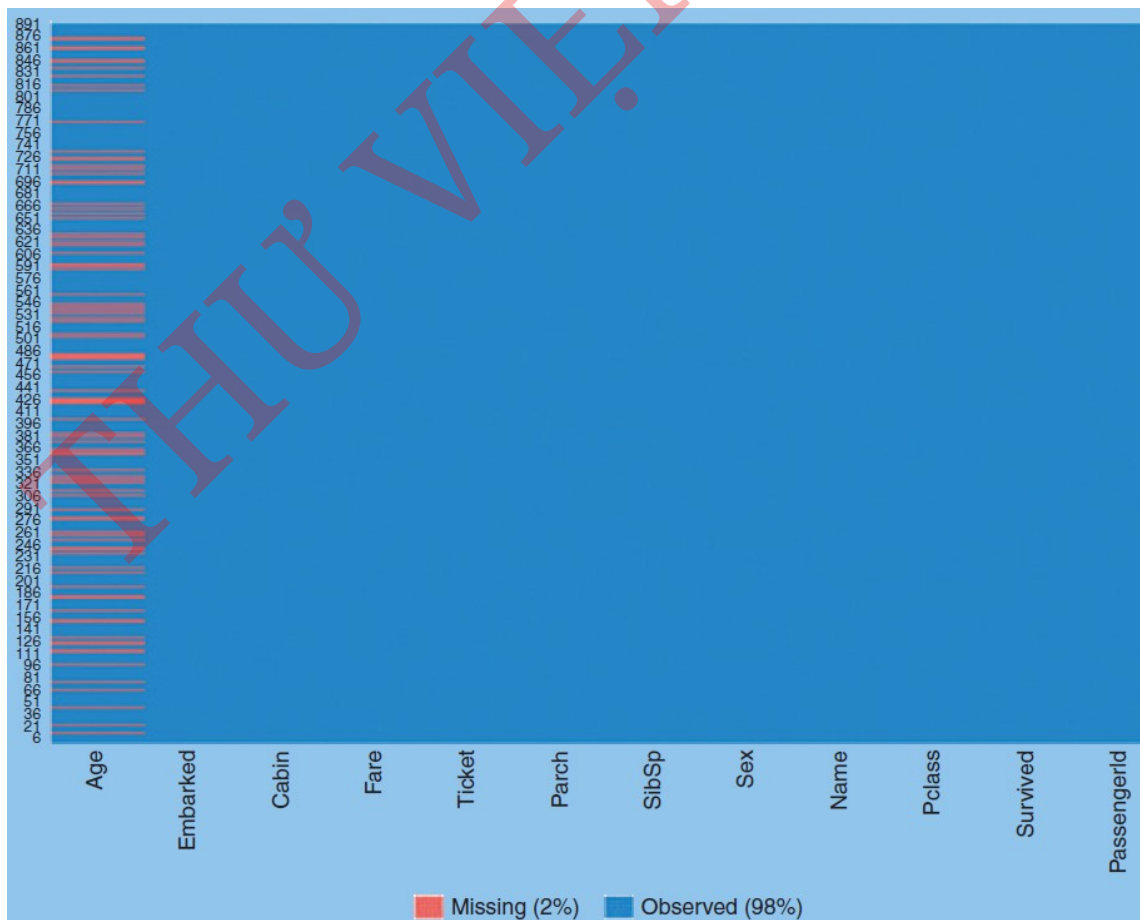
Copyright (C) 2005-2017 James Honaker, Gary King and
Matthew Blackwell

Refer to <http://gking.harvard.edu/amelia/> for more information

>missmap(titanic.data,main="Missingvaluesvsobserved")

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	1	0	3 Braund, Mr. Owen Harris	male	22.00	1	0	A/5 21171	7.2500		S
2	2	1	1 Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38.00	1	0	PC 17599	71.2833	C85	C
3	3	1	3 Heikkinen, Miss. Laina	female	26.00	0	0	STON/O2. 3101282	7.9250		S
4	4	1	1 Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.00	1	0	113803	53.1000	C123	S
5	5	0	3 Allen, Mr. William Henry	male	35.00	0	0	373450	8.0500		S
6	6	0	3 Moran, Mr. James	male	NA	0	0	330877	8.4583	Q	S
7	7	0	1 McCarthy, Mr. Timothy J	male	54.00	0	0	17463	51.8625	E46	S
8	8	0	3 Palsson, Master. Gosta Leonard	male	2.00	3	1	349909	21.0750		S
9	9	1	3 Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.00	0	2	347742	11.1333		S
10	10	1	2 Nasser, Mrs. Nicholas (Adele Achem)	female	14.00	1	0	237736	30.0708		C
11	11	1	3 Sandstrom, Miss. Marguerite Rut	female	4.00	1	1	PP 9549	16.7000	G6	S
12	12	1	1 Bonnell, Miss. Elizabeth	female	58.00	0	0	113783	26.5500	C103	S
13	13	0	3 Saendercock, Mr. William Henry	male	20.00	0	0	A/5 2151	8.0500		S
14	14	0	3 Andersson, Mr. Anders Johan	male	39.00	1	5	347082	31.2750		S
15	15	0	3 Vestrom, Miss. Hulda Amanda Adolfina	female	14.00	0	0	350406	7.8542		S

Hình 52: Bộ dữ liệu mẫu Titanic



Hình 53: Mô hình hóa dữ liệu bị thiếu

Như trên Hình 53 đã chỉ ra, cột tuổi (Age) có rất nhiều giá trị bị thiếu. Vì vậy, chúng ta phải xử lý các giá trị bị thiếu trước khi tiếp tục. Trong chương trước đã đề cập nhiều phương pháp để thực hiện việc làm sạch dữ liệu như vậy. Tương tự, các giá trị bị thiếu được trong cột tuổi thay thế bằng giá trị trung bình. Đây là cách thực hiện:

```
> titanic.data$Age[is.na(titanic.data$Age)] <- mean (titanic.data$Age,na.rm=T)
```

Ở đây, chúng ta đã thay thế các giá trị bị độ tuổi bị thiếu bằng độ tuổi trung bình. Nếu bất kỳ cột nào có một số lượng giá trị bị thiếu lớn, có thể cân nhắc loại bỏ cột đó hoàn toàn. Mục đích của bài tập này chỉ sử dụng các cột Age, Embarked, Fare, Ticket, Parch, SibSp, Sex, Pclass và Survived để đơn giản hóa mô hình.

```
> titanic.data <- subset(titanic.data, select=c(2,3,5,6,7,8,10,12))
```

Đối với các biến phân loại trong bộ dữ liệu, việc sử dụng hàm read.table() hoặc read.csv() mặc định sẽ mã hóa các biến phân loại dưới dạng các yếu tố (factors). Trong R, yếu tố là cách xử lý các biến phân loại.

Việc mã hóa thành các yếu tố được kiểm tra bằng cách sử dụng hàm is.factor(), hàm này sẽ trả về "true" cho tất cả các biến phân loại.

```
> is.factor(titanic.data$Sex)
```

```
[1] TRUE
```

Bây giờ cần chia bộ dữ liệu thành tập huấn luyện và tập kiểm tra trước khi huấn luyện mô hình. Ở đây, 800 mẫu được sử dụng với mục tiêu huấn luyện và 91 mẫu còn lại làm mẫu kiểm tra. Ngoài ra, có thể chọn các chiến lược phân chia khác nhau trong bộ dữ liệu này

```
> train <- titanic.data[1:800,]
```

```
> test <- titanic.data[801:891,]
```

Bây giờ bộ dữ liệu đã sẵn sàng để xây dựng mô hình. Chúng ta sẽ sử dụng tham số *family=binomial* (hai lớp hoặc nhị phân) trong hàm glm():

```
> model <- glm(Survived ~.,family=binomial(link='logit'), data=train)
```

```
> summary(model)
```

Các dòng mã trên sẽ tạo ra các kết quả sau:

Call:

```
glm(formula = Survived ~ ., family = binomial(link = "logit"), data = train)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.6059	-0.5933	-0.4250	0.6215	2.4148

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	15.947761	535.411378	0.030	0.9762
Pclass	-1.087576	0.151088	-7.198	6.10e-13***
Sexmale	-2.754348	0.212018	-12.991	<2e-16***
Age	-0.037244	0.008192	-4.547	5.45e-06***
SibSp	-0.293478	0.114660	-2.560	0.0105*
Parch	-0.116828	0.128113	-0.912	0.3618
Fare	0.001515	0.002352	0.644	0.5196
EmbarkedC	-10.810682	535.411254	-0.020	0.9839
EmbarkedQ	-10.812679	535.411320	-0.020	0.9839
EmbarkedS	-11.126837	535.411235	-0.021	0.9834

Signif. codes: ***, 0; **,0.001; *, 0.01.

Dispersion parameter for binomial family taken to be 1

Null deviance: 1066.33 on 799 degrees of freedom.

Residual deviance: 708.93 on 790 degrees of freedom.

242 Supervised Learning

AIC: 728.93.

Number of Fisher scoring iterations: 12.

Từ kết quả, rõ ràng là Fare và Embarked không có ý nghĩa thống kê do xấp xỉ 0 hoặc sai số chuẩn quá lớn (535). Điều này mang ý nghĩa rằng không có đủ căn cứ cho phép những yếu tố này đóng góp nhiều vào mô hình tổng thể. Còn đối với các biến có ý nghĩa thống kê, Sex có giá trị p thấp nhất, cho thấy có sự liên kết mạnh mẽ giữa giới tính với xác suất sống sót. Hệ số âm của yếu tố này cho thấy, với các biến khác không đổi, hành khách nam có khả năng sống sót thấp hơn. Lúc này, chúng ta nên dừng lại và suy ngẫm về nhận định này. Khi Titanic bắt đầu chìm và các xuồng cứu sinh được lấp đầy bằng những hành khách được cứu, ưu tiên đã được dành cho phụ nữ và trẻ em. Vì vậy, điều này hợp lý khi cho rằng hành khách nam, đặc biệt là nam giới trưởng thành, có ít cơ hội sống sót hơn.

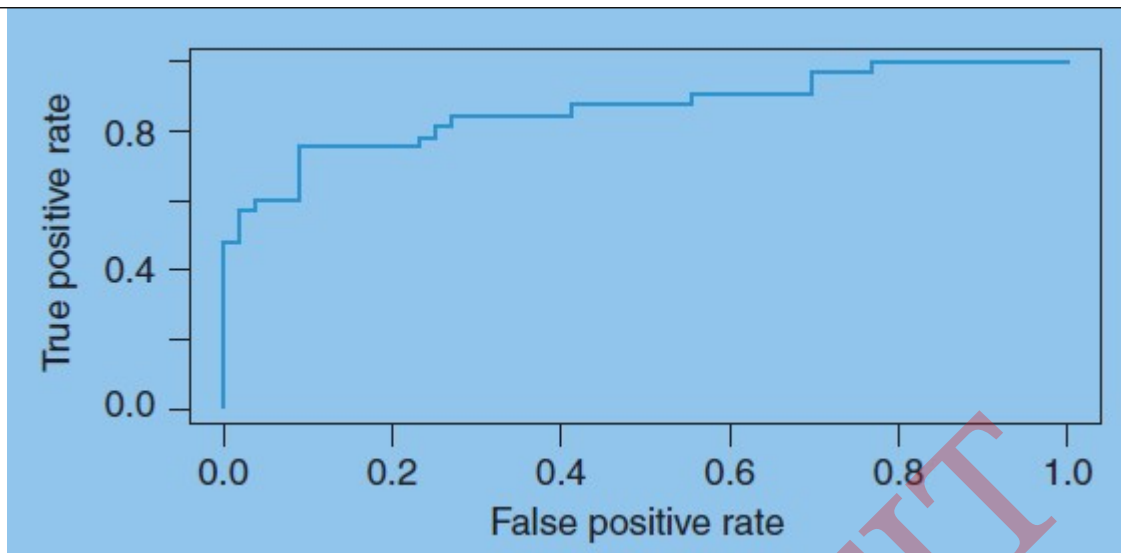
Bây giờ, chúng ta sẽ xem mô hình của mình dự đoán các giá trị cho các mẫu kiểm

tra tốt như thế nào. Bằng cách thiết lập tham số `type="response"`, R sẽ xuất ra xác suất dưới dạng $P(y = 1 | X)$. Ranh giới quyết định là 0.5. Nếu $P(y = 1 | X) > 0.5$ thì $y = 1$, nếu không thì $y = 0$.

```
> fitted.results <-  
predict(model,newdata=subset(test,select=c  
(2,3,4,5,6,7,8)), type='response')  
> fitted.results <- ifelse(fitted.results > 0.5,1,0)  
>  
> misClasificError <- mean(fitted.results != test$Survived)  
> print(paste('Accuracy',1-misClasificError))  
[1] "Accuracy 0.842696629213483"
```

Như chúng ta thấy từ kết quả trên, độ chính xác của mô hình trong việc dự đoán nhãn của các mẫu kiểm tra là 0.84, điều này cho thấy mô hình hoạt động khá tốt. Ở bước cuối cùng, chúng ta sẽ vẽ đường cong ROC và tính toán diện tích dưới đường cong (AUC), đây là những chỉ số hiệu suất điển hình cho một bộ phân loại nhị phân (Hình 54).

```
> library(ROCR)  
Loading required package: gplots  
Attaching package: 'gplots'  
The following object is masked from 'package:stats': lowess  
> p <- predict(model, newdata=subset(test,select=c  
(2,3,4,5,6,7,8)), type="response")  
> pr <- prediction(p, test$Survived)  
> prf <- performance(pr, measure = "tpr", x.measure = "fpr")  
> plot(prf)  
> auc <- performance(pr, measure = "auc")
```



Hình 54: ROC

```
>auc <- auc@y.values[[1]]> auc
[1] 0.866342
```

Đường cong ROC được tạo ra bằng cách vẽ tỷ lệ dương thực tế (TPR) so với tỷ lệ dương tính giả (FPR) tại các cài đặt ngưỡng khác nhau, trong khi AUC là diện tích dưới đường cong ROC. TPR chỉ ra mức độ chính xác được phát hiện là “1” thực sự là “1,” và FPR chỉ ra mức độ chính xác của những gì chúng ta phát hiện là “1” nhưng thực tế lại là “0”. Nếu một mô hình có khả năng dự đoán tốt nên có chỉ số AUC gần với 1 hơn là 0.5. Trong Hình 54, có thể thấy diện tích dưới đường cong khá lớn – chiếm khoảng 87% diện tích hình chữ nhật. Đây là một chỉ số hiệu suất rất tốt, cho thấy bộ phân loại này có độ chính xác cao và cân bằng.

1.12.3. Hồi quy Softmax

Cho đến phần này, chúng ta đã xem xét các mô hình hồi quy cho biến mục tiêu liên tục (numerical), cũng như hồi quy cho biến mục tiêu nhị phân ('có' hoặc 'không', '1' hoặc '0'). Nhưng điều gì sẽ xảy ra nếu có nhiều hơn hai lớp cần phân loại? Ví dụ trong trường hợp cần đánh giá hiệu suất của một học sinh dựa trên điểm số trong từng môn học là “xuất sắc”, “tốt”, “trung bình” hoặc “dưới trung bình”. Do đó cần phải sử dụng hồi quy logistic đa thức (multinomial logistic regression) cho trường hợp này. Hồi quy logistic đa thức hay hồi quy softmax là sự mở rộng của hồi quy logistic thông thường để xử lý nhiều (hơn hai) lớp.

Trong hồi quy softmax, hàm sigmoid được thay thế bằng hàm softmax. Hàm này nhận một vector gồm n số thực làm đầu vào và chuẩn hóa vector này thành một phân phối các xác suất n . Cụ thể, hàm này chuyển tất cả các thành phần n từ bất kỳ giá trị thực nào (dương hoặc âm) thành các giá trị trong khoảng $(0, 1)$.

Ví dụ

Chúng ta sẽ tìm hiểu hồi quy softmax thông qua một ví dụ trong R. Trong ví dụ này, bộ dữ liệu được sử dụng là “hsbdemo”. Bộ dữ liệu này được tải tại <https://stats.idre.ucla.edu/stat/data/hsb2.csv> và trên Kaggle <https://www.kaggle.com/code/saurabhbagchi/multinomial-logistic-regression-for-beginners>. Bộ dữ liệu này đề cập các học sinh vào cấp ba lựa chọn chương trình học nào giữa các chương trình chung, chương trình nghề nghiệp và chương trình học thuật. Lựa chọn của sinh viên được mô hình hóa dựa trên điểm viết và trạng thái kinh tế xã hội. Bộ dữ liệu chứa các giá trị thuộc tính của 200 học sinh. Biến độc lập ở đây là “prog” - loại chương trình mà học sinh chọn. Các biến phụ thuộc là tình trạng kinh tế xã hội (ses) là một biến phân loại ba lớp, và điểm viết (write) một biến liên phụ thuộc liên tục

Bắt đầu bằng cách lấy một số thống kê mô tả về các biến quan tâm.

```
> hsbdemo <- read.csv("hsbdemo.csv", header = TRUE, sep = ",")
```

```
> View(hsbdemo)
```

```
> with(hsbdemo, table(ses, prog))
```

prog

ses	academic	general	vocation
high	42	9	7
low	19	16	12
middle	44	20	31

```
> with(hsbdemo, do.call(rbind, tapply(write, prog, function (x) c(M = mean(x), SD = sd(x)))))
```

	M	SD
academic	56.257147	.943343
general	51.333339	.397775
vocation	46.760009	.318754

Có nhiều thư viện trong R có thể thực hiện hồi quy logistic đa thức. Ví dụ, có thể sử dụng gói “mlogit” để thực hiện hồi quy logistic đa thức. Tuy nhiên, đối với ví dụ này,

nếu muốn sử dụng gói “mlogit” thì phải chuyển đổi lại bộ dữ liệu trước. Một cách giải quyết khả thi cho bước tiền xử lý này là sử dụng hàm “multinom” từ gói “nnet” để ước lượng mô hình hồi quy logistic đa thức mà không yêu cầu bất kỳ việc chuyển đổi dữ liệu nào.

Trước khi chạy hồi quy đa thức, cần nhớ rằng biến kết quả không phải là thứ tự (ví dụ: “tốt,” “tốt hơn,” và “tốt nhất”). Vì vậy, để tạo mô hình, chọn mức của biến kết quả sử dụng làm cơ sở và chỉ định điều này trong hàm relevel. Đây là cách thực hiện:

```
> hsbdemo$prog2 <- relevel(hsbdemo$prog, ref = “academic”)
```

Ở đây, thay vì chuyển đổi biến độc lập “prog”, chúng ta đã khai báo một biến khác là “prog2” sử dụng hàm relevel, trong đó “academic” được khai báo làm cơ sở.

```
> library(nnet)
```

```
> model1 <- multinom(prog2 ~ ses + write, data = hsbdemo)
```

```
# weights: 15 (8 variable)
```

```
initial value 219.722458
```

```
iter 10 value 179.983731
```

```
final value 179.981726
```

```
Converged
```

Bỏ qua tất cả các thông báo cảnh báo. Như đã thấy, mô hình trong đó biến kết quả là “prog2” đã được xây dựng. Với mục đích minh họa, chúng ta chỉ sử dụng “ses” và “write” làm các biến phụ thuộc và bỏ qua các biến còn lại. Mô hình đã tạo ra một số đầu ra mặc dù chúng ta đang gán mô hình cho một đối tượng R mới. Kết quả của quá trình chạy mô hình này bao gồm các lịch sử chạy vòng lặp và bao gồm giá trị log-likelihood âm cuối cùng, 179.981726.

Tiếp theo, để khám phá thêm chi tiết về mô hình đã xây dựng cho đến nay, có thể thực hiện lệnh summary() trên mô hình của mình:

```
> summary(model1)
```

Call:

```
multinom(formula = prog2 ~ ses + write, data = hsbdemo)
```

Coefficients:

	(Intercept)	seslow	sesmiddle	write
general	1.689478	1.1628411	0.6295638	-
				0.05793086
vocation	4.235574	0.9827182	1.2740985	-

	(Intercept)	seslow	sesmiddle	write
				0.11360389
Std. Errors:				
	(Intercept)	seslow	sesmiddle	write
general	1.226939	0.5142211	0.4650289	0.02141101
vocation	1.204690	0.5955688	0.5111119	0.02222000
Residual Deviance: 359.9635				
AIC: 375.9635				

Kết quả tóm tắt được tạo ra bởi mô hình bao gồm hai khối: khối hệ số và khối sai số chuẩn. Mỗi khối này có một hàng giá trị tương ứng với một phương trình mô hình. Trước tiên với khối hệ số: hàng đầu tiên so sánh prog=general với cơ sở prog=academic. Tương tự, hàng thứ hai đại diện cho sự so sánh giữa prog=vocation và cơ sở.

Khai báo các hệ số từ hàng đầu tiên là “b₁” (b₁₀ cho hệ số chặn (intercept), b₁₁ cho “seslow,” b₁₂ cho “sesmiddle,” và b₁₃ cho “write”) và các hệ số từ hàng thứ hai là “b₂” (b₂₀ cho hệ số chặn, b₂₁ cho “seslow,” b₂₂ cho “sesmiddle,” và b₂₃ cho “write”). Sử dụng các hệ số này có thể tính toán log odds. Từ đó có thể so sánh mô hình đang xét với mô hình cơ sở và định lượng mức độ thay đổi của biến phụ thuộc khi biến độc lập tăng/giảm một đơn vị, so với sự thay đổi tương ứng trong mô hình cơ sở. Log odds là một khái niệm quan trọng trong thống kê, đặc biệt trong các mô hình hồi quy logistic, giúp mô tả tỷ lệ của một sự kiện xảy ra so với không xảy ra dưới dạng logarit. Các phương trình mô hình này có thể được viết như sau:

$$\ln\left(\frac{P(\text{prog2} = \text{general})}{P(\text{prog2} = \text{academic})}\right) = b_{10} + b_{11}(\text{ses} = 2) + b_{12}(\text{ses} = 3) + b_{13}(\text{write})$$

và

$$\ln\left(\frac{P(\text{prog2} = \text{vocation})}{P(\text{prog2} = \text{academic})}\right) = b_{20} + b_{21}(\text{ses} = 2) + b_{22}(\text{ses} = 3) + b_{23}(\text{write})$$

Sử dụng các phương trình này, chúng ta có thể nhận thấy rằng sự gia tăng một đơn vị trong biến “write” liên quan đến việc giảm log odds của việc chọn chương trình “general” so với chương trình “academic” với mức độ là 0.058.

Trong loại phân tích này, một thuật ngữ thường gặp là Rủi ro Tương đối (Relative Risk). Đây là Tỷ lệ Odds (Odds Ratio) giữa xác suất chọn bất kỳ cấp độ kết quả nào (ví dụ: 'vocation', 'general') so với xác suất của cấp độ cơ sở (baseline). Để tính toán Tỷ lệ Odds, có thể lấy lũy thừa tự nhiên của các hệ số (β) từ mô hình


```
> exp(coef(model1))
```

	(Intercept)	seslow	sesmiddle	write
general	5.416653	3.199009	1.876792	0.9437152
vocation	69.101326	2.671709	3.575477	0.8926115

Như đã thấy, tỷ lệ rủi ro tương đối cho sự gia tăng một đơn vị trong biến "write" là 0.9437 đối với việc chọn chương trình "general" so với chương trình "academic".

Trước đây, đã thấy rằng cần có một số mẫu kiểm tra để đánh giá độ chính xác của mô hình. Ngoài ra, trong trường hợp không có sẵn mẫu kiểm tra, cách chia toàn bộ dữ liệu thành các mẫu huấn luyện và kiểm tra cũng đã được sử dụng để chia tập dữ liệu

1.12.4. Phân loại với kNN

Các phần trên đã đề cập đến hai dạng hồi quy thực hiện một nhiệm vụ: phân loại. Phần này sẽ tiếp tục và xem xét các kỹ thuật khác để thực hiện phân loại. Nhiệm vụ phân loại là: học cách gán nhãn cho một tập hợp các điểm dữ liệu mới dựa trên mô hình đã được huấn luyện bằng tập dữ liệu và nhãn tương ứng có sẵn, qua đó đưa điểm dữ liệu mới vào đúng lớp.

Các bài toán được phân loại thành bài toán có giám sát (supervised) hoặc không có giám sát (unsupervised). Trường hợp đầu tiên là khi gán nhãn cho một bức ảnh, ví dụ, là "mèo" hoặc "chó." Ở đây, số lượng các lựa chọn có thể là đã được xác định trước. Khi chỉ có hai lựa chọn, nó được gọi là phân loại hai lớp hoặc phân loại nhị phân. Khi có nhiều hơn hai lớp còn phân loại thì được gọi là phân loại đa lớp hoặc phân loại đa thức. Có nhiều phương pháp và thuật toán để xây dựng bộ phân loại, trong đó thuật toán k-láng giềng gần nhất (kNN) là một trong những thuật toán phổ biến nhất.

Hãy cùng xem cách kNN hoạt động bằng cách liệt kê các bước chính của thuật toán:

1. Như trong vấn đề phân loại chung, chúng ta có một tập hợp các điểm dữ liệu mà chúng ta biết nhãn lớp đúng.
2. Khi chúng ta nhận được một điểm dữ liệu mới, chúng ta so sánh nó với từng điểm dữ liệu hiện có và tìm sự tương đồng.
3. Lấy k điểm dữ liệu tương đồng nhất (k láng giềng gần nhất).
4. Từ k điểm dữ liệu này, lấy phiếu bầu đa số từ nhãn. Nhãn thắng cuộc là nhãn/lớp của điểm dữ liệu mới. Số k thường là một số nhỏ, từ 2 đến 20.

Như bạn có thể tưởng tượng, số lượng láng giềng gần nhất (giá trị của k) càng lớn, thì thời gian xử lý càng lâu.

Ví dụ

Hãy lấy một ví dụ và thử hình dung cách thực hiện phân loại với kNN trong R. Trong ví dụ này, chúng ta sẽ sử dụng bộ dữ liệu “Iris”.

Bộ dữ liệu bao gồm một mẫu gồm 50 bông hoa từ ba loài diên vĩ (xanh - setosa, tím – versicolor và tím sẫm – virginica). Mỗi mẫu được đo bốn đặc trưng: chiều dài và chiều rộng của đài hoa và cánh hoa, tính bằng centimét. Dựa trên sự kết hợp của bốn thuộc tính này, chúng ta cần phân biệt các loài iris.

Bộ dữ liệu Iris đã được tích hợp sẵn trong R, vì vậy chúng ta có thể xem bộ dữ liệu này bằng cách gõ “iris” vào bảng điều khiển R (hoặc RStudio). Trước khi tiến hành phân loại, hãy xem phân phối giá trị trong bộ dữ liệu. Để trực quan hóa, chúng ta sẽ sử dụng gói ggvis trong R. Đây là cách thực hiện:

```
# Load the library “ggvis”
```

```
library(ggvis)
```



Hình 55: Bảng phân bố hoa diên vĩ dựa trên chiều dài và chiều rộng của đài hoa

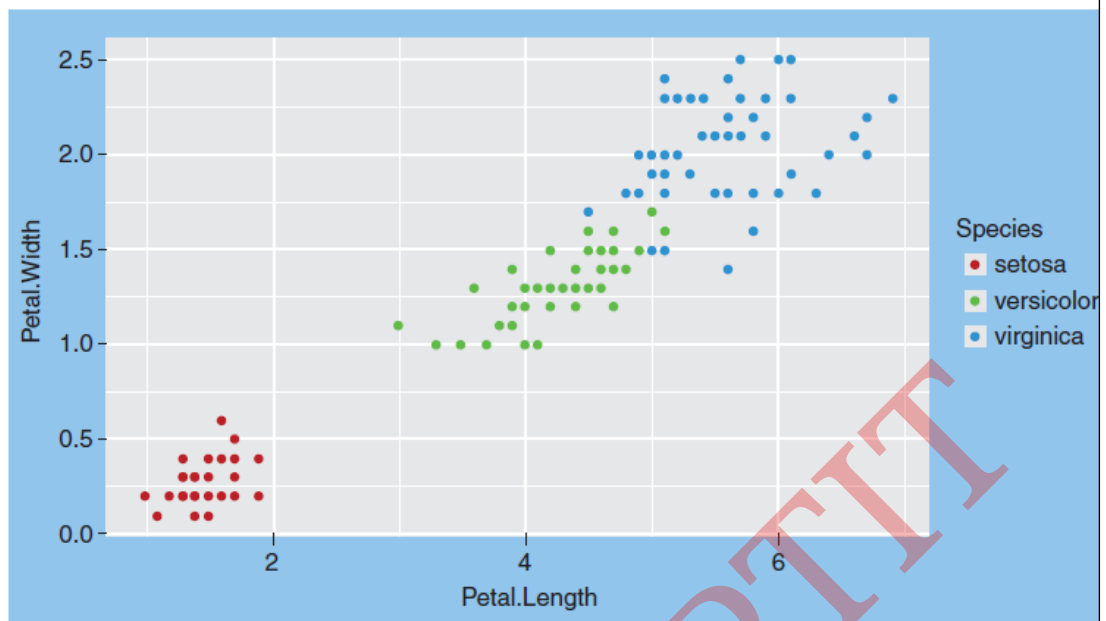
```
# Tạo biểu đồ phân bố
```

```
iris %>% ggvis(~Sepal.Length, ~Sepal.Width, fill = ~Species)
```

Từ Hình 55, chúng ta có thể thấy rằng có sự tương quan cao giữa chiều dài đài hoa và chiều rộng đài hoa diên vĩ, trong khi sự tương quan này có phần thấp hơn đối với hoa I. virginica và I. versicolor.

Nếu mô hình hóa về mối quan hệ giữa chiều dài cánh hoa và chiều rộng cánh hoa, biểu đồ phân bố được trình bày trên Hình 56

```
iris %>% ggvis(~Petal.Length, ~Petal.Width, fill = ~Species)
```



Hình 56: Bảng phân bố hoa diên vĩ dựa trên chiều dài và chiều rộng của cánh hoa

Biểu đồ chỉ ra một sự tương quan tích cực giữa chiều dài cánh hoa và chiều rộng cánh hoa cho tất cả các loài khác nhau có trong bộ dữ liệu Iris.

Trước khi tiến hành nhiệm vụ phân loại, cần có một bộ dữ liệu kiểm tra để đánh giá hiệu suất của mô hình sau này. Do chưa có bộ dữ liệu đó, bộ dữ liệu gốc sẽ được chia thành hai phần: phần huấn luyện và phần kiểm tra. Chúng ta có thể chia theo bất kỳ cách nào, nhưng cần nhớ rằng bộ huấn luyện phải đủ lớn để tạo ra một mô hình tốt. Ngoài ra, phải đảm bảo rằng tất cả ba lớp loài đều có mặt trong mô hình huấn luyện. Quan trọng hơn, số lượng các mẫu của cả ba loài cần phải tương đối đều nhau, để tránh việc thiên vị lớp này hay lớp khác trong dự đoán.

Để chia bộ dữ liệu thành bộ huấn luyện và bộ kiểm tra, chúng ta nên đặt một hạt giống (seed) trước. Đây là một số cho bộ sinh số ngẫu nhiên của R. Ưu điểm chính của việc đặt hạt giống là chúng ta có thể nhận được cùng một dãy số ngẫu nhiên mỗi khi cung cấp cùng một hạt giống trong bộ sinh số ngẫu nhiên. Đây là cách thực hiện:

```
> set.seed(1234)
```

Có thể chọn bất kỳ số nào khác ngoài 1234 trong dòng trên. Tiếp theo, chúng ta muốn đảm bảo rằng bộ dữ liệu Iris của chúng ta được xáo trộn và có số lượng đều các loài trong bộ huấn luyện và bộ kiểm tra. Một cách để đảm bảo điều này là sử dụng hàm `sample()` để lấy mẫu với kích thước được thiết lập bằng số hàng của bộ

dữ liệu Iris (ở đây là 150). Chúng ta lấy mẫu với thay thế: chọn từ một vector gồm hai phần tử và gán "1" hoặc "2" cho 150 hàng của bộ dữ liệu Iris. Việc phân bổ các phần tử này dựa trên trọng số xác suất là 0.67 và 0.33. Điều này dẫn đến khoảng hai phần ba dữ liệu được gán nhãn là "1" (huấn luyện) và phần còn lại là "2" (kiểm tra), tức 2 phần huấn luyện và 1 phần được sử dụng để kiểm tra:

```
ind <- sample(2, nrow(iris), replace=TRUE, prob=c(0.67, 0.33))
```

Chúng ta có thể sử dụng mẫu được lưu trữ trong biến "ind" để định nghĩa bộ huấn luyện và bộ kiểm tra, chỉ lấy bốn cột đầu tiên hoặc các thuộc tính từ dữ liệu.

```
iris.training <- iris[ind==1, 1:4]
```

```
iris.test <- iris[ind==2, 1:4]
```

Ngoài ra, chúng ta cần nhớ rằng "Species" là nhãn lớp, là biến độc lập và các thuộc tính còn lại là các biến phụ thuộc. Do đó, cần lưu trữ nhãn lớp trong các vector yếu tố và chia các lớp này cả bộ huấn luyện và bộ kiểm tra. Điều này có thể được thực hiện như sau:

```
iris.trainLabels <- iris[ind==1,5]
```

```
iris.testLabels <- iris[ind==2,5]
```

Khi tất cả các bước chuẩn bị này đã hoàn thành, đây chính là lúc sử dụng phương pháp phân loại k-láng giềng gần nhất (kNN). Để làm điều này cần sử dụng hàm knn() có sẵn trong R. Hàm knn() sử dụng khoảng cách Euclid để tìm sự tương đồng giữa các mẫu huấn luyện k và mẫu kiểm tra của bạn. Giá trị của k được cung cấp bởi người dùng nhằm xác định số lượng phần tử gần nhất để lấy nhãn cho việc phân loại.

Xây dựng mô hình dựa trên kNN bằng cách thực hiện như sau, giả sử lấy 3 phần tử láng giềng gần nhất

```
library(class)
```

```
iris_pred <- knn(train = iris.training, test = iris.test, cl = iris.trainLabels, k=3)
```

Chúng ta có thể yêu cầu một bản tóm tắt bằng cách sử dụng "summary(iris_pred)" và nhận được kết quả sau:

setosa	versicolor	virginica
12	13	15

Vì chúng ta đã xây dựng mô hình và dự đoán nhãn lớp cho các thuộc tính kiểm

tra, hãy đánh giá độ chính xác của những dự đoán đó. Để làm điều này, chúng ta sẽ sử dụng bảng chéo, có thể được tìm thấy trong thư viện “gmodels”. Trong trường hợp chưa cài đặt nó, hãy tiến hành cài đặt thư viện này như sau:

```
library(gmodels)
```

```
CrossTable(x=iris_pred, y=iris.testLabels, prop.chisq =FALSE)
```

Kết quả được hiển thị trên bảng dưới đây. Từ bảng này có thể thấy các dự đoán của mình (iris_pred) so khớp với sự thật (iris.testLabels) ra sao. Có vẻ như chỉ có một trường hợp dự đoán sai "versicolor" mà nhãn thật là "virginica."

Cell Contents

```
|-----|
| N      |
| N/Row Total |
| N/Col Total  |
| N/Table Total |
|-----|
```

Total Observations in Table: 40

iris_pred	setosa	versicolor	virginica	Row Total
setosa	12 1.000 1.000 0.300	0 0.000 0.000 0.000	0 0.000 0.000 0.000	12 0.300
versicolor	0 0.000 0.000 0.000	12 0.923 1.000 0.300	1 0.077 0.062 0.025	13 0.325
virginica (Thực tế)	0 0.000 0.000 0.000	0 0.000 0.000 0.000	15 1.000 0.938 0.375	15 0.375
Column Total	12 0.300	12 0.300	16 0.400	40

Sử dụng bảng này, bạn có thể dễ dàng tính toán độ chính xác của chúng ta. Trong 40 lần dự đoán chỉ sai 1 lần duy nhất. Vì vậy, độ chính xác của chúng ta là $39/40 = 97.5\%$.

1.12.5. Phân loại với cây quyết định

Trong Học máy (Machine Learning), Cây Quyết định (Decision Tree) là một thuật toán chính được sử dụng để giải quyết các bài toán phân loại và hồi quy. Trong các bài toán này, mục tiêu là xây dựng một mô hình có khả năng dự đoán giá trị của biến mục tiêu dựa trên một loạt biến đầu vào. Cây quyết định hoạt động bằng cách xây dựng cấu trúc cây thông qua việc phân tách một tập dữ liệu lớn thành các tập con nhỏ hơn một cách dần dần. Cấu trúc cuối cùng của mô hình là một cây bao gồm các nút quyết định (Decision Nodes) và các nút lá (Leaf Nodes).

Hãy xem xét bộ dữ liệu về bóng bay được trình bày trong Hình 57. Bộ dữ liệu này có bốn thuộc tính: màu sắc, kích thước, hành động, tuổi, và một nhãn lớp về đã bơm (T = đúng hoặc F = sai). Bộ dữ liệu này được sử dụng để hiểu cách thuật toán cây quyết định hoạt động như thế nào. Có nhiều thuật toán tạo ra cây quyết định, như ID3/4/5, CART, CLS, v.v. Trong số các thuật toán [xây dựng Cây Quyết định], ID3 (Iterative Dichotomiser 3), được phát triển bởi J. R. Quinlan, là thuật toán phổ biến nhất. Thuật toán này sử dụng phương pháp tìm kiếm tham lam theo hướng từ trên xuống dưới qua không gian các nhánh có thể có mà không quay lại (non-backtracking). Đặc tính "Không quay lại" (Non-backtracking) nhấn mạnh rằng khi ID3 chọn một đặc trưng để chia tách (splitting feature) tại một nút, thuật toán này sẽ không bao giờ xem xét lại lựa chọn đó ở các bước sau. ID3 sử dụng hai khái niệm khóa học bao gồm Độ rối loạn (Entropy) và Độ lợi thông tin (Information Gain) để xây dựng cây quyết định. Cần hiểu hai thuật ngữ này trước khi đi sâu vào thuật toán.

Color	Size	Act	Age	Inflated
Yellow	Small	Stretch	Adult	T
Yellow	Small	Stretch	Adult	T
Yellow	Small	Stretch	Child	F
Yellow	Small	Dip	Adult	F
Yellow	Small	Dip	Child	F
Yellow	Large	Stretch	Adult	T
Yellow	Large	Stretch	Adult	T
Yellow	Large	Stretch	Child	F
Yellow	Large	Dip	Adult	F
Yellow	Large	Dip	Child	F
Purple	Small	Stretch	Adult	T
Purple	Small	Stretch	Adult	T
Purple	Small	Stretch	Child	F
Purple	Small	Dip	Adult	F
Purple	Small	Dip	Child	F
Purple	Large	Stretch	Adult	T
Purple	Large	Stretch	Adult	T
Purple	Large	Stretch	Child	F
Purple	Large	Dip	Adult	F
Purple	Large	Dip	Child	F

Hình 57: Bộ dữ liệu về bóng bay

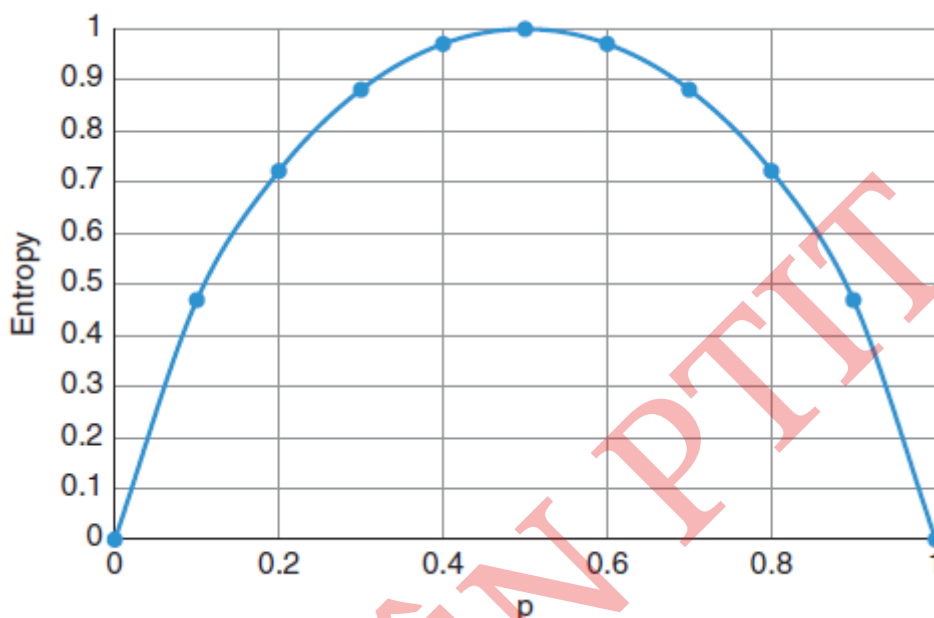
Độ rối loạn (Entropy): Entropy (E) là một thước đo sự rối loạn, không chắc chắn hoặc ngẫu nhiên. Nếu tung một đồng xu công bằng, xác suất tung đồng xu ra mặt ngửa hoặc mặt sấp là như nhau. Nói cách khác, khi các khả năng xảy ra là bằng nhau, sự không chắc chắn về kết quả là lớn nhất. Công thức của thước đo này là:

$$\text{Entropy}(E) = -\sum_{i=1}^k p_i \log_2(p_i).$$

Ở đây, k là số lượng giá trị lớp có thể có, và p_i là số lần xuất hiện của lớp $i=1$ trong bộ dữ liệu. Vì vậy, trong bộ dữ liệu "balloons", số lượng giá trị lớp có thể có là 2 (T hoặc F). Lý do cho dấu trừ là vì logarit của các phân số $p_1, p_2, \dots, p_n$ là số âm, vì vậy entropy trở thành số dương. Thông thường, các phép tính logarit được biểu diễn theo cơ số 2; khi đó, Entropy sẽ có đơn vị gọi là bits – đây chính là loại bits thông thường được sử dụng trong máy tính.

Hình 58 cho thấy đồ thị entropy theo các giá trị xác suất cho một sự kiện. Như có thể thấy, đồ thị đạt giá trị cao nhất (1) khi xác suất p là 0.5. Điều này đúng với bài toán cầm một đồng xu công bằng, xác suất để ra mặt ngửa hoặc mặt sấp là 0.5. Entropy cho đồng

xu này là cao nhất tại điểm này, phản ánh mức độ không chắc chắn cao nhất. Ngược lại, trong trường hợp đồng xu không công bằng và luôn ra mặt ngửa, xác suất ra mặt ngửa sẽ là 1 và entropy tương ứng sẽ là 0, chỉ ra rằng không có sự không chắc chắn nào về kết quả của sự kiện này.



Hình 58: Hình biểu diễn Entropy

Độ lợi thông tin (Information Gain): Nếu bạn nghĩ rằng hôm nay không mưa và tôi nói với bạn rằng thực sự có mưa, có phải nói rằng bạn đã nhận được một số thông tin từ tôi không? Ngược lại, nếu bạn đã biết rằng sẽ có mưa, thì dự đoán của tôi sẽ không thực sự ảnh hưởng nhiều đến kiến thức hiện tại của bạn. Có một cách toán học để đo lường sự gia tăng thông tin như vậy:

$$IG(A, B) = \text{Entropy}(A) - \text{Entropy}(A, B)$$

Ở đây, Độ lợi Thông tin đạt được khi biết B cùng với A là sự khác biệt giữa Entropy của A và Entropy chung của A và B.

Nhưng trước hết, hãy quay lại thuật toán cây quyết định. Cây quyết định là một cây phân cấp, xây dựng từ nút gốc đến các lá, và bao gồm việc phân chia dữ liệu thành các tập con nhỏ hơn chứa các đối tượng có giá trị tương tự (đồng nhất). Thuật toán ID3 sử dụng độ hỗn loạn để tính toán sự đồng nhất của một mẫu. Nếu mẫu hoàn toàn đồng nhất, độ hỗn loạn bằng không, và nếu mẫu được chia đều, độ hỗn loạn là một. Nói cách khác, độ hỗn loạn là một thước đo sự rối loạn trong dữ liệu.

Bây giờ, để xây dựng cây quyết định, chúng ta cần tính toán hai loại entropy bằng cách sử dụng bảng tần suất, như sau:

a. Độ hỗn loạn sử dụng bảng tần suất của một thuộc tính:

Inflated	
True	False
8	12

Từ đó

$$\begin{aligned}
E(\text{Inflated}) &= E(12, 8) \\
&= E(0.6, 0.4) \\
&= -(0.6 \log_2 0.6) - (0.4 \log_2 0.4) \\
&= 0.4422 + 0.5288 \\
&= 0.9710
\end{aligned}$$

b. Tương tự, Entropy được tính bằng cách sử dụng bảng tần số của hai thuộc tính là Entropy chung

$$E(A, B) = \sum_{k \in B} P(k) E(k)$$

Hãy cùng xem Bảng 9.2.

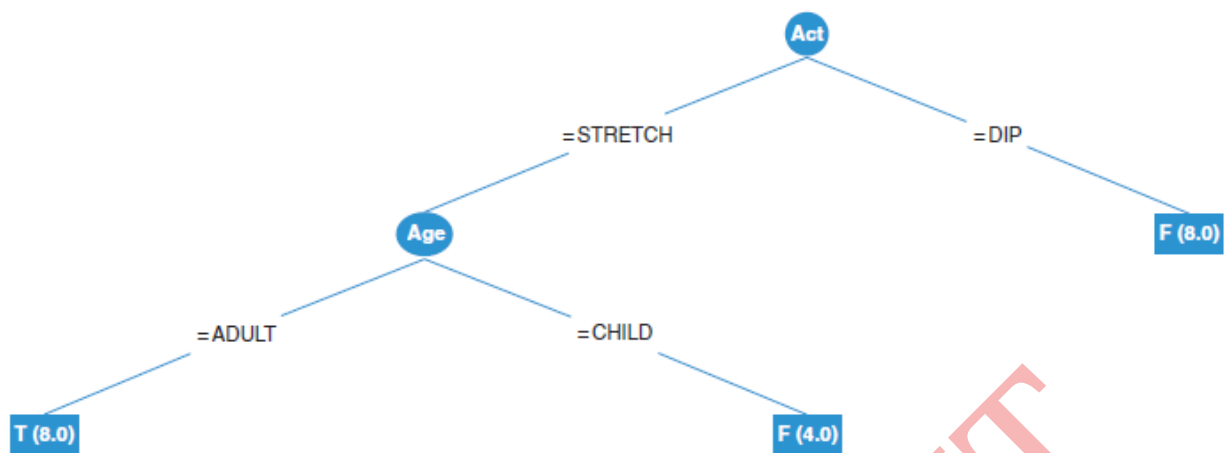
$$\begin{aligned}
&E(\text{Inflated}, \text{Act}) \\
&= P(\text{Dip}) \times E(8, 0) + P(\text{Stretch}) \times E(4, 8) \\
&= \left(\frac{8}{20}\right) \times 0.0 + \left(\frac{12}{20}\right) \times \{-(0.3 \log_2 0.3) - (0.7 \log_2 0.7)\} = \left(\frac{12}{20}\right) \times (0.5278 + 0.3813) \\
&= \left(\frac{12}{20}\right) \times 0.9090 \\
&= 0.5454
\end{aligned}$$

Như giá trị trên chỉ ra, có một sự giảm độ hỗn loạn so với trường hợp đầu tiên. Sự giảm độ hỗn loạn này được gọi là **độ lợi thông tin**. Ở đây, **độ lợi thông tin (IG)** là:

$$\begin{aligned}
IG &= E(\text{Inflated}) - E(\text{Inflated}, \text{Act}) \\
&= 0.9710 - 0.5454 \\
&= 0.4256
\end{aligned}$$

Bảng 11: Bảng tần suất của thuộc tính act và inflated

Act	Inflated True	Inflated False	Total
Dip	0	8	8
Stretch	8	4	12
			20



Hình 59: Cây quyết định cuối cùng

Nếu Entropy là thước đo độ rối loạn (hay mức độ không thuần khiết) của một tập dữ liệu, thì Độ lợi Thông tin là thước đo sự giảm rối loạn đó đạt được thông qua việc phân chia bộ dữ liệu ban đầu theo một thuộc tính.

Mục tiêu cốt lõi trong việc xây dựng cây quyết định là tại mỗi nút, tìm kiếm thuộc tính mang lại độ lợi thông tin cao nhất, qua đó tạo ra các tập con (nhánh) có độ đồng nhất (thuần khiết) cao nhất. Dưới đây là các bước tính xây dựng cây quyết định:

Bước 1: Tính entropy của biến mục tiêu hoặc lớp, trong trường hợp của chúng ta là 0.9710.

Bước 2: Bộ dữ liệu sau đó được chia nhỏ theo các thuộc tính khác nhau thành các bảng con nhỏ hơn, ví dụ, Inflated và Act, Inflated và Age, Inflated và Size, và Inflated và Color. Độ rối loạn cho mỗi bảng con được tính. Sau đó, chúng được cộng lại tỷ lệ với nhau, để có entropy tổng cho sự phân chia. Entropy kết quả được trừ đi từ entropy trước khi phân chia. Kết quả là độ lợi thông tin hoặc sự giảm độ hỗn loạn.

Bước 3: Chọn thuộc tính có độ lợi thông tin lớn nhất làm nút quyết định, chia bộ dữ liệu theo các nhánh của nó và lặp lại quy trình này trên mỗi nhánh.

Kết quả cây quyết định được hiển thị như trên Hình 59.

1.12.5.1. Quy tắc quyết định

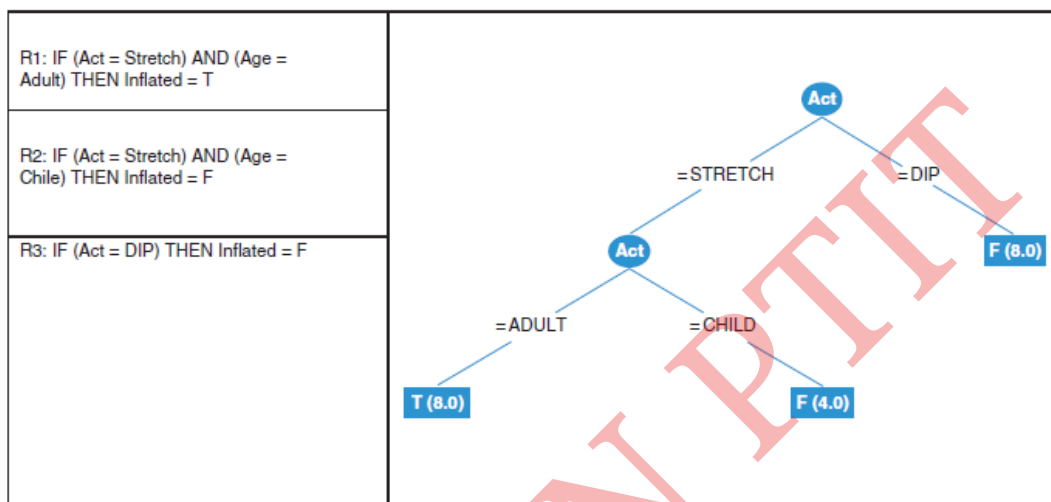
Các quy tắc là một lựa chọn phổ biến, thay thế cho cây quyết định trong học máy.

Một quy tắc thường được biểu diễn dưới dạng biểu thức IF-THEN (ví dụ: {IF "điều kiện" THEN "kết quả"}).

Thông thường, một quy tắc riêng lẻ không được xem là một mô hình hoàn chỉnh, vì quy tắc chỉ có thể áp dụng và đưa ra dự đoán khi điều kiện liên quan được thỏa mãn. Do đó, các phương pháp học máy dựa trên quy tắc thường xác định một tập hợp các quy tắc.

Tập hợp các quy tắc này khi được tổng hợp lại, sẽ tạo thành mô hình dự đoán đầy đủ hoặc một cơ sở tri thức để đưa ra các quyết định.

Để phù hợp với bất kỳ bộ dữ liệu nào, một tập hợp các quy tắc có thể dễ dàng được suy ra từ cây quyết định bằng cách theo dõi các đường đi từ nút gốc đến các nút lá. Đối với cây quyết định trên trong Hình 59, các quy tắc quyết định tương ứng được hiển thị ở bên trái của Hình 60.



Hình 60: Điều kiện dựa trên cây kết quả

1.12.5.2. Quy tắc phân loại

Việc đọc một tập hợp các quy tắc phân loại trực tiếp từ cây quyết định khá dễ dàng. Theo đó, mỗi nút lá trong cây sẽ sinh ra một quy tắc. Cụ thể, tiền đề (IF) của quy tắc bao gồm chuỗi các điều kiện (một điều kiện cho mỗi nút) trên con đường từ nút gốc đến lá đó, và hậu quả (THEN) của quy tắc chính là lớp được gán bởi nút lá đó.

Quá trình trích xuất quy tắc từ cây quyết định tạo ra các quy tắc rõ ràng (mutually exclusive). Điều này có nghĩa là mỗi mẫu dữ liệu chỉ có thể thỏa mãn tiền đề của một quy tắc duy nhất, nhờ đó thứ tự thực thi các quy tắc trở nên không quan trọng. Tuy nhiên, các quy tắc này thường phức tạp hơn mức cần thiết. Do đó, các quy tắc được suy ra từ cây thường phải được cắt tỉa để loại bỏ các điều kiện thừa. Mặt khác, do cây quyết định không thể dễ dàng biểu thị sự kết hợp phức tạp giữa các quy tắc khác nhau trong một tập hợp quy tắc tổng quát, nên việc chuyển đổi ngược lại—from một tập hợp quy tắc tổng quát thành một cấu trúc cây—không phải là một quá trình đơn giản. Một ví dụ rõ ràng về điều này xảy ra khi các quy tắc có cùng cấu trúc nhưng các thuộc tính khác nhau, chẳng hạn như:

- Nếu a và b thì x
- Nếu c và d thì x

Để xử lý cấu trúc này, cây quyết định buộc phải phá vỡ sự đối xứng và chọn một thuộc tính duy nhất làm bài kiểm tra tại nút gốc. Nếu, ví dụ, thuộc tính "a" được chọn làm bài kiểm tra đầu tiên, toàn bộ quy tắc thứ hai (Nếu c và d thì x) sẽ phải được lặp lại (sao chép) dưới **hai** nhánh con khác nhau (nhánh a và nhánh không phải a). Đây được gọi là vấn đề nhánh con bị sao chép. Sự sao chép này không chỉ làm cho cây trở nên phức tạp và lớn hơn mức cần thiết, mà còn làm giảm khả năng khái quát hóa và khó khăn trong việc giải thích mô hình.

1.12.5.3. Quy tắc kết hợp

Các quy tắc kết hợp (Association Rules) không khác biệt nhiều so với các quy tắc phân loại, ngoại trừ việc chúng có thể dự đoán bất kỳ thuộc tính nào, chứ không chỉ riêng thuộc tính lớp. Điều này cho phép chúng có khả năng dự đoán cả tổ hợp các thuộc tính.

Mặt khác, các quy tắc kết hợp không nhằm mục đích được sử dụng cùng nhau như một tập hợp—như cách các quy tắc phân loại được áp dụng. Các quy tắc kết hợp khác nhau thể hiện các quy luật khác nhau ẩn dưới tập dữ liệu và thường dự đoán những điều khác nhau.

Outlook	Temperature	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Hình 61: Bảng dữ liệu về thời tiết ảnh hưởng đến khả năng chơi golf

Vì có thể suy ra một số lượng rất lớn các quy tắc kết hợp khác nhau ngay cả từ một bộ dữ liệu nhỏ, nên sự quan tâm thường tập trung vào những quy tắc thỏa mãn hai tiêu chí chính: đầu tiên phải áp dụng cho một số lượng đáng kể các mẫu và thứ hai phải có độ chính xác khá cao đối với các mẫu mà chúng áp dụng.

Hai thước đo chính được sử dụng để đánh giá chất lượng của một quy tắc kết hợp là Độ phủ (thường được gọi là Coverage) và Độ tin cậy (thường được gọi là Accuracy). Độ phủ, hay còn được gọi là độ hỗ trợ (Support) của một quy tắc đo lường tần suất quy tắc đó xuất hiện trong tập dữ liệu, tức là số lượng mẫu thỏa mãn cả tiền đề và hậu quả của quy tắc (số mẫu mà quy tắc dự đoán chính xác) thể hiện số lượng trường hợp dự đoán đúng, trong khi đó, độ tin cậy, hay còn được gọi là độ chính xác (Accuracy) đo lường độ chính xác của quy tắc khi tiền đề được thỏa mãn. Độ tin cậy được tính bằng cách lấy số lượng mẫu dự đoán chính xác, sau đó biểu diễn dưới dạng tỷ lệ phần trăm của tất cả các mẫu mà quy tắc áp dụng (tức là số mẫu thỏa mãn tiền đề).

Ví dụ, xem xét dữ liệu thời tiết trong Hình 61, một bộ dữ liệu huấn luyện về thời tiết và biến độc lập tương ứng là "Play" - biến này gợi ý có chơi được trong thời tiết này không. Cây quyết định và các quy tắc quyết định được suy ra từ bộ dữ liệu này được trình bày trong Hình 62

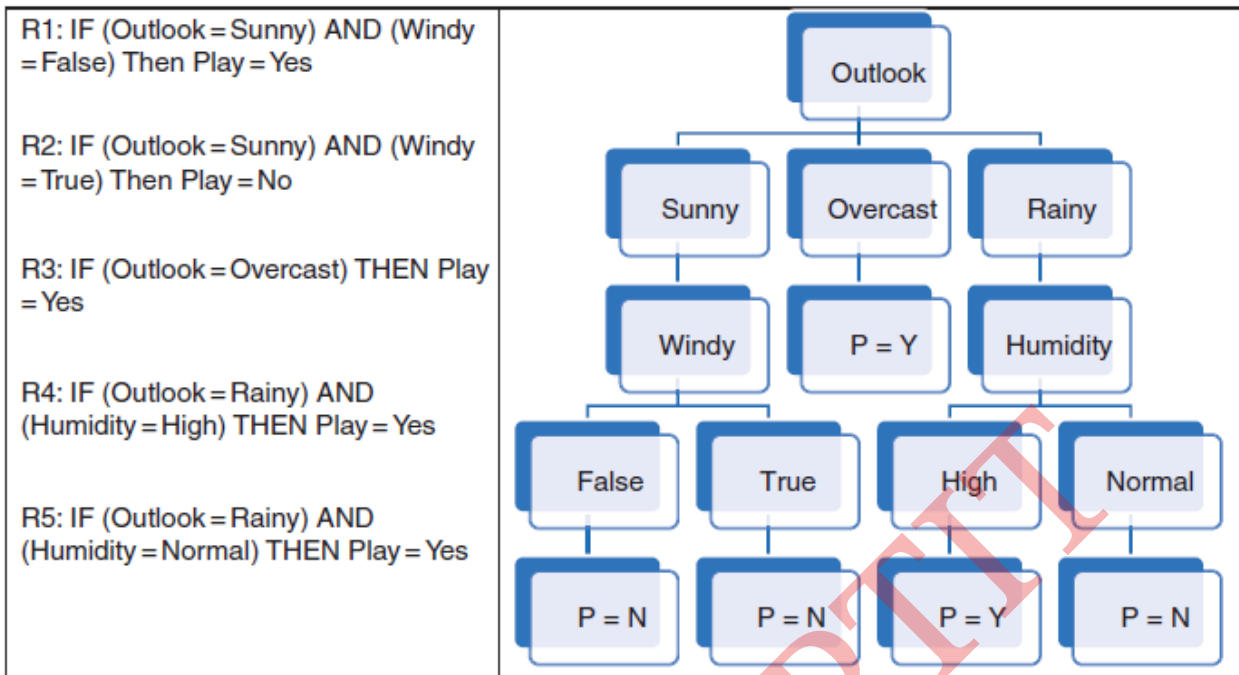
Hãy xem xét quy tắc sau:

Nếu **nhật độ** = **mát** thì **độ ẩm** = **bình thường**.

Độ phủ là số ngày có cả nhiệt độ mát và độ ẩm bình thường (bốn ngày trong dữ liệu của Hình 61), và **độ tin cậy** là tỷ lệ phần trăm của những ngày mát có độ ẩm bình thường (100% trong trường hợp này). Một số quy tắc kết hợp có chất lượng tốt từ bảng trên là:

- Nếu **độ ẩm** = **bình thường** và **có gió** = **sai** thì **chơi** = **có**.
- Nếu **thời tiết** = **nắng** và **chơi** = **không** thì **có gió** = **đúng**.
- Nếu **có gió** = **sai** và **chơi** = **không** thì **thời tiết** = **nắng** và **độ ẩm** = **cao**.

Hãy kiểm tra xem những quy tắc này có hợp lý không bằng cách đi qua cây quyết định trong Hình 62 và đặt câu hỏi logic về chúng (ví dụ, nếu trời nắng nhưng rất gió ngoài trời, liệu có nên chơi không?).



Hình 62: Cây quyết định về thời tiết

Ví dụ Thực hành 9.4: Cây Quyết Định

Hãy cùng xem cách một bộ phân loại dựa trên cây quyết định có thể được triển khai với R. Trong ví dụ này, chúng ta sẽ sử dụng bộ dữ liệu bóng bay khác. Đây là bộ dữ liệu từ kho lưu trữ mà chúng ta đã sử dụng trong ví dụ trước. Bộ dữ liệu này được tải ở <https://www.kaggle.com/datasets/juliusalphonso/balloons> , đặt tên là yellow-small.csv

Bước đầu tiên là nhập dữ liệu vào RStudio. Cần chú ý trước khi thực hiện bất kỳ thao tác nào, cần kiểm tra kiểu dữ liệu của tất cả các thuộc tính (một ví dụ như được trình bày ở đây).

```
> balloon <- read.csv("yellow-small.csv", sep = ',', header = TRUE)
> View(balloon)
> is.factor(balloon$Size)
```

Chúng ta sẽ sử dụng gói “party” ở đây. Bạn phải cài đặt gói này trước khi tiếp tục với bước tiếp theo.

```
> install.packages("party")
> library(party)
```

Trong bước tiếp theo, chúng ta sẽ tạo cây quyết định.

```
> View(balloon)
```

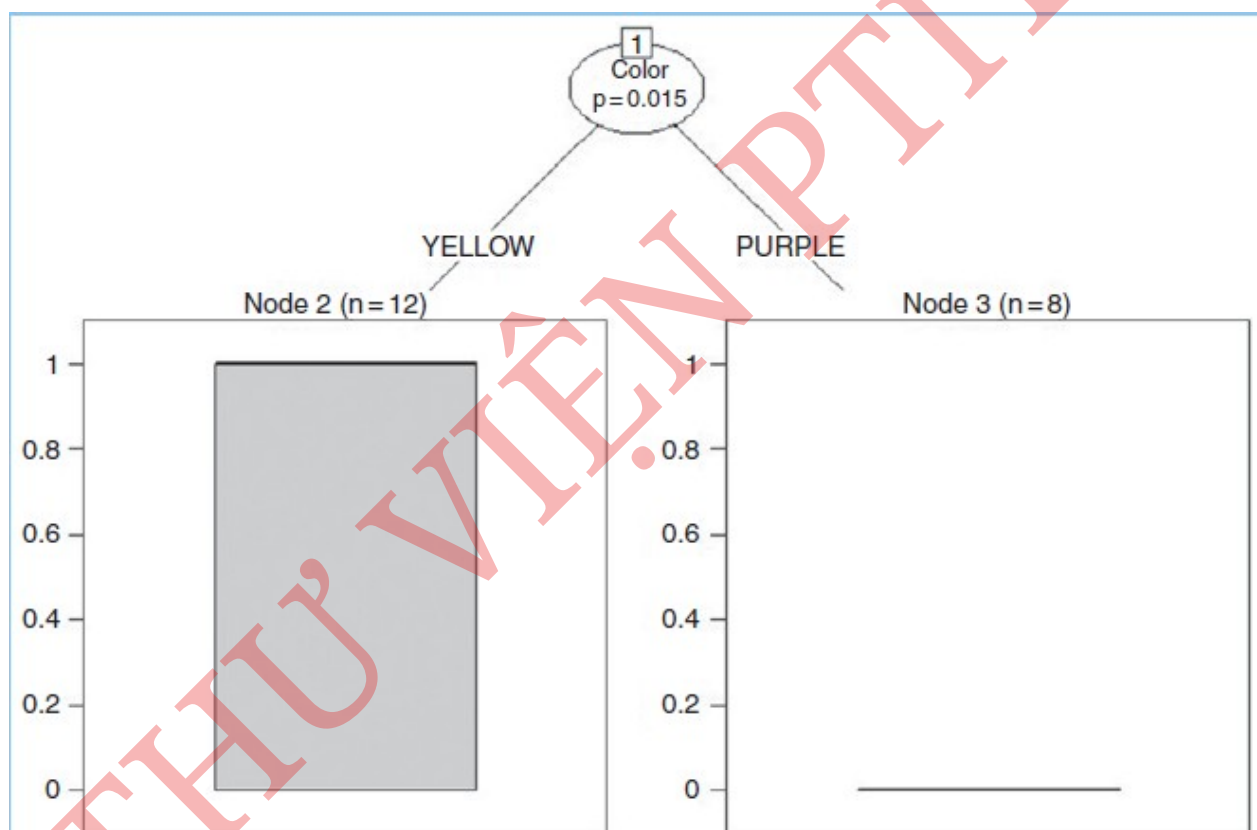


```
> inflated.Tree<- ctree(Inflated~., data = balloon)
```

Để vẽ cây quyết định, thực thi dòng lệnh sau:

```
> plot(inflated.Tree)
```

Nếu đã thực hiện đúng các bước, cây quyết định được hiển thị giống như trong Hình 63. Từ cây quyết định, có thể kết luận rằng màu sắc là một yếu tố dự đoán tốt liệu quả bóng có được thổi phồng hay không



Hình 63: Mô hình cây quyết định được hiển thị

1.12.6. Phân loại với phương pháp rừng ngẫu nhiên

Một cách hiệu quả để giải quyết các hạn chế của mô hình đơn lẻ (như vấn đề nhánh con bị sao chép trong cây quyết định) là sử dụng phương pháp học tập kết hợp (Ensemble Learning). Thay vì chỉ sử dụng một hoặc hai cây quyết định, phương pháp này sử dụng nhiều mô hình (hoặc cây), mỗi mô hình được huấn luyện và tạo ra một cách hơi khác biệt. Dự đoán cuối cùng sau đó được lấy bằng cách kết hợp (thường là lấy trung bình hoặc bỏ phiếu) từ những gì các mô hình này quyết định.

Trong học máy, các phương pháp học tập kết hợp dựa vào nhiều thuật toán học máy để đạt được độ chính xác dự đoán tốt hơn so với bất kỳ thuật toán đơn lẻ nào. Một thuật toán học tập kết hợp bao gồm một tập hợp hữu hạn các mô hình thay thế, được tích hợp thông qua một cấu trúc linh hoạt để ra quyết định chung. Một ví dụ về phương pháp học tập kết hợp là **mô hình rừng ngẫu nhiên**, có thể được sử dụng cho cả tác vụ hồi quy và phân loại.

Rừng ngẫu nhiên (Random Forest) là một phương pháp hoạt động mạnh mẽ bằng cách xây dựng một số lượng lớn các cây quyết định độc lập trong suốt quá trình huấn luyện. Khi đưa ra dự đoán, mô hình tổng hợp kết quả của các cây này: đối với tác vụ phân loại, phương pháp này chọn nhãn lớp được bầu chọn nhiều nhất làm kết quả cuối cùng; còn đối với tác vụ hồi quy, phương pháp này tính giá trị trung bình của các dự đoán từ tất cả các cây cá nhân. Lợi thế cốt lõi của việc sử dụng rừng ngẫu nhiên so với một cây quyết định đơn lẻ là phương pháp này hiệu quả trong việc sửa chữa thói quen quá khớp dữ liệu huấn luyện của cây quyết định, do đó giúp mang lại dự đoán tổng thể ổn định và chính xác hơn. Dưới đây là cách thức các mô hình này được xây dựng chi tiết:

1. Một mẫu từ N trường hợp huấn luyện được chọn ngẫu nhiên nhưng có thay thế từ bộ dữ liệu huấn luyện gốc. Mẫu này sẽ được sử dụng làm bộ huấn luyện để phát triển cây.
2. Nếu bộ dữ liệu có M biến đầu vào, một số m (m nhỏ hơn rất nhiều so với M) được chỉ định sao cho, tại mỗi nút, m biến được chọn ngẫu nhiên từ M . Trong số m này, cách chia tốt nhất được sử dụng để chia nút. Giá trị của m được giữ cố định trong suốt quá trình phát triển rừng ngẫu nhiên.
3. Theo các bước trên, mỗi cây được phát triển đến mức độ lớn nhất có thể và không có bước tỉa cây.
4. Dự đoán dữ liệu mới bằng cách tổng hợp các dự đoán từ n cây (tức là, bỏ phiếu đa số cho phân loại, trung bình cho hồi quy).

Giả sử bộ dữ liệu huấn luyện, N , có bốn quan sát về ba biến dự báo, cụ thể là A , B và C . Dữ liệu huấn luyện được cung cấp trong Bảng 12.

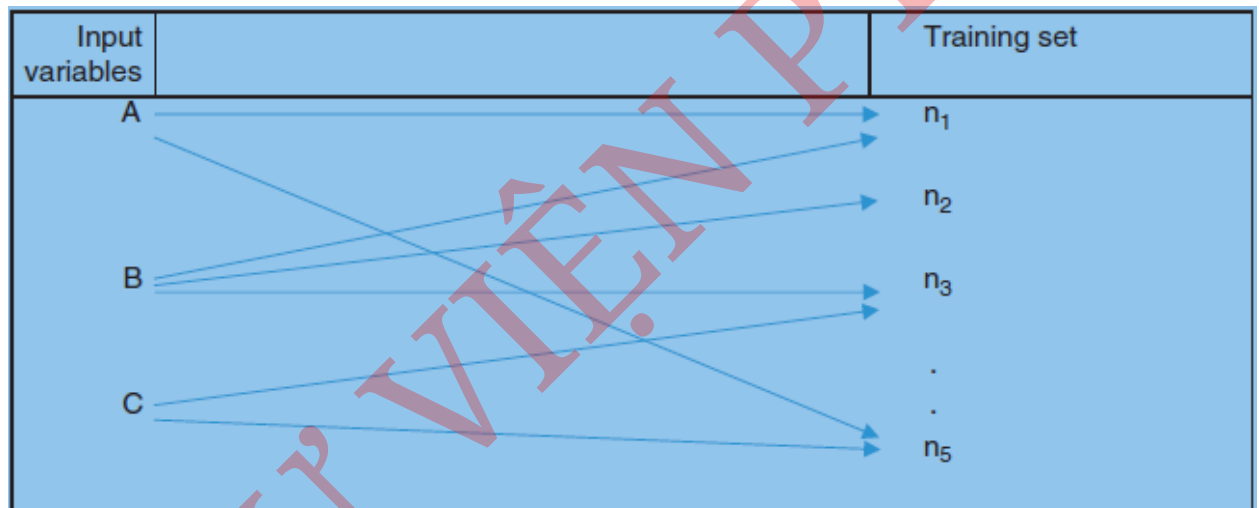
Chúng ta sẽ làm việc qua thuật toán rừng ngẫu nhiên trên bộ dữ liệu nhỏ này.

Bước 1: Lấy mẫu ngẫu nhiên các trường hợp huấn luyện từ N . Vì vậy, các tập con của N , ví dụ như $n_1, n_2, n_3, \dots, n_n$ (như minh họa trong Hình 65) được sử dụng để phát triển (huấn luyện) số lượng cây quyết định. Các mẫu này được rút ra ngẫu nhiên nhất có thể. Ví dụ, n_1 có thể gồm các trường hợp huấn luyện 1, 1, 1 và 4. Tương tự, n_2 có thể gồm các trường hợp 2, 3, 3 và 4, và tiếp tục tương tự.

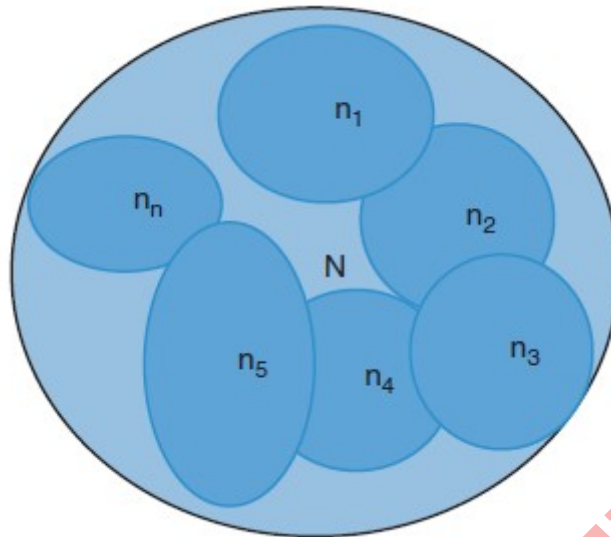
Bước 2: Trong số ba biến dự đoán, một số $m \ll 3$ được chỉ định sao cho, tại mỗi nút, m biến được chọn ngẫu nhiên từ M . Giả sử m ở đây là 2. Vậy, n_1 có thể được huấn luyện trên A, B; n_2 có thể được huấn luyện trên B, C, và cứ thế tiếp tục (xem Hình 64).

Bảng 12: Bộ dữ liệu mẫu

	A	B	C
	A1	B1	C1
	A2	B2	C2
	A3	B3	C3
Mẫu huấn luyện	A4	B4	C4



Hình 64: Lựa chọn thuộc tính của cây quyết định

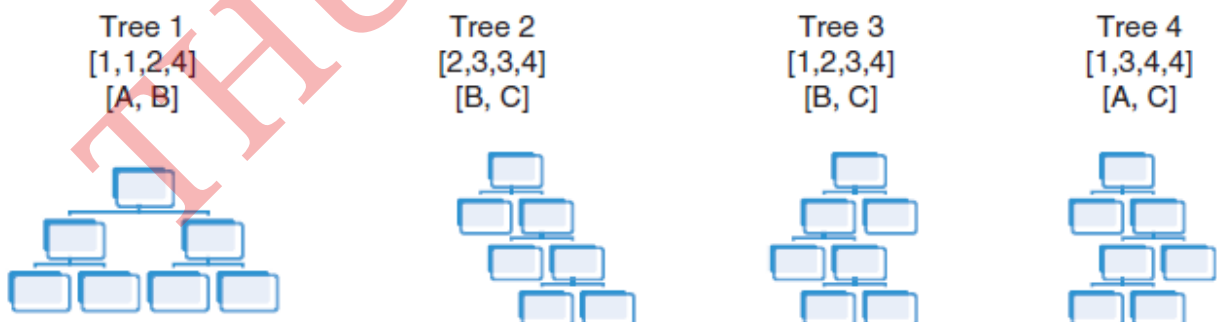


Hình 65: Lấy mẫu bộ dữ liệu huấn luyện

Vì vậy, kết quả các cây quyết định có thể tương tự như trong Hình 66.

Rừng ngẫu nhiên sử dụng kỹ thuật lấy mẫu Bootstrap, trong đó dữ liệu đầu vào được lấy mẫu có thay thế. Trước khi áp dụng thuật toán huấn luyện (đặc biệt là trong các mô hình dựa trên Bagging), một phần dữ liệu (thường là khoảng một phần ba) sẽ không được sử dụng để huấn luyện bất kỳ cây nào mà được sử dụng để kiểm tra. Những mẫu không được chọn này được gọi là mẫu Out-of-Bag (OOB).

Việc ước lượng lỗi trên tập hợp OOB—được gọi là lỗi Out-of-Bag (OOB Error)—cung cấp bằng chứng cho thấy ước tính OOB có thể chính xác như việc sử dụng một tập kiểm tra riêng biệt có kích thước bằng tập huấn luyện. Do đó, việc sử dụng ước tính lỗi OOB loại bỏ sự cần thiết phải dành riêng một tập kiểm tra tách biệt cho mô hình này.



Hình 66: Các cây quyết định cho dữ liệu rừng ngẫu nhiên

Vậy câu hỏi lớn là tại sao rừng ngẫu nhiên làm tốt hơn so với từng cây quyết định cá nhân? Mặc dù không có sự đồng thuận rõ ràng giữa các nhà nghiên cứu, nhưng có hai niềm tin chính đằng sau điều này:

1. Như câu ngạn ngữ "Không ai biết mọi thứ, nhưng mọi người đều biết điều gì đó," không phải tất cả các cây trong rừng đều hoàn hảo hay chính xác tuyệt đối. Tuy nhiên, hầu hết các cây quyết định đều cung cấp dự đoán đúng cho phần lớn dữ liệu. Do Rừng ngẫu nhiên lấy lớp chiếm đa số của các dự đoán đầu ra để xác định lớp cuối cùng, các dự đoán sai của một số cây riêng lẻ sẽ bị áp đảo bởi dự đoán đúng của lớp đa số. Về mặt trực giác, tính hiệu quả của nguyên tắc này phụ thuộc vào tính ngẫu nhiên trong kỹ thuật lấy mẫu (bootstrap). Mẫu càng ngẫu nhiên, các cây càng ít tương quan với nhau, và khả năng các lỗi từ cây này ảnh hưởng đến cây khác càng thấp.
2. Quan trọng hơn, các cây khác nhau thường mắc lỗi ở những vị trí khác nhau trong không gian dữ liệu chứ không phải tất cả đều mắc lỗi tại cùng một điểm. Về mặt trực giác, niềm tin này phụ thuộc vào tính ngẫu nhiên trong việc lựa chọn thuộc tính tại các nút chia tách: thuộc tính càng được chọn ngẫu nhiên, các cây càng ít có khả năng mắc cùng một loại lỗi tại cùng một vị trí.

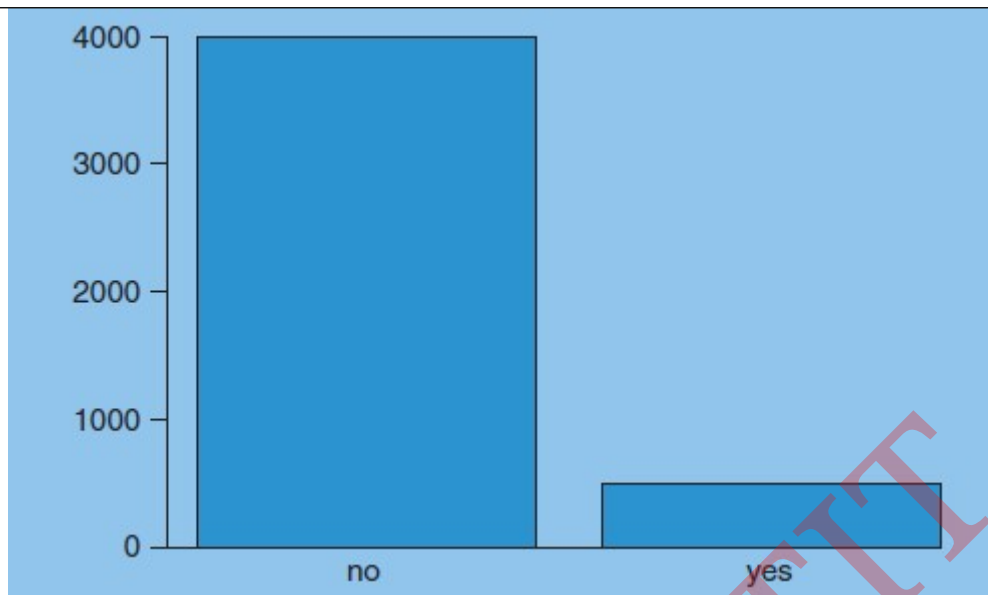
Ví dụ thực hành: Rừng ngẫu nhiên

Bây giờ, chúng ta sẽ lấy một ví dụ và xem cách sử dụng rừng ngẫu nhiên trong R. Để làm điều này, chúng ta sẽ sử dụng bộ dữ liệu **Bank Marketing** từ **Đại học California, Irvine** (University of California, Irvine). Với bộ dữ liệu này, mục tiêu là dự đoán xem khách hàng sẽ đăng ký gửi một khoản tiền có kỳ hạn hay không. Bộ dữ liệu này được tải từ <https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset>

Hãy bắt đầu bằng cách nhập bộ dữ liệu vào **RStudio**. Lưu ý rằng trong bộ dữ liệu gốc, các cột được phân tách bằng dấu chấm phẩy. Nếu bạn không quen với cách xử lý dữ liệu phân tách bằng dấu chấm phẩy, bạn có thể muốn chuyển đổi nó thành định dạng .csv hoặc .tsv, hoặc định dạng mà bạn quen thuộc nhất.

```
> bank <- read.csv(file.choose(), header = TRUE, sep = ";")  
> View(bank)  
> barplot(table(bank$y))
```

Dòng mã trên tạo ra biểu đồ cột như được minh họa trong Hình 9.14.



Hình 67: Đồ thị mô tả các nhãn có gửi tiết kiệm hay không:

Như biểu đồ cột thể hiện, phần lớn các điểm dữ liệu trong bộ dữ liệu này có nhãn lớp là "no." Bây giờ, trước khi xây dựng mô hình, hãy tách bộ dữ liệu thành các trường hợp huấn luyện và kiểm tra:

```
> set.seed(1234)
> population <- sample(nrow(bank), 0.75 * nrow(bank))
> train <- bank[population, ]
> test <- bank[-population, ]
```

Như đã trình bày, bộ dữ liệu được chia thành hai phần: 75% dùng cho mục đích huấn luyện và phần còn lại để đánh giá mô hình của chúng ta. Để xây dựng mô hình, thư viện **randomForest** được cài đặt và sử dụng. Tiếp theo, sử dụng các tập dữ liệu huấn luyện để xây dựng mô hình:

```
> install.packages("randomForest")
> library(randomForest)
> model <- randomForest(y ~ ., data = train)
> model
```

Chúng ta có thể sử dụng **ntree** và **mtry** để chỉ định tổng số cây cần xây dựng (mặc định = 500), và số lượng các biến dự báo sẽ được chọn ngẫu nhiên tại mỗi điểm phân

chia, tương ứng. Các dòng mã trên sẽ tạo ra kết quả sau:

Call:

```
randomForest(formula = y ~ ., data = train)
```

Type of random forest: classification

Number of trees: 500

No. of variables tried at each split: 4

OOB estimate of error rate: 9.94%

Confusion matrix:

	no	yes	class.error
no	2901	97	0.032549
yes	240	152	0.6122449

Có thể thấy rằng 500 cây đã được xây dựng và mô hình đã lấy ngẫu nhiên bốn biến dự đoán tại mỗi lần phân chia (split). Ngoài ra cũng hiển thị một ma trận nhầm lẫn (confusion matrix) chứa dự đoán so với giá trị thực tế, cùng với lỗi phân loại (classification error) cho mỗi lớp. Hãy cùng xem mô hình thể hiện với tập dữ liệu kiểm tra ra sao:

```
> prediction <- predict(model, newdata = test)
```

```
> table(prediction, test$y)
```

Kết quả sau đây được tạo ra:

prediction	no	yes
no	964	84
yes	38	45

Dựa vào số liệu trên, có thể đánh giá thêm độ chính xác như sau:

```
> (964 + 45) / nrow(test)
```

```
[1] 0.8921309
```

Độ chính xác đạt được là 89%

Trong cộng đồng những người làm khoa học dữ liệu, Rừng ngẫu nhiên được xem như một "liều thuốc bách bệnh" cho mọi vấn đề. Có một niềm tin rằng khi bạn không nghĩ ra được thuật toán nào khác, bất kể tình huống ra sao, hãy sử dụng rừng ngẫu nhiên.

Điều này có phần phi lý, vì không có thuật toán nào vượt trội hơn tất cả các thuật toán khác trong mọi ứng dụng (nguyên tắc "một kích cỡ không phù hợp với tất cả").

Tuy nhiên, mọi người đều có những thuật toán yêu thích của riêng mình. Và có những lý do chính đáng giải thích tại sao, đối với nhiều nhà khoa học dữ liệu, rừng ngẫu nhiên lại là lựa chọn ưu tiên hàng đầu:

1. Rừng ngẫu nhiên có thể giải quyết cả hai loại vấn đề, đó là phân loại và hồi quy, và đưa ra kết quả khá tốt cho cả hai.
2. Rừng ngẫu nhiên hầu như không yêu cầu chuẩn bị dữ liệu đầu vào. Phương pháp này có thể xử lý các đặc trưng nhị phân, đặc trưng phân loại và đặc trưng số mà không cần bất kỳ thao tác chia tỷ lệ (scaling) nào.
3. Rừng ngẫu nhiên không quá nhạy cảm với bộ tham số cụ thể được sử dụng. Kết quả là, phương pháp này không yêu cầu phải điều chỉnh nhiều để có được một mô hình tốt; chỉ cần sử dụng một số lượng cây lớn sẽ đạt được kết quả khả quan.
4. Rừng ngẫu nhiên là một phương pháp hiệu quả để ước tính dữ liệu thiếu và duy trì độ chính xác khi một tỷ lệ lớn dữ liệu bị thiếu.

Vậy, liệu rừng ngẫu nhiên có phải là phương pháp "viên đạn bạc" hay không? Chắc chắn là không. Thứ nhất, phương pháp này làm tốt công việc phân loại nhưng không tốt bằng đối với thuật toán hồi quy, vì thuật toán này không đưa ra các dự đoán chính xác về tính chất liên tục như mong muốn. Đối với các nhà mô hình hóa thống kê, rừng ngẫu nhiên có thể bị coi là một phương pháp hộp đen, do đó có rất ít quyền kiểm soát đối với cách mô hình đưa ra quyết định. Tốt nhất, có thể thử các tham số khác nhau và hy vọng rằng điều đó sẽ thay đổi kết quả đầu ra.

1.12.7. Phân loại với Naïve Bayes

Phân loại Bayes là một phương pháp học có giám sát cũng như một phương pháp thống kê cho phân loại. Đây là một kỹ thuật phân loại dựa trên định lý Bayes với giả định về sự độc lập giữa các biến dự báo. Ở đây, tất cả các thuộc tính đóng góp một cách cân bằng và độc lập vào quyết định dự đoán. Nói một cách đơn giản, bộ phân loại Naïve Bayes giả định rằng sự hiện diện của một đặc tính cụ thể trong một lớp không liên quan đến sự hiện diện của bất kỳ đặc tính nào khác. Ví dụ, một quả táo cây có thể được coi là một quả táo nếu nó có màu đỏ, hình tròn và có đường kính khoảng ba inch. Ngay cả khi những đặc tính này phụ thuộc vào nhau hoặc vào sự tồn tại của những đặc tính khác, tất cả các đặc tính này độc lập đóng góp vào xác suất rằng quả táo cây này là táo, và đó là lý do tại sao nó được gọi là naïve (ngây thơ). Hóa ra, trong hầu hết các trường hợp, mặc dù giả định này không đúng, các mô hình phân loại kết quả lại hoạt động rất tốt.

Hãy cùng nhìn vào định lý Bayes cung cấp một cách tính xác suất hậu nghiệm $P(c|x)$ từ $P(c)$, $P(x)$ và $P(x|c)$. Xem phương trình dưới đây:

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

Ở đây:

- $P(c|x)$ là xác suất hậu nghiệm của lớp c (mục tiêu) dựa trên biến phụ thuộc x (thuộc tính).
- $P(c)$ là xác suất tiên nghiệm của lớp.
- $P(x|c)$ Khả năng xảy ra (Likelihood) chính là xác suất của biến dự đoán (predictor) khi biết trước lớp (class).
- $P(x)$ là xác suất tiên nghiệm của bộ dữ liệu.

Và đây là giả định naïve: có thể chia $P(c|x)$ thành các phần độc lập, trong đó $x_1, x_2, x_3, \dots, x_n$ là các xác suất tiên nghiệm độc lập.

$$P(c|x) = \frac{P(x_1|c)P(x_2|c)P(x_3|c) \cdots P(x_n|c)P(c)}{P(x)}$$

Để hiểu cách Naïve Bayes hoạt động, hãy cùng xem lại bộ dữ liệu **golf** để xem cách thuật toán này hoạt động từng bước.

Outlook	Temperature	Humidity	Windy	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Hình 68: Bảng dữ liệu về thời tiết ảnh hưởng đến khả năng chơi golf

Như được thể hiện trong bảng trên, bộ dữ liệu có bốn thuộc tính, đó là **Outlook**, **Temperature**, **Humidity**, và **Windy**, tất cả đều là các yếu tố khác nhau của điều kiện

thời tiết. Dựa trên bốn thuộc tính này, mục tiêu là dự đoán giá trị của biến kết quả, **Play** (có hoặc không) – liệu thời tiết có phù hợp để chơi golf hay không. Dưới đây là các bước của thuật toán mà chúng ta có thể sử dụng để đạt được mục tiêu đó.

- **Bước 1:** Đầu tiên, chuyển đổi bộ dữ liệu thành bảng tần suất (Hình 69)
- **Bước 2:** Tạo bảng xác suất bằng cách tìm các xác suất, chẳng hạn xác suất có nhiệt độ nóng là 0.29 và xác suất chơi golf là 0.64 như thể hiện trong Hình 69.
- **Bước 3:** Bây giờ, sử dụng phương trình **Naïve Bayesian** để tính toán xác suất hậu nghiệm cho mỗi lớp. Lớp có xác suất hậu nghiệm cao nhất sẽ là kết quả của dự đoán.

Temperature	Play	Frequency Table			Likelihood Table				
		Temperature	No	Yes	Temperature	No	Yes		
Hot	no	Hot	2	2	Hot	2	2	4/14	0.29
Hot	yes	Mild	2	4	Mild	2	4	6/14	0.43
Mild	yes	Cool	1	3	Cool	1	3	4/14	0.29
Cool	yes	Total	5	9	All	5	9		
Cool	no					5/14	9/14		
Cool	yes					0.36	0.64		
Mild	no								
Cool	yes								
Mild	yes								
Mild	yes								
Mild	yes								
Hot	yes								
mild	no								

Hình 69: Bảng tần suất

Để kiểm tra điều này trong thực tế, giả sử chúng ta cần quyết định liệu có nên ra ngoài chơi khi thời tiết ôn hòa dựa trên bộ dữ liệu. Chúng ta có thể giải quyết điều này bằng cách sử dụng phương pháp xác suất hậu nghiệm đã thảo luận ở trên. Sử dụng định lý **Bayes** thu được

$$P(\text{Yes} | \text{Mild}) = \frac{P(\text{Mild} | \text{Yes}) \times P(\text{Yes})}{P(\text{Mild})}$$

Ở đây chúng ta có:

$$P(\text{Mild} | \text{Yes}) = 4/9 = 0.44$$

$$P(\text{Mild}) = 6/14 = 0.43$$

Bây giờ, chúng ta có:

$$P(\text{Yes} | \text{Mild}) = (0.44 \times 0.64) / 0.43 = 0.65.$$

Nói cách khác, có thể suy luận được rằng xác suất chơi (play) khi thời tiết ôn hòa là 65%. Nếu muốn chuyển điều này thành một quyết định "Có-Không" (Yes-No), có thể

thấy xác suất này cao hơn điểm giữa là 50%. Do đó, chúng ta có thể đưa ra câu trả lời là "Có".

Thuật toán Naïve Bayes sử dụng một phương pháp tương tự để dự đoán xác suất của các lớp khác nhau dựa trên nhiều thuộc tính. Thuật toán này thường được sử dụng trong phân loại văn bản và các bài toán có từ hai lớp trở lên. Một ví dụ nổi bật là phát hiện thư rác (spam detection). Lọc thư rác bằng Naïve Bayes là một bài toán hai lớp, tức là xác định một tin nhắn hoặc email là thư rác (spam) hay không. Dưới đây là cách thức hoạt động của thuật toán:

Giả sử có một số từ (ví dụ, “viagra”, “rich” và “friend”) chỉ ra rằng một tin nhắn là thư rác. Chúng ta có thể áp dụng định lý Bayes để tính xác suất một email là thư rác dựa trên các từ trong email như sau:

$$\begin{aligned} &P(\text{spam} \mid \text{words}) \\ &= \frac{P(\text{words} \mid \text{spam}) \times P(\text{spam})}{P(\text{words})} \\ &= \frac{P(\text{spam}) \times P(\text{viagra, rich, ..., friend} \mid \text{spam})}{P(\text{viagra, rich, ..., friend})} \\ &\propto P(\text{spam}) \times P(\text{viagra, rich, ..., friend} \mid \text{spam}) \end{aligned}$$

Ở đây, $\propto$ là ký hiệu tỷ lệ.

Theo Naïve Bayes, các sự kiện từ hoàn toàn độc lập với nhau; do đó, việc đơn giản hóa công thức trên bằng cách sử dụng định lý Bayes sẽ trông như sau:

$$P(\text{spam} \mid \text{words}) \propto P(\text{viagra} \mid \text{spam}) \times P(\text{rich} \mid \text{spam}) \times \dots \times P(\text{friend} \mid \text{spam})$$

Bây giờ, chúng ta có thể tính $P(\text{viagra} \mid \text{spam})$, $P(\text{rich} \mid \text{spam})$, và $P(\text{friend} \mid \text{spam})$ mỗi giá trị một cách riêng biệt nếu chúng ta có một bộ dữ liệu huấn luyện đủ lớn gồm các tin nhắn thư rác đã được phân loại trước và các lần xuất hiện của những từ này (“viagra,” “rich,” “friend,” v.v.) trong bộ dữ liệu huấn luyện. Vì vậy, có thể xác định xác suất một email từ bộ kiểm tra là thư rác dựa trên các giá trị này.

Naïve Bayes hoạt động rất tốt mặc dù giả định về sự độc lập rõ ràng là không đúng, vì phân loại không cần các ước lượng xác suất chính xác miễn là xác suất lớn nhất được gán cho lớp đúng. Naïve Bayes cho phép xây dựng và đánh giá mô hình nhanh chóng và có thể được sử dụng cho cả vấn đề phân loại nhị phân và phân loại nhiều lớp.

Ví dụ: Naïve Bayes

Hãy sử dụng dữ liệu **golf** trong để khám phá cách thực hiện phân loại Naïve Bayes trong R. Bộ dữ liệu này được tải tại đây: <https://www.kaggle.com/dsheemanthbhat/simple-weather-forecast>. Chú ý cần sửa dữ liệu sau khi tải: Windy với Weak là False, Strong là True.

Đầu tiên, bạn cần nhập bộ dữ liệu vào **RStudio**:

```
> golf <- read.csv(file = "golf.csv", header = TRUE, sep = ",")  
> View(golf)
```

Có một số gói thư viện trong **R** hỗ trợ phân loại **Naïve Bayes**. Trong ví dụ này, bạn sẽ sử dụng gói thư viện "**e1071**". Trong trường hợp bạn chưa có gói này trong hệ thống, hãy chắc chắn cài đặt thư viện này trước.

```
> install.packages("e1071")  
> library(e1071)
```

Khi bạn đã cài đặt xong, việc xây dựng mô hình **Naïve Bayes** trong **R** là đơn giản và dễ dàng hơn. Tuy nhiên, khi xây dựng mô hình, bạn sẽ cần dữ liệu kiểm tra để đánh giá mô hình của mình. Vì bạn không có dữ liệu kiểm tra ở đây, chúng ta sẽ tách dữ liệu huấn luyện và kiểm tra từ bộ dữ liệu gốc. Dưới đây là cách thực hiện điều đó:

```
> set.seed(123)  
> id <- sample(2, nrow(golf), prob = c(0.7, 0.3), replace = T)  
> golfTrain <- golf[id == 1,]  
> golfTest <- golf[id == 2,]
```

Hãy kiểm tra các bộ dữ liệu huấn luyện và kiểm tra vừa xây dựng. Từ dữ liệu gốc, có 14 điểm dữ liệu, có 9 điểm dữ liệu dành cho huấn luyện, chiếm 70%, phần còn lại vào bộ kiểm tra. Các điểm dữ liệu riêng lẻ sẽ vào bộ huấn luyện và bộ kiểm tra phụ thuộc vào cách lấy mẫu dữ liệu.

```
> View(golfTest)  
> View(golfTrain)
```

Tiếp theo, chúng ta sẽ xây dựng mô hình trên dữ liệu huấn luyện và tiến hành đánh giá.

```
> golfModel <- naiveBayes(PlayGolf~., data = golfTrain)  
> print(golfModel)
```

Kết quả này sẽ tạo ra đầu ra tương tự như sau.

Naive Bayes Classifier for Discrete Predictors

Call:

```
naiveBayes.default(x = X, y = Y, laplace = laplace)
```

A-priori probabilities:

Y

No	Yes
0.3333333	0.6666667

Conditional Probabilities

Outlook

Y	Overcast	Rainy	Sunny
No	0.000000	0.333333	0.666667
Yes	0.666667	0.166667	0.166667

Temp

Y	Cool	Hot	Mild
No	0.333333	0.333333	0.333333
Yes	0.333333	0.333333	0.333333

Humidity

Y	High	Normal
No	0.666667	0.333333
Yes	0.333333	0.666667

Windy

Y	FALSE	TRUE
No	0.333333	0.666667
Yes	0.666667	0.333333

Kết quả bao gồm một bảng xác suất khả năng cũng như các xác suất tiên nghiệm. Các xác suất tiên nghiệm này tương đương với xác suất tiên nghiệm trong định lý Bayes. Cụ thể, đó là tần suất xuất hiện của mỗi mức lớp trong bộ dữ liệu huấn luyện. Lý thuyết cơ bản đằng sau xác suất tiên nghiệm là, nếu một mức lớp hiếm trong bộ huấn luyện, thì ít có khả năng mức lớp đó sẽ xuất hiện trong bộ dữ liệu kiểm tra. Nói cách khác, dự đoán về một kết quả không chỉ bị ảnh hưởng bởi các biến dự báo, mà còn bởi sự phổ biến của kết quả đó.

Hãy tiếp tục với việc đánh giá mô hình này bằng cách sử dụng bộ kiểm tra cho việc đánh giá kết quả huấn luyện.

```
> prediction <- predict(golfModel, newdata = golfTest)
```

Hãy kiểm tra nhãn lớp mà mô hình đã dự đoán cho tất cả các điểm dữ liệu trong bộ kiểm tra:

```
> print(prediction)
```

Tuy nhiên, sẽ dễ dàng hơn nếu so sánh các nhãn lớp dự đoán với các nhãn thực tế

bên cạnh nhau, hoặc trong một ma trận nhầm lẫn. May mắn thay, trong **R** có một gói thư viện hỗ trợ tính năng này, gọi là **caret**. Một lần nữa, nếu chưa có gói này, hãy chắc chắn cài đặt nó trước. Lưu ý rằng việc cài đặt **caret** yêu cầu một số gói thư viện phụ thuộc như **lattice** và **ggplot2**. Khi đã cài đặt tất cả các gói cần thiết, sử dụng lệnh sau:

```
> confusionMatrix(prediction, golfTest$PlayGolf)
```

Và bạn sẽ có một ma trận nhầm lẫn đẹp mắt cùng với các giá trị p và tất cả các chỉ số đánh giá khác.

Confusion Matrix and Statistics

Reference

Prediction No Yes

No 2 2

Yes 0 1

Accuracy : 0.6

95% CI : (0.1466, 0.9473)

No Information Rate : 0.6

P-Value [Acc > NIR] : 0.6826

Kappa : 0.2857

Mcnemar's Test P-Value : 0.4795

Sensitivity : 1.0000

Specificity : 0.3333

Pos Pred Value : 0.5000

Neg Pred Value : 1.0000

Prevalence : 0.4000

Detection Rate : 0.4000

Detection Prevalence : 0.8000

Balanced Accuracy : 0.6667

'Positive' Class : No

Như có thể thấy, độ chính xác không cao, chỉ 60%, điều này có thể do chỉ có chín ví dụ trong bộ huấn luyện. Đây cũng là cách xây dựng bộ phân loại **Naïve Bayes** trong **R**.

1.12.8. Phân loại với Máy vector hỗ trợ (SVM)

Đây cũng là phương pháp phân loại cuối cùng trong phần này. Một điểm chung trong tất cả các mô hình phân loại tính đến thời điểm này là các mô hình này giả định việc phân tách các lớp theo tuyến tính. Nói cách khác, các mô hình này cố gắng tìm ra một ranh giới quyết định là một đường thẳng (hoặc một siêu phẳng trong không gian cao hơn). Tuy nhiên, nhiều vấn đề không có đặc tính tuyến tính như vậy. Máy Vector Hỗ trợ (Support Vector Machine - SVM) là một phương pháp phân loại có khả năng xử lý cả dữ liệu tuyến tính và phi tuyến tính (nonlinear). SVM được nhiều người coi là bộ phân loại cơ bản tốt nhất để thực hiện các tác vụ học máy. "Cơ bản" ở đây có nghĩa là ở dạng nguyên thủy, chưa được tùy chỉnh. Điều này có nghĩa là có thể lấy dạng cơ bản của bộ phân loại này, chạy trên dữ liệu, và kết quả thường sẽ có tỷ lệ lỗi thấp.

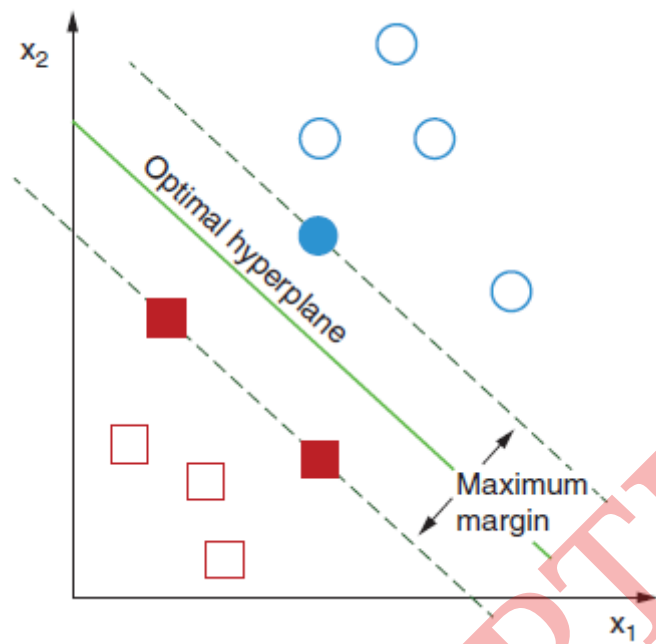
Tóm lại, SVM là một thuật toán sử dụng ánh xạ phi tuyến để chuyển đổi dữ liệu huấn luyện ban đầu thành không gian cao hơn. Trong không gian mới này, phương pháp này tìm kiếm siêu phẳng phân tách tối ưu tuyến tính (tức là một đường biên quyết định phân tách các véc-tơ của lớp này với lớp khác). Với một ánh xạ phi tuyến tính phù hợp đến một chiều đủ cao, dữ liệu từ hai lớp luôn có thể được phân tách bằng một siêu phẳng. SVM tìm siêu phẳng này bằng cách sử dụng các véc-tơ hỗ trợ (các véc-tơ huấn luyện "thiết yếu") và biên độ (được xác định bởi các véc-tơ hỗ trợ).

Để hiểu hơn về thuật toán, hãy bắt đầu với một ví dụ đơn giản phân loại hai lớp.

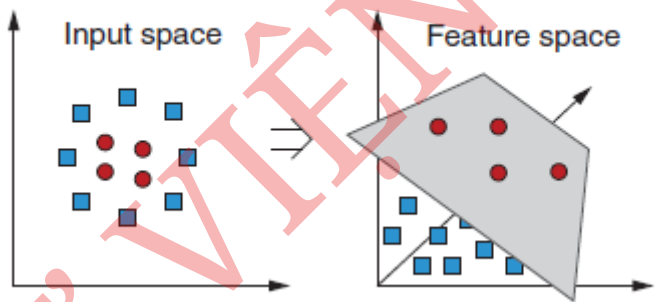
Giả sử bộ dữ liệu D được cho dưới dạng $(X_1, y_1), (X_2, y_2), \dots, (X_{|D|}, y_{|D|})$, trong đó X_i là tập hợp các bộ dữ liệu huấn luyện với nhãn lớp tương ứng y_i . Mỗi y_i có thể nhận một trong hai giá trị, là +1 hoặc -1 (tức là, $y_i \in \{+1, -1\}$), tương ứng với các lớp được biểu diễn bởi các hình vuông đỏ rỗng và các hình tròn xanh (bỏ qua các điểm dữ liệu được biểu diễn bằng các ký hiệu đặc trong lúc này), như trong **Hình 70**. Từ đồ thị, ta thấy rằng dữ liệu trên biểu đồ 2D dễ dàng được phân tách bởi một đường thẳng, đường này tách các lớp +1 khỏi tất cả các bộ dữ liệu của lớp -1.

Trong không gian n chiều với n thuộc tính, mục tiêu là tìm siêu phẳng $(n-1)$ chiều tối ưu để làm ranh giới quyết định. Tuy nhiên, thuật ngữ siêu phẳng thường được sử dụng để chỉ ranh giới quyết định này, không phụ thuộc vào số lượng thuộc tính đầu vào. Vấn đề đặt ra là: xác định siêu phẳng tối ưu như thế nào?

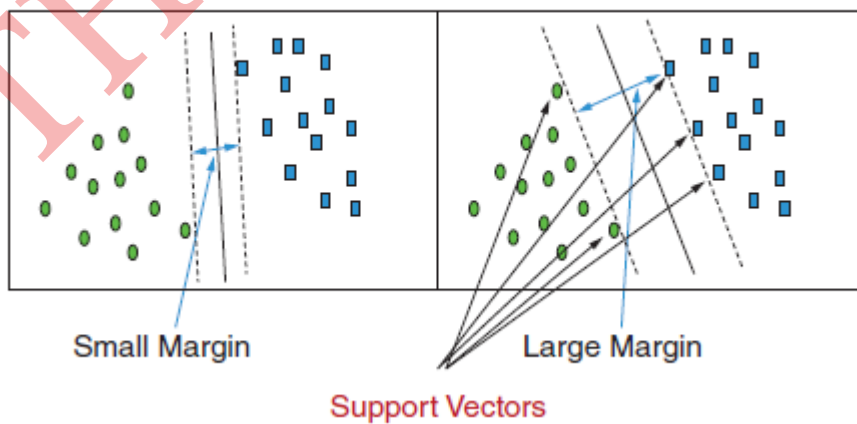
SVM tiếp cận bài toán này bằng cách tìm kiếm siêu phẳng có **biên độ** lớn nhất. Hãy xem xét Hình 72, minh họa hai siêu phẳng phân tách khả thi cùng với các biên độ tương ứng của chúng. Trước khi đi sâu vào định nghĩa biên độ, hãy quan sát trực quan Hình 72. Cả hai siêu phẳng đều có khả năng phân loại chính xác tất cả các véc-tơ dữ liệu đã cho.



Hình 70: Siêu phẳng(Hyperplane) phân tách dữ liệu



Hình 71: Từ đường đến mặt phẳng phân tách trên không gian ba chiều



Hình 72: Các mặt phẳng và các đường biên có thể được xác định.

Tuy nhiên, theo trực giác, chúng ta kỳ vọng rằng siêu phẳng có **biên độ lớn hơn** sẽ có độ chính xác cao hơn trong việc phân loại các véc-tơ dữ liệu tương lai so với siêu phẳng có biên độ nhỏ hơn. Đó là lý do tại sao (trong giai đoạn học hoặc huấn luyện), **SVM** tìm kiếm siêu phẳng với biên độ lớn nhất, tức là **siêu phẳng biên độ tối đa (MMH)**. Biên độ tương ứng này cung cấp sự phân tách lớn nhất giữa các lớp.

Các điểm dữ liệu gần nhất với siêu phẳng phân tách được gọi là **véc-tơ hỗ trợ**. Mục tiêu là có được **biên độ** lớn nhất có thể, vì nếu có sai sót hoặc mô hình được huấn luyện trên dữ liệu hạn chế, chúng ta muốn mô hình đạt độ vững chắc cao nhất. Khi đã xác định được mục tiêu là tối đa hóa khoảng cách từ đường phân tách đến các véc-tơ hỗ trợ, chúng ta cần tìm một phương pháp để tối ưu hóa bài toán này; cụ thể là, làm thế nào để tìm ra một **SVM** với **siêu phẳng biên độ tối đa (MMH)** và **các véc-tơ hỗ trợ** tương ứng. Siêu phẳng được thể hiện qua hàm dưới đây

$$f(x) = \beta_0 + \beta^T x$$

Trong đó, T là **véc-tơ trọng số**, với $T = \{1, 2, 3, \dots\}$; n là số lượng thuộc tính; và β_0 là một hằng số **vô hướng**, thường được gọi là **độ lệch (bias)**. Siêu phẳng tối ưu có thể được biểu diễn theo một số cách vô hạn bằng cách thay đổi tỷ lệ của β và β_0 . Theo quy ước, trong tất cả các cách biểu diễn có thể của siêu phẳng, siêu phẳng được chọn là:

$$\beta_0 + \beta^T x = 1$$

Trong đó, x tượng trưng cho các mẫu huấn luyện nằm gần nhất với siêu phẳng. Nhìn chung, các mẫu huấn luyện gần siêu phẳng nhất được gọi là véc-tơ hỗ trợ. Cách biểu diễn này được gọi là siêu phẳng chuẩn tắc.

Bây giờ, từ hình học, chúng ta biết rằng khoảng cách d giữa một điểm $(\mathbf{m}, \mathbf{n})$ và một đường thẳng được biểu diễn bởi $\mathbf{Ax} + \mathbf{By} + \mathbf{C} = 0$ được cho bởi công thức:

$$d = \frac{|Am + Bn + C|}{\sqrt{A^2 + B^2}}$$

Do đó, mở rộng phương trình tương tự cho một siêu phẳng sẽ cho ta công thức tính khoảng cách giữa một điểm x và siêu phẳng đó:

$$d = \frac{|\beta_0 + \beta^T x|}{\|\beta\|}$$

Cụ thể, đối với siêu phẳng chuẩn tắc, từ số bằng một và khoảng cách đến các vector hỗ trợ là:

$$d_{\text{support vectors}} = \frac{|\beta_0 + \beta^T x|}{\|\beta\|}$$

Bây giờ, **biên độ M** là gấp đôi khoảng cách đến các ví dụ gần nhất. Vì vậy,

$$M = \frac{2}{\|\beta\|}$$

Cuối cùng, bài toán tối đa hóa biên độ M tương đương với bài toán tối thiểu hóa một hàm số $L()$ phải tuân theo một số ràng buộc. Các ràng buộc này mô hình hóa yêu cầu siêu phẳng phải phân loại chính xác tất cả các mẫu huấn luyện x_i

$$\min_{\beta, \beta_0} L(\beta) = \frac{1}{2} \|\beta\|^2 \quad \text{subject to} \quad y_i(\beta^T x_i + \beta_0) \geq 1 \quad \forall i$$

Phần trên là tóm tắt lý thuyết về **Máy Véc-tơ Hỗ trợ (SVM)**, được trình bày với mục đích chính là xây dựng trực giác về cách hoạt động của **SVM** và tổng quát hơn là các **bộ phân loại biên độ tối đa**. Hãy cùng xem xét ví dụ thực tế sau

Ví dụ thực hành 9.7: SVM

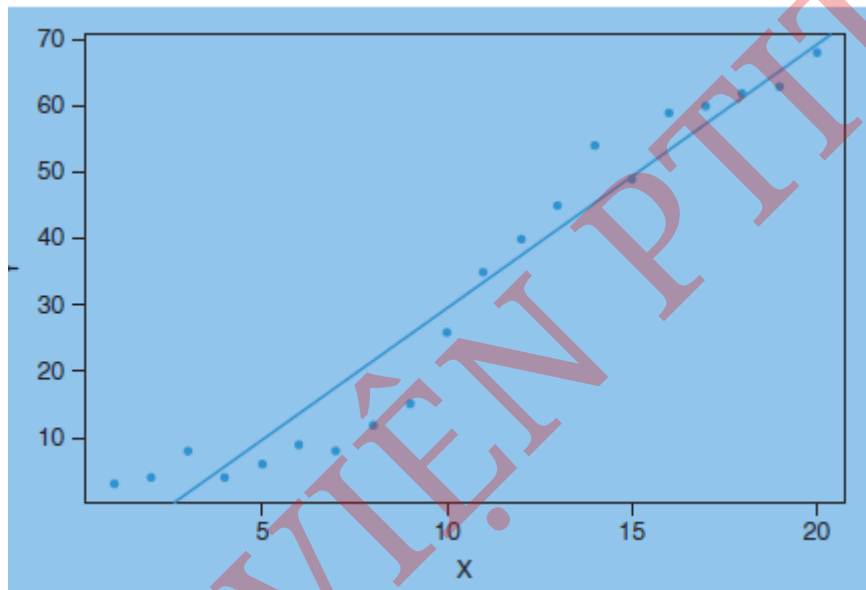
Xem xét một bài toán phân loại đơn giản sử dụng dữ liệu **regression.csv** với chỉ hai thuộc tính **X** và **Y**. Chúng ta có thể sử dụng **R** để hiển thị dữ liệu:

```
# Nạp dữ liệu từ file csv
data <- read.csv('regression.csv', header = TRUE, sep=";")
# Vẽ dữ liệu
plot(data, pch=16)
# Tạo mô hình hồi quy
model <- lm(y ~ x, data)
# Add the fitted line
abline(model)
```

Bảng 13: Bộ dữ liệu regression.csv

X	Y
1	3
2	4
3	8
4	4
5	6
6	9
7	8
8	12
9	15
10	26
11	35

X	Y
12	40
13	45
14	54
15	49
16	59
17	60
18	62
19	63
20	68



Hình 73: Đường thẳng hồi quy được vẽ trên dữ liệu

Ở đây, chúng ta đã thử sử dụng một mô hình hồi quy tuyến tính đầu tiên để phân loại dữ liệu như được thể hiện trong **Hình 73**. Để có thể so sánh hồi quy tuyến tính với **máy vector hỗ trợ (SVM)**, trước tiên cần một cách để đo lường độ chính xác của mô hình.

Để làm điều đó, chúng ta sẽ thay đổi một chút mã của mình để trực quan hóa mỗi dự đoán được đưa ra bởi mô hình:

```
# Vẽ lại các điểm dữ liệu
plot(data, pch=16)
# Dự đoán cho từng điểm X
predictedY <- predict(model, data)
# Hiển thị dự đoán
points(data$x, predictedY, col = "blue", pch=4)
```

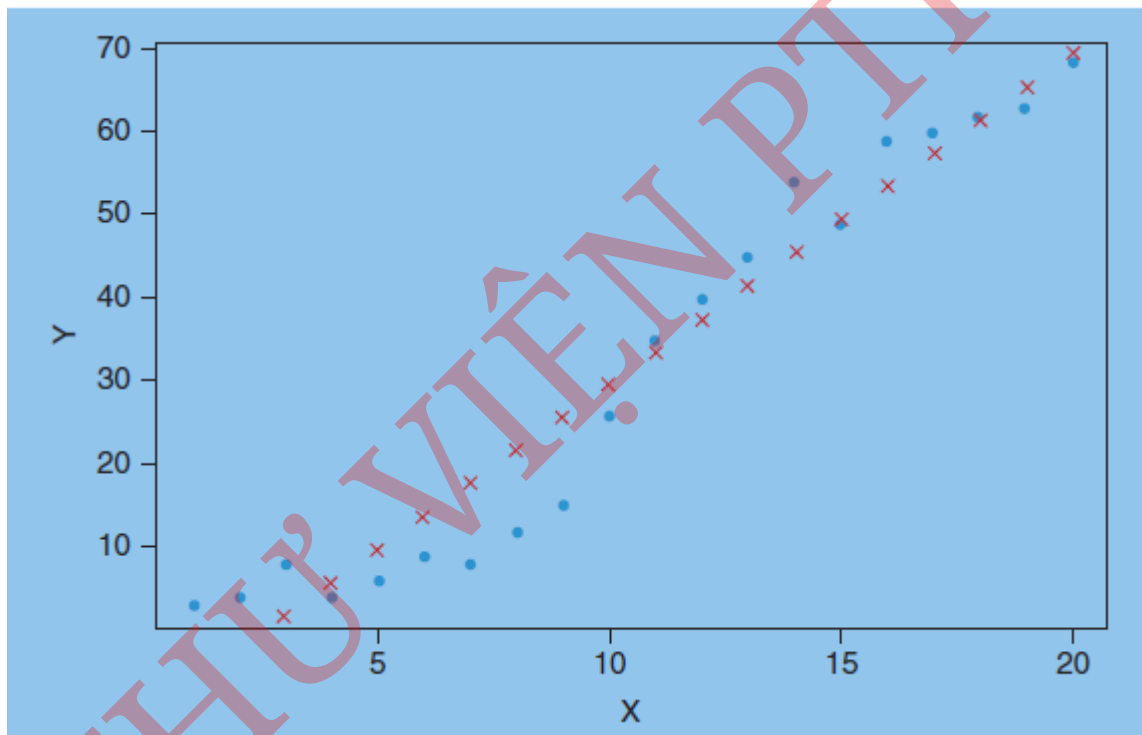
Điều này tạo ra đồ thị như được hiển thị trong **Hình 74**.

Đối với mỗi điểm dữ liệu x_i , mô hình đưa ra một dự đoán y_i được hiển thị dưới dạng các điểm màu xanh trên đồ thị. Sự khác biệt duy nhất so với đồ thị trước là các chấm không được nối với nhau.

Để đánh giá mức độ hiệu quả của mô hình, chúng ta sẽ tính toán số lượng lỗi mà mô hình mắc phải, điều này được thực hiện bằng cách tính **Sai số Căn bậc hai Trung bình Bình phương (Root Mean Square Error - RMSE)**.

Cách tính **RMSE** trong **R** là:

```
rmse <- function(error)
{
  sqrt(mean(error^2))
}
```



Hình 74: Kết quả dự đoán sử dụng mô hình hồi quy tuyến tính với dữ liệu gốc

```
error <- model$residuals # tương tự như data$Y – predictedY
lrPredictionRMSE <- rmse(error)
```

Giá trị **RMSE** của mô hình hồi quy tuyến tính mà chúng ta đã thu được từ **R** là 5.70. Hãy thử cải thiện hiệu quả với **SVM**.

Để tạo một mô hình **SVM** với **R**, cần cài đặt gói thư viện "**e1071**". Vì vậy, trước khi tiếp tục với các bước tiếp theo, hãy chắc chắn cài đặt gói này và thêm vào thư viện ngay từ đầu trong tệp của bạn.

Dưới đây là mã để tạo một mô hình với **SVM** trong **R**, sau khi gói "**e1071**" đã được cài đặt và tải vào phiên làm việc hiện tại:

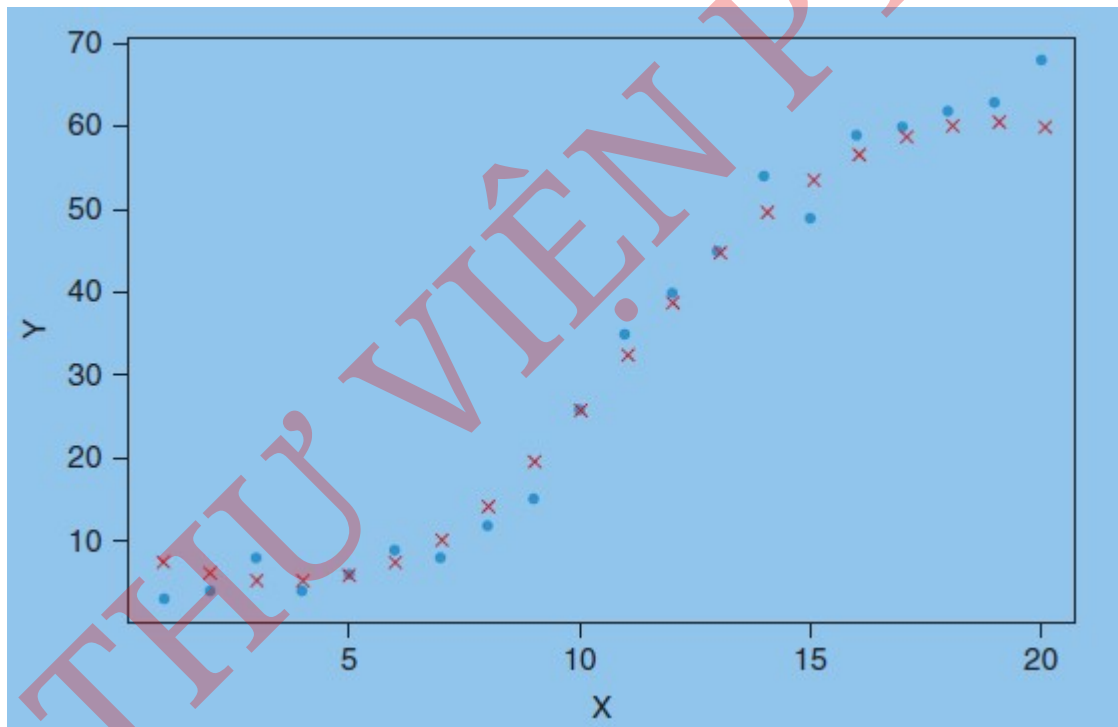
```
model2 <- svm(y ~ x, data)
predictedY <- predict(model2, data)
points(data$x, predictedY, col = "red", pch=4)
```

Đồ thị được hiển thị trong **Hình 75**.

Lần này, dự đoán gần với các giá trị thực tế hơn so với phương pháp trước. Hãy tính toán **RMSE** của mô hình **hồi quy máy vector hỗ trợ (SVR)** của chúng ta.

```
error <- data$y - predictedY
svmPredictionRMSE <- rmse(error) # 3.157061
```

Có thể thấy rằng **SVM** cho giá trị **RMSE** thấp hơn: 3.15 so với 5.70 của hồi quy tuyến tính.



Hình 75: Sử dụng phương pháp SVM dự đoán trên tập dữ liệu gốc

1.13. Học không giám sát

1.13.1. Giới thiệu

Trong phần trước, các mô hình học máy được xây dựng dựa trên dữ liệu đã có nhãn hoặc giá trị thực. Nói cách khác, thông tin đúng/sai đã được cung cấp để huấn luyện mô hình hồi quy hoặc phân loại, từ đó mô hình có thể dự đoán cho dữ liệu mới. Quá trình này được gọi là học có giám sát (supervised learning).

Phần này sẽ tìm hiểu một lĩnh vực khác của học máy: học không giám sát (unsupervised learning). Khác với học có giám sát, dữ liệu trong học không giám sát không có nhãn thực. Mục tiêu của các thuật toán ở đây là khám phá và mô tả cấu trúc tiềm ẩn trong dữ liệu. Điều này có thể bao gồm việc phân nhóm các điểm dữ liệu tương tự nhau (clustering), hoặc tìm cách biểu diễn dữ liệu phức tạp sao cho dễ hiểu và có tổ chức hơn.

Phân cụm là quá trình gán một tập hợp quan sát vào các nhóm con (cụm) sao cho các quan sát trong cùng cụm có đặc điểm tương đồng. Đây là một trong những kỹ thuật phổ biến nhất của học không giám sát, được ứng dụng rộng rãi trong phân tích dữ liệu thống kê và nhiều lĩnh vực khác.

Trong phần này, chúng ta sẽ xem xét hai loại thuật toán phân cụm chính:

- Phân cụm kết hợp (Agglomerative – đi từ dưới lên)
- Phân cụm phân chia (Divisive – đi từ trên xuống)

Ngoài ra, sẽ thảo luận thêm về một kỹ thuật quan trọng là Cực đại hóa kỳ vọng (Expectation Maximization – EM). EM đặc biệt hữu ích khi dữ liệu chứa nhiều biến ẩn và không có đủ thông tin hướng dẫn để giải thích trực tiếp; khi đó, mô hình sẽ ước lượng tham số theo cách lặp lại để dần tiệm cận nghiệm tốt nhất.

Tất cả các phương pháp trên đều minh họa cho phương pháp học không giám sát, nơi mà dữ liệu không có nhãn nhưng mục tiêu vẫn là khám phá cấu trúc tiềm ẩn và tìm ra cách diễn giải hợp lý.

1.13.2. Phân cụm tích hợp

Phân cụm tích hợp (Agglomerative Clustering) là một phương pháp tiếp cận từ dưới lên trong việc xây dựng các cụm dữ liệu. Quá trình này bắt đầu bằng việc coi mỗi điểm dữ liệu là một cụm riêng lẻ, sau đó lặp lại việc gộp dần các cụm gần nhau nhất cho đến khi đạt đến số cụm mong muốn hoặc thỏa mãn một tiêu chí dừng nào đó.

1. Sử dụng một phép đo độ tương đồng giữa các cụm $d(C_i, C_j)$ có thể tính toán được, ví dụ: khoảng cách Euclidean, độ tương đồng cosine, v.v.
2. Với n đối tượng $v_1, v_2, \dots, v_n$ gán mỗi đối tượng vào một cụm đơn lẻ $C_i = \{v_i\}$.
3. Lặp lại {

- Xác định hai cụm tương đồng nhất C_j và C_k (có thể có nhiều cặp tương đồng nhất - chọn một cặp)
 - Xóa C_j và C_k và thêm cụm hợp nhất $(C_j \cup C_k)$ vào tập hợp các cụm.
- } cho đến khi chỉ còn lại duy nhất một cụm.
4. Sử dụng biểu đồ cây quyết định nhị phân (dendogram) để hiển thị trình tự các lần hợp nhất cụm.

Ví dụ

Trong ví dụ này, chúng ta sẽ sử dụng năm điểm dữ liệu. Khoảng cách giữa từng cặp điểm được thể hiện trong ma trận khoảng cách trên.

Ở đây, không cần quan tâm đến ý nghĩa cụ thể của các dữ liệu; bảng được đưa ra chỉ nhằm minh họa cách hoạt động của thuật toán phân cụm đã mô tả ở phần trên.

Đúng như dự kiến, ma trận khoảng cách là đối xứng, vì khoảng cách từ điểm x đến điểm y bằng khoảng cách từ y đến x . Trên đường chéo chính, các giá trị bằng 0, bởi khoảng cách từ một điểm đến chính nó luôn bằng không. Do tính chất đối xứng, trên bảng chỉ trình bày tam giác dưới của ma trận; phần tam giác trên chỉ là phản chiếu và không cần lặp lại.

Bảng 14: Tính toán khoảng cách ma trận

	1	2	3	4	5
1	0				
2	8	0			
3	3	6	0		
4	5	5	8	0	
5	13	10	2	7	0

Vì đã có sẵn khoảng cách giữa từng cặp điểm, ta bắt đầu quá trình phân cụm bằng cách gộp các cặp có khoảng cách nhỏ nhất. Theo bảng trên, cặp điểm dữ liệu 3 và 5 gần nhau hơn bất kỳ cặp nào khác, với khoảng cách là 2 (giá trị nhỏ nhất, được gạch chân). Do đó, ở bước đầu tiên, hai điểm này được hợp nhất thành một cụm duy nhất, ký hiệu là 35. Sau bước này, tập hợp các cụm còn lại gồm: 1, 2, 4, và 35. Vì dữ liệu đã thay đổi nên cần tính toán lại ma trận khoảng cách, do đó, lúc này phải có một quy tắc để xác định khoảng cách giữa cụm 35 và các điểm dữ liệu khác. Cách thực hiện ở đây là lấy giá trị lớn nhất trong hai khoảng cách: khoảng cách từ điểm bất kỳ đến 3 và khoảng cách từ điểm đó đến 5. Từ đó, khoảng cách mới giữa cụm 35 và các điểm khác được xác định.

$$ds\ t_{35,i} = \max(ds\ t_{3,i}, ds\ t_{5,i})$$

Dựa trên công thức trên, ta tính toán lại ma trận khoảng cách mới, kết quả được trình bày trong Bảng 15.

Nếu tiếp tục lặp lại quá trình này bằng cách áp dụng cùng công thức cho đến khi toàn bộ các điểm dữ liệu được gộp thành một cụm duy nhất, sẽ thu được sơ đồ phân cụm như minh họa trong Hình 76. Trong sơ đồ này, trục tung (Oy) biểu diễn chiều cao cụm, tức là giá trị khoảng cách giữa các đối tượng tại thời điểm được hợp nhất.

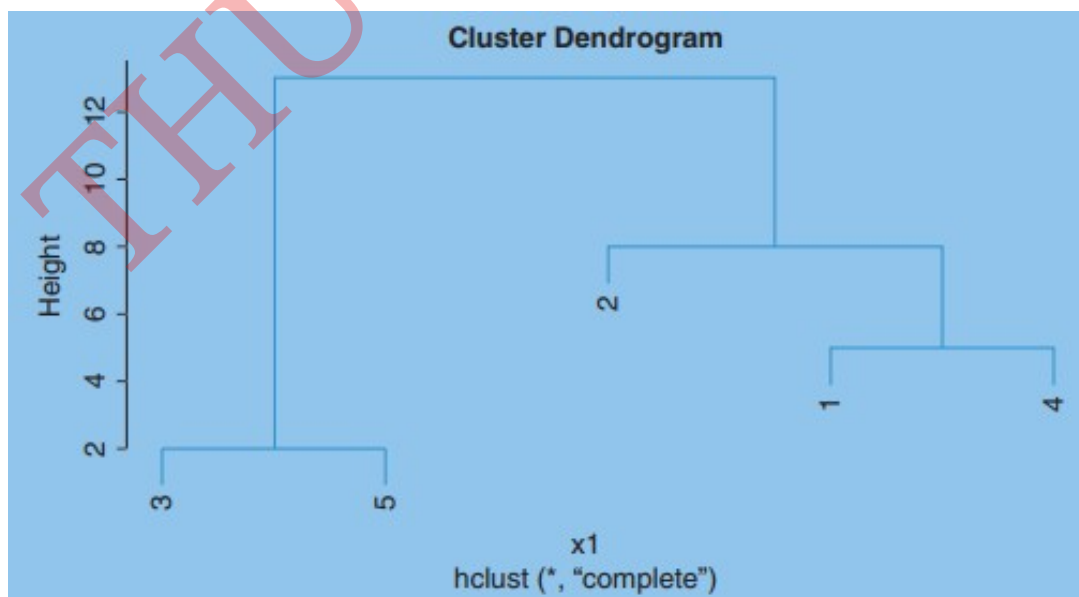
Bảng 15: Tính toán khoảng cách ma trận kế tiếp

	35	1	2	4
35	0			
1	13	0		
2	10	8	0	
4	9	5	5	0

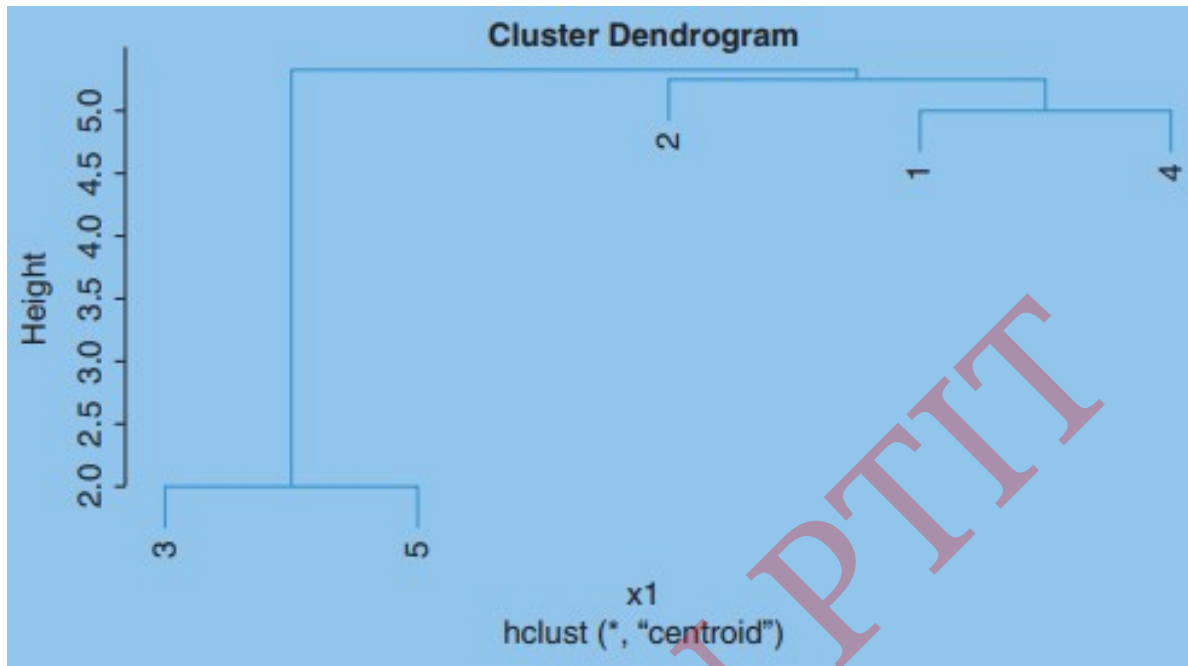
Một lưu ý: Công thức tính khoảng cách cho ma trận có thể khác nhau tùy theo từng bài toán, và tùy thuộc vào công thức được chọn, có thể dẫn đến kết quả phân cụm hoàn toàn khác biệt. Ví dụ, đối với cùng tập dữ liệu trên, nếu tính khoảng cách theo công thức:

$$ds\ t_{35,i} = \min(ds\ t_{3,i}, ds\ t_{5,i})$$

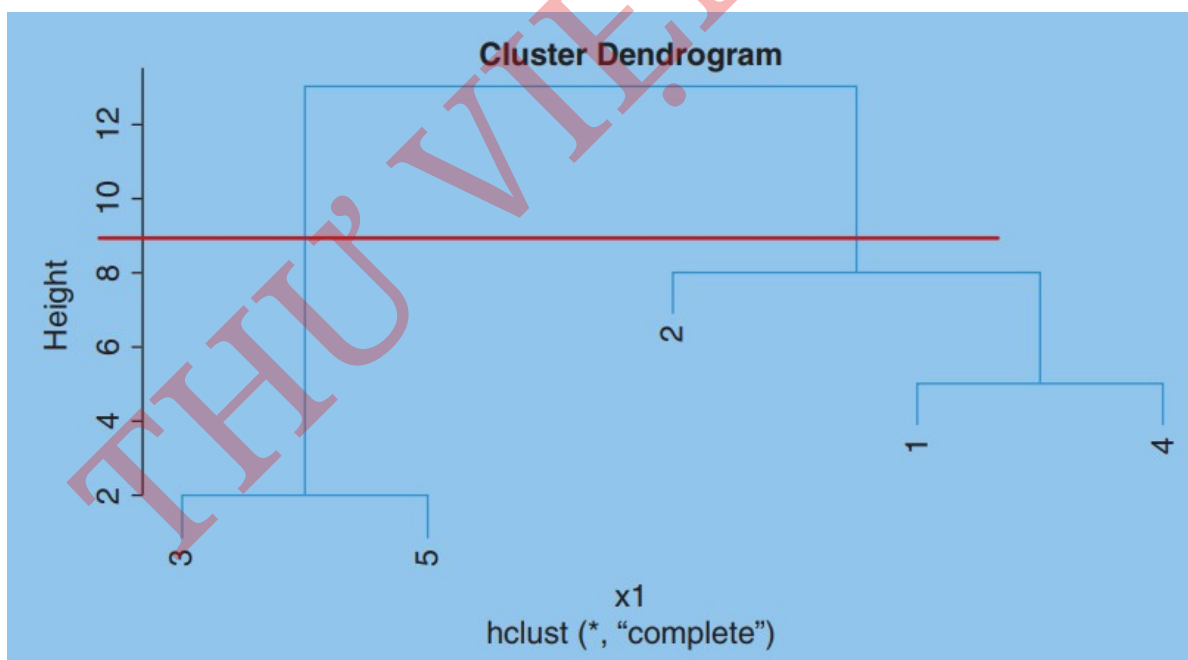
Kết quả được hiển thị như trên Hình 77



Hình 76: Các cụm từ thuật toán phân cụm tích hợp với khoảng cách tối đa



Hình 77: Sơ đồ cụm từ cụm kết tụ với khoảng cách tối thiểu

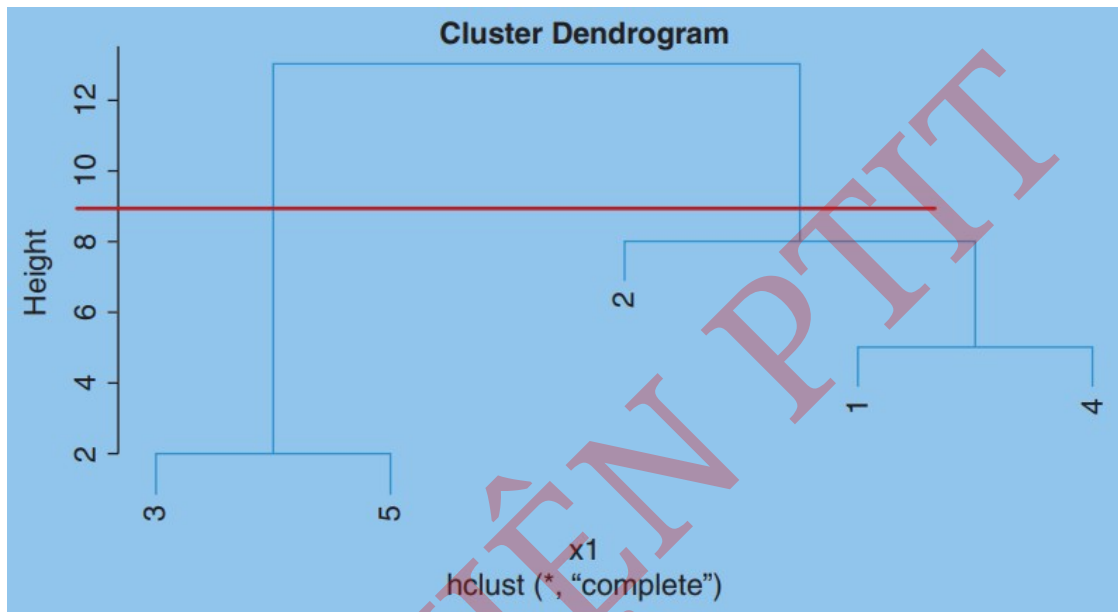


Hình 78: Số lượng cụm khi ngưỡng được đặt ở mức 9.

Một trong những vấn đề thường gặp với phân cụm phân cấp (chẳng hạn như phân cụm tích hợp) là không có một tiêu chuẩn chung để xác định chính xác số lượng cụm k tồn tại. Kết quả phân cụm phụ thuộc hoàn toàn vào cách định nghĩa ngưỡng khoảng

cách tối thiểu để hợp nhất hoặc phân chia hai cụm. Việc này biến quá trình chọn k thành một quyết định chủ quan, thường được thực hiện bằng cách quan sát biểu đồ cây phân cấp (dendrogram). Ví dụ, trong cây liên kết đơn đầu tiên, nếu chúng ta đặt ngưỡng khoảng cách tại 9, ta sẽ cắt cây thành hai cụm như đường kẻ màu đỏ trong Hình 78, và kết quả là hai cụm lớn: (1, 2, 4) và (3, 5).

Trong khi đó, nếu đặt số cụm là 6, kết quả thu được như Hình 79



Hình 79: Số cụm thu được khi ngưỡng đạt mức 6

1.13.3. Phân cụm phân chia

Phân cụm phân chia là phép nghịch đảo của phân cụm tích hợp. Đây là phương pháp tiếp cận từ trên xuống, trong đó tất cả các đối tượng ban đầu được đặt trong một cụm duy nhất và sau đó được chia dần thành các cụm nhỏ hơn.

Cách tiếp cận chung:

1. Đặt tất cả các đối tượng vào một cụm duy nhất.
2. Lặp lại cho đến khi mỗi cụm chỉ còn một phần tử đơn lẻ:
 - Chọn một cụm để tách dựa trên một tiêu chí nhất định.
 - Thay thế cụm đã chọn bằng các cụm con được hình thành.

Mặc dù ý tưởng cơ bản khá đơn giản, nhưng trong thực tế có nhiều vấn đề cần giải quyết, chẳng hạn như: nên tách một cụm thành bao nhiêu cụm con và cách thức thực hiện việc tách đó.

Một trong những thuật toán phổ biến và hiệu quả thường được sử dụng để triển khai cách tiếp cận này chính là k-means clustering. Đây là một thuật toán phân cụm nổi tiếng,

được sử dụng rộng rãi để phân loại hoặc nhóm các đối tượng dựa trên đặc trưng thành k cụm, trong đó k là một số nguyên dương do người dùng xác định.

Thuật toán k -means hoạt động bằng cách tối thiểu hóa tổng bình phương khoảng cách giữa các điểm dữ liệu và tâm cụm (cluster centroid) tương ứng, qua đó đạt được sự phân loại dữ liệu thành các cụm rõ ràng.

Quy trình hoạt động của k -means gồm ba bước cơ bản và được lặp lại cho đến khi hội tụ:

1. Xác định số cụm k : Người dùng quyết định số lượng cụm mong muốn.
2. Khởi tạo cụm ban đầu:
 - Có thể chọn ngẫu nhiên k điểm dữ liệu làm tâm cụm, hoặc chọn k phần tử đầu tiên trong tập dữ liệu.
 - Sau đó, gán mỗi điểm dữ liệu còn lại vào cụm có tâm gần nhất. Sau khi gán xong, cập nhật lại tâm cụm.
3. Lặp lại việc gán và cập nhật:
 - Với mỗi điểm dữ liệu, tính khoảng cách đến tất cả các tâm cụm.
 - Nếu điểm không thuộc cụm có tâm gần nhất, di chuyển điểm sang cụm đó và cập nhật lại tâm của cả hai cụm liên quan.
 - Tiếp tục quá trình cho đến khi không còn sự thay đổi nào trong một lượt quét toàn bộ dữ liệu.

Thuật toán dừng khi đạt đến hội tụ (convergence), tức là không còn điểm dữ liệu nào bị chuyển cụm sau một vòng lặp.

Ví dụ: Phân cụm phân chia

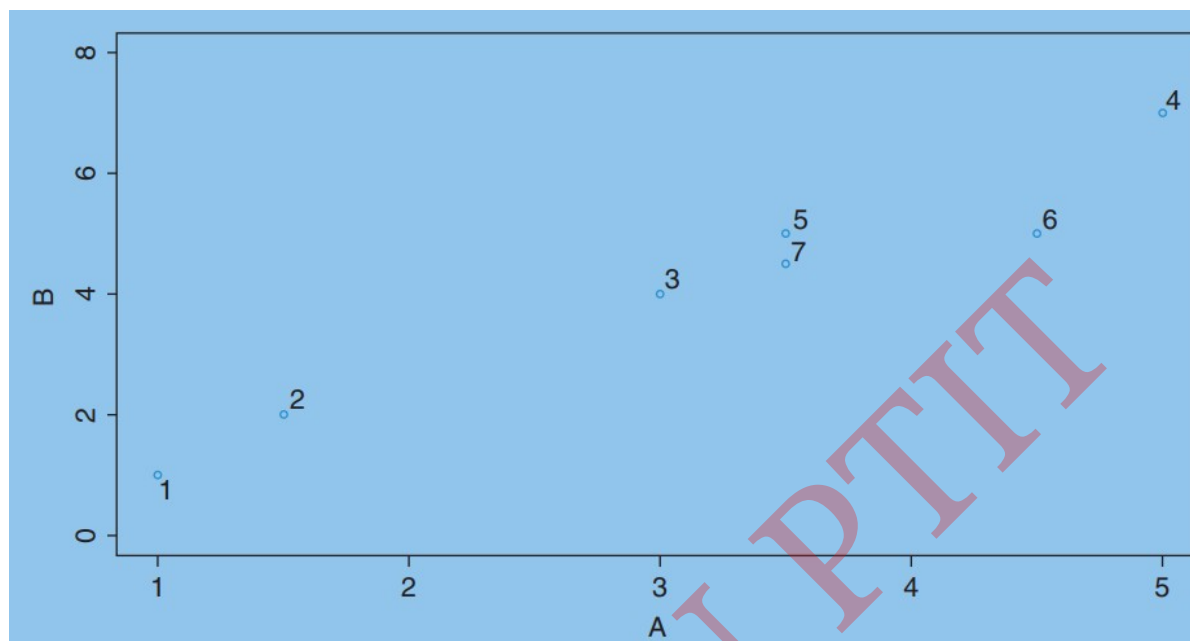
Xét bộ dữ liệu ở Bảng 16 gồm điểm số của hai biến cho bảy cá thể. Ban đầu, cả bảy cá thể được xem như một cụm duy nhất. Mục tiêu là chia dữ liệu thành hai cụm.

Bảng 16: Bộ dữ liệu ví dụ cho thuật toán k -means

Individual	A	B
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Một cách tách hợp lý là chọn hai cá thể xa nhau nhất (theo khoảng cách Euclid)

làm tâm cụm khởi tạo. Khi biểu diễn bảy điểm trên mặt phẳng 2D (Hình 80), có thể thấy cá thể 1 và cá thể 4 cách xa nhau nhất, vì vậy chúng được chọn làm hai tâm cụm ban đầu (centroids); xem Bảng 17.



Hình 80: A so với B được biểu diễn trên đồ thị 2D

Bảng 17: Khởi tạo hai cụm.

	Individual	Vectơ trung bình (trọng tâm)
Cụm 1	1	(1.0, 1.0)
Cụm 2	4	(5.0, 7.0)

Ở bước tiếp theo, lần lượt xét các cá thể còn lại và gán vào cụm có tâm gần nhất (đo bằng khoảng cách Euclid tới trung bình cụm hiện tại). Sau mỗi lần gán, cập nhật vectơ trung bình của cụm vừa nhận thành viên. Lặp lại quy trình cho tới khi không còn cá thể nào cần gán thêm. Toàn bộ các bước được trình bày từng bước trong Bảng 18

Bảng 18: Bước đầu tiên thực hiện với thuật toán k-means

Bước	Cụm 1		Cụm 2	
	Cá nhân	Vectơ trung bình (trọng tâm)	Cá nhân	Vectơ trung bình (trọng tâm)
1	1	(1.0, 1.0)	4	(5.0, 7.0)
2	1, 2	(1.2, 1.5)	4	(5.0, 7.0)
3	1, 2, 3	(1.8, 2.3)	4	(5.0, 7.0)

4	1, 2, 3	(1.8, 2.3)	4, 5	(4.2, 6.0)
5	1, 2, 3	(1.8, 2.3)	4, 5, 6	(4.3, 5.7)
6	1, 2, 3	(1.8, 2.3)	4, 5, 6, 7	(4.1, 5.4)

Sau bước trước, sự phân vùng ban đầu đã thay đổi, và hai cụm hiện tại được thể hiện trong Bảng 19

Bảng 19: Kết quả của bước đầu tiên với thuật toán k-means

	Cá nhân	Vector trung bình (trọng tâm)
Cụm 1	1, 2, 3	(1.8, 2.3)
Cụm 2	4, 5, 6, 7	(4.1, 5.4)

Tuy nhiên, chưa thể khẳng định chắc chắn rằng tất cả các cá thể đều đã được gán đúng vào cụm của mình. Vì vậy, cần so sánh khoảng cách của mỗi cá thể đến trung bình cụm mà nó đang thuộc về với khoảng cách đến trung bình của cụm còn lại. Kết quả của phép so sánh này được trình bày trong Bảng 20.

Như thể hiện trong bảng, cá thể số 3 hiện được gán vào cụm 1, nhưng thực tế lại gần cụm 2 hơn. Do đó, hợp lý khi chuyển cá thể này sang cụm 2. Sự phân vùng mới sau điều chỉnh được trình bày trong Bảng 21.

Bảng 20: Bước thứ hai với thuật toán k-means

Cá nhân	Khoảng cách đến trung bình (trọng tâm) của cụm 1	Khoảng cách đến trung bình (trọng tâm) của cụm 2
1	1.5	5.4
2	0.4	4.3
3	2.1	1.8
4	5.7	1.8
5	3.2	0.7
6	3.8	0.6
7	2.8	1.1

Bảng 21: Kết quả sau khi phân chia k-means

	Cá nhân	Vector trung bình (trọng tâm)
Cụm 1	1, 2	(1.3, 1.5)
Cụm 2	3, 4, 5, 6, 7	(3.9, 5.1)

1.13.4. Tối đa hóa kỳ vọng.

Cho đến hiện tại, chúng ta đã xem xét các thuật toán phân cụm (clustering), phân loại (classification) và các mô hình xác suất (probabilistic models), vốn dựa trên giả định rằng luôn tồn tại những quy trình hiệu quả và mạnh mẽ để học tham số từ dữ liệu quan sát. Tuy nhiên, trong thực tế, dữ liệu dùng để huấn luyện mô hình thường không đầy đủ.

Ví dụ, giá trị bị thiếu có thể xuất hiện trong chẩn đoán y tế, nơi bệnh sử bệnh nhân chỉ bao gồm kết quả từ một số xét nghiệm giới hạn. Hoặc trong phân cụm biểu hiện gen, dữ liệu không đầy đủ phát sinh từ việc cố ý bỏ qua một số gán gen vào cụm trong mô hình xác suất.

Một phương pháp tiếp cận mạnh mẽ để xử lý tình huống này là Thuật toán Tối đa hóa Kỳ vọng (Expectation Maximization – EM). EM cho phép ước lượng các tham số trong mô hình xác suất ngay cả khi dữ liệu quan sát không đầy đủ.

Hãy xét một ví dụ đơn giản với thử nghiệm tung đồng xu. Giả sử được cung cấp một cặp đồng xu A và B, với các độ chệch (biases) chưa biết là θ_A và θ_B tương ứng (tức là, trong bất kỳ lần tung nào, đồng xu A sẽ ngửa với xác suất θ_A và sấp với xác suất $1-\theta_A$. Tương tự, đối với đồng xu B, các xác suất là θ_B và $1-\theta_B$). Mục tiêu của thử nghiệm này là ước tính $\theta = (\theta_A, \theta_B)$ bằng cách lặp lại năm lần các bước sau: chọn ngẫu nhiên một trong hai đồng xu (với xác suất bằng nhau), và thực hiện 10 lần tung đồng xu độc lập với đồng xu đã chọn. Như vậy, toàn bộ quy trình bao gồm tổng cộng 50 lần tung đồng xu.

Trong thí nghiệm này, ta đếm được một vector quan sát:

$$x = (x_1, x_2, \dots, x_5),$$

Trong đó $x_i \in \{0, 1, \dots, 10\}$ là số lần ngửa được quan sát trong vòng tung thứ i . Ước lượng tham số của thí nghiệm này được gọi là trường hợp dữ liệu đầy đủ khi các giá trị của tất cả các biến ngẫu nhiên liên quan trong mô hình (tức là, kết quả của mỗi lần tung đồng xu và loại đồng xu được sử dụng cho mỗi lần tung) đều được biết.

Một cách đơn giản để ước tính θ_A và θ_B là trả về tỷ lệ mặt ngửa quan sát được của mỗi đồng xu:

$$\theta_A = \frac{\text{số mặt ngửa sử dụng đồng xu A}}{\text{tổng số lần tung đồng xu A}}$$

và

$$\theta_B = \frac{\text{số mặt ngửa sử dụng đồng xu B}}{\text{tổng số lần tung đồng xu B}}$$

Phỏng đoán trực giác trên, trong tài liệu thống kê, được biết đến với tên gọi Ước lượng Khả năng Hợp lý Tối đa (Maximum Likelihood Estimation – MLE). Nói một cách đơn giản, MLE đánh giá chất lượng của một mô hình thống kê dựa trên xác suất mà mô hình đó gán cho dữ liệu quan sát được. Nếu $\log P_{(x,y)}$ là logarit của xác suất đồng thời (hay log khả năng hợp lý - log likelihood) để thu được bất kỳ vector cụ thể nào của số lần ngửa được quan sát x và loại đồng xu y , thì các phương trình trên giải ra các tham số (θ_A, θ_B) sao cho $\log P_{(x,y)}$ được tối đa hoá.

Tuy nhiên, trong nhiều trường hợp, các phương trình này không thể giải trực tiếp. Khi đó, Thuật toán tối đa hóa kì vọng (Expectation Maximization – EM) được sử dụng để tìm nghiệm (cục bộ) của các tham số MLE trong mô hình thống kê.

Các mô hình áp dụng EM thường bao gồm các biến tiềm ẩn bên cạnh các tham số chưa biết và các quan sát đã có. Điều này có nghĩa là:

- Dữ liệu có thể chứa giá trị bị thiếu, hoặc
- Mô hình có thể được biểu diễn dưới dạng đơn giản hơn bằng cách giả định sự tồn tại của các biến không quan sát được.

Ví dụ: EM

Bây giờ, hãy xem một ví dụ minh họa cách thuật toán hoạt động trong **R**. Trong thí nghiệm này, chúng ta sẽ sử dụng bộ dữ liệu **diabetes** (bệnh tiểu đường), vốn được cung cấp sẵn trong gói **mclust** (phiên bản 5.3).

Để trích dẫn gói **R**, có thể sử dụng lệnh:

```
> library(mclust)
> data("diabetes")
> summary(diabetes)
```

	class	glucose	insulin	sspg
Chemical	:36	Min. : 70	Min. : 45	Min. : 10.0
Normal	:76	1st Qu.: 90	1st Qu.: 352.0	1st Qu.: 118.0
Overt	:33	Median : 97	Median : 403.0	Median : 156.0

Mean :122	Mean : 540.8	Mean :186.1
3rd Qu.:112	3rd Qu.: 558.0	3rd Qu.:221.0
Max. :353	Max. :1568.0	Max. :748.0

Để áp dụng thuật toán EM (Expectation Maximization), sẽ sử dụng cùng gói “mclust”. Lệnh summary với đối tượng mclust tạo ra các thông số sau:

- log.likelihood (Log khả năng hợp lý): Đây là log khả năng hợp lý của giá trị Tiêu chí Thông tin Bayesian (BIC) – cho biết mức độ phù hợp của mô hình.
- n: Đây là số lượng điểm dữ liệu X.
- df: Đây là bậc tự do (degrees of freedom).
- BIC (Bayesian Information Criterion): Đây là Tiêu chí Thông tin Bayesian; giá trị thấp là tốt.
- ICL (Integrated Complete X Likelihood): Khả năng hợp lý X đầy đủ tích hợp – một phiên bản phân loại của BIC.

Đầu tiên, cần trích xuất dữ liệu phù hợp trước khi có thể sử dụng thuật toán EM trên dữ liệu đó.

```
> data("diabetes")
```

```
> head(diabetes)
```

	class	glucose	insulin	sspg
1	Normal	80	356	124
2	Normal	97	289	117
3	Normal	105	319	143
4	Normal	90	356	199
5	Normal	90	323	240
6	Normal	86	381	157

```
> class.dia = diabetes$class
```

```
> table(class.dia)
```

```
class.dia
```

	Normal	Overt
Chemical		
36	76	33

```
> X = diabetes[,-1]
```

```
> head(X)
```

	glucose	insulin	speg
1	80	356	124
2	97	289	117
3	105	319	143
4	90	356	199
5	90	323	240
6	86	381	157

```
> clPairs(X, class.dia)
```

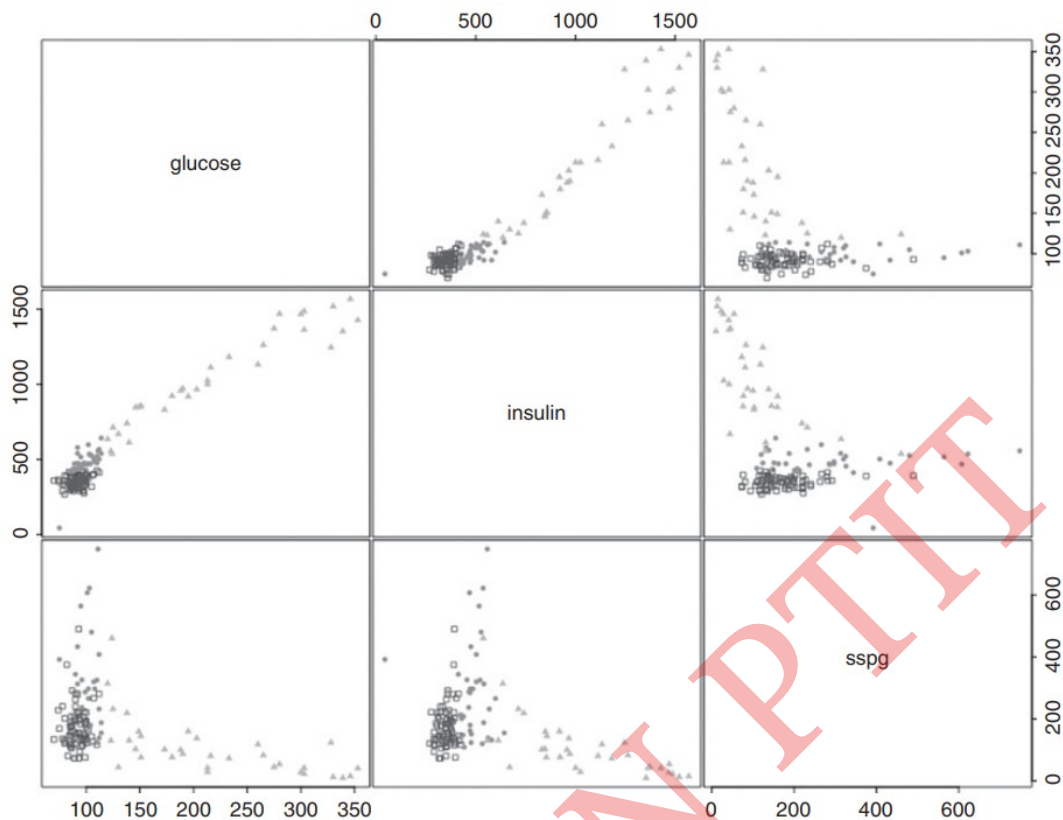
Các bước trên tạo ra đầu ra được hiển thị trong Hình 10.7. Ở đây, hàm “clPairs” là một hàm dùng để tạo ra các biểu đồ phân tán theo cặp (pairwise scatterplots) hiển thị sự phân loại. Đây là một biểu diễn bằng hình ảnh về mối quan hệ giữa hai dạng danh mục dữ liệu (trong trường hợp này là lớp bệnh tiểu đường và các biến glucose, insulin, speg).

Bây giờ, sẽ sử dụng hàm Mclust để fit một mô hình (ước lượng mật độ - density estimation).

```
> fit <- Mclust(X)
```

```
fitting ...
```

```
|=====| 100%
```



Hình 81: Biểu đồ phân tán từng cặp thể hiện phân loại.

```
> fit
```

'Mclust' model object:

best model: ellipsoidal, varying volume, shape, and orientation (VVV) with 3 components

```
> summary(fit)
```

Gaussian finite mixture model fitted by EM algorithm

Mclust VVV (ellipsoidal, varying volume, shape, and orientation) model with 3 components:

	n	df	BIC	ICL
log.likelihood				
-2307.883	145	29	-4760.091	-4776.086

Clustering table:

	2	3
1		
82	33	30

Sau khi áp dụng thuật toán EM để khớp dữ liệu, ta đã xây dựng được một mô hình cho bộ dữ liệu. Bước tiếp theo là đánh giá mức độ phù hợp của mô hình

Có nhiều tiêu chí thường được sử dụng cho việc đánh giá này và phổ biến là các tiêu chí thông tin Akaike (AIC), tiêu chí thông tin Bayesian (BIC), và log khả năng hợp lý (log likelihood), v.v.

AIC cung cấp một ước tính về mất mát thông tin tương đối của một mô hình đã cho khi mô hình đó đại diện cho quá trình đã tạo ra dữ liệu. Giả sử, với một số dữ liệu, một mô hình đã được tạo, trong đó k là số lượng tham số được ước tính. Nếu $\hat{L}$ là giá trị tối đa của hàm khả năng hợp lý (likelihood function) cho mô hình, thì giá trị AIC được tính như sau:

$$AIC = 2k - 2\ln(\hat{L})$$

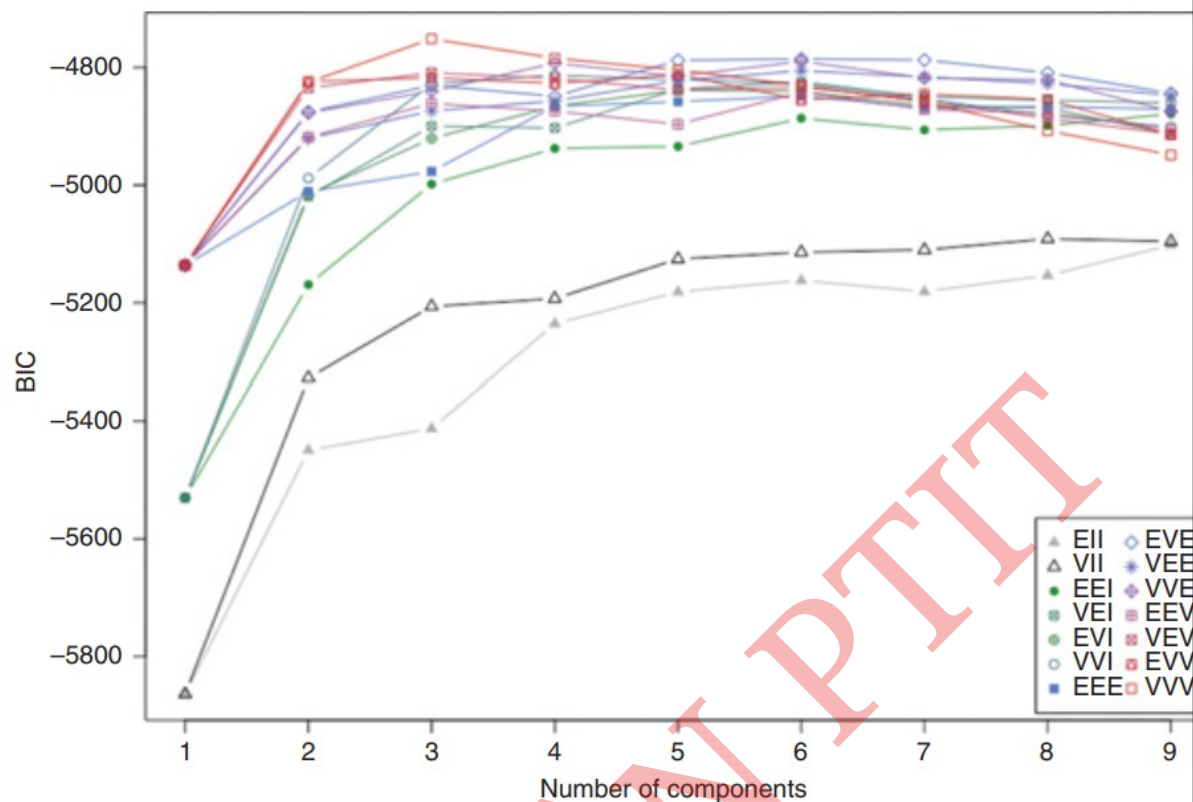
Mặt khác, BIC (Bayesian Information Criterion - Tiêu chí Thông tin Bayesian) ước tính xác suất hậu nghiệm (posterior probability) của một mô hình là đúng từ một thiết lập Bayesian nhất định. Giá trị BIC càng thấp có nghĩa là mô hình đó được coi là có khả năng là một mô hình tốt hơn. Công thức tính BIC là:

$$BIC = (\ln n)k - 2\ln(\hat{L})$$

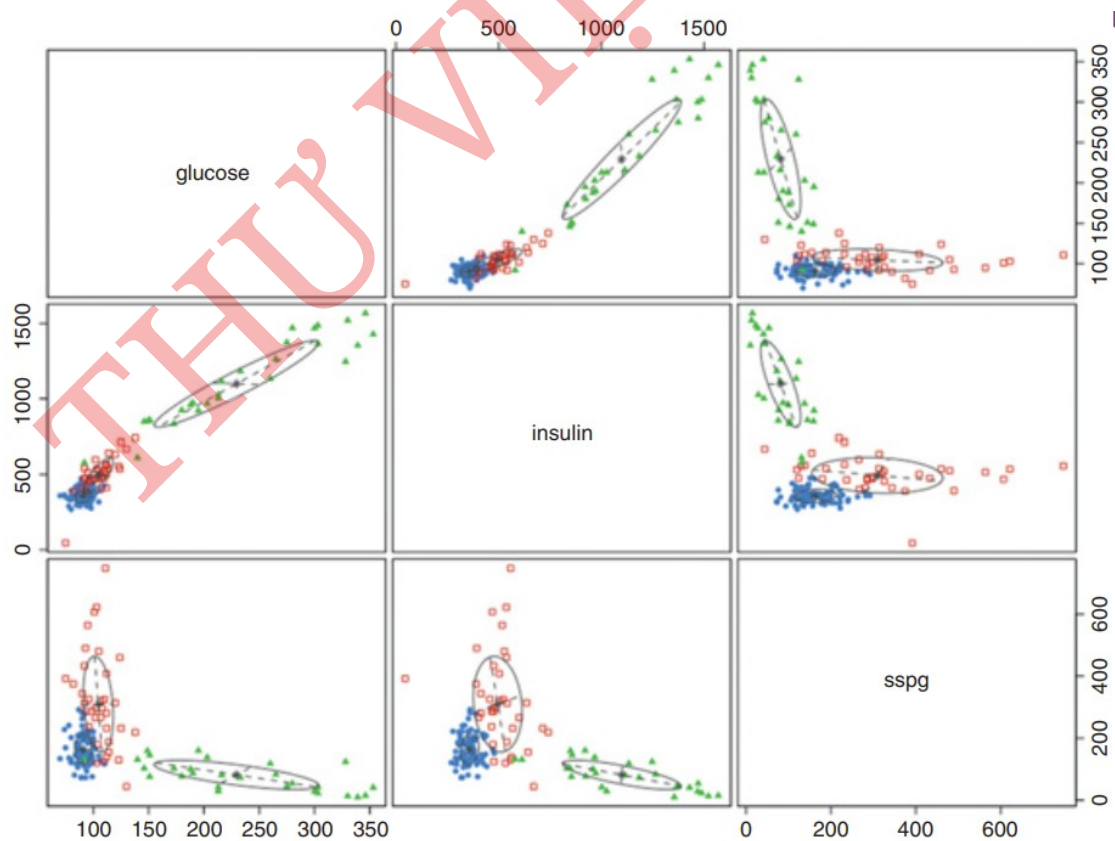
Cả AIC và BIC đều là các tiêu chí khả năng hợp lý bị phạt (penalized-likelihood criteria). Điều này có nghĩa là giá trị càng cao, mô hình càng tệ. Điểm khác biệt đáng kể duy nhất giữa AIC và BIC là sự lựa chọn giữa $\log n$ (trong BIC) và 2 (trong AIC) để áp dụng hình phạt.

Vì vậy, hãy trực quan hóa mô hình này với các thành phần sau:

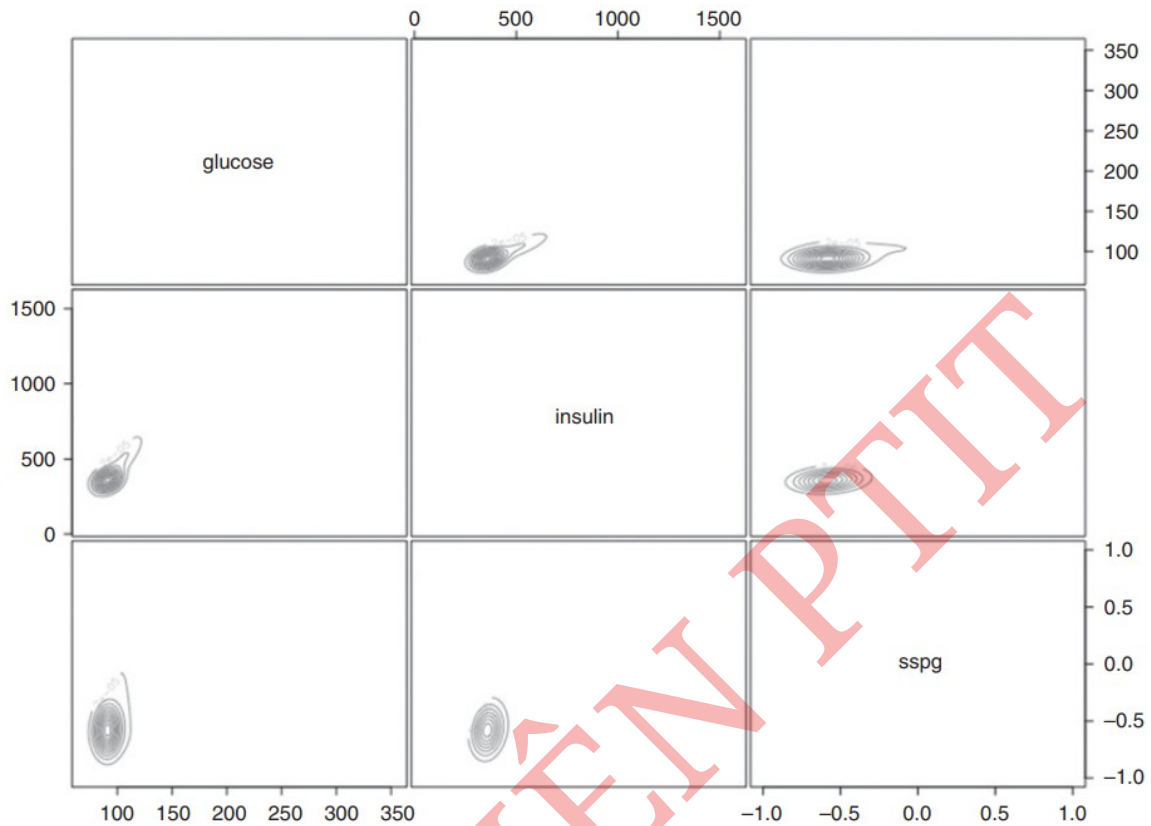
- Các giá trị BIC được sử dụng để chọn số lượng cụm/thành phần Hình 82
- Một biểu đồ về phân cụm (clustering)
- Một biểu đồ về độ bất định của phân loại (Hình 83)
- Biểu đồ orbital của các cụm (Hình 84)



Hình 82: Biểu đồ hiển thị số lượng thành phần và BIC tương ứng.



Hình 83: Biểu đồ phân loại.

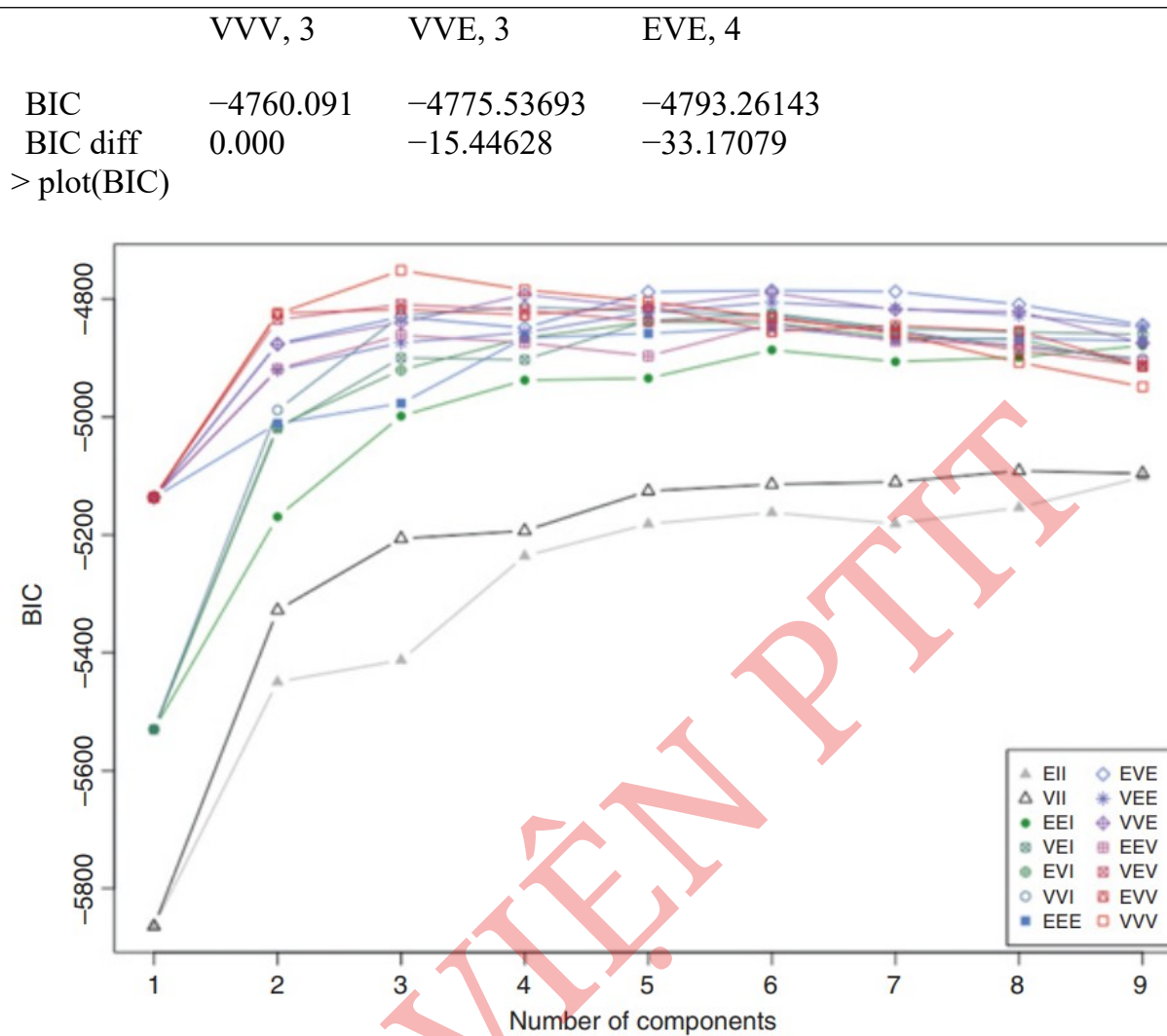


Hình 84: Biểu đồ mật độ

```
> plot(fit, what = "BIC")
> #table(class, fit$BIC)
> #2: classification
> plot(fit, what = "classification")
> plot(fit, what = "density")
```

Ngoài việc đánh giá mức độ phù hợp của mô hình, cũng có thể trực quan hóa độ bất định trong phân loại. Trong gói mclust, hàm mclustBIC sử dụng tiêu chí BIC để lựa chọn mô hình. Thuật toán tối đa hóa kì vọng trong trường hợp này được khởi tạo thông qua phân cụm phân cấp dựa trên mô hình, áp dụng cho các mô hình hỗn hợp Gaussian tham số hóa (parameterized Gaussian mixture models), như minh họa trong Hình 85

```
> BIC = mclustBIC(X)
fitting ...
=====| 100%
> summary(BIC)
Best BIC values:
```



Hình 85: Hình ảnh hóa tiêu chí BIC

Hàm `mclustICL` từ gói `mclust` sử dụng Khả năng hợp lý dữ liệu hoàn chỉnh tích hợp (Integrated Complete-data Likelihood - ICL) cho các mô hình hỗn hợp Gaussian được tham số hóa được khớp bằng thuật toán tối đa hóa kì vọng và được khởi tạo bằng phương pháp phân cụm phân cấp dựa trên mô hình (Hình 86).

```
> ICL = mclustICL(X)
fitting ...
```

```
=====
```

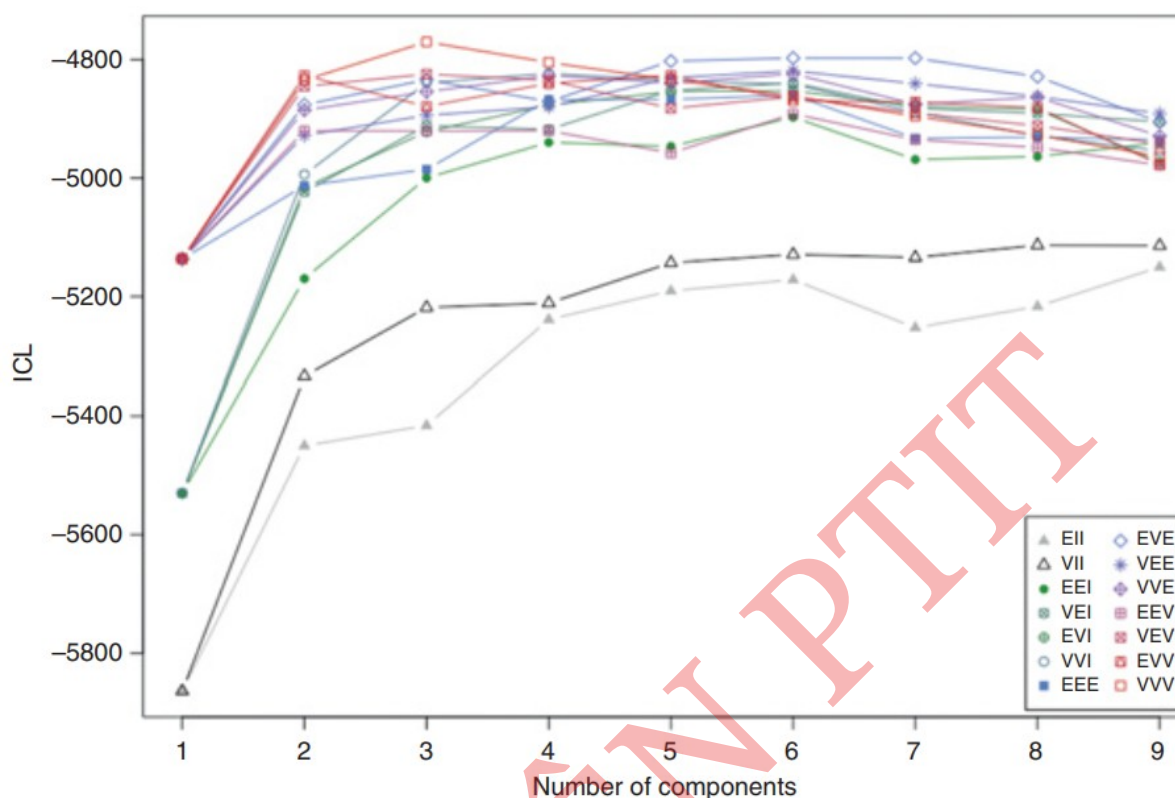
```
100%
```

```
> summary(ICL)
```

```
Best ICL values:
```

	VVV, 3	VVE, 3	EVE, 4
ICL	-4776.086	-4793.27143	-4809.16868
ICL diff	0.000	-17.18553	-33.08278

> plot(ICL) # Only ICL plot



Hình 86: Hình dung tiêu chí khả năng hoàn thành tích hợp.

Bỏ qua các chỉ số và biểu đồ nâng cao, có thể khẳng định rằng ít nhất ba tiêu chí cơ bản – (log likelihood), AIC và BIC – là những công cụ quan trọng. Các giá trị này có thể được tính toán trực tiếp cho một mô hình, qua đó cung cấp cơ sở để đánh giá mức độ mà mô hình đó phản ánh đúng cấu trúc dữ liệu nền tảng.

1.13.5. Học tăng cường

Học Tăng cường (Reinforcement Learning – RL) bắt nguồn từ các thí nghiệm trong tâm lý học hành vi gần một thế kỷ trước. Phương pháp này tìm cách mô hình hóa cách mà các tác nhân phần mềm nên đưa ra hành động trong một môi trường nhằm tối đa hóa phần thưởng tích lũy theo thời gian.

Ví dụ, giả sử muốn huấn luyện một máy tính chơi cờ vua với con người. Việc xác định nước đi tối ưu trong trò chơi này phụ thuộc vào rất nhiều yếu tố, và số lượng trạng thái có thể xảy ra trong một ván cờ là cực kỳ lớn. Nếu tiếp cận theo phương pháp dựa trên quy tắc truyền thống, cần phải xây dựng một số lượng khổng lồ các quy tắc được mã hóa cứng. Học tăng cường loại bỏ sự cần thiết này bằng cách cho phép tác nhân tự học

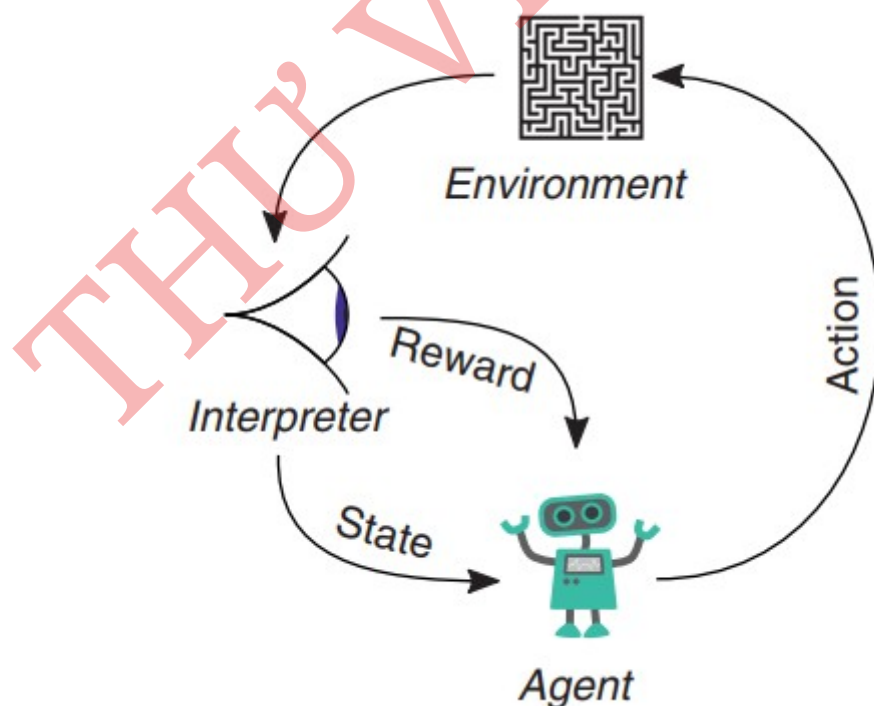
thông qua trải nghiệm, ví dụ bằng cách liên tục chơi trò chơi. Với các trò chơi hai người như cờ tào cáo (backgammon), tác nhân học tăng cường có thể được huấn luyện bằng cách chơi với con người hoặc thậm chí với những tác nhân học tăng cường khác.

Trong học tăng cường, thuật toán phải quyết định hành động kế tiếp ngay khi nhận được một quan sát mới từ môi trường. Sau đó, tùy vào mức độ phù hợp của hành động, tác nhân sẽ nhận được một tín hiệu phản hồi dưới dạng phần thưởng hoặc hình phạt. Quá trình này được lặp lại, và tác nhân liên tục điều chỉnh chính sách của mình theo hướng tối đa hóa phần thưởng.

Học tăng cường được ứng dụng rộng rãi trong các ngành robot, nơi đầu vào của mô hình là tập hợp các giá trị cảm biến tại một thời điểm và tác nhân phải quyết định hành động tiếp theo cho robot. Ngoài ra, phương pháp này cũng phù hợp với nhiều kịch bản Internet vạn vật (IoT), nơi các thiết bị thông minh liên tục tương tác với môi trường.

Một mô hình RL cơ bản bao gồm các thành phần chính được thể hiện trên Hình 87

1. Tập hợp các trạng thái (S) đại diện cho môi trường và tác nhân.
2. Tập hợp các hành động (A) mà tác nhân có thể thực hiện.
3. Chính sách (policy) xác định cách tác nhân lựa chọn hành động dựa trên trạng thái hiện tại.
4. Các quy tắc xác định phần thưởng tức thời khi tác nhân chuyển trạng thái
5. Các quy tắc mô tả những gì tác nhân quan sát được.



Hình 87: Khung điển hình của một kịch bản học tăng cường

Một tác nhân học tăng cường tương tác với môi trường của nó theo các bước thời gian rời rạc và theo một trình tự cụ thể:

1. Tại mỗi thời điểm t , tác nhân nhận được một quan sát o_t , thường bao gồm phần thưởng r_t .
2. Sau đó, tác nhân chọn một hành động a_t từ tập hợp các hành động có sẵn, hành động này sau đó được gửi đến môi trường.
3. Môi trường chuyển sang một trạng thái mới s_{t+1} từ trạng thái s_t , và phần thưởng r_{t+1} liên quan đến quá trình chuyển đổi s_t, a_t, s_{t+1} được xác định.
4. Mục tiêu của một tác nhân học tăng cường là thu thập càng nhiều phần thưởng càng tốt. Tác nhân có thể chọn bất kỳ hành động nào dưới dạng một hàm của lịch sử và thậm chí có thể ngẫu nhiên hóa việc lựa chọn hành động.

Tiếp theo có thể minh họa cách RL hoạt động bằng một quy trình đơn giản xây dựng mô hình trong R như sau:

Ví dụ: Học tăng cường

Trước tiên, cần tải và cài đặt các gói cần thiết trong R. Gói **RL** hiện vẫn ở giai đoạn thử nghiệm và được phân loại như một phần của các công cụ phát triển, do đó cần được cài đặt thủ công trước khi sử dụng.

```
install.packages("devtools")
library(devtools)
install_github("nproellochs/ReinforcementLearning")
library(ReinforcementLearning)
```

Gói RL đi kèm với một trò chơi có sẵn là tic-tac-toe (cờ ca-rô), cùng với bộ dữ liệu đã được tạo sẵn trong thư viện. Bộ dữ liệu này gồm hơn 400.000 bản ghi, mô tả chi tiết các bước đi trong trò chơi. Phần minh họa tiếp theo sẽ sử dụng chính bộ dữ liệu này.

Tải bộ dữ liệu và xem bộ dữ liệu sử dụng

```
data("tictactoe")
```

```
View(tictactoe)
```

Bộ dữ liệu cung cấp thông tin về các trạng thái khác nhau trong trò chơi, các hành động có thể thực hiện và phần thưởng tương ứng cho từng hành động. Nói cách khác, dữ liệu đóng vai trò như một cơ chế hướng dẫn, chỉ ra hành động nào phù hợp hoặc không phù hợp trong từng tình huống cụ thể của trò chơi cờ ca-rô.

Sử dụng thông tin này, có thể huấn luyện một mô hình như sau

Thực hiện học tăng cường

```
tic_tac_toe_model <- ReinforcementLearning(tictactoe, s = "State", a = "Action", r  
= "Reward", s_new = "NextState", iter = 1)
```

Dòng này sử dụng bộ dữ liệu đã cung cấp để huấn luyện một mô hình dựa trên cách tiếp cận học tăng cường. Vì bộ dữ liệu thực sự lớn nên việc huấn luyện sẽ mất vài phút để hoàn thành. Khi quá trình huấn luyện hoàn tất, có thể hỏi về chính sách và phần thưởng liên quan mà mô hình đã học được.

Để truy vấn về chính sách tối ưu, nhập:

```
tic_tac_toe_model$Policy
```

Kết quả đầu ra cho thấy một tập hợp lớn các quy tắc hoặc chính sách mà mô hình đã học được. Cụ thể, mô hình xác định được các hành động tối ưu cần thực hiện (được biểu diễn dưới dạng **c1, c2, ...**) khi rơi vào một trạng thái nhất định trong trò chơi. Cần lưu ý rằng các kết quả có thể thay đổi giữa các lần chạy; ở mỗi lần, mô hình sẽ in ra một ma trận lớn mô tả toàn bộ các bước hành động khả dĩ trong từng trạng thái tương ứng. Để tìm phần thưởng (reward) cho lần chạy này, thực hiện dòng lệnh sau:

```
tic_tac_toe_model$Reward  
[1] 5449
```

Trong trường hợp này, phần thưởng được tính là 5449, nhưng giá trị này có thể thay đổi tùy theo mỗi lần chạy.

Có thể thấy rằng việc chạy mô hình trong ví dụ này khá đơn giản. Tuy nhiên, điều quan trọng cần rút ra là trong tương lai, nếu muốn áp dụng một dạng học tăng cường, có thể xây dựng một bộ dữ liệu tương tự như bộ dữ liệu đã sử dụng ở đây. Bộ dữ liệu đó cần bao gồm:

- Các trạng thái (states),
- Các hành động (actions),
- Các trạng thái kế tiếp (next states) mà hành động dẫn tới, và
- Các phần thưởng (rewards) tương ứng.

Bộ dữ liệu này có thể được đưa vào một quy trình huấn luyện tương tự như quy trình đã trình bày, từ đó xây dựng và đánh giá một mô hình RL cho các ứng dụng cụ thể.

1.14. Câu hỏi và bài tập Chương 3

Bài 1: (*Câu hỏi lý thuyết*) Phân biệt học có giám sát (supervised learning) và học không giám sát (unsupervised learning).

Bài 2: (*Câu hỏi lý thuyết*) Cho tập dữ liệu với 2 đặc trưng, hãy tính toán thủ công bước cập nhật trọng số trong thuật toán hồi quy tuyến tính (linear regression).

Bài 3: (*Câu hỏi ứng dụng*) Ứng dụng của thuật toán phân cụm (clustering) trong phân khúc khách hàng của một công ty viễn thông.

Bài 4: (*Bài tập lập trình*) Sử dụng thư viện scikit-learn (Python) để xây dựng mô hình phân loại văn bản với Naïve Bayes.

Bài 5: (*Câu hỏi so sánh*) So sánh ưu nhược điểm giữa cây quyết định (Decision Tree) và Random Forest.

Bài 6: (*Bài tập thực hành*) Giải thích ngắn gọn ý nghĩa của tham số C trong Support Vector Machine.

Bài 7: (*Câu hỏi tình huống*) Khi nào nên sử dụng học tăng cường (Reinforcement Learning) thay vì các phương pháp học có giám sát?

Bài 8: (*Bài tập nhóm*) Xây dựng mô hình dự đoán lượng tiêu thụ data của thuê bao di động sử dụng hồi quy logistic.

Bài 9: (*Câu hỏi thực hành*) Giải thích các chỉ số Precision, Recall và F1-score trong bài toán phân loại.

Bài 10: (*Bài tập tổng hợp*) Đề xuất một giải pháp học máy để phát hiện gian lận trong cuộc gọi viễn thông, mô tả các bước từ thu thập dữ liệu đến đánh giá mô hình.

Bài 11: (*Bài tập nhóm*) Xây dựng hệ thống recommendation cho nền tảng streaming nhạc/video.

Bài 12: (*Bài tập nhóm*) Nghiên cứu ứng dụng học máy trong phát hiện gian lận giao dịch ngân hàng.

Bài 13: (*Bài tập nhóm*) So sánh hiệu quả các thuật toán phân loại trong bài toán chẩn đoán bệnh từ dữ liệu y tế.

Bài 14: (*Bài tập nhóm*) Triển khai mô hình NLP để phân tích cảm xúc (sentiment analysis) từ review sản phẩm.

Bài 15: (*Bài tập nhóm*) Xây dựng hệ thống dự báo chuỗi thời gian (time series) cho dự báo giá cổ phiếu/tiền điện tử.

Bài 16: (*Bài tập nhóm*) Nghiên cứu ứng dụng học tăng cường (RL) trong tối ưu hóa hệ

thống giao thông đô thị.

Bài 17: (*Bài tập nhóm*) Triển khai hệ thống computer vision để nhận diện biển số xe tự động.

Bài 18: (*Bài tập nhóm*) Nghiên cứu các kỹ thuật xử lý mất cân bằng dữ liệu (imbalanced data) trong bài toán phân loại.

Bài 19: (*Bài tập nhóm*) Xây dựng pipeline MLOps để tự động hóa training và deployment mô hình học máy.

Bài 20: (*Bài tập nhóm*) Nghiên cứu ứng dụng học máy trong nông nghiệp thông minh (smart agriculture).

THƯ VIỆN PTIT

CHƯƠNG IV: THU THẬP, THỬ NGHIỆM VÀ ĐÁNH GIÁ DỮ LIỆU

4.1. Giới thiệu chung

Trong nghiên cứu dữ liệu, việc thu thập, thử nghiệm và đánh giá dữ liệu đóng vai trò quan trọng trong việc đảm bảo kết quả phân tích chính xác và đáng tin cậy. Quá trình thu thập dữ liệu yêu cầu lựa chọn các phương pháp phù hợp để thu thập thông tin từ các nguồn khác nhau, trong khi thử nghiệm giúp kiểm tra các giả thuyết và thu thập dữ liệu thực tế. Đánh giá dữ liệu sau khi thu thập và thử nghiệm sẽ giúp xác định chất lượng, độ chính xác của kết quả phân tích và đảm bảo tính hiệu quả của các mô hình hoặc phương pháp đã áp dụng. Đây là những bước không thể thiếu trong việc xây dựng một nghiên cứu dữ liệu khoa học, giúp rút ra những kết luận có giá trị và ứng dụng được trong thực tế.

4.2. Các phương pháp thu thập dữ liệu

Phương pháp thu thập dữ liệu là bước đầu tiên và quan trọng trong quá trình phân tích dữ liệu, giúp đảm bảo thông tin được thu thập đầy đủ, chính xác và phù hợp với mục tiêu nghiên cứu. Các phương pháp thu thập dữ liệu có thể được chia thành nhiều loại, từ các phương pháp định lượng đến định tính, tùy thuộc vào mục đích và yêu cầu của nghiên cứu.

Một trong những phương pháp phổ biến nhất là khảo sát. Phương pháp này có thể thực hiện trực tiếp hoặc trực tuyến. Khảo sát giúp thu thập thông tin từ một nhóm đối tượng lớn một cách nhanh chóng và hiệu quả, đặc biệt khi cần dữ liệu số lượng lớn từ nhiều người tham gia.

Ngoài ra, phỏng vấn và nhóm trọng tâm (focus group) là các phương pháp thu thập dữ liệu định tính, giúp khai thác sâu hơn về suy nghĩ, cảm nhận và quan điểm của người tham gia. Phương pháp này thường được sử dụng khi cần hiểu rõ hơn về các yếu tố ảnh hưởng đến hành vi hoặc nhận thức của người tham gia.

Các phương pháp khác như quan sát trực tiếp, nghiên cứu tài liệu, hoặc ghi chép tự động sử dụng các thiết bị công nghệ cũng đóng vai trò quan trọng trong việc thu thập dữ liệu trong nhiều lĩnh vực nghiên cứu.

Tùy vào mục đích và tính chất của vấn đề nghiên cứu, việc lựa chọn phương pháp thu thập dữ liệu phù hợp là yếu tố then chốt giúp đảm bảo chất lượng và độ tin cậy của dữ liệu thu được.

4.2.1. Khảo sát

Giả sử có một tình huống cần thu thập dữ liệu cụ thể cho một vấn đề nào đó khi chưa có bất kỳ dữ liệu gì về tình huống đó. Do vậy, một trong những phương pháp hiệu quả để thu thập dữ liệu trong trường hợp này là thực hiện khảo sát. Một số ví dụ điển hình của khảo sát có thể kể đến điều tra dân số, xếp hạng Nielsen, hay các cuộc thăm dò ý kiến vào đêm bầu cử. Các cuộc khảo sát có thể được thực hiện theo hình thức trực tiếp, trên giấy hoặc trực tuyến.

Snap Surveys Ltd. đã liệt kê bốn lý do chính để thực hiện khảo sát:

1. Khám phá câu trả lời
2. Kích thích thảo luận
3. Căn cứ quyết định dựa trên thông tin khách quan
4. So sánh kết quả

Việc khảo sát sẽ cung cấp các kết quả dựa theo mục tiêu đã được xác định, từ đó tạo ra một kho tàng thông tin tiềm năng. Để bắt đầu, điều quan trọng nhất là xác định rõ câu hỏi hoặc vấn đề cần giải quyết, từ đó xây dựng các bộ câu hỏi phù hợp. Nếu cần thu thập số liệu (ví dụ: số lượng người nghĩ rằng "ABC" hoặc "XYZ"), việc sử dụng câu hỏi nhiều lựa chọn hoặc câu hỏi bắt buộc có thể là một phương án hiệu quả. Ngược lại, nếu mục tiêu là thu thập ý kiến công chúng về một vấn đề, việc yêu cầu người tham gia cung cấp câu trả lời mở sẽ mang lại kết quả có giá trị hơn.

4.2.2. Các loại câu hỏi khảo sát

Hãy xem xét các loại câu hỏi khảo sát đang được sử dụng hiện tại.

Câu hỏi chọn nhiều lựa chọn là loại câu hỏi cho phép người tham gia trả lời nhanh chóng và dễ dàng, nhưng nếu các lựa chọn không phản ánh chính xác ý kiến của họ thì sao? Một giải pháp là mở đầu câu hỏi bằng cách yêu cầu người tham gia chọn câu trả lời gần nhất với quan điểm của mình. Mặc dù có thể nghĩ rằng càng nhiều lựa chọn càng tốt, nhưng điều này cũng có thể khiến người tham gia cảm thấy mệt mỏi khi phải đọc qua quá nhiều lựa chọn, từ đó mất kiên nhẫn và không trả lời cẩn thận.

Một số câu hỏi nhân khẩu học thường được thiết kế dưới dạng câu hỏi trắc nghiệm nhiều lựa chọn. Ví dụ, các câu hỏi về giới tính (Nam/Nữ), nhóm tuổi (18–25, 26–30, 31–40, 41–50, trên 51), hoặc tình trạng việc làm (Đang làm việc, Thất nghiệp, Tự làm chủ). Mỗi câu hỏi nên bổ sung thêm lựa chọn “Không muốn trả lời” để đảm bảo tính đầy đủ và tôn trọng người tham gia,

Tuy nhiên, các câu hỏi nhiều lựa chọn cũng thường được sử dụng để thu thập phản hồi về những câu hỏi phức tạp hơn. Các câu hỏi như vậy thường xuất hiện khi thu thập thông tin về các chỉ số sử dụng, bao gồm mức độ hiệu quả, sự hài lòng, mức độ dễ học và mức độ tham gia. Ví dụ, khi khảo sát khách hàng/người dùng về những yếu tố không hài

lòng nhất đối với một dịch vụ hoặc giao diện mới bất kì, có thể thiết kế câu hỏi chọn nhiều với năm lựa chọn: (1) hộp thoại nhập liệu; (2) màn hình xác nhận giao dịch; (3) tốc độ tương tác; (4) phản hồi âm thanh và hình ảnh trong mỗi tương tác; và (5) độ dễ đọc của các thông báo. Đặc biệt, khảo sát có thể được thiết kế sao cho khi người tham gia chọn một lựa chọn nhất định, họ sẽ nhận được câu hỏi tiếp theo phụ thuộc vào lựa chọn giúp thu thập thêm thông tin chi tiết về vấn đề nào đó.

Các câu hỏi dạng xếp hạng yêu cầu người tham gia chọn một lựa chọn trong số các tùy chọn theo thứ tự ưu tiên. Những câu hỏi này thường xuất hiện trong các vấn đề so sánh cặp, chẳng hạn như việc so sánh giữa giá cả và chất lượng. Đây là loại câu hỏi dễ trả lời, nhưng có thể không mang lại giá trị thực tiễn lớn trừ khi đang kiểm tra một giả thuyết cụ thể. Ví dụ, nếu nghĩ rằng mọi người có thể thích một yếu tố nào đó hơn một yếu tố khác, ý tưởng này có thể được đưa vào một câu hỏi xếp hạng và giả thuyết có thể được xác nhận. Thông thường, chỉ nên có một câu hỏi xếp hạng trong khảo sát, vì người tham gia có thể không đủ kiên nhẫn để trả lời cẩn thận với quá nhiều câu hỏi dạng này.

Dưới đây là một ví dụ về câu hỏi xếp hạng thiên hướng về cảm nhận hơn là sự thật.

Vui lòng xếp hạng năm mục dưới đây theo thứ tự, từ nhiều nhất đến ít nhất, sử dụng 1 cho mục nhiều nhất và 5 cho mục ít nhất.

Phương thức nào là cách đi lại thân thiện với môi trường nhất khi đi từ Miami đến New York City, sắp xếp theo thứ tự từ thân thiện nhất với môi trường (1) đến ít thân thiện nhất (5):

Lái xe ô tô Toyota Prius; đi tàu Amtrak; bay trên máy bay jumbo; đi xe buýt Greyhound; hoặc đi xe máy cho hai người. (Điền vào chỗ trống với câu trả lời ngắn gọn như “Prius”, “Amtrak”, “Jet”, “Bus” và “Motorcycle”).

- 1.
- 2.
- 3.
- 4.
- 5.

Câu hỏi đánh giá hoặc câu hỏi mở đều có thể có ích trong mục đích khảo sát. Câu hỏi thang Likert (từ 1 đến 5) – thường được sử dụng trong các khảo sát để đánh giá phản hồi – đưa ra năm ý kiến và yêu cầu người tham gia chọn một câu gần nhất với cảm nhận của người được phỏng vấn, dao động từ (1) hoàn toàn đồng ý đến (5) hoàn toàn không đồng ý. Người tham gia thường được yêu cầu chọn một ý. Câu hỏi này cũng tương tự như câu hỏi đồng ý/không đồng ý, nhưng không cung cấp lý do đằng sau lựa chọn đó.

Dưới đây là một ví dụ về câu hỏi thang Likert:

Vui lòng đánh dấu vào ô thể hiện rõ nhất ý kiến hoặc cảm nhận về câu hỏi sau.

1. *Cấu trúc của trang web hợp lý từ góc độ người sử dụng.*

☐ ☐ ☐ ☐ ☐
hoàn toàn đồng ý đồng ý trung lập không đồng ý hoàn toàn không đồng ý

Câu hỏi mở được thiết kế sao cho người tham gia không thể trả lời "có" hoặc "không" mà phải cung cấp bổ sung thông tin. Câu hỏi mở rất hữu ích khi đang phỏng vấn một cách trực tiếp. Trong loại khảo sát này, cần tạo không gian đủ cho người tham gia viết một hoặc hai câu trả lời. Các câu trả lời này có thể được sử dụng để trích dẫn trong bài viết. Ví dụ về câu hỏi mở có thể là:

"Đêm qua các bạn đã làm gì?"

hoặc

"Bạn nghĩ điều gì nên làm để trang web này thân thiện với người dùng hơn?"

Mặt khác, câu hỏi mở sẽ khó định lượng hơn, nhưng có thể mang lại những thông tin hữu ích để giải thích các lý do đằng sau. Ví dụ, nếu muốn biết trải nghiệm kỳ nghỉ của ai đó tại Cộng hòa Dominican, trong trường hợp khi hỏi *"Trải nghiệm của bạn như thế nào?"* người tham gia có thể trả lời *"Tốt!"*. Tuy nhiên câu trả lời đó không mang lại nhiều thông tin. Thay vào đó, nếu hỏi *"Điều gì là điều bạn thích nhất ở Cộng hòa Dominican?"* có thể nhận được nhiều câu trả lời thú vị và đầy đủ hơn. Dựa trên câu trả lời trên, có thể hỏi tiếp một câu hỏi *"Điều gì sẽ làm cho kỳ nghỉ của bạn còn tuyệt vời hơn nữa?"* để có những thông tin giá trị.

Câu hỏi nhị phân (câu hỏi đóng) yêu cầu câu trả lời "có" hoặc "không", hoặc "đúng" "sai". Ví dụ,

"Trong 30 ngày qua, bạn có thấy bất kì quảng cáo nào liên quan đến giày dép trên Facebook không?" ☐ có ☐ không

hoặc

"Ba cộng năm bằng tám." ☐ đúng ☐ sai

Có thể lập trình khảo sát sao cho câu hỏi sẽ thay đổi tùy theo câu trả lời của người tham gia. Ví dụ, nếu người tham gia trả lời "có" về việc đã thấy quảng cáo giày, có thể hỏi họ về thương hiệu họ đã xem là gì. Nếu họ trả lời "không", có thể đưa ra một số tên thương hiệu và xem liệu một trong số đó có phải là câu trả lời hay không. Tuy nhiên, điều này có thể không hoàn toàn chính xác, vì sự quen thuộc với một thương hiệu có thể ảnh hưởng đến câu trả lời của người tham gia.

4.2.3. Đối tượng khảo sát

Cũng như việc xác định các loại câu hỏi, một yếu tố không kém phần quan trọng là đối tượng tham gia khảo sát. Khi sử dụng dịch vụ khảo sát trực tuyến và phát tán rộng rãi, chúng ta sẽ rất khó kiểm soát ai là người tiếp cận và trả lời. Vì không gặp trực tiếp người tham gia, việc xác định liệu câu trả lời có đến từ đúng đối tượng mục tiêu hay không là một thách thức lớn. Thực tế, rất khó để đảm bảo người tham gia là con người thật, bởi hiện nay có rất nhiều chương trình tự động thực hiện tham gia khảo sát. Liệu có cách nào kiểm tra xem một người có tham gia khảo sát nhiều lần không? Hay họ chỉ đang ngẫu nhiên chọn các câu trả lời trong danh sách câu hỏi trắc nghiệm? Đây đều là những vấn đề khó giải quyết, do đó cần suy nghĩ và lên kế hoạch cẩn thận. Một giải pháp có thể kể đến là sử dụng dịch vụ như CAPTCHA để xác nhận người tham gia thực sự là người thật. Ngoài ra, có thể chèn thêm một vài câu hỏi để kiểm tra sự chú ý của người tham gia trong suốt quá trình khảo sát, ví dụ: “1 + 1 bằng bao nhiêu?”.

Bên cạnh đó, cần nhớ rằng đây là một quá trình yêu cầu người tham gia chia sẻ suy nghĩ hoặc ý kiến của họ. Các câu hỏi khi khảo sát bao gồm ai là đối tượng sẽ tham gia khảo sát? Phương thức nào sẽ giúp thu hút sự tham gia một cách hiệu quả? Có thể tận dụng cộng đồng trên Facebook hay Twitter, hoặc kết nối qua các cộng đồng học thuật trên Google Docs? Liệu việc gửi email đến đúng mục tiêu sẽ dễ dàng thu hút đúng đối tượng hơn? Hay liệu có nên mở rộng phạm vi để thu thập câu trả lời từ những người lạ hay không?

4.2.4. Các dịch vụ khảo sát

Có thể thực hiện khảo sát qua nhiều kênh khác nhau, từ các nền tảng di động, email truyền thống, cho đến phương pháp thủ công như phát phiếu giấy tại lối ra của một sự kiện để thu thập ý kiến ngay lập tức. Một số công ty thậm chí vẫn sử dụng Dịch vụ Bưu chính Mỹ để gửi khảo sát bản cứng qua thư. Việc lựa chọn phương thức phụ thuộc vào đối tượng và mục tiêu cần tiếp cận.

Hiện nay có rất nhiều dịch vụ khảo sát trực tuyến miễn phí và hiệu quả. Bên cạnh đó cũng có rất nhiều dịch vụ cao cấp, phù hợp với các dự án lớn được tài trợ hoặc cần thực hiện khảo sát đa nền tảng, đa ngôn ngữ. Dưới đây là một số lựa chọn phổ biến:

- Microsoft Office 365: Cung cấp tùy chọn khảo sát dành riêng cho di động, sử dụng Excel để tổng hợp dữ liệu, tiện lợi cho những người đã có sẵn gói đăng ký Office 365.
- SmartSurvey: Nền tảng này có các gói miễn phí và giảm giá cho sinh viên. Với gói miễn phí, có thể tự tạo khảo sát hoặc sử dụng hơn 50 mẫu câu hỏi có sẵn thuộc nhiều lĩnh vực. Nền tảng này cũng cho phép tùy chỉnh các khảo sát có sẵn trên web và chia sẻ qua mạng xã hội hoặc website.

- Google Forms: Đây là công cụ quen thuộc, hoàn toàn miễn phí, cho phép tạo khảo sát cá nhân, công việc hay học tập một cách dễ dàng. Nền tảng này cũng cung cấp các công cụ phân tích và trực quan hóa dữ liệu khá mạnh mẽ.
- SurveyMonkey và Qualtrics: Nếu có ngân sách hoặc trường học/công ty đã đăng ký, đây là hai công cụ khảo sát cao cấp với nhiều tính năng chuyên sâu.
- Các mạng xã hội: Nhiều nền tảng mạng xã hội cho phép tạo các cuộc thăm dò ý kiến nhanh (poll). Tuy nhiên, các công cụ này thường không đủ nếu cần tùy chọn nâng cao hoặc muốn thêm hình ảnh vào cuộc thăm dò.

4.2.5. Phân tích dữ liệu khảo sát

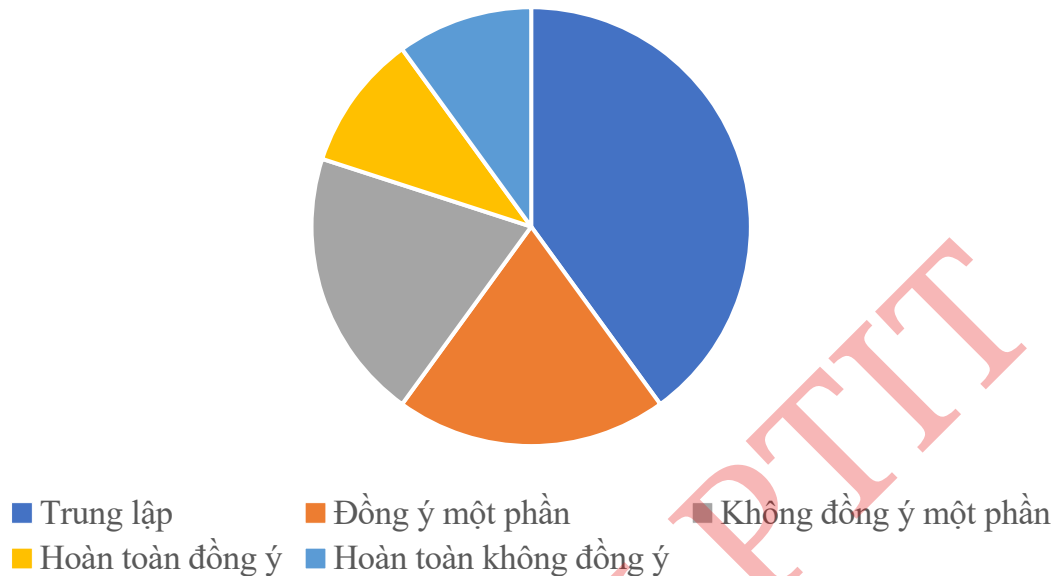
Rất có thể người dùng đã quen thuộc và có một chương trình bảng tính yêu thích như Excel hoặc Google Sheets. Dữ liệu từ hầu hết các công cụ khảo sát có thể dễ dàng nhập vào các chương trình này. Nếu khảo sát được thực hiện bằng Google Forms, dữ liệu đã có sẵn trong Google Sheets. Biểu đồ cũng là một cách hiệu quả để trực quan hóa phạm vi các câu trả lời, như minh họa tại hình dưới. Có nhiều phương pháp để phân tích và trình bày kết quả, chỉ cần tìm kiếm trên Google thì kết quả trả về sẽ cung cấp vô số lựa chọn để tham khảo.

Bảng 22: Bảng khảo sát ý kiến người tham gia về trải nghiệm trang web

Nhân	Số lượng người dùng trả lời cho câu hỏi “Tôi nghĩ trang web này dễ sử dụng.”
Trung lập	4
Đồng ý một phần	2
Không đồng ý một phần	2
Hoàn toàn đồng ý	1
Hoàn toàn không đồng ý	1
Tổng cộng	10

Từ đó có biểu đồ sau

Số lượng người dùng trả lời cho câu hỏi “Tôi nghĩ trang web này dễ sử dụng.”



Hình 88: Kết quả được hiển thị từ dạng câu hỏi Likert

4.2.6. Ưu điểm và nhược điểm của khảo sát

Khảo sát mang đến một góc nhìn mới cho nghiên cứu. Dù không nên bảo vệ toàn bộ luận án chỉ dựa trên kết quả khảo sát do dễ sai lệch bởi tâm trạng của người trả lời, tuy nhiên dữ liệu thu được vẫn có thể gợi ý cho một hướng đi tiềm năng mới.

Khi kết quả không như mong đợi, có thể cần thực hiện một cuộc khảo sát khác để kiểm tra do những người tham gia có thể trả lời không trung thực. Mặc dù không thể bắt buộc người được trả lời phải cam kết cho câu trả lời của họ, nhưng có thể tăng khả năng trung thực bằng cách đảm bảo tính ẩn danh và sau đó tổng hợp câu trả lời lại, điều này có nghĩa không ai có thể biết ai đã trả lời gì. Một hạn chế khác là một số câu hỏi có thể không làm rõ được lý do "tại sao" đằng sau câu trả lời mà họ đưa ra.

Do đó, nên kết hợp nhiều loại câu hỏi và đưa ra nhiều phương thức trả lời khác nhau. Để thu thập thông tin cụ thể, hãy thử hỏi cùng một câu hỏi hoặc một vài câu hỏi tương tự theo nhiều cách và từ nhiều góc độ. Trước khi tiến hành khảo sát chính thức, nên chạy thử nghiệm với một mẫu câu hỏi nhỏ để đánh giá xem kết quả có khả quan hay không.

4.2.7. Phỏng vấn và nhóm trọng tâm

Phỏng vấn và nhóm trọng tâm có thể mang lại nhiều thông tin phong phú, có mục tiêu rõ ràng, với ưu điểm là thu thập được chính xác lời nói của người được phỏng vấn. Khi gặp mặt trực tiếp, người tham gia có thể đưa ra những thông tin giá trị không thể có

được từ một cuộc khảo sát. Phỏng vấn thường được thực hiện một-một, trong khi nhóm trọng tâm đúng như tên gọi của nó—một nhóm người cùng trả lời các câu hỏi hoặc phản ứng với các một hay một số câu hỏi cụ thể. Phỏng vấn có thể phù hợp hơn cho những câu hỏi mang tính cá nhân, còn nhóm trọng tâm rất lý tưởng để kiểm tra một một ý kiến nào đó dễ dàng được thảo luận công khai. Có nhiều lý do, cũng như ưu và nhược điểm, khi sử dụng mỗi phương pháp này.

4.2.8. Tại sao nên thực hiện phỏng vấn?

Khi nghĩ đến một người phỏng vấn giỏi, chúng ta thường hình dung ra một nhân vật truyền hình hay một giọng nói trên radio. Điều gì làm họ tài năng trong việc khơi gợi những sự thật hay câu chuyện sâu sắc? Hầu hết mọi người đều có kinh nghiệm khi được phỏng vấn xin việc hoặc khi nộp đơn vào một trường học. Tất nhiên, với tư cách là người được phỏng vấn, chúng ta luôn chuẩn bị và thể hiện bản thân tốt nhất.

Tương tự, khi chuẩn bị phỏng vấn người khác, việc lên kế hoạch cũng vô cùng cần thiết. Nếu có một luận điểm cần làm rõ, xác nhận hoặc khám phá, cần chuẩn bị câu hỏi dưới nhiều hình thức. Không chỉ khác nhau về loại câu hỏi, mà còn nên hỏi cùng một vấn đề từ nhiều góc độ khác nhau. Mục đích là để người được phỏng vấn không thể đoán trước được mục tiêu của chúng ta, từ đó tránh đưa ra những câu trả lời rập khuôn theo ý họ nghĩ. Để cuộc trò chuyện diễn ra tự nhiên, có thể đan xen một vài "câu hỏi phụ" giữa những câu hỏi cốt lõi thực sự muốn làm rõ.

Nếu muốn tìm hiểu suy nghĩ thật của một người, cần đào sâu hơn và tìm kiếm những sắc thái ý nghĩa ở nhiều góc độ. Như đã đề cập trong phần khảo sát, một số câu hỏi mở sẽ khuyến khích người trả lời suy nghĩ và trình bày chi tiết thông qua một hay nhiều cuộc hội thoại, trong khi các câu hỏi khác chỉ cần câu trả lời có/không hoặc một đáp án duy nhất.

4.2.9. Tại sao nên thực hiện nhóm tập trung?

Giả sử một quản lý thương hiệu cần thử nghiệm một loại ngũ cốc mới đang trong giai đoạn phát triển, được bảo chứng bởi một thương hiệu uy tín và có áp lực phải đảm bảo hương vị và kết cấu đạt chuẩn. Bộ phận Nghiên cứu và Phát triển đã đưa ra công thức và chế biến sản phẩm. Các loại sữa khác nhau cũng được chuẩn bị để đáp ứng mọi sở thích. Bây giờ, nhiệm vụ là thử nghiệm sản phẩm này với một nhóm người tham gia với mục tiêu là tìm ra chính xác những yếu tố giúp sản phẩm trở nên tuyệt vời, chứ không chỉ "tốt", liệu họ có thích nó, có sẵn sàng mua hay không, và cần cải thiện những gì.

Nhóm tập trung cũng rất hữu ích cho việc lên ý tưởng. Ví dụ cần phản hồi để cải thiện về logo mới hoặc cập nhật thương hiệu, có thể đưa ra cho nhóm đó các biến thể và thu

thập phần hồi của mọi người trong nhóm. Mục tiêu là một biểu tượng dễ nhớ, không phải dễ bị bỏ qua.

Giống như khi phỏng vấn, người tham gia cần ký một thỏa thuận cho phép sử dụng ý kiến và lời nói, nhưng vẫn bảo vệ quyền riêng tư. Họ sẽ không tham gia nếu không đồng ý với điều khoản này từ trước. Ngoài ra, cần có một hình thức thù lao để khuyến khích sự tham gia.

4.2.10. Quy trình phỏng vấn

Giả sử bạn muốn biết cảm giác lớn lên trong một gia đình đơn thân, hoặc cảm nhận của một người về môn toán trung học. B Giả sử bạn muốn biết cảm giác lớn lên trong một gia đình đơn thân, hoặc cảm nhận của một người về môn toán trung học. Những câu hỏi này có thể phù hợp với một cuộc phỏng vấn.

Khi các câu hỏi đã được soạn, có thể thử nghiệm với một vài người bạn để xem liệu các câu hỏi có tạo ra những câu trả lời có thể sử dụng được không.

Quy trình phỏng vấn bao gồm các công đoạn sau

- **Thỏa thuận:** Yêu cầu người tham gia viết và ký tên cùng ngày tháng và thông tin liên hệ của họ dưới một tuyên bố rằng họ có đủ năng lực và sẵn sàng chia sẻ suy nghĩ / ý kiến của mình trong cuộc phỏng vấn. Đảm bảo rằng danh tính và quyền riêng tư của họ sẽ được bảo vệ. Ngoài ra, bước này có thể được thực hiện ngoài phòng phỏng vấn bởi một người khác nếu bạn có nhân viên trợ giúp.
- **Bắt đầu cuộc trò chuyện:** Giúp người tham gia thư giãn bằng cách mời họ uống nước và hỏi họ về chuyến đi đến đây. Chia sẻ một điều gì đó tương tự về bản thân. Cung cấp cho họ một chiếc ghế ở bàn sao cho quyền lực giữa hai bên là ngang nhau và đảm bảo ánh sáng không chiếu vào mắt họ, v.v. Mục tiêu là khiến họ cảm thấy an toàn và thoải mái.
- **Xin ý kiến chân thành:** Nói với họ ngay từ đầu rằng bạn hy vọng họ sẽ sẵn lòng nói thật. Hỏi họ có đồng ý nếu bạn ghi âm cuộc phỏng vấn không. Nhắc họ rằng danh tính của họ sẽ được bảo vệ và bạn sẽ hỏi họ cho phép trích dẫn ý kiến của họ một cách ẩn danh sau khi quá trình phỏng vấn hoàn tất, nếu họ đưa ra những nhận xét có giá trị.
- **Kế hoạch.** Nói với họ rằng bạn sẽ hỏi các câu hỏi khác nhau từ danh sách chuẩn bị và họ có thể trả lời từ từ, nhưng cuộc phỏng vấn sẽ không kéo dài, ví dụ, nửa giờ. Nếu cuộc trò chuyện kéo dài vì có những thông tin hữu ích, hãy xin phép nếu muốn tiếp tục.

Khi cuộc phỏng vấn kết thúc, hãy cảm ơn họ và cung cấp cho họ một hình thức thù lao. Thù lao đó có thể là tiền mặt khi mọi người đều nhận số tiền như nhau – hoặc thẻ quà tặng Starbucks.

4.2.11. Phân tích dữ liệu phỏng vấn

Sau khi hoàn thành các cuộc phỏng vấn, việc phân tích kết quả giúp rút ra những thông tin quan trọng và nhận diện các xu hướng chính. Các câu trả lời của người tham gia đã làm rõ những suy nghĩ, cảm nhận và quan điểm của họ về các vấn đề được đặt ra. Một số câu trả lời phản ánh sự đồng thuận trong nhóm, trong khi một số khác chỉ ra những sự khác biệt rõ rệt, điều này giúp hiểu rõ hơn về các yếu tố ảnh hưởng đến quyết định của họ. Qua việc phân tích các ý kiến, chúng ta có thể xác định được các điểm mạnh và yếu trong quan điểm của người tham gia, đồng thời phát hiện ra những chủ đề hoặc câu hỏi cần được nghiên cứu thêm. Những thông tin thu thập được không chỉ giúp tạo ra những nhận định sâu sắc hơn mà còn mở ra cơ hội cải thiện hoặc điều chỉnh các chiến lược trong tương lai. Kết quả phỏng vấn cung cấp cái nhìn toàn diện và chi tiết về cách người tham gia tiếp cận và suy nghĩ về vấn đề, từ đó giúp đưa ra các quyết định chính xác và phù hợp hơn.

4.2.12. Ưu điểm và nhược điểm của phỏng vấn và nhóm trọng tâm

Cả phỏng vấn và nhóm trọng tâm đều là những phương pháp nghiên cứu định tính mạnh mẽ, nhưng mỗi phương pháp đều có những ưu và nhược điểm riêng.

Phỏng vấn mang lại chiều sâu ý kiến vượt trội. Đây là cách tốt nhất để thu thập thông tin chi tiết và cá nhân hóa, đặc biệt khi cần khám phá những chủ đề nhạy cảm hoặc phức tạp mà người tham gia không muốn chia sẻ công khai. Bằng cách tương tác trực tiếp, người phỏng vấn có thể đặt câu hỏi đào sâu, làm rõ những câu trả lời mơ hồ và nắm bắt các sắc thái cảm xúc. Tuy nhiên, phương pháp này tiêu tốn nhiều chi phí và thời gian hơn. Việc phân tích dữ liệu cũng khó hơn vì mỗi câu trả lời là duy nhất và không thể so sánh bằng số liệu.

Nhóm trọng tâm lại hiệu quả hơn trong việc thu thập nhiều ý kiến trong thời gian ngắn. Ưu điểm lớn nhất của phương pháp này là sự tương tác nhóm. Khi mọi người thảo luận và trao đổi, họ có thể khơi gợi những ý tưởng hoặc quan điểm bất ngờ, tạo ra một luồng thông tin phong phú. Đây là phương pháp lý tưởng để kiểm tra phản ứng ban đầu của công chúng đối với một ý tưởng hoặc sản phẩm mới, do có thể quan sát cách mọi người phản ứng và tương tác với nhau. Ngược lại, nhược điểm là một vài cá nhân có thể chiếm lĩnh cuộc trò chuyện, trong khi những người khác lại ngại bày tỏ ý kiến. Ngoài ra, "hiệu ứng nhóm" (groupthink) có thể xảy ra, khiến các thành viên có xu hướng đồng ý với ý kiến chung thay vì đưa ra quan điểm cá nhân.

4.2.13. Thu thập dữ liệu thông qua nhật ký hoặc nhật trình

Một trong những cách hiệu quả nhất để ghi lại trải nghiệm của người tham gia nghiên cứu là sử dụng nhật ký (diary) hoặc nhật trình (log), nhờ tính linh hoạt cao. Đây là lựa

chọn lý tưởng cho những người cần chủ động về thời gian, ví dụ như sinh viên hay người đi làm toàn thời gian. Họ có thể ghi chép dễ dàng trên ứng dụng điện thoại thông minh hoặc máy tính xách tay ở nhà. Hơn nữa, một cấu trúc ghi chép đều đặn sẽ rất phù hợp với loại dữ liệu cần thu thập.

Về bản chất, nhật trình và nhật ký đều có chung mục đích: thu thập dữ liệu theo các khoảng thời gian cố định. Ví dụ, một thuyền trưởng dùng nhật trình để ghi lại thời tiết, tọa độ và các sự kiện bất thường vào những thời điểm nhất định trong ngày, giúp họ có thể xem lại để tìm nguyên nhân nếu chẳng may đâm phải tảng băng. Tương tự, một vận động viên cũng có thể dùng nhật trình để theo dõi tiến độ luyện tập cho Olympic. Trong nghiên cứu, nhật trình hoạt động có thể giúp ghi lại dữ liệu cụ thể qua các lần kiểm tra định kỳ để đánh giá một yếu tố nào đó theo thời gian.

Trong khi đó, nhật ký thích hợp hơn khi muốn tìm hiểu trải nghiệm hoặc nhận thức của ai đó hàng ngày trong một giai đoạn nhất định. Có thể yêu cầu người tham gia viết nhật ký mỗi ngày, vào cuối ngày, trong suốt quá trình nghiên cứu. Hãy thử gợi ý họ viết vài câu hoặc một đoạn văn bằng chính lời lẽ của mình

Bảng 23: Mẫu nhật trình hoạt động

Thời gian	Hoạt động/Công việc	Cảm nhận/Quan sát	Thời gian/Đo lường
10:00 sáng			
1:00 chiều			
4:00 chiều			
8:00 tối			

Bảng 24: Mẫu nhật ký chung

Bảng tự suy ngẫm cho (Tên):	
(Ngày): / /	
Câu hỏi	Trả lời
1. Trải nghiệm của bạn khi làm “XYZ” hôm nay như thế nào?	
2. Nếu bạn phải tóm gọn nó trong một từ, từ đó là gì?	
3. Làm thế nào để công việc này có thể hiệu quả/hợp lý hơn nếu bạn làm điều gì đó khác đi? Chia sẻ những suy nghĩ khác nếu có. Cảm ơn!	

Hãy cùng xem cách phân tích loại dữ liệu này. Cũng như khi thực hiện phỏng vấn hay nhóm trọng tâm, có thể tuyển người tham gia qua thông báo, ví dụ trong lớp học, các báo sinh viên, mạng xã hội, email, phát tờ rơi, dán thông báo trên bảng tin, hoặc gọi điện thoại. Cần những người sẵn lòng tham gia và nên có khoản thù lao cho họ. Đồng thời,

cần chuẩn bị cẩn thận các câu hỏi hoặc hoạt động (hoặc tìm/tạo một ứng dụng) để ghi lại nhật trình hoặc nhật ký.

Vậy, làm thế nào để định lượng và phân tích kết quả? Một cách là gán các giá trị số cho những câu trả lời (hoặc từ ngữ) dự kiến thu thập. Khi đó, những con số này sẽ trở thành dữ liệu hữu ích để tiến hành phân tích.

4.2.14. Các nghiên cứu trong phòng thí nghiệm

Thu thập dữ liệu qua nghiên cứu người dùng hoặc trong phòng thí nghiệm là một yếu tố quan trọng trong việc phát triển và cải tiến các sản phẩm, dịch vụ, và hệ thống. Những nghiên cứu này giúp các nhà thiết kế, nhà phát triển, và các nhà nghiên cứu hiểu rõ hơn về hành vi và nhu cầu của người dùng, từ đó tạo ra những sản phẩm tốt hơn, phù hợp hơn với người sử dụng. Việc thu thập dữ liệu có thể thực hiện theo nhiều cách khác nhau, nhưng chủ yếu được chia thành hai phương pháp chính: nghiên cứu người dùng trong môi trường thực tế và nghiên cứu trong phòng thí nghiệm.

Nghiên cứu người dùng trong môi trường thực tế, hay còn gọi là nghiên cứu ngoài thực tế, là phương pháp thu thập dữ liệu khi người dùng tương tác với sản phẩm hoặc dịch vụ trong môi trường sống hàng ngày của họ. Phương pháp này mang lại cái nhìn chân thực và khách quan nhất về cách người dùng thực sự sử dụng sản phẩm trong các tình huống thực tế, chẳng hạn như khi làm việc, học tập, giải trí, hay tham gia vào các hoạt động khác. Một ví dụ điển hình là nghiên cứu về cách người dùng sử dụng các ứng dụng di động trong cuộc sống hàng ngày của họ. Các nhà nghiên cứu có thể thu thập dữ liệu từ phản hồi tự báo cáo của người dùng, ghi chép hành vi sử dụng, hoặc sử dụng các công cụ theo dõi như ứng dụng di động hoặc thiết bị theo dõi hoạt động.

Ưu điểm lớn nhất của nghiên cứu ngoài thực tế là khả năng phản ánh chính xác hành vi và cảm nhận của người dùng trong điều kiện tự nhiên, không bị tác động bởi các yếu tố môi trường. Dữ liệu thu thập được trong môi trường thực tế có thể giúp các nhà phát triển hiểu rõ hơn về những khó khăn, thách thức mà người dùng gặp phải khi sử dụng sản phẩm, từ đó đưa ra những cải tiến hợp lý. Tuy nhiên, nghiên cứu ngoài thực tế cũng có một số nhược điểm. Một trong số đó là khó kiểm soát các yếu tố bên ngoài, chẳng hạn như sự thay đổi trong môi trường sử dụng, hành vi người dùng không đồng nhất, hay các yếu tố tác động từ các thiết bị hoặc phần mềm khác. Điều này có thể dẫn đến việc thu thập dữ liệu không chính xác hoặc thiếu nhất quán.

Một ví dụ điển hình của nghiên cứu ngoài thực tế là việc nghiên cứu hành vi người dùng khi sử dụng một ứng dụng di động trong suốt một ngày làm việc của họ. Các nhà nghiên cứu có thể theo dõi xem người dùng tương tác với ứng dụng như thế nào, có gặp phải khó khăn hay vấn đề gì không, và cách họ sử dụng các tính năng của ứng dụng. Qua

đó, các nhà nghiên cứu có thể thu thập được những phản hồi tự nhiên từ người dùng và tìm ra những điểm mạnh và điểm yếu của sản phẩm, từ đó đưa ra những đề xuất cải tiến.

Ngược lại, nghiên cứu trong phòng thí nghiệm cung cấp môi trường kiểm soát chặt chẽ, nơi các yếu tố tác động có thể được kiểm soát và quan sát một cách dễ dàng. Môi trường phòng thí nghiệm thường được sử dụng khi cần giám sát chi tiết hành vi người dùng hoặc khi sản phẩm cần phải được thử nghiệm trong điều kiện lý tưởng, không bị ảnh hưởng bởi các yếu tố ngoài ý muốn. Trong một nghiên cứu phòng thí nghiệm, người tham gia có thể được yêu cầu thực hiện các nhiệm vụ cụ thể trong một không gian được thiết kế để đo lường và phân tích hành vi của họ.

Một trong những ưu điểm lớn của nghiên cứu trong phòng thí nghiệm là khả năng kiểm soát các yếu tố bên ngoài, giúp dữ liệu thu thập được trở nên chính xác và đáng tin cậy hơn. Các nhà nghiên cứu có thể tạo ra các tình huống thí nghiệm cụ thể để kiểm tra các giả thuyết và đo lường hiệu quả của các tính năng hoặc giao diện người dùng. Chẳng hạn, nếu một công ty phát triển một giao diện người dùng mới cho ứng dụng di động, họ có thể mời người dùng đến phòng thí nghiệm để thực hiện một loạt các nhiệm vụ, và từ đó thu thập dữ liệu về mức độ dễ sử dụng, sự hài lòng và mức độ hiệu quả của giao diện mới. Ngoài ra, trong môi trường phòng thí nghiệm, các nhà nghiên cứu có thể sử dụng các công cụ hỗ trợ như máy theo dõi mắt, máy đo nhịp tim, máy đo độ dẫn điện và các công cụ đo lường khác để thu thập dữ liệu chi tiết về hành vi và cảm xúc của người dùng khi sử dụng sản phẩm.

Tuy nhiên, một nhược điểm của nghiên cứu trong phòng thí nghiệm là môi trường giả lập có thể làm thay đổi cách người dùng tương tác với sản phẩm. Người tham gia có thể cảm thấy lo lắng, căng thẳng hoặc bị giám sát, điều này có thể ảnh hưởng đến hành vi tự nhiên của họ. Do đó, mặc dù dữ liệu thu thập được trong phòng thí nghiệm có thể rất chính xác và chi tiết, nhưng nó không phải lúc nào cũng phản ánh chính xác cách mà người dùng sẽ tương tác với sản phẩm trong môi trường thực tế. Ví dụ, trong khi một người dùng có thể thực hiện các nhiệm vụ trên ứng dụng một cách nhanh chóng và hiệu quả trong môi trường phòng thí nghiệm, họ có thể gặp khó khăn hơn khi sử dụng ứng dụng trong cuộc sống hàng ngày, khi có nhiều yếu tố tác động khác như công việc, gia đình hay môi trường xung quanh.

Mặc dù mỗi phương pháp nghiên cứu có những ưu điểm và nhược điểm riêng, nhưng việc kết hợp cả nghiên cứu ngoài thực tế và nghiên cứu trong phòng thí nghiệm có thể mang lại những kết quả tối ưu. Ví dụ, một nghiên cứu có thể bắt đầu với việc thu thập dữ liệu trong môi trường phòng thí nghiệm để kiểm tra các giả thuyết và tính năng của sản phẩm. Sau đó, nghiên cứu có thể được mở rộng ra ngoài thực tế để xem xét cách người

dùng thực sự sử dụng sản phẩm trong môi trường tự nhiên của họ. Việc kết hợp cả hai phương pháp này giúp các nhà nghiên cứu thu thập được dữ liệu toàn diện hơn, vừa có độ chính xác cao từ phòng thí nghiệm, vừa có được những phản hồi thực tế từ người dùng trong môi trường sống hàng ngày.

4.3. Các phương pháp phân tích

4.3.1. Giới thiệu chung

Có hai nhóm phương pháp phân tích dữ liệu chính: phương pháp định lượng và phương pháp định tính. Phương pháp định lượng sử dụng các kỹ thuật đo lường để thu thập kết quả dưới dạng các con số hoặc dữ liệu. Ngược lại, phương pháp định tính liên quan đến việc quan sát hành vi, thái độ, hoặc ý kiến, từ đó đánh giá các phẩm chất.

Việc hình thành lý thuyết thường bắt đầu từ một quan sát dẫn đến một giả thuyết. Đây là quá trình quy nạp trong nghiên cứu định tính. Trong khi đó, trong phân tích định lượng, giả thuyết được kiểm tra và các con số, thông tin hoặc dữ liệu được thu thập để phân tích, từ đó suy ra một giả thuyết khác chính xác hơn, tiếp tục phát triển lý thuyết thành mô hình.

4.3.2. Phương pháp phân tích định lượng

Phương pháp phân tích định lượng (Quantification) là quá trình thu thập và phân tích dữ liệu dưới dạng số liệu nhằm đo lường một hiện tượng, sự kiện hoặc vấn đề cụ thể, với đặc điểm chính bao gồm: khả năng đo lường chính xác, tính khách quan cao, dễ so sánh và phân tích bằng các công cụ thống kê. Phương pháp này thường áp dụng cho mẫu lớn, giúp xác định mối quan hệ nhân quả và xu hướng một cách rõ ràng. Kết quả định lượng mang tính tổng quát, phù hợp để giải quyết các vấn đề cần sự chính xác và kiểm chứng dữ liệu.

Các ứng dụng định lượng trong thực tế có thể kể đến bao gồm

- Trong kinh doanh: Đo lường hiệu quả tài chính, doanh số và mức độ hài lòng khách hàng.
- Trong Marketing: Phân tích hiệu suất các chiến dịch qua các chỉ số ROI, CTR.
- Trong khoa học: Kiểm tra giả thuyết và rút ra kết luận trên cơ sở dữ liệu lớn.

Phương pháp phân tích định lượng là phương pháp phân tích sử dụng dữ liệu số liệu để thu thập thông tin và kiểm tra giả thuyết. Thay vì thăm dò ý nghĩa chủ quan bằng câu hỏi như “làm thế nào?” và “tại sao?” phương pháp phân tích định lượng cung cấp những giải thích nhân quả chính xác có thể đo lường và truyền đạt bằng toán học.

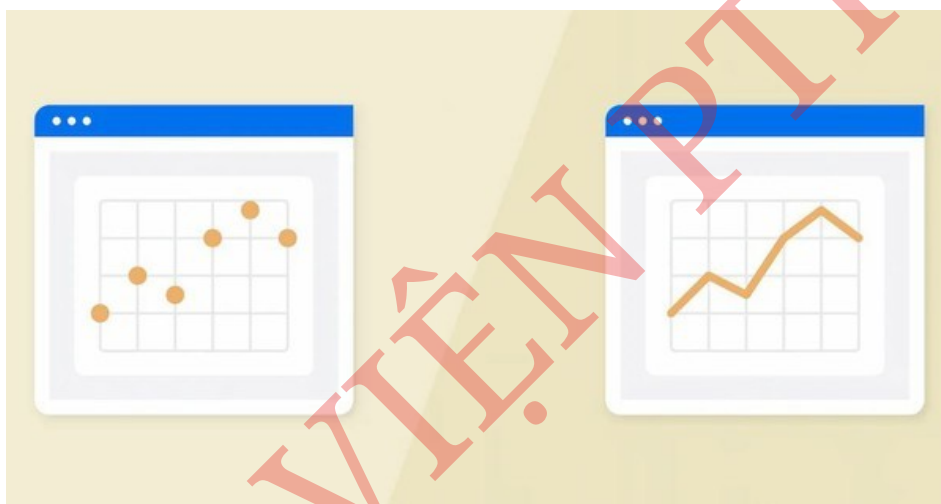
Phương pháp phân tích định lượng thường được tiến hành trong một môi trường được kiểm soát. Thay vì đạt được cái nhìn sâu sắc hoặc hiểu biết về một vấn đề chủ quan, phụ thuộc vào bối cảnh như trường hợp nghiên cứu định tính, mục tiêu của phương pháp

phân tích định lượng là thu thập thông tin khách quan, khái quát hóa kết quả nghiên cứu, đưa ra kết luận chính xác và có thể kiểm chứng được.

Dữ liệu định lượng là loại dữ liệu có thể được đo lường và biểu diễn thông qua số lượng, bao gồm bất kỳ thông tin nào có thể được đếm hoặc đo lường, cho phép các nhà nghiên cứu thu được kết quả có thể so sánh và tính toán được. Dữ liệu này thường được sử dụng để xác định mối quan hệ, mức độ chênh lệch, hoặc các xu hướng số lượng.

Có hai loại chính của dữ liệu định lượng:

- Dữ liệu liên tục: Có thể nhận bất kỳ giá trị nào trong một khoảng. Ví dụ: chiều cao, cân nặng, nhiệt độ và thời gian.
- Dữ liệu rời rạc: Chỉ nhận các giá trị cụ thể, thường là các số đếm. Ví dụ: số lượng sinh viên trong một lớp học, số lần xảy ra một sự kiện.



Hình 89: Dữ liệu rời rạc (bên trái) và dữ liệu liên tục (bên phải)

Dữ liệu định lượng thường được thu thập thông qua các phương pháp như thăm dò, thí nghiệm, phân tích số liệu và có thể được phân tích sử dụng các kỹ thuật thống kê để tạo ra kết quả dễ hiểu và dễ trình bày. Dữ liệu định lượng rất quan trọng trong các lĩnh vực như khoa học tự nhiên, kinh tế và y tế, nơi mà sự chính xác và khả năng đo lường là cần thiết.

Kỹ thuật phân tích dữ liệu định lượng có thể kể đến bao gồm thống kê mô tả và thống kê suy luận

- Thống kê mô tả là tập hợp các phương pháp được sử dụng để tóm tắt và mô tả các đặc điểm chính của một tập dữ liệu. Thống kê mô tả giúp chúng ta hiểu rõ hơn về dữ liệu, bao gồm:
 - Dữ liệu tập trung như thế nào? (Trung bình, trung vị, mốt - mode)

- Dữ liệu phân tán như thế nào? (Độ lệch chuẩn, phương sai)
- Dữ liệu có dạng hình gì? (Biểu đồ tần suất, biểu đồ cột)
- Mối liên hệ giữa các biến là gì? (Hệ số tương quan)

Thống kê mô tả cung cấp cho chúng ta một bức tranh tổng quan về dữ liệu, giúp xác định các xu hướng, mẫu hình và điểm bất thường.

- Thống kê suy luận là tập hợp các phương pháp được sử dụng để đưa ra kết luận về một tập dữ liệu lớn hơn dựa trên một mẫu nhỏ. Thống kê suy luận cho phép kiểm tra các giả thuyết, dự đoán xu hướng và đánh giá mức độ tin cậy của kết quả giúp đưa ra những kết luận chính xác và đáng tin cậy từ dữ liệu, hỗ trợ việc ra quyết định hiệu quả trong nhiều lĩnh vực khác nhau.

Trong nghiên cứu và thu thập dữ liệu, các phương pháp định lượng đóng vai trò quan trọng, giúp cung cấp những con số chính xác và có thể phân tích được, từ đó đưa ra những kết luận rõ ràng về các hiện tượng, hành vi hoặc xu hướng. Các phương pháp này bao gồm khảo sát, thí nghiệm kiểm soát, phân tích thống kê từ dữ liệu thứ cấp, và quan sát định lượng, mỗi phương pháp mang lại những ưu điểm và ứng dụng khác nhau tùy vào mục đích nghiên cứu.

- Khảo sát và bảng hỏi là phương pháp thu thập dữ liệu định lượng phổ biến, thường được thực hiện qua các câu hỏi có sẵn và đáp án có thể định lượng được. Phương pháp này có thể được thực hiện dưới nhiều hình thức khác nhau, bao gồm khảo sát trực tiếp, qua email, điện thoại hoặc nền tảng trực tuyến. Các câu hỏi thường sử dụng thang đo Likert (ví dụ từ 1 đến 5 hoặc từ 1 đến 10) hoặc các lựa chọn với nhiều đáp án, giúp thu thập ý kiến của số lượng lớn người trong thời gian ngắn. Khảo sát thường được dùng để đánh giá mức độ hài lòng của khách hàng, nhận xét về sản phẩm hoặc dịch vụ, và thu thập ý kiến từ người tiêu dùng trên diện rộng. Một trong những ưu điểm của phương pháp này là khả năng chuẩn hóa và phân tích nhanh chóng nhờ vào sự thống nhất trong cấu trúc và cách thức định lượng.
- Trong khi đó, thí nghiệm và thử nghiệm kiểm soát là phương pháp chủ chốt trong nghiên cứu khoa học và phát triển sản phẩm. Các nhà nghiên cứu tạo ra một môi trường kiểm soát và tác động các biến độc lập lên biến phụ thuộc, từ đó đo lường sự thay đổi và rút ra kết luận khoa học. Ví dụ, trong nghiên cứu y tế, các thử nghiệm lâm sàng kiểm soát giả dược giúp đánh giá hiệu quả của thuốc mới. Phương pháp này mang lại độ chính xác cao nhờ khả năng kiểm soát và loại trừ các yếu tố ngoại lai ảnh hưởng đến kết quả.

- Một phương pháp định lượng khác là phân tích thống kê từ dữ liệu thứ cấp, là các dữ liệu đã được thu thập trước đó từ các tổ chức, cơ quan nhà nước hoặc các nhà nghiên cứu khác. Việc phân tích thống kê từ dữ liệu này giúp tiết kiệm thời gian và chi phí trong việc thu thập dữ liệu mới, đồng thời cho phép các nhà nghiên cứu tận dụng thông tin đã có sẵn để thực hiện các phân tích sâu rộng và đưa ra kết luận. Doanh nghiệp thường sử dụng phương pháp này để dự báo các xu hướng thị trường từ các báo cáo của chính phủ.
- Cuối cùng, quan sát định lượng là phương pháp thu thập dữ liệu thông qua việc đo lường các đặc điểm và hành vi có thể định lượng được, như tần suất, thời gian và tỷ lệ của các hành động cụ thể. Ví dụ, trong nghiên cứu thị trường, nhà nghiên cứu có thể quan sát số lượng người mua hàng trong một giờ, thời gian mà họ dành ở từng khu vực của cửa hàng, hoặc tỷ lệ người mua sản phẩm này so với sản phẩm kia. Dữ liệu thu thập được có thể chuyển thành con số để phân tích và đưa ra dự đoán, rất phù hợp cho các nghiên cứu về hành vi người tiêu dùng hoặc các hoạt động trong môi trường tự nhiên mà không cần phải hỏi trực tiếp người tham gia.

Phương pháp phân tích định lượng có ưu điểm như sau

- Độ chính xác đáng tin cậy: Sử dụng dữ liệu số liệu đo lường, giúp đạt được mức độ chính xác cao trong kết quả nghiên cứu. Phương pháp này thường sử dụng các công cụ, kỹ thuật và phương pháp thống kê để đảm bảo tính chính xác và đáng tin cậy của dữ liệu.
- Khả năng so sánh và tổng quát hóa: Kết quả của phương pháp phân tích định lượng thường có thể được so sánh và tổng quát hóa cho một quần thể lớn hơn. Bằng cách sử dụng các phương pháp thống kê, phương pháp phân tích định lượng có thể rút ra những kết luận chung cho một tập hợp lớn người dùng hoặc quần thể tương tự.
- Phân tích sâu, khảo sát chi tiết: Cho phép phân tích sâu và khám phá các mối quan hệ phức tạp giữa các biến số. Khi sử dụng các phương pháp thống kê và mô hình, phương pháp phân tích định lượng có thể xác định tác động và mức độ ảnh hưởng của các yếu tố khác nhau đối với hiện tượng nghiên cứu.

Bên cạnh đó, phương pháp này cũng có các nhược điểm có thể kể đến bao gồm

- Thiếu chiều sâu: Không thể hiểu được động cơ và lý do đằng sau hành vi của con người.

- Tính gò bó: Người tham gia bị giới hạn bởi các câu hỏi và lựa chọn được cung cấp.
- Có thể bị sai lệch: Những mô hình định lượng thường tập trung vào việc thu thập dữ liệu số liệu, bỏ qua những khía cạnh phức tạp và sự phong phú của hiện thực.
- Chi phí và thời gian: Việc thu thập và phân tích dữ liệu định lượng tốn kém nhiều chi phí, nguồn lực. Song đó, việc thiết kế và thực hiện một phương pháp phân tích định lượng có thể mất nhiều thời gian.
- Giới hạn của mô hình và giả định: Phương pháp phân tích định lượng thường dựa trên các mô hình và giả định để phân tích dữ liệu. Tuy nhiên, các mô hình này có thể không thể hiện đầy đủ mọi khía cạnh của hiện tượng nghiên cứu và có thể gây ra sai lệch trong kết quả nghiên cứu nếu giả định không chính xác.

4.3.3. Phương pháp phân tích định tính

Phương pháp phân tích định tính (Qualitative) là phương pháp nghiên cứu tập trung vào việc hiểu sâu sắc hiện tượng thông qua thu thập và phân tích dữ liệu định tính, các thông tin phi số liệu như văn bản, hình ảnh hoặc âm thanh. Phương pháp này bao gồm nghiên cứu định tính, trong đó sử dụng các phương pháp như phỏng vấn sâu, nhóm trọng tâm, quan sát tham gia và phân tích tài liệu. Kỹ thuật phân tích dữ liệu định tính như phân tích nội dung, phân tích chủ đề, lý thuyết nền tảng và phân tích diễn ngôn giúp diễn giải và rút ra ý nghĩa từ dữ liệu. Nghiên cứu định tính có ưu điểm khi cung cấp cái nhìn chi tiết, linh hoạt và bối cảnh hóa, nhưng cũng có nhược điểm như khó tổng quát hóa kết quả, tính chủ quan, khó tái lập và đòi hỏi nhiều thời gian và nguồn lực.

Phương pháp phân tích định tính là phương pháp phân tích sử dụng dữ liệu phi số liệu để thu thập thông tin và hiểu rõ hơn về một vấn đề hoặc hiện tượng. Nghiên cứu định tính nhằm mục đích đạt được những hiểu biết sâu sắc về các hiện tượng, nhóm hoặc trải nghiệm không thể đo lường hoặc định lượng một cách khách quan bằng toán học. Thay vì tìm cách khám phá các câu trả lời hoặc số liệu thống kê chính xác trong môi trường được kiểm soát như nghiên cứu định lượng, nghiên cứu định tính mang tính khám phá nhiều hơn, dựa trên các nguồn dữ liệu như ảnh, quan sát, phân tích văn bản, đoạn phim video và các cuộc phỏng vấn. Một số ví dụ về các câu hỏi nghiên cứu định tính được cung cấp, chẳng hạn như cảm giác của một trải nghiệm như thế nào, mọi người nói về điều gì đó như thế nào, họ hiểu trải nghiệm đó như thế nào và các sự kiện diễn ra như thế nào đối với mọi người. Nghiên cứu theo cách tiếp cận định tính mang tính khám phá và tìm cách giải thích “làm thế nào” và “tại sao” một hiện tượng hoặc hành vi cụ thể lại hoạt động như vậy trong một bối cảnh cụ thể. Nó có thể được sử dụng để tạo ra các giả thuyết và lý thuyết từ dữ liệu.

Dữ liệu định tính là loại dữ liệu không thể được đo lường bằng số mà thường được mô tả bằng ngôn từ. Dữ liệu định tính chủ yếu dùng để mô tả đặc điểm, thuộc tính, hoặc hiện tượng theo cách không định lượng, tập trung vào "chất" hơn là "lượng". Dữ liệu này thường liên quan đến mô tả về màu sắc, kích thước, hình dạng, cảm xúc, kinh nghiệm và cảm nhận, hoặc bất kỳ khía cạnh nào không thể được tính toán chính xác bằng số.

Dữ liệu định tính có thể được thu thập thông qua các phương pháp như phỏng vấn và nhóm trọng tâm, quan sát và phân tích nội dung. Loại dữ liệu này thường được sử dụng trong các nghiên cứu xã hội, nhân văn và một số lĩnh vực khoa học tự nhiên khi mục tiêu nghiên cứu tập trung vào việc hiểu các nguyên tắc cơ bản hay ngữ cảnh của một hiện tượng chứ không chỉ đơn thuần là đo lường.

Kỹ thuật phân tích dữ liệu định tính là các phương pháp giúp nhà nghiên cứu xử lý và diễn giải dữ liệu phi số liệu, nhằm rút ra các chủ đề, mẫu hình hoặc ý nghĩa từ dữ liệu.

- Lý thuyết nội dung (Content theory – CT): Đây là một phương pháp nghiên cứu nhằm hiểu và giải thích ý nghĩa của dữ liệu văn bản. CT tập trung vào việc xác định các chủ đề, khái niệm và mối quan hệ trong nội dung dữ liệu để từ đó xây dựng hoặc củng cố lý thuyết về chủ đề nghiên cứu.
- Lý thuyết nền tảng (Grounded theory – GT): Lý thuyết nền tảng (GT) là một phương pháp nghiên cứu quy nạp được sử dụng để xây dựng lý thuyết từ dữ liệu định tính. Nó tập trung vào việc khám phá và phát triển các khái niệm, mối liên hệ và mô hình từ dữ liệu thu thập được, thay vì áp dụng các lý thuyết hoặc khuôn khổ có sẵn.
- Phân tích theo chủ đề (Thematic analysis – TA): Phân tích theo chủ đề (TA) là một phương pháp phân tích dữ liệu định tính phổ biến, được sử dụng để xác định, phân tích và diễn giải các chủ đề, mô típ và ý nghĩa trong dữ liệu. TA có thể áp dụng cho nhiều loại dữ liệu khác nhau, bao gồm văn bản, phỏng vấn, ghi chép quan sát, hình ảnh và video.
- Phân tích biện luận (Discourse analysis – DA): DA là một phương pháp nghiên cứu nhằm hiểu và giải thích ý nghĩa của các văn bản, bao gồm cả lời nói, văn viết và hình ảnh. DA tập trung vào cách thức mà ngôn ngữ được sử dụng để thực hiện các hành động xã hội, thiết lập các mối quan hệ quyền lực và hình thành các ý thức hệ.

Các phương pháp thu thập dữ liệu định tính đóng vai trò quan trọng trong việc hiểu sâu về suy nghĩ, cảm xúc và hành vi của người tham gia nghiên cứu. Những phương pháp này, bao gồm phỏng vấn sâu, thảo luận nhóm trọng tâm, quan sát thực địa và phân tích tài liệu, giúp nhà nghiên cứu khai thác thông tin chi tiết và phong phú mà các phương pháp định lượng khó có thể nắm bắt được. Mỗi phương pháp mang lại một góc

nhìn độc đáo, giúp giải mã các yếu tố tinh tế trong hành vi và quan điểm của đối tượng nghiên cứu.

- **Phỏng vấn sâu:** Phỏng vấn sâu là một trong những phương pháp thu thập dữ liệu định tính phổ biến, giúp nhà nghiên cứu tìm hiểu kỹ càng về suy nghĩ, cảm xúc và quan điểm của người tham gia. Phỏng vấn thường được thực hiện theo hình thức trò chuyện trực tiếp, nhưng không theo một khuôn mẫu cố định, cho phép người tham gia tự do chia sẻ và nhà nghiên cứu có thể điều chỉnh câu hỏi theo diễn biến của cuộc trò chuyện. Phương pháp này thường được dùng trong nghiên cứu xã hội, tâm lý, và nghiên cứu thị trường, giúp phát hiện những động lực sâu xa, các yếu tố cảm xúc hoặc những yếu tố khó nhận biết từ các phương pháp định lượng.
- **Thảo luận nhóm trọng tâm:** Thảo luận nhóm trọng tâm là phương pháp thu thập dữ liệu định tính thông qua việc mời một nhóm người tham gia vào cuộc trò chuyện có hướng dẫn bởi một người điều phối. Phương pháp này giúp nhà nghiên cứu khám phá các quan điểm đa dạng và tương tác giữa các thành viên trong nhóm, từ đó hiểu rõ hơn về cách suy nghĩ và hành vi của nhóm đối tượng mục tiêu. Mỗi thành viên có thể chia sẻ ý kiến cá nhân và phản hồi lẫn nhau, điều này tạo nên một không gian trao đổi phong phú và năng động. Phương pháp thảo luận nhóm thường được áp dụng trong nghiên cứu marketing và truyền thông, giúp tìm hiểu sâu hơn về sự yêu thích của khách hàng, các ý tưởng sản phẩm mới hoặc những yếu tố ảnh hưởng đến quyết định mua hàng.
- **Quan sát Thực địa (Field Observation):** Quan sát thực địa là phương pháp nhà nghiên cứu tự tham gia hoặc quan sát trực tiếp hoạt động của người được nghiên cứu trong môi trường tự nhiên của họ. Phương pháp này không chỉ thu thập dữ liệu về hành vi mà còn bao gồm cả những yếu tố môi trường, bối cảnh xã hội xung quanh đối tượng nghiên cứu. Ví dụ, trong nghiên cứu hành vi tiêu dùng, nhà nghiên cứu có thể quan sát cách khách hàng di chuyển và chọn sản phẩm trong cửa hàng, giúp phát hiện các yếu tố ảnh hưởng đến quyết định mua sắm mà không cần can thiệp hay hỏi trực tiếp. Quan sát thực địa rất hữu ích trong nghiên cứu xã hội và nhân chủng học, vì nó cung cấp cái nhìn sâu sắc từ chính trải nghiệm thực tế của người nghiên cứu.
- **Phân tích tài liệu và nội dung:** Phân tích tài liệu là phương pháp thu thập dữ liệu định tính qua việc đọc và phân tích các tài liệu đã có sẵn như báo cáo, tài liệu lịch sử, ghi chép cá nhân, hoặc nội dung truyền thông. Từ đó, nhà nghiên cứu rút ra các chủ đề, mô hình hoặc ý nghĩa mà những tài liệu này phản ánh. Phương pháp phân tích nội

dung thường được áp dụng trong nghiên cứu truyền thông, giáo dục, và văn hóa. Ví dụ, một nhà nghiên cứu về truyền thông có thể phân tích bài báo, mạng xã hội và blog để hiểu cách mà một vấn đề xã hội cụ thể được truyền tải và tiếp nhận bởi cộng đồng. Phương pháp này đặc biệt hữu ích khi cần so sánh quan điểm qua các giai đoạn thời gian hoặc qua các nguồn khác nhau.

Phương pháp phân tích định tính có ưu điểm sau:

- Nhờ sự tham gia chặt chẽ của nhà nghiên cứu, nghiên cứu định tính giúp giúp có được cái nhìn sâu sắc hơn về lĩnh vực này. Nghiên cứu định tính cho phép nhà nghiên cứu tìm ra những vấn đề thường bị bỏ sót (chẳng hạn như sự tinh tế và phức tạp) bởi các câu hỏi mang tính khoa học, thực tế hơn.
- Những mô tả định tính có thể quan trọng trong việc gợi ý các mối quan hệ, nguyên nhân, tác động và các quá trình có thể có.
- Phân tích định tính cho phép phát hiện sự mơ hồ/mâu thuẫn trong dữ liệu, phản ánh thực tế.
- Nghiên cứu định tính sử dụng phong cách miêu tả, tường thuật; nghiên cứu này có thể mang lại lợi ích đặc biệt cho người thực hiện vì họ có thể chuyển sang các báo cáo định tính để kiểm tra các dạng kiến thức có thể không có sẵn, từ đó đạt được cái nhìn sâu sắc mới.
- Nghiên cứu định tính cho phép khám phá chi tiết các khía cạnh của một vấn đề, thu thập những quan điểm, trải nghiệm và ý kiến cá nhân của người tham gia. Nhờ vậy, nhà nghiên cứu có thể hiểu rõ hơn về các yếu tố ảnh hưởng đến hành vi, thái độ và suy nghĩ của họ.

Bên cạnh đó, phương pháp này cũng tồn tại những nhược điểm sau:

- Do tốn thời gian và chi phí nên các thiết kế định tính thường không lấy mẫu từ các tập dữ liệu quy mô lớn.
- Vấn đề về tính giá trị hoặc độ tin cậy đầy đủ là một vấn đề bị chỉ trích lớn. Do tính chất chủ quan của dữ liệu định tính và nguồn gốc của nó trong các bối cảnh riêng lẻ nên rất khó áp dụng các tiêu chuẩn thông thường về độ tin cậy và giá trị.
- Thời gian cần thiết để thu thập, phân tích và giải thích dữ liệu rất dài. Phân tích dữ liệu định tính là khó khăn và cần có kiến thức chuyên môn về lĩnh vực đó để diễn giải dữ liệu định tính.

4.3.4. Kết hợp cả hai phương pháp

Chiến lược phương pháp hỗn hợp có thể khắc phục những hạn chế của từng phương pháp riêng lẻ bằng cách kết hợp cả phương pháp định lượng và định tính vào nghiên cứu

để mang lại góc nhìn toàn diện hơn. Thay vì thiên về một bên, phương pháp hỗn hợp nhấn mạnh vào chính vấn đề nghiên cứu và sử dụng mọi cách tiếp cận có thể để tạo ra sự hiểu biết đầy đủ hơn.

Trong thiết kế nghiên cứu hỗn hợp, dữ liệu định lượng thường bao gồm những thông tin đóng (closed-ended), sau đó được phân tích, thống kê và so sánh để đưa ra các biểu diễn dưới dạng con số. Ngược lại, dữ liệu định tính thường mang tính chủ quan và mở (open-ended), cho phép “tiếng nói” của người tham gia được lắng nghe, từ đó việc quan sát trở nên dễ diễn giải hơn.

Ví dụ sau đây cho thấy cách hai phương pháp có thể được kết hợp để làm rõ hơn một vấn đề nghiên cứu:

Hãy tưởng tượng bạn là một nhà nghiên cứu trẻ trong lĩnh vực truy xuất thông tin muốn tìm hiểu mối liên hệ giữa ý định của con người khi tham gia vào những phiên tìm kiếm mất nhiều thời gian và hành vi tìm kiếm trên web. Từ đây có thể thiết kế một nghiên cứu người dùng để thu thập dữ liệu bằng công cụ định lượng, chẳng hạn như nhật ký truy vấn (query logs), thời gian sử dụng truy vấn hay chiến lược chỉnh sửa lại truy vấn, v.v. Các dữ liệu này được nhóm lại thành các hành vi tìm kiếm tương tự nhau, với giả định rằng mỗi nhóm phản ánh mục đích tìm kiếm khác nhau, và mục đích khác nhau sẽ dẫn đến hành vi tìm kiếm khác nhau. Tuy nhiên, rất khó để biết chính xác mỗi nhóm tương ứng với từng loại nào. Nếu may mắn, bạn có thể tìm được vài gợi ý từ các nghiên cứu trước, nhưng trong những nghiên cứu mới nhất thì thường không có sẵn. Trong tình huống như vậy, bạn – với tư cách là nhà nghiên cứu – có thể tiếp tục phỏng vấn một nhóm nhỏ những người tham gia để hiểu rõ hơn về chiến lược và ý định của họ trong quá trình tìm kiếm. Kết hợp kết quả định lượng với dữ liệu định tính sẽ giúp bạn có câu trả lời toàn diện hơn cho vấn đề nghiên cứu.

4.4. Đánh giá

Mục đích chính của phân tích dữ liệu thường là để đưa ra quyết định hoặc phát triển một cái nhìn sâu sắc. Và để đạt được những mục tiêu này, điều quan trọng là cần biết mức độ tin cậy của dữ liệu mà mình đang sử dụng. Cụ thể hơn, chúng ta cần biết mức độ tin tưởng vào việc phân tích mà chúng ta đã thực hiện hoặc mô hình mà chúng ta đã xây dựng có đủ vững chắc để chấp nhận những kết luận hoặc khuyến nghị từ đó không? Ví dụ, nếu chúng ta đang xây dựng một bộ phân loại, chúng ta cần biết độ chính xác của nó. Nhưng liệu chỉ số độ chính xác có đủ không? Nếu một mô hình bộ phân loại đã học quá mức dữ liệu thì sao? Trong trường hợp này, mô hình có thể cho chúng ta một giá trị độ chính xác tương đối tốt đối với dữ liệu huấn luyện nhưng lại hoạt động kém trên dữ liệu

mới (dữ liệu kiểm tra). Tương tự, chúng ta có thể tạo ra một mô hình quá phức tạp đến mức khó tổng quát hoặc giải thích mặc dù mô hình đó có thể hoạt động tốt.

Khi đánh giá độ tốt của kỹ thuật hoặc mô hình, chúng ta cần xem xét nhiều chỉ số và mức độ hiệu quả. Điều này cũng áp dụng khi chúng ta cố gắng so sánh các mô hình khác nhau.

4.4.1. So sánh mô hình

Khi giải quyết một bài toán học máy, việc chọn mô hình phù hợp đóng vai trò quan trọng trong việc đạt được kết quả tối ưu. Tuy nhiên, không có một mô hình duy nhất nào phù hợp cho tất cả các loại dữ liệu hay bài toán, vì vậy việc so sánh các mô hình khác nhau là một bước thiết yếu trong quy trình phát triển mô hình học máy. Vậy câu hỏi đặt ra là làm thế nào để so sánh các mô hình khác nhau và chọn được mô hình tốt nhất cho vấn đề của mình? Rõ ràng, cần phải có một phương pháp để kiểm tra hiệu quả của mô hình, để xác định mức độ mô hình có thể giải thích dữ liệu tốt như thế nào. May mắn thay, có rất nhiều chỉ số để đánh giá chất lượng của mô hình. Một số chỉ số đại diện bao gồm:

- Độ chính xác (Accuracy) là một chỉ số đánh giá hiệu suất của mô hình phân loại, được định nghĩa là tỷ lệ giữa số lượng mẫu được mô hình dự đoán đúng (bao gồm cả True Positive - TP và True Negative - TN) trên tổng số mẫu trong tập dữ liệu. Độ chính xác giúp ta đánh giá hiệu quả dự báo của mô hình trên một bộ dữ liệu. Độ chính xác càng cao thì mô hình của chúng ta càng chuẩn xác. Độ chính xác là chỉ số đơn giản và trực quan, thường được sử dụng làm điểm khởi đầu để so sánh các mô hình hoặc điều chỉnh tham số. Chỉ số này cung cấp cái nhìn tổng quan về hiệu suất trước khi đi sâu vào các chỉ số chuyên biệt hơn như độ nhạy (Sensitivity) hoặc độ đặc hiệu (Specificity). Độ chính xác được tính theo công thức

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Độ chính xác của lớp dương (Precision) là một chỉ số đánh giá hiệu suất của mô hình phân loại, đo lường tỷ lệ các mẫu được dự đoán là dương (positive) trong số các mẫu thực sự thuộc lớp dương. Chỉ số này cho biết trong tất cả những trường hợp mà mô hình dự đoán là dương tính, có bao nhiêu trường hợp thực sự đúng là dương tính. Độ chính xác của lớp dương là một chỉ số đánh giá quan trọng trong bài toán phân loại, thể hiện mức độ chính xác trong các dự đoán dương tính của mô hình và đặc biệt quan trọng trong các tình huống mà dự đoán sai dương tính sẽ gây hậu quả rất nghiêm trọng, chẳng hạn trong các bài toán như chẩn đoán bệnh, lọc thư rác, hay hệ

thống tuyến dụng tự động. Giả sử nếu mô hình dự đoán bệnh nhân mắc bệnh rối loạn nhịp tim trong khi thực sự bệnh nhân đó không gặp bất kì các bệnh nào hoặc mắc các bệnh khác, điều này dẫn đến việc điều trị trở nên không cần thiết. Chỉ số này càng cao đồng nghĩa với việc mô hình càng ít mắc lỗi khi xác định một đối tượng là dương tính, đem lại sự tin cậy cao trong các dự đoán dương tính. Tuy nhiên, chỉ số này cần được xem xét cùng với độ nhạy để có cái nhìn toàn diện hơn về hiệu quả của mô hình. Độ chính xác của lớp dương được tính theo công thức

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Độ nhạy (Sensitivity, còn gọi là Recall hoặc True Positive Rate) là một chỉ số quan trọng trong việc đánh giá hiệu suất của mô hình phân loại. Chỉ số này thể hiện tỷ lệ các mẫu dương tính thực sự được mô hình dự đoán đúng trên tổng số mẫu dương tính trong tập dữ liệu. Độ nhạy cho biết tỷ lệ các mẫu bất thường được mô hình nhận diện đúng so với tổng số mẫu bất thường thực tế. Giá trị cao thể hiện mô hình ít bỏ sót các trường hợp quan trọng. Giả sử trong chẩn đoán rối loạn nhịp tim dựa trên tín hiệu điện tâm đồ ECG, số âm tính giả cao (Chỉ số FN cao, điều này đồng nghĩa với mô hình dự đoán bình thường trong khi thật sự tình trạng bệnh nhân đang gặp bất thường) có thể dẫn đến việc không phát hiện các tình trạng nguy hiểm, gây rủi ro cho sức khỏe bệnh nhân. Độ nhạy cao giúp giảm thiểu rủi ro này.

Độ nhạy thường được ưu tiên cao các ứng dụng y tế, nơi một vài chẩn đoán sai bất thường có thể yêu cầu kiểm tra lại thường quan trọng hơn so với bỏ sót bệnh. Điều này đảm bảo an toàn cho bệnh nhân, thậm chí với một số báo động giả vẫn có thể được chấp nhận. Ngoài ra, trong dữ liệu mất cân bằng (ví dụ: nhịp tim bình thường chiếm 90%, nhịp bất thường chiếm 10%), độ nhạy tập trung vào lớp thiểu số (bất thường) là lớp dương, giúp đánh giá hiệu suất của mô hình trên các trường hợp hiếm gặp mà độ chính xác có thể không phản ánh đúng.

Độ nhạy và độ chính xác của lớp dương có thể mâu thuẫn: tăng độ nhạy có thể làm giảm độ chính xác của lớp dương. Do đó, cần cân nhắc cả hai chỉ số, thường thông qua điểm F1 để có đánh giá toàn diện.

Độ nhạy được tính bằng công thức

$$\text{Recall} = \frac{TP}{TP + FN}$$

- AIC: Tiêu chí thông tin Akaike (AIC) là một phương pháp toán học dùng để đánh giá mức độ phù hợp của mô hình với dữ liệu mà nó được tạo ra. Trong thống kê, AIC

được sử dụng để so sánh các mô hình khả thi khác nhau và xác định mô hình nào phù hợp nhất với dữ liệu. Tiêu chí này được tính toán từ:

- Số lượng biến độc lập được sử dụng để xây dựng mô hình.
- Ước tính độ tin cậy tối đa của mô hình (mô hình tái tạo dữ liệu tốt như thế nào).

Theo AIC, mô hình phù hợp nhất là mô hình giải thích được mức độ biến thiên lớn nhất khi sử dụng ít biến độc lập nhất có thể. Ví dụ: “*Bạn muốn biết liệu việc uống đồ uống có đường có ảnh hưởng đến cân nặng hay không dựa vào dữ liệu đã thu thập được từ một cuộc khảo sát sức khỏe quốc gia, trong đó có các quan sát về mức tiêu thụ đồ uống có đường, độ tuổi, giới tính và BMI (chỉ số khối cơ thể). Để tìm ra yếu tố nào trong số các yếu tố này quan trọng trong việc dự đoán mối quan hệ giữa lượng tiêu thụ đồ uống có đường và cân nặng cơ thể, bạn hãy tạo một số mô hình khả thi và so sánh chúng thông qua AIC.*”

Trong thống kê, AIC thường được sử dụng để lựa chọn mô hình phù hợp. Bằng cách tính toán và so sánh điểm AIC của một số mô hình khả thi, bạn có thể chọn mô hình phù hợp nhất với dữ liệu.

Khi kiểm định một giả thuyết, có thể thu thập dữ liệu về các biến chưa chắc chắn. Để muốn biết biến độc lập nào đã đo lường giải thích được sự biến thiên của biến phụ thuộc, một trong những cách để tìm hiểu là tạo một tập hợp các mô hình, mỗi mô hình chứa một tổ hợp khác nhau từ các biến độc lập mà bạn đã đo lường. Các tổ hợp này nên dựa trên:

- Kiến thức về hệ thống nghiên cứu – tránh sử dụng các thông số không có mối liên hệ logic!
- Thiết kế thử nghiệm – ví dụ, nếu bạn chia hai phương pháp điều trị cho các đối tượng thử nghiệm, thì có lẽ không có lý do gì để kiểm tra sự tương tác giữa hai phương pháp điều trị.

Sau khi đã tạo ra một số mô hình khả thi, bạn có thể sử dụng AIC để so sánh. Điểm AIC thấp hơn sẽ tốt hơn, và AIC sẽ “phạt” các mô hình sử dụng nhiều tham số hơn. Vì vậy, nếu hai mô hình giải thích cùng một lượng biến thiên, mô hình có ít tham số hơn sẽ có điểm AIC thấp hơn và sẽ là mô hình phù hợp hơn

- BIC: BIC (Bayesian Information Criterion) là một tiêu chí thống kê quan trọng được sử dụng để lựa chọn mô hình, đặc biệt trong phân tích dữ liệu và học máy. BIC xuất phát từ nền tảng lý thuyết Bayes, nhằm tìm sự cân bằng giữa mức độ phù hợp của mô hình với dữ liệu và độ phức tạp của mô hình. Công thức tính của BIC là $BIC = k \cdot \log(n) - 2 \cdot \log(L)$, trong đó n là số quan sát, k là số tham số ước lượng, còn L là giá trị cực đại của hàm hợp lý. Điểm nổi bật của BIC là thành phần phạt $k \cdot \log(n)$. Khi kích thước mẫu

n càng lớn thì hệ số phạt càng lớn, dẫn đến việc BIC thường ưu tiên các mô hình đơn giản và tránh tình trạng quá khớp. Khác với AIC, vốn thường thiên về chọn các mô hình có khả năng dự báo tốt, BIC lại có xu hướng chọn mô hình “đúng” hơn trong số các mô hình ứng viên, miễn là mô hình đó thực sự tồn tại trong tập so sánh. Chính vì vậy, BIC được xem là nhất quán: khi số quan sát tiến về vô hạn, xác suất chọn đúng mô hình thật sẽ tiến về 1.

Trong thực tế, BIC thường được sử dụng khi nhà nghiên cứu muốn có một mô hình gọn nhẹ, dễ diễn giải, và quan tâm nhiều đến bản chất cấu trúc dữ liệu hơn là chỉ tối ưu dự báo. Tuy nhiên, BIC cũng không phải là tuyệt đối, bởi BIC phụ thuộc vào tập mô hình ứng viên và giả định về phân phối dữ liệu. Do đó, trong nhiều trường hợp, người ta kết hợp BIC với các tiêu chí khác hoặc phương pháp như xác thực chéo để đảm bảo lựa chọn mô hình hợp lý và tin cậy hơn.

4.4.2. Huấn luyện – kiểm thử và kiểm thử A/B

Trong nhiều ví dụ ở các chương trước, sau khi xây dựng một mô hình bằng dữ liệu huấn luyện, người ta thường kiểm tra hiệu quả mô hình trên một tập dữ liệu kiểm tra. Tập kiểm tra này thường nhỏ hơn so với tập huấn luyện, và quan trọng là mô hình chưa từng “thấy” dữ liệu này trước đó. Lý do của chiến lược này là để đánh giá khả năng khái quát hóa của mô hình. Ở dạng cơ bản nhất, khái quát hóa chính là việc dự đoán dựa trên những quan sát đã biết. Khái quát hóa có thể xảy ra theo hai cách bao gồm

- Đầu tiên khi các nhà nghiên cứu nói về khái quát hóa, họ muốn ám chỉ việc dự đoán cho cả một quần thể lớn dựa trên kết quả của một mẫu nghiên cứu đại diện. Ví dụ: để trả lời câu hỏi “Có bao nhiêu phần trăm dân số Mỹ ủng hộ Đảng Dân chủ?”, ta không thể hỏi tất cả mọi người có quyền bầu cử. Thay vào đó, nhà nghiên cứu khảo sát một số người và mở rộng kết quả sang cả quần thể. Trong trường hợp này, điều quan trọng là mẫu khảo sát phải đại diện cho quần thể, nghĩa là phải bao gồm đúng các nhóm có liên quan theo tỷ lệ hợp lý.

- Thứ hai, khái niệm khái quát hóa cũng giúp chuyển từ quan sát khoa học sang lý thuyết hoặc giả thuyết. Chẳng hạn, vào thập niên 1940–1950, hai nhà nghiên cứu Richard Doll và Bradford Hill ở Anh nhận thấy 647 trong số 649 bệnh nhân ung thư phổi ở các bệnh viện London đều là người hút thuốc. Quan sát này đã thúc đẩy nhiều nghiên cứu tiếp theo với kích cỡ mẫu lớn hơn, nhiều nhóm người khác nhau và mức độ hút thuốc khác nhau. Khi kết quả ở nhiều bối cảnh khác nhau đều nhất quán, quan sát ban đầu được khái quát hóa thành một lý thuyết: hút thuốc lá gây ung thư phổi.

Trong trường hợp này, kiểu khái quát hóa thứ nhất được áp dụng. Để kiểm tra tính khái quát hóa, bước quan trọng là tách tập dữ liệu ban đầu thành tập huấn luyện và tập

kiểm tra. Thông thường, phần lớn dữ liệu được dùng để huấn luyện, còn một phần nhỏ dành cho kiểm tra. Việc tách phải ngẫu nhiên và không thiên lệch, để cả hai tập có sự tương đồng về mức độ biến thiên và đại diện cho quần thể.

Đôi khi, thay vì chỉ chia thành hai tập, người ta còn bổ sung tập xác thực. Tập này được dùng để giảm tình trạng quá khớp. Ví dụ, trong mô hình cây quyết định, quá trình huấn luyện có thể liên tục điều chỉnh ranh giới phân chia để giảm lỗi. Tuy nhiên, nếu điều chỉnh quá nhiều, mô hình có thể chỉ phù hợp với dữ liệu huấn luyện mà không dự đoán tốt với dữ liệu khác. Lúc này, tập xác thực được sử dụng để kiểm chứng rằng việc cải thiện độ chính xác trên tập huấn luyện cũng đi kèm với cải thiện trên dữ liệu chưa từng được mô hình học trước đó.

Ngoài ra, trong những tình huống chưa xác định rõ biến nào quan trọng hơn hoặc phiên bản nào mang lại hiệu quả cao hơn, ta có thể áp dụng phương pháp kiểm thử A/B. Đây là một thí nghiệm có kiểm soát với hai biến thể (A và B), nhằm so sánh phản ứng của người dùng để xác định biến thể nào tối ưu hơn. Ví dụ, một công ty giao đồ ăn muốn thu hút khách hàng thông qua mã giảm giá; tuy nhiên, họ chưa biết nên quảng cáo qua email hay mạng xã hội. Do đó, công ty tạo hai mẫu quảng cáo giống nhau, chỉ khác kênh phân phối và mã khuyến mãi. Bằng cách theo dõi mã nào được sử dụng nhiều hơn, công ty có thể biết được kênh hiệu quả hơn. Trong thực tế, kiểm thử A/B không chỉ giới hạn ở hai điều kiện mà có thể mở rộng ra nhiều biến thể khác. Điều này đặc biệt hữu ích trong thương mại điện tử và các dịch vụ trực tuyến với số lượng người dùng lớn.

4.4.3. Xác thực chéo

Một cách khác thay thế cho việc tách riêng tập huấn luyện và tập kiểm tra như đã đề cập để đánh giá khả năng khái quát hóa là kỹ thuật xác thực chéo (cross-validation), đôi khi còn được gọi là ước lượng xoay vòng. Trong kỹ thuật này, tập dữ liệu gốc lại được chia thành một tập để huấn luyện mô hình và một tập để kiểm tra, cách này thường được áp dụng khi không có sẵn tập kiểm tra riêng. Trong thực tế có nhiều biến thể của xác thực chéo.

Hình thức đơn giản nhất của phương pháp này là phương pháp giữ lại (hold-out). Ở đây, tập dữ liệu được chia thành hai tập tách biệt: tập huấn luyện và tập kiểm tra. Mô hình được xây dựng dựa trên tập huấn luyện, sau đó dùng để dự đoán giá trị đầu ra của tập kiểm tra chưa từng thấy trước đó. Các lỗi dự đoán được cộng dồn, được gọi là lỗi trung bình tuyệt đối trên tập kiểm tra. Hiển nhiên, sai số này càng nhỏ càng tốt, và được sử dụng làm thước đo đánh giá mô hình.

Một phương pháp khác cải tiến hơn so với phương pháp giữ lại là phương pháp xác thực chéo k lần (k-fold cross-validation). Trong trường hợp này, tập dữ liệu được chia

thành k phần bằng nhau. Quá trình đánh giá được thực hiện k lần: mỗi lần, một phần dữ liệu được giữ lại để kiểm tra, còn $k - 1$ phần còn lại được dùng để huấn luyện. Có thể xem đây là việc lặp lại phương pháp giữ lại k lần. Sai số trung bình trên cả k lần được tính toán và sử dụng như độ chính xác tổng thể của mô hình. Ưu điểm của phương pháp này là mô hình ít bị ảnh hưởng bởi cách chia dữ liệu, vì mỗi điểm dữ liệu đều có một lần làm dữ liệu kiểm tra và $k - 1$ lần làm dữ liệu huấn luyện. Do đó, khi k tăng, phương pháp này giúp giảm phương sai của ước lượng. Tuy nhiên, nhược điểm là chi phí tính toán cao hơn: thuật toán huấn luyện phải chạy k lần thay vì chỉ một lần như ở phương pháp holdout.

Một biến thể khác của phương pháp k -fold là xác thực chéo loại bỏ một phần tử (Leave-one-out cross-validation hay LOO-CV), trong đó k bằng với số lượng điểm dữ liệu N . Khi đó, mô hình được huấn luyện trên toàn bộ dữ liệu ngoại trừ một tập duy nhất làm tập kiểm tra, và quá trình này lặp lại cho đến khi tất cả các tập đều được dùng làm tập kiểm tra đúng một lần. Sai số trung bình trên N lần đánh giá được tính toán và dùng làm sai số tổng thể của mô hình. Phương pháp này cho kết quả đánh giá rất tốt, nhưng chi phí tính toán lại cực kỳ lớn, đặc biệt khi số lượng dữ liệu rất lớn nên phương pháp này thường ít gặp trong thực tế.

4.5. Câu hỏi và bài tập Chương 4

Bài 1: (*Câu hỏi lý thuyết*) So sánh phương pháp thu thập dữ liệu định lượng và định tính.

Bài 2: (*Bài tập thiết kế*) Thiết kế bảng câu hỏi khảo sát (survey) để thu thập ý kiến người dùng về chất lượng mạng 5G.

Bài 3: (*Câu hỏi thực tế*) Khi thu thập dữ liệu từ log hệ thống, cần lưu ý những vấn đề gì về tính toàn vẹn và bảo mật?

Bài 4: (*Bài tập phân tích*) Cho kịch bản A/B testing để đánh giá hiệu quả giao diện ứng dụng, hãy nêu các bước triển khai và chỉ số đánh giá.

Bài 5: (*Câu hỏi lý thuyết*) Giải thích ý nghĩa của phương pháp xác minh chéo (cross-validation) và ưu điểm so với chia tập train/test thông thường.

Bài 6: (*Câu hỏi bài tập*) Một cột dữ liệu tuổi trong dataset bị thiếu 30% giá trị. Bạn sẽ xử lý thế nào?

Bài 7: (*Câu hỏi đánh giá*) Trong bài toán hồi quy, khi nào thì sử dụng MAE, khi nào dùng RMSE để đánh giá mô hình?

Bài 8: (*Bài tập tình huống*) Đề xuất phương pháp thu thập và phân tích dữ liệu để đánh giá mức độ hài lòng của sinh viên với môn học trực tuyến.

Bài 9: (*Bài hỏi tích hợp*) Làm thế nào để kết hợp dữ liệu từ khảo sát và dữ liệu log hệ thống để cải thiện trải nghiệm người dùng?

Bài 10: (*Bài tập tổng hợp*) Cho một vấn đề thực tế (ví dụ: tối ưu hóa lưu lượng mạng), hãy phác thảo quy trình từ thu thập dữ liệu đến đánh giá mô hình phân tích.

Bài 11: (*Bài tập nhóm*) Thiết kế nghiên cứu hỗn hợp (mixed methods) để đánh giá trải nghiệm người dùng ứng dụng mobile.

Bài 12: (*Bài tập nhóm*) Triển khai A/B testing để tối ưu hóa tỷ lệ chuyển đổi (conversion rate) trên website thương mại điện tử.

Bài 13: (*Bài tập nhóm*) Xây dựng hệ thống giám sát chất lượng dữ liệu (Data Quality Monitoring) tự động.

Bài 14: (*Bài tập nhóm*) Nghiên cứu các phương pháp thu thập dữ liệu không xâm phạm (non-intrusive data collection).

Bài 15: (*Bài tập nhóm*) Thiết kế chiến lược sampling cho nghiên cứu thị trường quy mô lớn.

Bài 16: (*Bài tập nhóm*) Phân tích và xử lý bias trong dữ liệu thu thập từ mạng xã hội.

Bài 17: (*Bài tập nhóm*) Triển khai hệ thống feature store để quản lý và versioning dữ liệu cho học máy.

Bài 18: (*Bài tập nhóm*) Nghiên cứu phương pháp đánh giá mô hình học máy trong môi trường production.

Bài 19: (*Bài tập nhóm*) Xây dựng framework đánh giá tác động kinh doanh (business impact) của các dự án dữ liệu.

Bài 20: (*Bài tập nhóm*) Thiết kế hệ thống feedback loop để cải tiến liên tục chất lượng dữ liệu và mô hình.

TÀI LIỆU THAM KHẢO

- [1]. Joe Reis and Matt Housley, Fundamentals of Data Engineering: Plan and Build Robust Data Systems, O'Reilly Media, 2022.
- [2]. Chirag Shah, A Hands-On Introduction to Data Science, Cambridge University Press, 2020.
- [3]. Francesco Corea, An Introduction to Data, Springer Nature Switzerland AG, 2019.
- [4]. Avrim Blum, John Hopcroft, Ravindran Kannan, Foundations of Data Science, Cambridge University Press, 2020.
- [5]. Paul Murrell, Introduction to data technologies, Chapman & Hall/CRC, 2009.

THƯ VIỆN PTIT